A Durable Cache for Pingora®[1]

Requirements Specification

Luis Malheiro

2026

© 2026 Luis Malheiro. Licensed under CC BY 4.0.

[1] *Cloudflare* and *Pingora* are trademarks or registered trademarks of Cloudflare, Inc. The Pingora framework is open-source software, © Cloudflare, Inc., released under the Apache License, Version 2.0. This document is an independent requirements specification; it is not affiliated with, endorsed by, or sponsored by Cloudflare, Inc. All other trademarks are the property of their respective owners.

Contents

1 Introduction **2**
- 1.1 The Pingora Framework . . . 2
- 1.2 Purpose of This Document . . . 2
- 1.3 The Refinement Model . . . 3

2 System Context **7**
- 2.1 HTTP Caching in `pingora-cache` . . . 7
- 2.2 How `HttpCache` Coordinates the `CacheLock`, `Storage`, and the `EvictionManager` . . . 7

3 High-Level Requirements **15**
- [HLR-SCOPE.V1-01] First Functional Version . . . 15
- [HLR-PERSISTENCE.DURABLE-01] Durable Persistence . . . 15
- [HLR-PERSISTENCE.CAPACITY-01] Capacity beyond RAM . . . 17
- [HLR-PERSISTENCE.CRASHSAFE-01] Host-crash safety . . . 17
- [HLR-EFFICIENCY.SPACE-01] Space Efficiency . . . 19
- [HLR-PERFORMANCE.LOOKUP-01] Lookup latency flat with size . . . 20
- [HLR-COMPAT.PINGORA-01] Compatible with Pingora . . . 21
- [HLR-COMPLIANCE.RFC-01] No added RFC 9111/9110 deviation . . . 22

4 High-Level Refinement **26**
- [FRE-COMPAT.UNSIZED-01] Bodies of unknown length . . . 26
- [FRE-COMPAT.CONCURRENT-01] Concurrent same-key writes . . . 27
- [FRE-COMPAT.EVICTION-01] Eviction state consistency . . . 29
- [FRE-COMPAT.HOTUPGRADE-01] Hot-upgrade cache consistency . . . 31
- [FRE-COMPAT.STREAMFLAG-01] Advertise streaming writes . . . 34
- [FRE-COMPAT.STREAMING-01] Partial hits on lookup . . . 36
- [FRE-COMPAT.STREAMDELIVERY-01] Partial-hit chunk delivery . . . 38
- [FRE-COMPAT.STREAMEND-01] Partial-hit completion . . . 39
- [FRE-COMPAT.FORMATUPGRADE-01] Records across Pingora changes 40
- [FRE-COMPAT.WRITEFAIL-01] Write failures fail the store . . . 41
- [FRE-COMPAT.ABORTSPACE-01] Aborted stores release space . . . 42
- [FRE-COMPAT.STARTRECLAIM-01] Reclaim leftovers at start . . . 44
- [FRE-COMPAT.EXPIRE-01] Expire keeps the body . . . 45
- [ADR-PERSISTENCE.MECHANISM-01] Plain file system storage . . . 47
- [REQ-EFFICIENCY.FRAGMENTATION-01] Sub-linear small-object overhead . . . 52
- [REQ-EFFICIENCY.DIRSCALE-01] Directory-entry scaling . . . 53

5 Detailed Refinement **56**
- [ADR-STORAGE.OBJECTFILE-01] Combined metadata and body file . . . 56
- [ADR-PERSISTENCE.DURABILITY-01] Host-crash safety via sync and CRC . . . 58
- [ADR-STORAGE.ATOMICPUBLISH-01] Publish by same-directory rename 62
- [ADR-STORAGE.WRITERID-01] Writer identity in temp names . . . 64
- [FDS-STORAGE.PUBLISH-01] Atomic publication via rename . . . 67

[FDS-STORAGE.PARTIAL-01] Streaming partial hits 72
[FDS-STORAGE.METAUPDATE-01] Metadata update: in place or withdrawn 74
[FDS-OBSERVABILITY.HEADROOM-01] Headroom-exceeded observability 80
[ADR-STORAGE.METAMOVE-01] Metadata moves deferred for v1 . . . 82
[ADR-OBSERVABILITY.METRICS-01] Metrics via the prometheus crate 84
[ADR-STORAGE.EXPIRE-01] Expire by rewriting the metadata 86
[FDS-STORAGE.PURGE-01] Purge via canonical-name unlink 89
[FDS-STORAGE.EXPIRE-01] Expire by updating the metadata 95
[ADR-STORAGE.EXECUTOR-01] File I/O on the blocking pool 99
[FDS-STORAGE.EXECUTOR-01] Storage calls never block the worker . . 101
[ADR-STORAGE.RECLAIM-01] Reclaim via the restart-time scan 104
[FDS-STORAGE.RECLAIM-01] Reclaiming orphaned files 108
[ADR-EFFICIENCY.COMPACTION-01] Compaction deferred for v1 . . . 112
[ADR-COMPAT.EVICTION-01] Eviction-state reconstruction strategy . . 114
[ADR-COMPAT.ENTRYID-01] No storage-assigned entry IDs 117
[ADR-COMPAT.READONLY-01] Read-only on the upgrade signal 119
[FRE-COMPLIANCE.HEADERS-01] Faithful header round-trip 121
[FRE-COMPLIANCE.TRAILERS-01] Trailer fields kept distinct 123
[FRE-COMPLIANCE.BODY-01] Faithful round-trip of the body 124
[FRE-COMPLIANCE.FRESHNESS-01] Preserve freshness and age inputs 125
[FRE-COMPLIANCE.VALIDATORS-01] Preserve validator fields 126
[FRE-COMPLIANCE.REVALIDATION-01] Freshen metadata 127
[FRE-COMPLIANCE.METAATOMIC-01] Atomic metadata update 129
[FRE-COMPLIANCE.METAPERSIST-01] Freshened metadata persists . . 130
[FRE-COMPLIANCE.METAABSENT-01] Update of an absent key 132
[FRE-COMPLIANCE.REPLACE-01] Replace stored responses atomically . 133
[FRE-COMPLIANCE.READERISOLATION-01] Readers survive replace/purge 134
[FRE-COMPLIANCE.VARY-01] Store and select Vary variants 136
[FRE-COMPLIANCE.VARYISOLATION-01] Variant isolation 137
[FRE-COMPLIANCE.PURGE-01] Honour purge and invalidation 137
[FRE-COMPLIANCE.PURGEABSENT-01] Purge or expiry of absent key . 139
[FRE-COMPLIANCE.PURGEDURABLE-01] Purge acknowledged once durable 140
[FRE-COMPLIANCE.INCOMPLETE-01] Never serve incomplete responses 142
[FRE-COMPLIANCE.PARTIALABORT-01] Aborted partial hits end in error144

6 Structural and Functional Design 146
[SDS-LAYOUT.SHARDING-01] Directory sharding 146
[FDS-LAYOUT.STARTUP-01] Startup: shard directory cache 149
[SDS-LAYOUT.OBJECTFILE-01] Object-file record layout 151
[SDS-LAYOUT.KEYSEGMENT-01] Key segment encoding 160
[FDS-EVICTION.STARTUP-01] Startup: reconstruction procedure 162
[FDS-COMPAT.READONLY-01] Read-only on the upgrade signal 166

7 Test Case Specifications **169**
[TC-COMPLIANCE.RFC-01] No regression against MemCache 169
[TC-COMPLIANCE.HEADERS-01] Header fields round-trip faithfully . . 170
[TC-COMPLIANCE.TRAILERS-01] Trailer fields kept distinct 171
[TC-COMPLIANCE.BODY-01] Body round-trips byte for byte 172
[TC-COMPLIANCE.FRESHNESS-01] Freshness and age inputs preserved 173
[TC-COMPLIANCE.VALIDATORS-01] Validator fields preserved 174
[TC-COMPLIANCE.REVALIDATION-01] Metadata freshened atomically . 175
[TC-COMPLIANCE.VARY-01] Vary variants stored and selected 177
[TC-COMPLIANCE.PURGE-01] Purge makes all variants unreachable . . 179
[TC-COMPLIANCE.INCOMPLETE-01] Incomplete responses never served 180
[TC-COMPLIANCE.REPLACE-01] Stores replace atomically 181
[TC-PERSISTENCE.CRASHSAFE-01] Host crash serves no bad content . 182
[TC-PERSISTENCE.RESTART-01] Objects survive a process exit 184
[TC-PERSISTENCE.CAPACITY-01] Cache larger than host RAM 185
[TC-EFFICIENCY.SPACE-01] Space overhead measured 186
[TC-PERFORMANCE.LOOKUP-01] Lookup latency flat with size 187
[TC-COMPAT.EVICTION-01] Exact eviction state after restart 188
[TC-COMPAT.HOTUPGRADE-01] Cache consistent across hot upgrade . 189
[TC-COMPAT.STREAMING-01] Followers stream a partial hit 190
[TC-COMPAT.UNSIZED-01] Unsized body accepted and persisted 192
[TC-COMPAT.CONCURRENT-01] Same-key writes stay isolated 194
[TC-COMPAT.FORMATUPGRADE-01] Foreign records are misses 195
[TC-COMPAT.WRITEFAIL-01] Write failures fail the store 196
[TC-COMPAT.ABORTSPACE-01] Aborted stores release space 197
[TC-EVICTION.RECONSTRUCT-01] Reconstruction converges exactly . 198
[TC-STORAGE.PUBLISH-01] Atomic same-directory publication 199
[TC-STORAGE.PURGE-01] Purge unlinks canonical name only 200
[TC-STORAGE.EXPIRE-01] Expire keeps the body readable 203
[TC-STORAGE.EXECUTOR-01] Storage never blocks the worker 204
[TC-STORAGE.RECLAIM-01] Orphan reclaim at restart 206
[TC-LAYOUT.SHARDING-01] Shard layout and directory bound 208
[TC-LAYOUT.DIRCACHE-01] Shard cache matches disk at start 209
[TC-LAYOUT.OBJECTFILE-01] Damaged records treated as a miss . . . 211
[TC-LAYOUT.KEYSEGMENT-01] Key segment round trip 212

8 Traceability Analysis **215**

A Storage Backend Alternatives **256**
A.1 Raw Device / Cyclone Store: Apache Traffic Server 256
A.2 Embeddable Cache Engine: Meta's CacheLib 256
A.3 Embedded Database: libmdbx/LMDB, RocksDB 257

B The `http-tests/cache-tests` Test Suite **259**
B.1 What It Is 259
B.2 Architecture 259
B.3 Mapping to `FRE-COMPLIANCE.*` 260
B.4 Coverage Gaps 261

1. Introduction

1.1 The Pingora Framework

Pingora is an open-source Rust framework, developed and open-sourced by Cloudflare, for building fast, reliable, and programmable networked systems.[1,2,3] It was created as a modern, memory-safe alternative to C/C++-based proxies and has been battle-tested in production at Cloudflare, where it serves more than 40 million Internet requests per second. Pingora is built entirely in Rust on top of an asynchronous runtime, and provides the building blocks for HTTP/1 and HTTP/2 proxies, gRPC and WebSocket proxying, load balancing, graceful reloads, and rich observability.

The framework is organised as a collection of crates, each addressing a distinct concern. The most relevant to this specification are:

- `pingora-core` — transport protocols and the core traits that define server and service behaviour.
- `pingora-proxy` — the HTTP proxy state machine and the `ProxyHttp` extension points used to customise proxy logic.
- `pingora-cache` — the HTTP caching layer, including the storage backend abstraction that this work targets.

1.2 Purpose of This Document

This document specifies the requirements for a *file-system-backed* implementation of the `pingora-cache` `Storage` trait. By persisting cached objects on disk rather than solely in memory, the cache can retain entries across process restarts and scale well beyond the limits of main memory, while remaining a drop-in backend behind the existing `HttpCache` state machine. The driving goals are:

- **Persistent storage** — use the file system as a durable medium for cached HTTP objects so that the cache survives restarts and grows beyond RAM capacity.
- **Standards compliance** — preserve the RFC 9111 and HTTP caching semantics already enforced by `pingora-cache`, so persistence does not compromise correctness.
- **Efficient use of space** — store and reclaim cached objects economically, supporting eviction and avoiding unnecessary disk overhead.

[1] cloudflare/pingora on GitHub
[2] Cloudflare blog: "Open sourcing Pingora"
[3] Cloudflare changelog, 4 May 2026

1.3 The Refinement Model

Artifacts and Relationships

The artifact types and the relationships between artifacts take inspiration from SysML©, the Object Management Group's Systems Modeling Language,[4][5][6] without aiming to follow it strictly.

Every artifact has a type, given by the prefix of its ID, and every type carries a *stereotype*: a label written between guillemets, such as «stakeholder requirement» or «design constraint», that names the type's category. Table 1.1 lists every artifact type with its prefix and stereotype. Two types can share a stereotype — functional designs (FDS) and functional requirements (FRE) are both «functional requirement» — so it is the prefix that tells an artifact's kind. The diagrams draw each artifact as a card headed by its stereotype, and each relationship as an arrow labelled with the relationship's name, pointing from the artifact that depends to the one it depends on.

Each relationship has one job in this model:

- *Satisfies* («satisfy»): a decision or a design satisfies the requirements it answers.
- *Derives* («deriveReqt»): a requirement derives from the artifact it comes from. Usually that is a broader requirement it breaks down, as a functional requirement breaks down a stakeholder need. It can also be a decision: choosing a mechanism creates constraints of its own that no stakeholder asked for, and those derive from the decision that created them (for example, choosing a plain file system, ADR-PERSISTENCE.MECHANISM-01, is what makes the number of entries in one directory matter, so REQ-EFFICIENCY.DIRSCALE-01 derives from that decision). A requirement can have both kinds of parent.
- *Refines* («refine»): a decision or a design refines the broader one it works out in more detail. A design normally refines the decision it carries out. It may instead refine another design, when it builds on terms that design defines and no decision of its own lies between the two (for example, the design that logs and counts metadata that outgrew its record, FDS-OBSERVABILITY.HEADROOM-01, refines the design that defines when that happens, FDS-STORAGE.METAUPDATE-01). The link to the decision is not lost: the other design refines its own decision in turn.
- *Verifies* («verify»): a test case verifies the requirement, decision, or design it checks.
- *Traces* («trace»): records any other dependency that none of the above fits.
- *Conflicts with* («conflicts with»): marks two artifacts that cannot both be met as stated; the artifact that declares the conflict also states how it is to be resolved.

[4] OMG©

[5] SysML©

[6] P. Roques, "Modeling Requirements with SysML©", *Requirements Engineering Magazine*, IREB, April 2015, section "Requirement Relationships".

Table 1.1: Artifact types and their stereotypes

Prefix	Stereotype	Artifact type
HLR	*«stakeholder requirement»*	High-Level Requirement: a stakeholder need or expectation, stated without commitment to a solution.
REQ	*«requirement»*	Base Requirement: a solution-neutral statement of capability or constraint — here, chiefly the derived constraints a foundational decision generates.
FRE	*«functional requirement»*	Functional Requirement: a solution-neutral statement of a verifiable function the backend must perform — the explicitly functional counterpart to the unbranded Base Requirement (REQ).
ADR	*«design constraint»*	Architecture Decision Record: a choice among alternatives, closing design freedom; argued by context, alternatives, and consequences — ranging from the foundational, solution-shaping decision for a domain down to a single focused mechanism.
SDS	*«structural requirement»*	Structural Design: the static aspect of a design — the shape realising the decisions above it (layouts, modules, deployment); it specifies rather than decides.
FDS	*«functional requirement»*	Functional Design: the dynamic aspect of a design — the behaviour realising the decisions above it (flows, sequence diagrams); it specifies rather than decides.
TC	*«test case»*	Test Case: verifies that a requirement, decision, or design holds.

Tied together this way, the artifacts form a chain that can be read forwards (goal to test) or traced backwards (test to goal); the Traceability Analysis chapter renders it explicitly. Table 1.2 summarises the relationships and the direction of their arrows.

Flow of Refinement

Rather than grouping its contents by artifact type — all requirements in one chapter, all architecture decisions in another — this document is organised to follow the *flow of refinement*: the order in which the artifacts are discovered as a need is progressively elaborated from an abstract goal down to a verifiable design. Each chapter is one stage in that flow, and any stage may contain artifacts of *any* type; what a chapter groups is a *degree of refinement*, not a kind of artifact. The stages are preceded by a *System Context* chapter that documents, non-normatively, the `pingora-cache` behaviour the artifacts cite as their basis.

The stages, and the artifact types that typically appear in each, are:

1. **High-Level Requirements** (HLR, and use cases UC) — the stakeholder needs the cache must meet, stated without commitment to a solution.

Table 1.2: Relationships between artifacts

Relationship	Arrow (from → to)	Meaning
«satisfy»	Decision, design, or implementation → requirement	Fulfils the requirement
«deriveReqt»	Derived requirement → its source	Follows from the source — a broader requirement it breaks down, or a decision whose choice created it
«refine»	Detailed artifact → broader artifact	Works the broader artifact out in more detail
«verify»	Test case → artifact	Checks that the artifact holds
«trace»	Any → any	A dependency no more specific relationship describes
«conflicts with»	Either way	The two cannot both be met as stated; the declaring artifact states how the conflict is to be resolved

2. **High-Level Refinement** — the first elaboration of those needs: the foundational architecture decision that shapes the whole solution (an ADR, *Architecture Decision Record*, at its broadest scope). A decision, here or later, *generates* requirements of its own — stated as base requirements (REQ) when left unbranded, or as functional requirements (FRE) when they state, in verifiable terms, *what* the backend must do; either kind may instead derive directly from the stakeholder need itself. The later, more detailed decisions must satisfy whichever of these the earlier ones generated.

3. **Detailed Refinement** — the decisions that follow from the high-level one: further architecture decisions (ADR) at a narrower scope, down to a single mechanism.

4. **Structural and Functional Design** — concrete designs realising those decisions: structural designs (SDS, e.g. the on-disk layout) and functional designs (FDS, e.g. sequence diagrams).

5. **Test Case Specifications** (TC) — the checks that verify the requirements and designs of the preceding stages.

Table 1.1 in Section 1.3 lists the artifact types in this same order.

The stages order the artifacts; two further axes tell them apart. The first is *altitude*: an ADR can sit anywhere from the foundational choice of storage medium down to a single mechanism. The second is *kind*: a *requirement* (HLR, REQ, FRE) states a need without choosing a solution; a *decision* (ADR) argues a choice among alternatives — recorded as context, alternatives, and consequences — and closes design freedom; a *design* specifies the chosen result — SDS its static shape, FDS its dynamic behaviour — and no longer decides anything; and a *test case* (TC) verifies that what was required, decided, or designed actually holds.

Two tests keep the kinds apart in practice. The *deletion test*: if removing an artifact would force a trade-off to be argued again, it is a decision; if the trade-off would stay settled but the implementer would lose the blueprint, it is a design.

The *change-trigger test*: a decision is reopened when its forces change — a new constraint, a new alternative, a shifted trade-off — while a design is revised when the decision stands but its realisation needs adjustment.

2. System Context

This chapter documents the environment the specified backend must fit into: the `pingora-cache` caching layer of Pingora v0.9.0 — its `Storage` abstraction and the components that `HttpCache` coordinates around it. The requirements chapters cite this material as the *basis* of their artifacts; nothing in it is normative on its own.

2.1 HTTP Caching in `pingora-cache`

The `pingora-cache` crate provides a standards-aware HTTP caching layer for proxies built on Pingora.[1] Its central component, `HttpCache`, is the state machine that drives the caching lifecycle, advancing a request through the phases tracked by `CachePhase` (cache key, miss, stale, expired, hit, revalidated, and so on). Although it does not claim full RFC 9111 conformance, the crate applies HTTP caching semantics defined by RFC 9111 (*HTTP Caching*) and RFC 9110 (*HTTP Semantics*) — parsing `Cache-Control` directives, computing freshness, and performing conditional revalidation — so that cache behaviour follows the HTTP standards rather than being ad hoc.

Crucially for this specification, `pingora-cache` decouples the caching *policy* from the caching *storage* through the `Storage` trait. `Storage` is a backend-agnostic interface: an implementation is responsible for reading cached objects (returning a `HitHandler`), for writing new ones (returning a `MissHandler`), and for removing them — either deleting an object outright (`purge`) or, since v0.9.0, keeping it but marking it stale so it revalidates against the origin rather than being refetched (`expire`) — keyed by `CacheKey`. Cached responses are identified by a `CacheKey` that, together with `VarianceBuilder`, supports multiple variants of the same URL (e.g. content negotiated by `Vary`). Around this core the crate adds a cache lock to prevent cache stampedes, an eviction module for capacity management, and a predictor for cacheability decisions.

The crate ships with `MemCache`, a hash-map-based, in-memory implementation of `Storage`. While convenient, an in-memory store does not survive process restarts and is bounded by available RAM, which makes it unsuitable as the system of record for a large, long-lived cache.

2.2 How `HttpCache` Coordinates the `CacheLock`, `Storage`, and the `EvictionManager`

Four components cooperate on every cached request, and their division of labour underpins the requirements that follow. `HttpCache` is the *orchestrator* — the per-request state machine that decides what to do at each `CachePhase`. The `Storage` backend, the subject of this document, is a largely passive *object store* that reads and writes cached representations on demand. The `EvictionManager` is the *capacity policy*: an in-memory index of which objects exist, their sizes, and their recency, from which it selects victims when space must be reclaimed. The `CacheLock` is the *concurrency coordinator*: an in-memory table of in-flight cache keys that serialises

[1] cloudflare/pingora: the `pingora-cache` crate (v0.9.0)

the work of populating or revalidating any one object, so a burst of simultaneous requests for the same uncached URL does not become a stampede of identical upstream fetches and Storage writes.[2]

These collaborators are handed to HttpCache when the proxy configures it — the enable call set up in the proxy's ProxyHttp hooks takes the Storage backend (the only mandatory collaborator) together with the EvictionManager, the cacheability predictor, and the CacheLock, each *optional*[3] — and from then on each is invoked through HttpCache rather than by the backend. The three are *siblings*: Storage, the EvictionManager, and the CacheLock never call one another directly. It is the proxy that drives HttpCache through these phases — calling cache_lookup, cache_miss, set_miss_handler, finish_miss_handler, and (for waiters) cache_lock_wait in a lookup loop — while HttpCache in turn acquires and releases the lock and calls Storage and the EvictionManager. So the lock's *wait-and-retry* (block on a lock another request holds, then retry the lookup) is the proxy's loop, not HttpCache's own.[4] (Figures 2.1, 2.3 and 2.4).

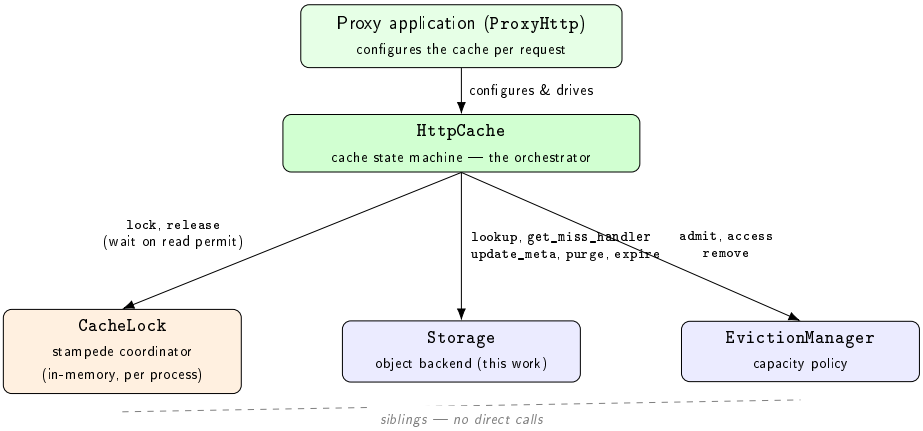


Figure 2.1: The proxy configures and drives HttpCache, which in turn drives the CacheLock, the Storage backend, and the EvictionManager — handed to it, already constructed, by the proxy's enable call; the three are siblings and never call one another directly.

The interactions follow the cache lifecycle:

- **Hit.** HttpCache asks Storage to lookup the key;[5] on a hit it receives a HitHandler to stream the stored body and records the touch against the eviction manager (access)[6] so that recency ordering stays current. A hit does not engage the lock (Figure 2.2).

[2] Source references in this section are to Pingora v0.9.0. The lock lives in pingora-cache/src/lock.rs: a per-key CacheLock table with lock / release operations and a LockStatus returned to waiters.

[3] HttpCache::enable takes all four collaborators; only the Storage backend is mandatory — the eviction manager, the predictor, and the cache lock are each an Option.

[4] The proxy's cache loop lives in HttpProxy::proxy_cache: it calls cache_lookup, cache_lock_wait and retries; the body-write path calls set_miss_handler and finish_miss_handler.

[5] HttpCache::cache_lookup calls storage.lookup(key, &span.handle()) directly; the lock is only taken when the result is None — a hit never reaches that branch.

[6] eviction.access(&cache_key, size, meta.0.internal.fresh_until) runs inside

- **Miss (under the lock).** A `lookup` that finds nothing acquires the `CacheLock` for the key.[7] Exactly one request receives the write permit and becomes the *leader* and fetches upstream; a concurrent request for the same key does *not* re-fetch. A streaming-capable backend's `lookup` — such as the stock `MemCache`'s — prefers a *partial* read, so once the leader's write is open a concurrent request gets a streaming reader and serves the body *as the leader writes it*; only a request that looks up *before* the write target exists takes a read lock and waits — and then only until the leader opens the write.[8] The leader calls `get_miss_handler` on `Storage` and writes the upstream response through it. The lock is released *as soon as the object is readable* — after the write finishes, or, for a backend that supports streaming partial writes, as early as right after the miss handler opens, so followers can stream the partial body while it is still being written.[9] Crucially the `EvictionManager` is updated *after* the lock is released and outside it: `admit` records the new entry and returns the victims to evict, which `HttpCache` then `purge`s from `Storage` asynchronously (Figure 2.3).[10] What the manager is told is not the bare cache key but a `CacheEntryKey`[11] — the cache key, optionally paired with an identifier the backend assigns to the particular stored generation — so that a backend retaining several generations of one key can have eviction account for the exact one. A backend that publishes a single object per key, as the backend specified by this document does (ADR-COMPAT.ENTRYID-01), returns no identifier and the entry key is the cache key alone. Releasing the lock wakes the waiters with a status[12]: normally *done*, so each re-runs `lookup` and now hits (Figure 2.4); if the leader failed, or held the lock past its age limit, the waiters recompete and one becomes the new leader; if the asset proved uncacheable — or a waiter's own wait timed out — the affected waiters fetch independently with caching disabled.[13] In the normal case the upstream fetch and the `Storage` write thus happen once, not once per waiter (a timed-out or given-up waiter fetches for itself, bypassing the cache).

`HttpCache::cache_found` — the separate call the proxy makes once it decides the lookup is a genuine, non-stale hit — and only when `hit_handler.should_count_access()`.

[7] `HttpCache::cache_lookup` takes the lock when storage returns nothing; the stale/treated-as-miss path does so in `cache_found`.

[8] `MemCache::lookup` "always prefer[s] partial read", returning a reader that streams the in-progress write; a partial read does not count an eviction `access` (`should_count_access`).

[9] Non-streaming backends release in `finish_miss_handler` once the body is written; streaming backends release right after opening the miss handler (`lib.rs L896`).

[10] In `finish_miss_handler` the order is `finish` → `release` → `admit` → spawn `purge`; eviction runs after, and outside, the lock. A store that created a new object `admit`s it; one that only appended to an object already admitted — a streaming partial write completing — `increment_weight`s it instead (the two arms of `MissFinishType`).

[11] `enum CacheEntryKey` is either `KeyOnly(CompactCacheKey)` or an `Identified` pair of that key and a storage-defined `CacheEntryId`; `HttpCache` builds it from `HandleMiss::entry_id`, which defaults to `None`.

[12] `finish_miss_handler` chooses `LockStatus::Done` or `LockStatus::TransientError` according to whether `finish()` succeeded.

[13] The waiter's next move is chosen from the `LockWaitOutcome` its wait returned, by `handle_lock_wait_outcome`: `Done` re-looks up; `TransientError`/`Dangling` elect a new writer; `AgeTimeout` lets waiters recompete for the write permit; `GiveUp`, `WaitTimeout` and `Abandoned` — the last carrying the reason a fill this waiter cannot use was abandoned — fetch independently with caching disabled. The retry loop around that choice is bounded: a waiter that has re-looked up more than `cache_lock_max_retries` times gives up and goes to the origin (`cache_lock_retry_limit_exceeded`).

- **Purge.** An explicit purge — e.g. a purge request the proxy accepts — removes the object. The proxy chooses between two actions: a *delete*, which makes the next lookup a miss, and an *expire*, which keeps the object stored but marks it stale so it revalidates against the origin and reuses the stored body on a 304.[14] `HttpCache` asks `Storage` to `purge` the key, marked as an *invalidation* to distinguish it from the purges that capacity eviction issues, or to `expire` it. The backend answers with a `PurgeOutcome` rather than a bare boolean, and only the `Purged` outcome — the entry is gone — also `remove`s the entry from the eviction manager; an `Expired` entry is still stored, so the manager keeps tracking it, and `NotFound` changes nothing.[15] The two purge kinds also address the backend differently: an invalidation names the logical key (`PurgeTarget::Active`), leaving a backend that retains several generations free to pick one, while an eviction victim names the exact entry the manager admitted (`PurgeTarget::Exact`).[16] A purge engages no cache lock: it is keyed directly off the cache key, with no lookup and no lock (Figure 2.5).[17]

- **Revalidation.** A stale entry is revalidated under the lock in the same way as a miss, so a single leader revalidates upstream while followers wait. On a 304 (`Not Modified`) the proxy merges the response into the stored header and `HttpCache` issues `update_meta` to `Storage` — the metadata is replaced, with the original admission time preserved, while the stored body stays untouched — and then releases the lock whether or not the update succeeded. A metadata update involves no eviction bookkeeping: the object's identity and body are unchanged.[18]

[14] `ProxyHttp::purge_action` returns `PurgeAction`, defaulting to `Delete`; `proxy_purge` dispatches to `HttpCache::purge` or `HttpCache::expire` accordingly.

[15] `PurgeOutcome` is `NotFound`, `Purged(Option<CacheEntryId>)` or `Expired`; `purge_impl` calls `eviction.remove` only on `Purged`.

[16] `PurgeTarget`; `purge_impl` targets `Active`, while the eviction loop targets `Exact`.

[17] `HttpProxy::proxy_purge` answers a purge request by calling `HttpCache::purge`, whose `purge_impl` passes the backend `PurgeType::Invalidation` (eviction victims instead get `PurgeType::Eviction`) and, only when the backend reports an asset removed, calls `eviction.remove`.

[18] On an upstream 304, `HttpProxy::revalidate_or_stale` merges the response into the cached header (`revalidate_merge_header`) and calls `revalidate_cache_meta`; `HttpCache::revalidate_cache_meta` preserves the original admission time — both the `created` timestamp and the `provenance` the stale entry carried (L1201) — calls `update_meta`, and releases the lock regardless of the result (L1230); it makes no eviction call. The stale path takes the lock in `cache_found`.

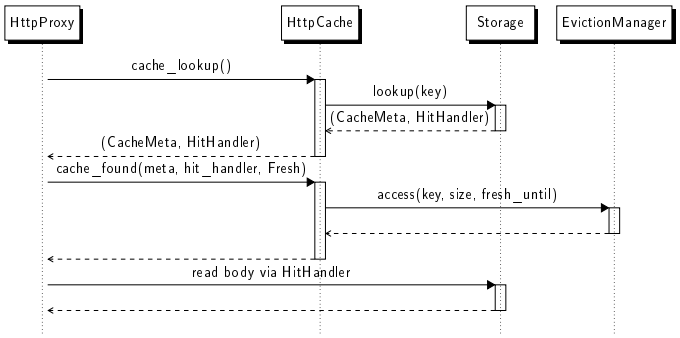


Figure 2.2: A plain hit: no lock, no writer to race. `cache_lookup` finds the object already published and returns it to `HttpProxy`, which calls `cache_found` to accept the hit — recording the touch against the `EvictionManager` so recency ordering stays current — before reading the body straight from `Storage`, the same direct-read pattern Figure 2.4's concurrent follower uses once its own wait resolves.

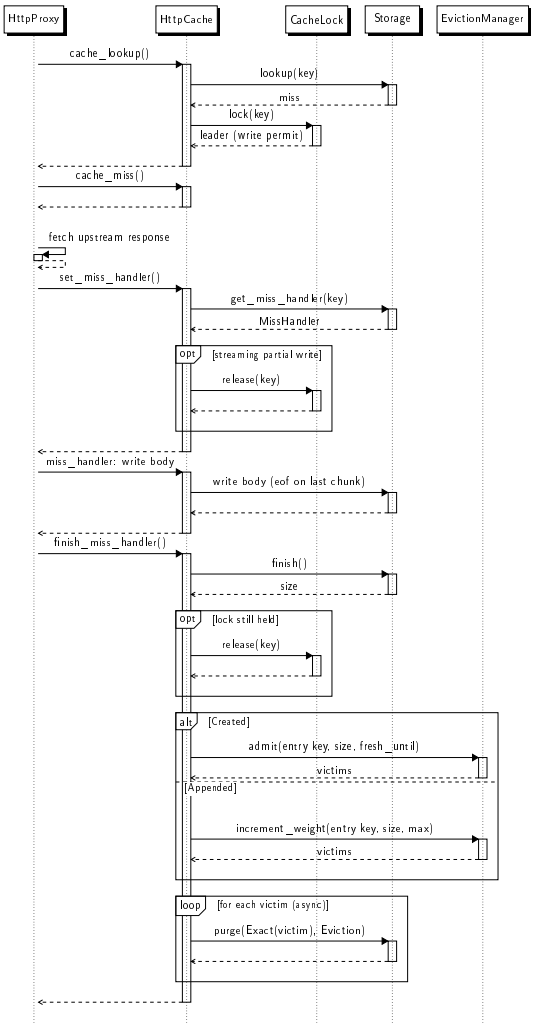


Figure 2.3: A cache miss, driven by `HttpProxy` and mediated by `HttpCache`, which alone hands out access to the backends (the body bytes actually flow through the `MissHandler` object the proxy obtains from `HttpCache` — shown here as calls through `HttpCache` for simplicity). `cache_lookup` reads `Storage` and takes the `CacheLock`; here the request wins the write permit and becomes the *leader*. After fetching upstream, `set_miss_handler`/`finish_miss_handler` write the body through `Storage` and release the lock as soon as the object is readable; eviction then runs *after* and outside the lock — a store that created the object `admits` it, one that only appended to an object already admitted `increment_weights` it, and either returns the victims, `purged` asynchronously and named as exact entries rather than as logical keys. The two *opt* fragments are two ends of the same choice: a backend supporting streaming partial writes — as the backend specified by this document does, FRE-COMPAT.STREAMFLAG-01 — releases the lock early, right after `get_miss_handler`, letting followers stream the body as it is written, and `finish_miss_handler` then finds the lock already released and skips its own `release`.

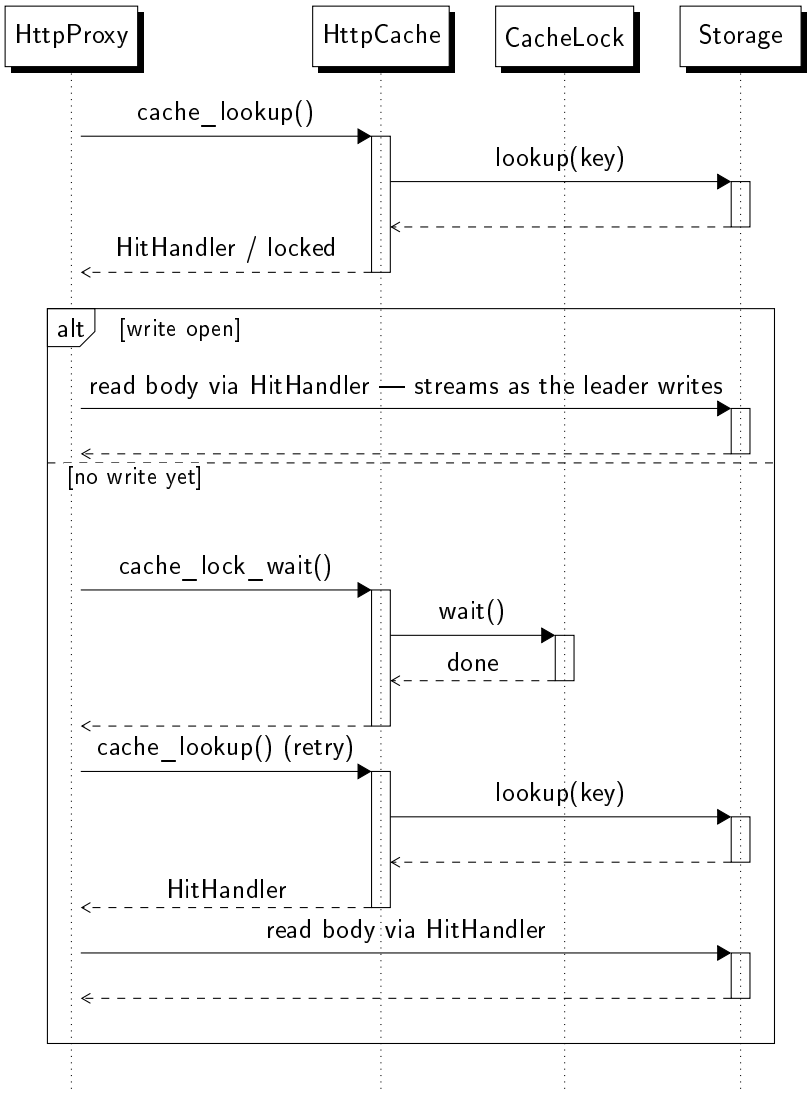


Figure 2.4: A concurrent request (*follower*). `Storage`'s `lookup` prefers a partial read, so the outcome depends on whether the leader's write is already open. **If it is**, `cache_lookup` returns a `HitHandler` at once and the follower streams the body *as the leader writes it* — no lock, no wait. **If not**, the request instead holds a read lock and blocks in `cache_lock_wait`, but only until the leader *opens* the write (in streaming mode the lock is released right after `get_miss_handler`, not when the body is done); it then retries `cache_lookup`, gets its `HitHandler`, and streams the same way. The backend specified by this document is of that streaming kind (FRE-COMPAT.STREAMFLAG-01, FRE-COMPAT.STREAMING-01): it registers the leader's write when the miss handler opens and notifies partial readers of each chunk appended to the per-writer temporary; the atomic rename that publishes the canonical, durable name still happens only once the object is complete (FDS-STORAGE.PUBLISH-01). (Only a non-streaming backend, having no partial read, would hold its followers for the complete object.) Either way the upstream fetch and the `Storage` write happen once, on the leader (Figure 2.3).

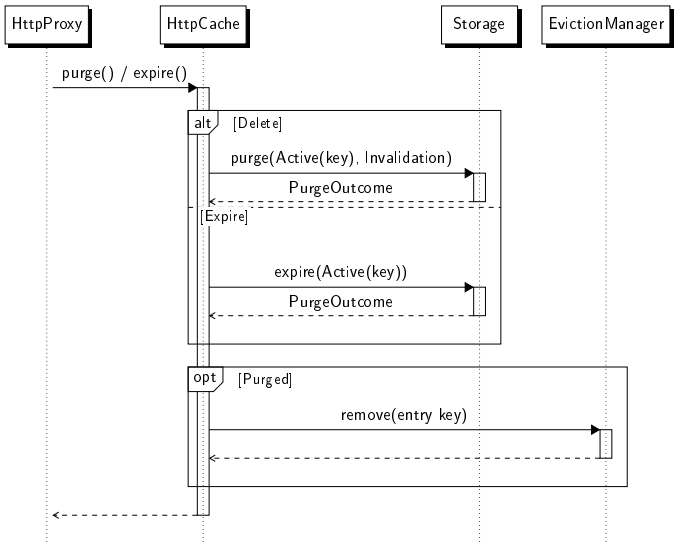


Figure 2.5: A purge, driven directly off the cache key: no lookup, no `CacheLock`. The proxy's `purge_action` hook chooses between deleting the object and expiring it — two different `Storage` calls, `purge` and `expire`, both naming the logical key as a `PurgeTarget::Active`. `purge` is additionally told which kind of purge this is (`Invalidation`, distinct from the `Eviction` kind capacity-driven purges use, which instead name the exact entry as a `PurgeTarget::Exact`). The backend reports a `PurgeOutcome`, and only `Purged` — the entry is gone — makes `HttpCache` drop it from the `EvictionManager`'s own bookkeeping; an `Expired` entry is still stored and stays tracked.

This structure — `HttpCache` owning the eviction calls while the backend merely serves object reads, writes, and purges — is why persistence is not a pure `Storage` concern. Because the eviction manager's in-memory index does not survive a restart, a persistent backend must arrange for that index to be rebuilt, and (as the framework, not the backend, owns the eviction calls) it cannot do so in isolation. The `CacheLock` is in-memory and per-process in the same way: it bounds same-key write concurrency *within* a process, but two processes sharing the cache during a hot upgrade hold independent locks, so the backend must still tolerate concurrent same-key writes from a peer. The requirements chapters return to both points.

3. High-Level Requirements

[HLR-SCOPE.V1-01]

First Functional Version ⟨⟨*stakeholder requirement*⟩⟩ `draft`

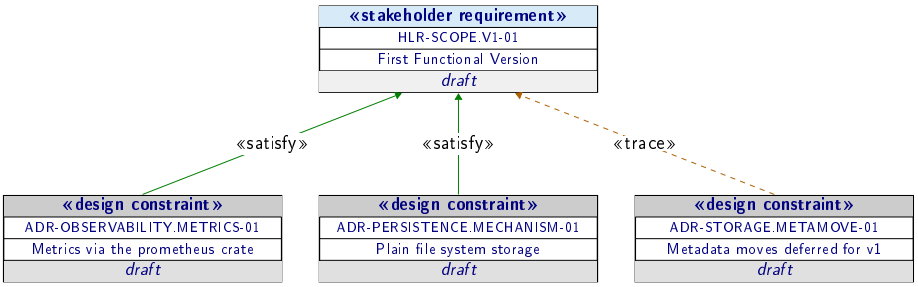


Figure 3.1: First Functional Version

WHEN two or more alternatives each satisfy the requirements an architecture decision must meet, the decision shall adopt the alternative that requires this project to build the fewest mechanisms of its own — counted as components, public interfaces, background tasks, and on-disk structures built rather than adopted — even at the cost of peak space or I/O efficiency.

This specification targets a *first functional version*: proven components doing the heavy lifting are preferred to bespoke ones. The requirement is verified by inspection — each architecture decision record's *Alternatives considered* section states what an alternative rejected on these grounds would have required the project to build. A later version of this specification may replace this requirement.

View complete diagram.

Required coverage	✓ ADR via ⟨⟨*satisfy*⟩⟩
Satisfied by	ADR-PERSISTENCE.MECHANISM-01 ADR-OBSERVABILITY.METRICS-01
Traced from	ADR-STORAGE.METAMOVE-01

[HLR-PERSISTENCE.DURABLE-01]

Durable Persistence ⟨⟨*stakeholder requirement*⟩⟩ `draft`

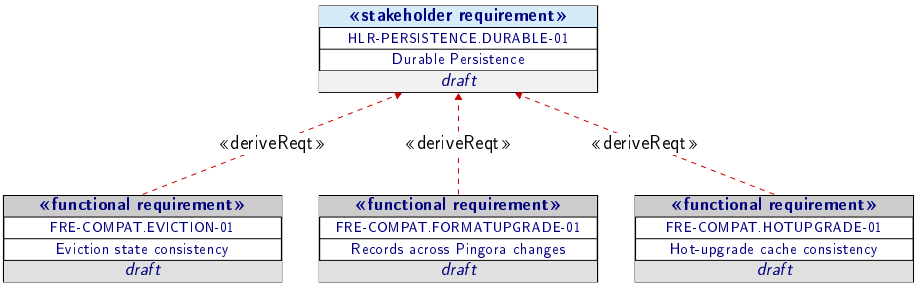


Figure 3.2: Durable Persistence

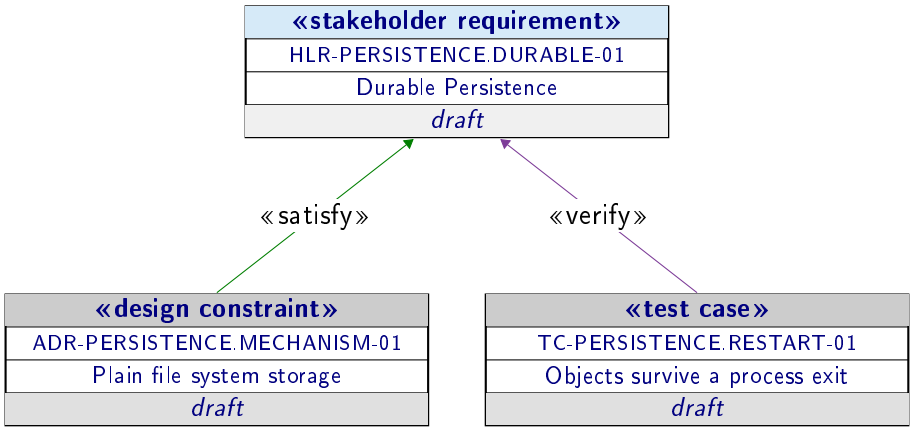


Figure 3.2: Durable Persistence (cont.)

WHEN the cache process exits — in an orderly way or by crashing — the cache shall serve, from the process that starts next, every cached HTTP object whose storage had completed before the exit.

A process exit leaves the host's own state intact; what the cache owes after the host itself crashes or loses power is stated separately in HLR-PERSISTENCE.CRASHSAFE-01, and the size of the cache it can hold in HLR-PERSISTENCE.CAPACITY-01.

View complete diagram.

Required coverage	✓ ADR-PERSISTENCE.MECHANISM-01 via ⟨⟨*satisfy*⟩⟩ — *Establish the storage substrate and mechanism that persists cached objects across an exit of the cache process.* ✓ TC via ⟨⟨*verify*⟩⟩
Derived from	FRE-COMPAT.EVICTION-01 FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.FORMATUPGRADE-01
Satisfied by	ADR-PERSISTENCE.MECHANISM-01
Verified by	TC-PERSISTENCE.RESTART-01

[HLR-PERSISTENCE.CAPACITY-01]

Capacity beyond RAM ⟨⟨*stakeholder requirement*⟩⟩ `draft`

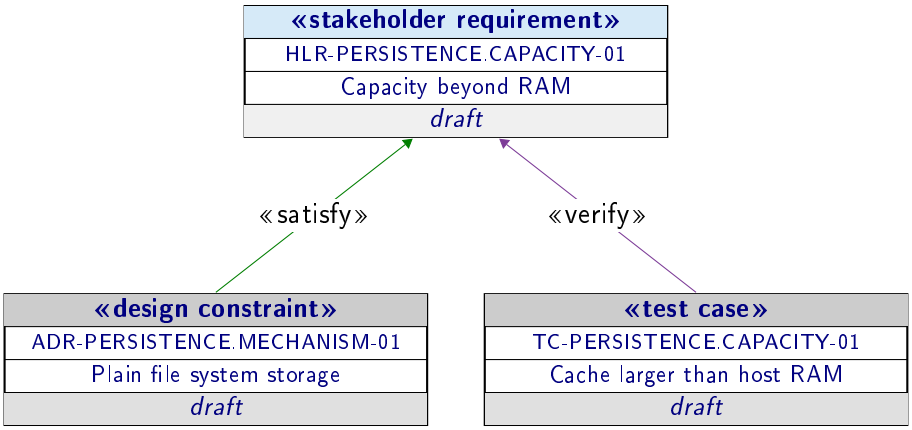


Figure 3.3: Capacity beyond RAM

The cache shall hold a set of cached HTTP objects whose total size exceeds the RAM installed on its host.

View complete diagram.

Required coverage	✓ ADR-PERSISTENCE.MECHANISM-01 via ⟨⟨*satisfy*⟩⟩ — *Choose a storage substrate whose capacity is bounded by persistent storage rather than by process RAM.* ✓ TC via ⟨⟨*verify*⟩⟩
Satisfied by	ADR-PERSISTENCE.MECHANISM-01
Verified by	TC-PERSISTENCE.CAPACITY-01

[HLR-PERSISTENCE.CRASHSAFE-01]

Host-crash safety ⟨⟨*stakeholder requirement*⟩⟩ `draft`

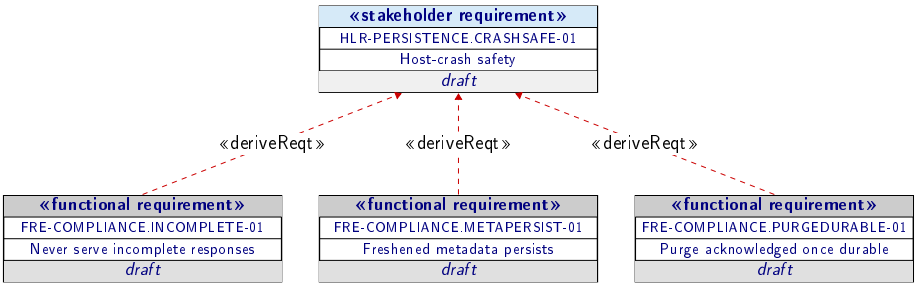


Figure 3.4: Host-crash safety

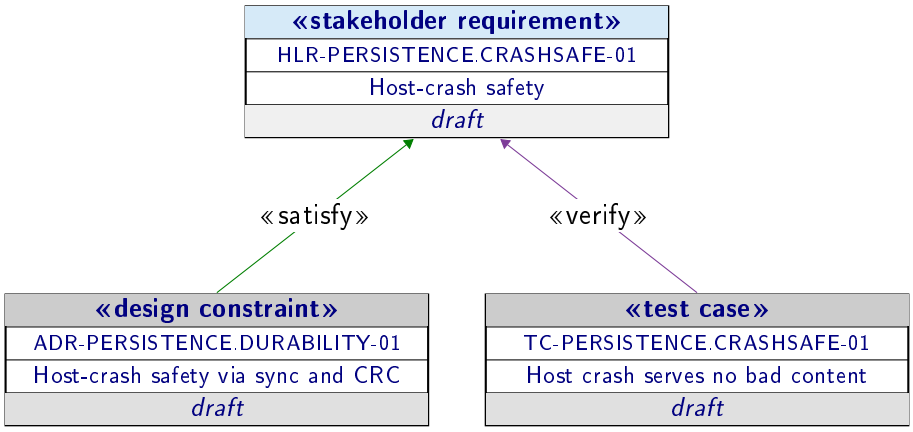


Figure 3.4: Host-crash safety (cont.)

IF the cache's host crashes or loses power, THEN the cache, once running again, shall serve no object whose content differs from what was stored and no object whose purge it had acknowledged before the crash.

Cached objects can be fetched again from their origin, so losing some of them in a host crash — each then simply a cache miss — is acceptable; serving wrong or purged content is not.

This assumes a storage stack that honours flush requests — what it acknowledges as flushed is on stable storage — and that returns the data written to it or an error. Detecting data corrupted at rest, or lost by a device that acknowledges flushes it has not made, is the storage layer's responsibility, not the cache's.

View complete diagram.

Required coverage	✓ ADR-PERSISTENCE.DURABILITY-01 via ⟨⟨*satisfy*⟩⟩ — *Decide which writes are synchronised to stable storage, and what integrity check detects the rest, so that a host crash can lose objects but never make the cache serve corrupted or purged content.* ✓ TC via ⟨⟨*verify*⟩⟩
Satisfied by	ADR-PERSISTENCE.DURABILITY-01
Derived from	FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.INCOMPLETE-01
Verified by	TC-PERSISTENCE.CRASHSAFE-01

[HLR-EFFICIENCY.SPACE-01]

Space Efficiency ⟨⟨*stakeholder requirement*⟩⟩ `draft`

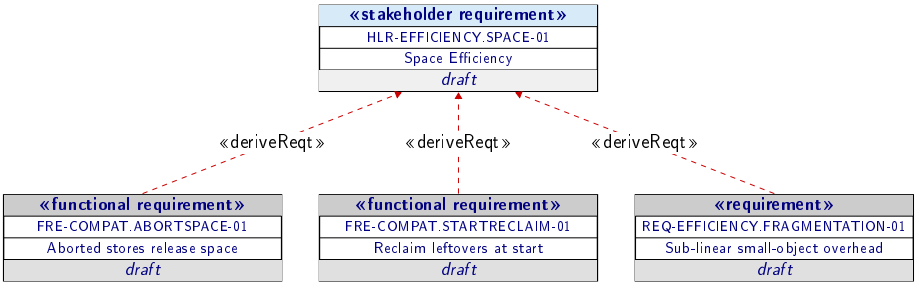


Figure 3.5: Space Efficiency

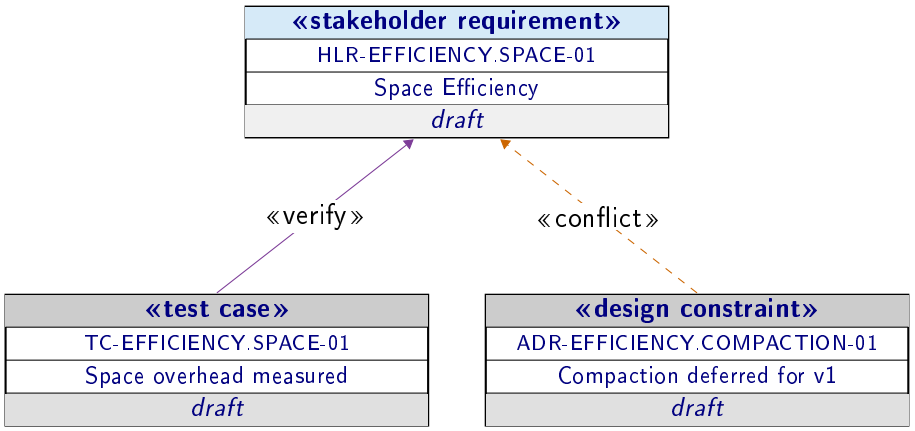


Figure 3.5: Space Efficiency (cont.)

The cache shall hold the objects it caches in operation with an on-disk space overhead of no more than 25 % of those objects' own metadata and body bytes.

The overhead is the storage the cache's directory tree occupies on disk — allocated blocks, not file lengths — minus the metadata and body bytes of the objects it holds, so it counts record headers, reserved headroom, block rounding, per-file and

per-directory costs, and leftover files alike. It is measured over what the deployment actually caches, because overhead depends strongly on object sizes: a 500-byte object in a 4 KiB block carries roughly 700 % overhead, a 1 MiB object well under 1 %. The first functional version knowingly does not meet this requirement for workloads dominated by small objects: ADR-EFFICIENCY.COMPACTION-01 defers the machinery that would, and declares the conflict.

View complete diagram.

Required coverage	✓ REQ via ⟨⟨*deriveReqt*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩
Derived from	FRE-COMPAT.ABORTSPACE-01 FRE-COMPAT.STARTRECLAIM-01 REQ-EFFICIENCY.FRAGMENTATION-01
Conflicts with	ADR-EFFICIENCY.COMPACTION-01
Verified by	TC-EFFICIENCY.SPACE-01

[HLR-PERFORMANCE.LOOKUP-01]

Lookup latency flat with size ⟨⟨*stakeholder requirement*⟩⟩ `draft`

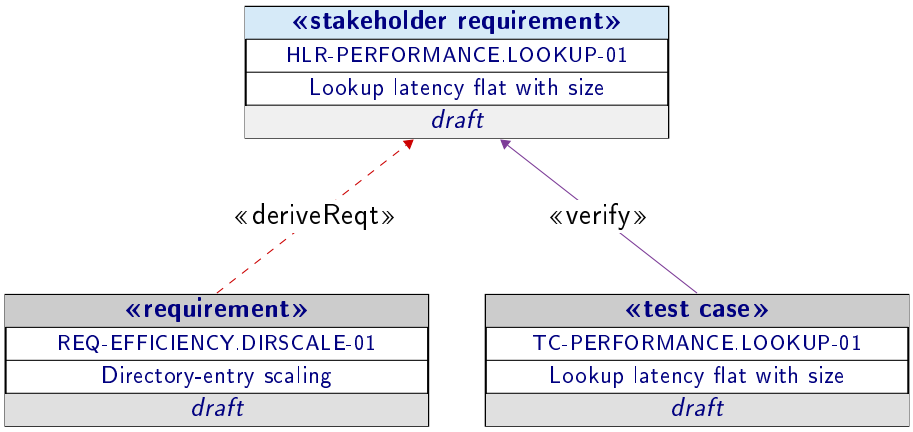


Figure 3.6: Lookup latency flat with size

WHILE the cache holds up to 10^7 cached HTTP objects, the cache shall keep its 99th-percentile lookup latency no more than 25 % above the 99th-percentile lookup latency it has while holding 10^4 objects, on the same host under the same conditions.

Lookup latency is the time from the start of a cache lookup until the cache knows whether the request is a hit — with the stored response head in hand — or a miss; delivering the body is not part of it. “The same conditions” means the same storage device and file system, the same page-cache state, and the same mix of hits and misses over uniformly chosen keys. The bound is relative rather than absolute because absolute lookup latency depends mostly on the host’s storage device and memory, which the deployment chooses; what the cache itself owes is that growing

towards its capacity does not make every request slower. 10^7 objects is a size a test host can hold; derived design targets may reach further.

View complete diagram.

Required coverage	✓ REQ via ⟨⟨*deriveReqt*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩
Derived from	REQ-EFFICIENCY.DIRSCALE-01
Verified by	TC-PERFORMANCE.LOOKUP-01

[HLR-COMPAT.PINGORA-01]

Compatible with Pingora ⟨⟨*stakeholder requirement*⟩⟩ `draft`

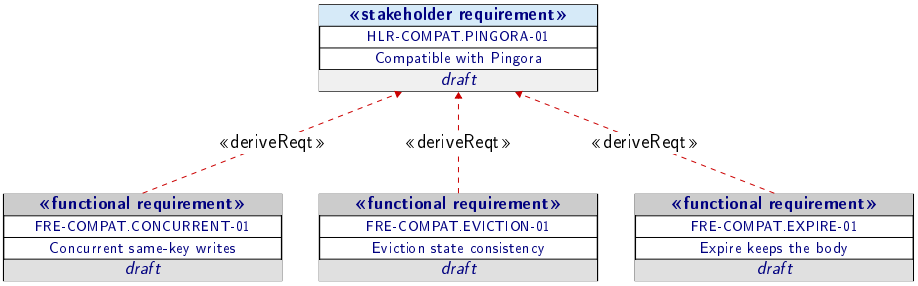


Figure 3.7: Compatible with Pingora

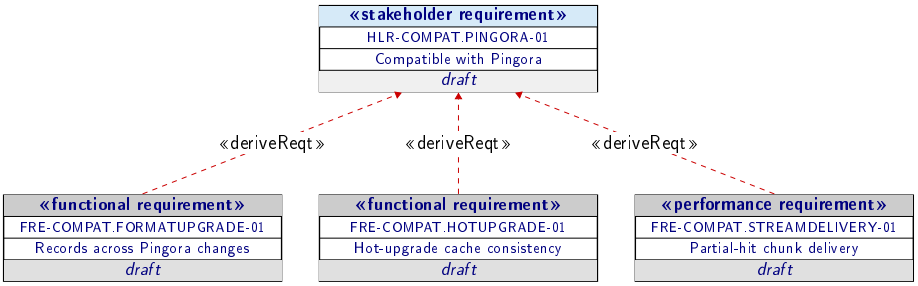


Figure 3.7: Compatible with Pingora (cont.)

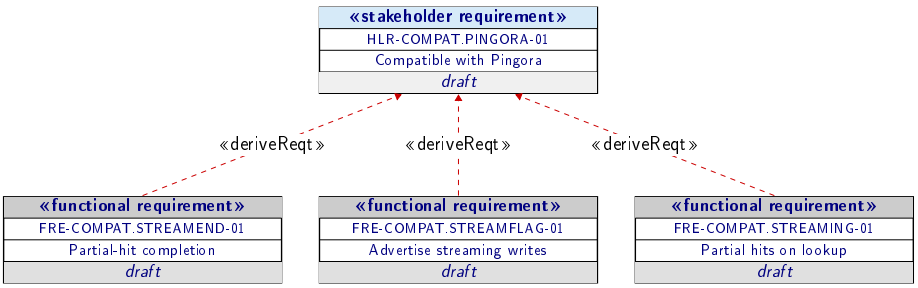


Figure 3.7: Compatible with Pingora (cont.)

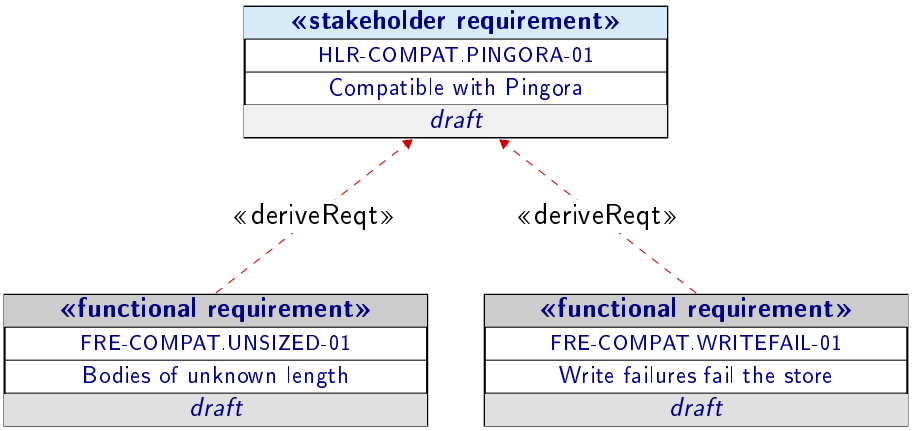


Figure 3.7: Compatible with Pingora (cont.)

The cache shall integrate with the `pingora-cache` 0.9 release line — version 0.9.0 and its patch releases — through only the extension points those releases expose publicly, without modification of the Pingora source.

The `Storage` backend is the primary integration point, but preserving cache semantics across restarts may require additional, separately-configured components — for example eviction-state coordination (FRE-COMPAT.EVICTION-01) — which meet this requirement as long as they, too, use only public extension points. Verification is by inspection of the deliverable's dependency manifest (Pingora crates constrained to 0.9.x, none patched or vendored) and of the interfaces it uses. A patch release of the 0.9 line is taken as it comes; adopting a later minor release is a revision of this requirement, not an implicit obligation. What the persisted cache owes across any Pingora change is stated in FRE-COMPAT.FORMATUPGRADE-01. This version is such a revision: it moves the target from 0.8.1 to the 0.9 line, whose `Storage` and `EvictionManager` surfaces differ in ways the requirements below name explicitly.

View complete diagram.

Required coverage	✓ FRE via ⟨⟨*deriveReqt*⟩⟩
Derived from	FRE-COMPAT.UNSIZED-01 FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.EVICTION-01 FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.STREAMFLAG-01 FRE-COMPAT.STREAMING-01 FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 FRE-COMPAT.FORMATUPGRADE-01 FRE-COMPAT.WRITEFAIL-01 FRE-COMPAT.EXPIRE-01

[HLR-COMPLIANCE.RFC-01]
No added RFC 9111/9110 deviation ⟨⟨*stakeholder requirement*⟩⟩ `draft`

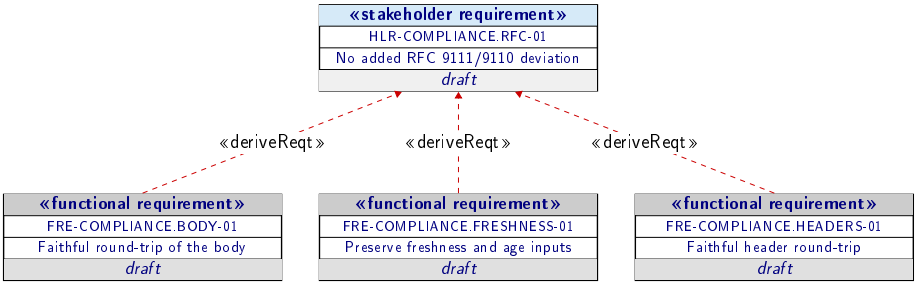


Figure 3.8: No added RFC 9111/9110 deviation

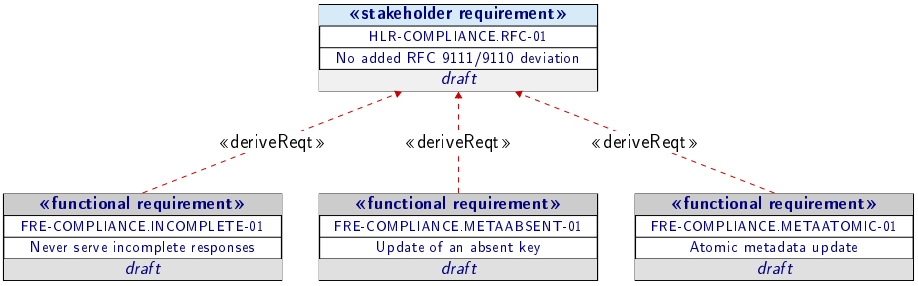


Figure 3.8: No added RFC 9111/9110 deviation (cont.)

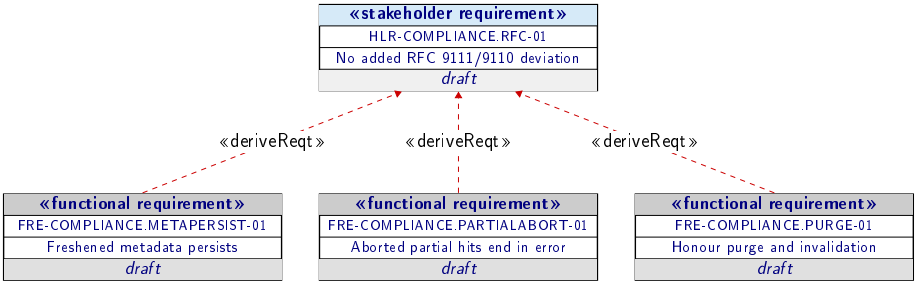


Figure 3.8: No added RFC 9111/9110 deviation (cont.)

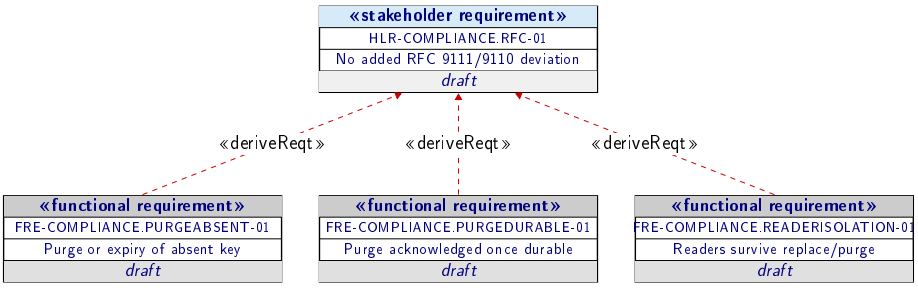


Figure 3.8: No added RFC 9111/9110 deviation (cont.)

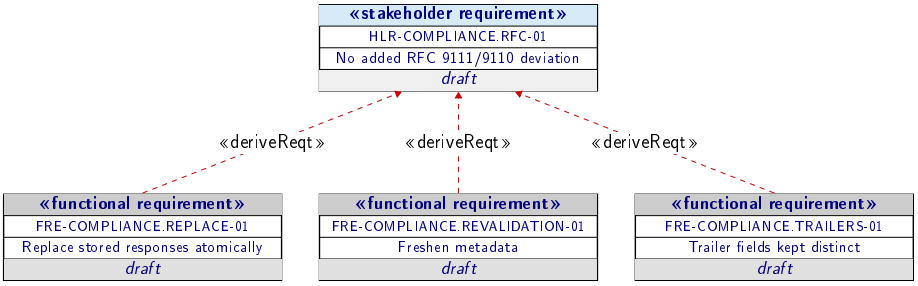


Figure 3.8: No added RFC 9111/9110 deviation (cont.)

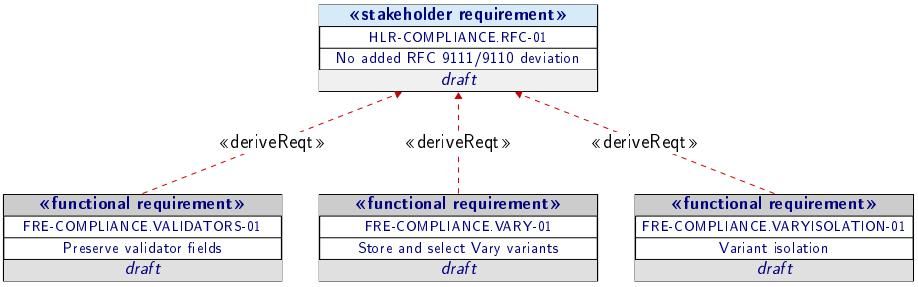


Figure 3.8: No added RFC 9111/9110 deviation (cont.)

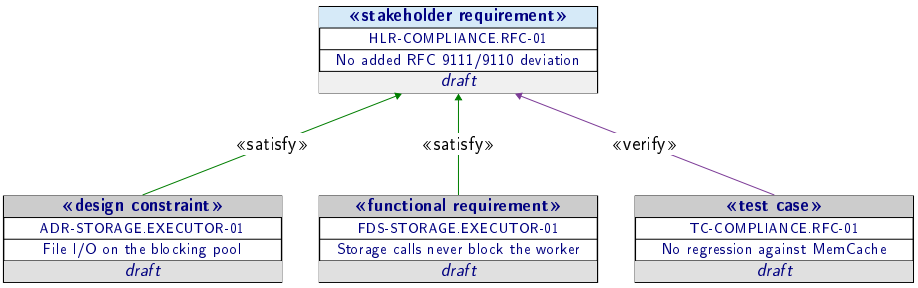


Figure 3.8: No added RFC 9111/9110 deviation (cont.)

The cache's observable HTTP caching behaviour shall match that of the same

proxy configured with the stock `MemCache` backend of `pingora-cache`, in every respect governed by RFC 9111 (*HTTP Caching*) and RFC 9110 (*HTTP Semantics*).

`pingora-cache` applies these semantics — freshness calculation, conditional revalidation, `Cache-Control` handling — without claiming full conformance. This requirement therefore asks the backend to add no deviation of its own, not to remove the framework's. Behaviour observed across a process restart, which `MemCache` cannot exhibit, is compared against the behaviour `MemCache` shows within a single process run.

View complete diagram.

Required coverage	✓ FRE via ⟨⟨*deriveReqt*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩
Satisfied by	ADR-STORAGE.EXECUTOR-01 FDS-STORAGE.EXECUTOR-01
Derived from	FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.TRAILERS-01 FRE-COMPLIANCE.BODY-01 FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.VALIDATORS-01 FRE-COMPLIANCE.REVALIDATION-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.METAABSENT-01 FRE-COMPLIANCE.REPLACE-01 FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPLIANCE.VARY-01 FRE-COMPLIANCE.VARYISOLATION-01 FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.PARTIALABORT-01
Verified by	TC-COMPLIANCE.RFC-01
Traces to	RFC 9111 (HTTP Caching) RFC 9110 (HTTP Semantics)

4. High-Level Refinement

[FRE-COMPAT.UNSIZED-01]

Bodies of unknown length ⟨⟨*functional requirement*⟩⟩ `draft`

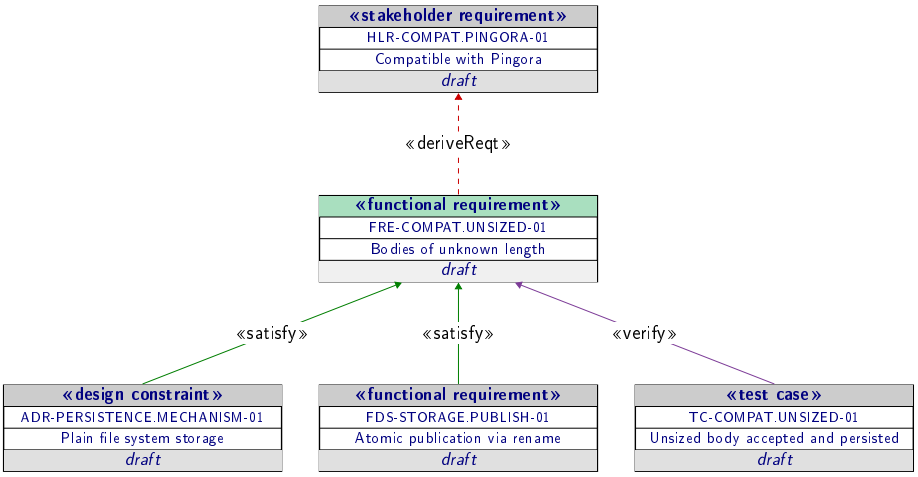


Figure 4.1: Bodies of unknown length

`pingora-cache` hands the backend a response body as it arrives from the upstream: the miss handler obtained from `get_miss_handler` receives the body chunk by chunk, and the end of the body is signalled only with the final chunk. Nothing in the `Storage` contract declares the body's total size when the store opens — a chunked upstream response carries no `Content-Length` to declare.

WHEN a store begins without the body's total length being declared, the backend shall persist the complete body while holding in process memory no more of that body than the chunk its current `write_body` call is writing, independent of the body's length.

The bound rules out buffering a whole body before writing it, and any per-store buffer besides: the operating system's page cache already batches writes to the device, so a buffer in the backend would save only system calls, at the cost of memory per concurrent store. The chunk being written is allowed because `pingora-cache` has already allocated it when the backend receives it, and its size depends on the upstream protocol — HTTP/1 bodies are read through a 64 KiB buffer,[1] while an HTTP/2 data frame can be larger.

Basis: `Storage::get_miss_handler` and the `MissHandler` chunked write path.[2]

View complete diagram.

[1] `BODY_BUFFER_SIZE` is `1024 * 64`.

[2] `Storage::get_miss_handler` returns a `MissHandler`, whose `HandleMiss::write_body(data, eof)` is called once per chunk, with `eof` set only on the final call.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩
Derives from	HLR-COMPAT.PINGORA-01
Satisfied by	ADR-PERSISTENCE.MECHANISM-01 FDS-STORAGE.PUBLISH-01
Verified by	TC-COMPAT.UNSIZED-01

[FRE-COMPAT.CONCURRENT-01]

Concurrent same-key writes ⟨⟨*functional requirement*⟩⟩ `draft`

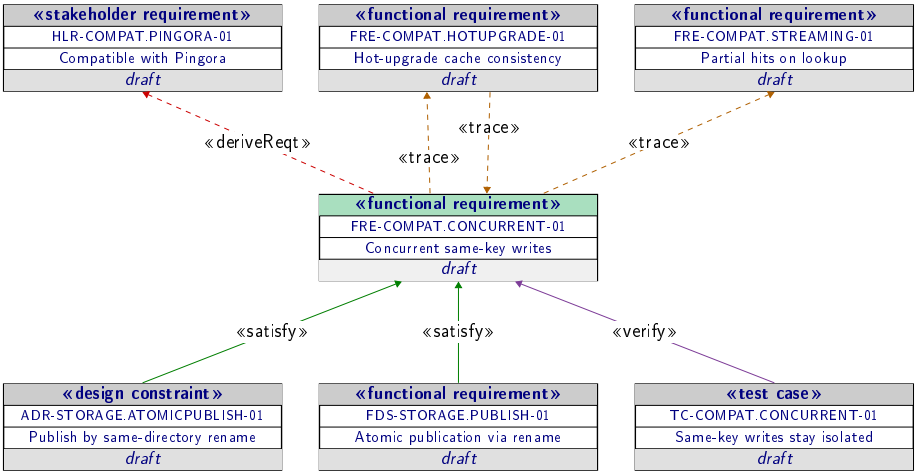


Figure 4.2: Concurrent same-key writes

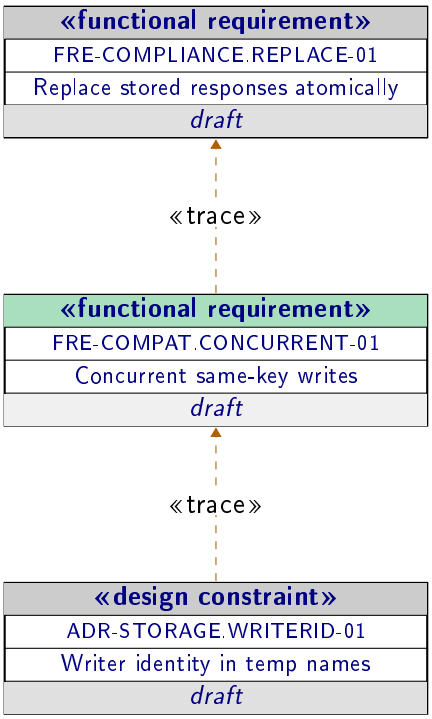


Figure 4.2: Concurrent same-key writes (cont.)

`pingora-cache` permits several writes of the same key to be in flight at once. The `CacheLock` that serialises miss handling is an optional collaborator, and even when configured its age timeout deliberately lets a new writer start while a previous one may still be writing; during a hot upgrade, a peer process holds its own, independent lock table (FRE-COMPAT.HOTUPGRADE-01), so same-key writes can also race across processes.

While several writes of the same key are in flight at once, the backend shall keep each in isolation — no writer's bytes ever interleaved with another's — and shall make each write *durably published* — visible by name to a scan, a restart, or a peer process — as a complete object or not at all. This is the general form of the atomicity guarantee whose replacement-specific case is detailed in FRE-COMPLIANCE.REPLACE-01; a lookup within the writing process may additionally be served a partial hit of an in-flight write, as governed by FRE-COMPAT.STREAMING-01. When those writes complete, the backend shall converge on a single winner; the atomic-replacement semantics of that convergence are also owned by FRE-COMPLIANCE.REPLACE-01.

Basis: `HttpCache`'s optional cache lock and its age timeout;[3] *the stock `MemCache`'s per-writer keying of in-flight writes.*[4]

View complete diagram.

[3] Once a lock ages out or is abandoned, `CacheLock::lock` stops treating it as an in-progress write and hands out a fresh write permit instead of making the caller wait — the mechanism behind the age timeout described above.

[4] `MemCache::get_miss_handler` assigns each new write its own incrementing id and inserts it into the per-key map alongside any other writes already in flight.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩
Derives from	HLR-COMPAT.PINGORA-01
Traces to	FRE-COMPLIANCE.REPLACE-01 FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.STREAMING-01
Traced from	FRE-COMPAT.HOTUPGRADE-01 ADR-STORAGE.WRITERID-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01
Verified by	TC-COMPAT.CONCURRENT-01

[FRE-COMPAT.EVICTION-01]

Eviction state consistency ⟨⟨*functional requirement*⟩⟩ `draft`

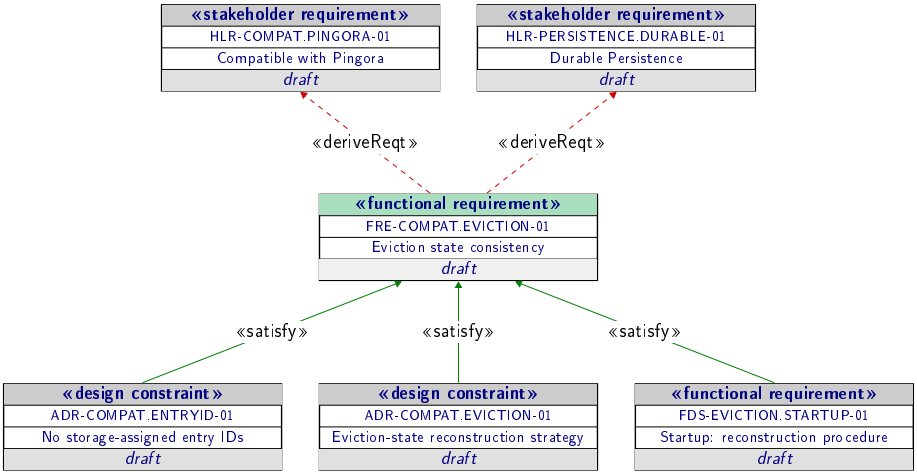


Figure 4.3: Eviction state consistency

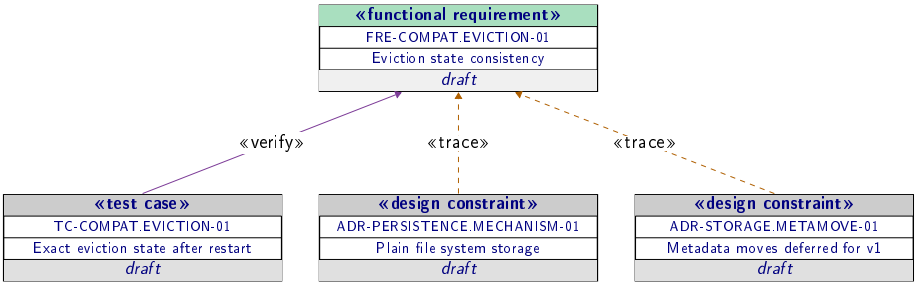


Figure 4.3: Eviction state consistency (cont.)

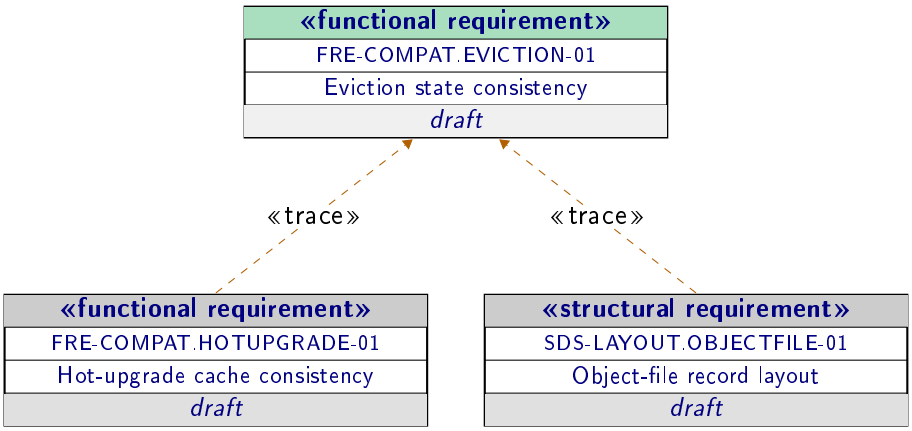


Figure 4.3: Eviction state consistency (cont.)

`pingora-cache` manages capacity through an eviction manager (the `EvictionManager` trait, e.g. its `simple_lru` and `lru` implementations) that holds, in memory, the set of cached items, their on-disk sizes, and the recency/freshness ordering used to choose eviction victims. The eviction manager is a component *distinct* from the `Storage` backend, and it is the framework — not the backend — that drives it: both objects are handed to `HttpCache` (e.g. through the cache's `enable` configuration set up in the proxy), after which `HttpCache` records admissions and accesses against the manager as objects are written and read, and purges the victims it returns through the backend's removal path. The two are configured independently and the backend never calls the manager itself.

Because this backend persists cached objects across process restarts (HLR-PERSISTENCE.DURABLE-01) while the eviction manager's in-memory state does not survive on its own, a naive restart leaves capacity accounting and eviction decisions out of step with the objects actually persisted: objects the manager no longer knows about are never counted against the size budget and never evicted. The discrepancy exists *only at restart* — in steady state `HttpCache` keeps a single manager in step — so the requirement is to close that startup gap.

A direct consequence of the framework owning the runtime `admit`, `access`, and purge calls is that *a `Storage` implementation cannot, on its own, keep the eviction manager synchronised*: its own contribution is to persist objects durably and to expose enumeration of the persisted entries (key, size, freshness), but bringing the manager back in step with that persisted set after a restart requires cooperation beyond what the `Storage` trait itself can provide. Meeting this requirement therefore demands more than a pure drop-in `Storage` backend — a compatibility consideration for HLR-COMPAT.PINGORA-01. This requirement thus derives from both parents: HLR-PERSISTENCE.DURABLE-01 creates the gap by making objects outlive the manager's memory, and HLR-COMPAT.PINGORA-01 fixes the framework-owned eviction contract the gap must be closed within.

WHEN the backend reports its restart procedure complete, the eviction manager's view of admitted objects shall be consistent with the durably published objects: every durably published object represented once — keyed by its `CompactCacheKey`, with its record size and stored freshness deadline — no entry remaining for an object that is no longer present, and the manager's reported

`total_items` and `total_size` equal to the count and aggregate record size of those objects.

An object's *record size* is the full length of the file that stores it, metadata and body together. The backend reports that same size both when a store completes and when the restart procedure admits the object, so the two admission paths agree and `total_size` measures the disk the cache occupies. Two things are deliberately left open. Before the restart procedure completes — serving may already have begun — capacity accounting may be inaccurate. And the recency order the manager used to choose victims before the restart need not survive it: afterwards, victims may be chosen in a different order.

Basis: `HttpCache` drives the eviction manager on the hit and miss paths;[5] *the `EvictionManager` trait's `admit`/`access` contract;*[6] *item identity via `CacheEntryKey`;*[7] *the manager's aggregate accounting.*[8]

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ ADR via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPAT.PINGORA-01 HLR-PERSISTENCE.DURABLE-01
Traced from	FRE-COMPAT.HOTUPGRADE-01 ADR-PERSISTENCE.MECHANISM-01 ADR-STORAGE.METAMOVE-01 SDS-LAYOUT.OBJECTFILE-01
Satisfied by	ADR-COMPAT.EVICTION-01 ADR-COMPAT.ENTRYID-01 FDS-EVICTION.STARTUP-01
Verified by	TC-COMPAT.EVICTION-01

[5] `eviction.access(...)` is called on a hit and `eviction.admit(...)` on a completed miss.

[6] `trait EvictionManager` declares `admit` and `access`.

[7] `enum CacheEntryKey` wraps a `struct CompactCacheKey`, optionally paired with a storage-defined `CacheEntryId`. A backend that publishes a single object per key supplies no identifier (ADR-COMPAT.ENTRYID-01), so the entry key is `KeyOnly` and carries the cache key alone.

[8] `total_size` and `total_items` report the manager's tracked byte size and item count.

[FRE-COMPAT.HOTUPGRADE-01]

Hot-upgrade cache consistency ⟨⟨*functional requirement*⟩⟩ `draft`

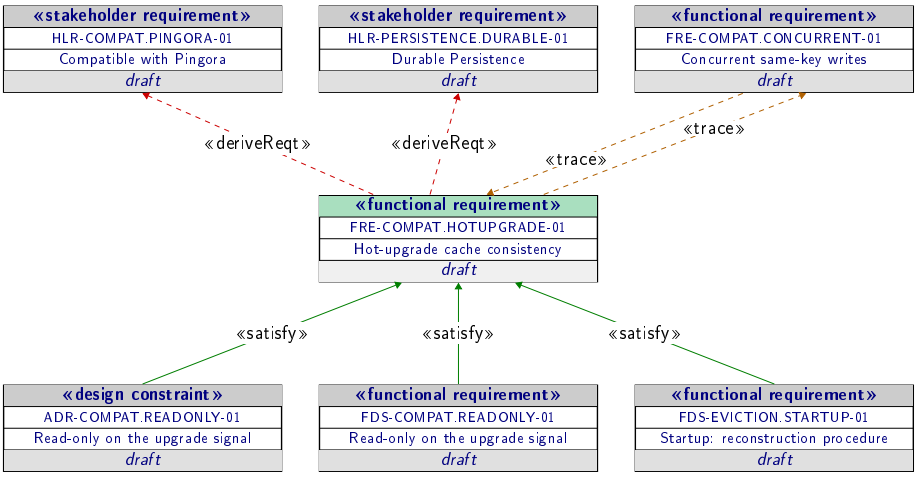


Figure 4.4: Hot-upgrade cache consistency

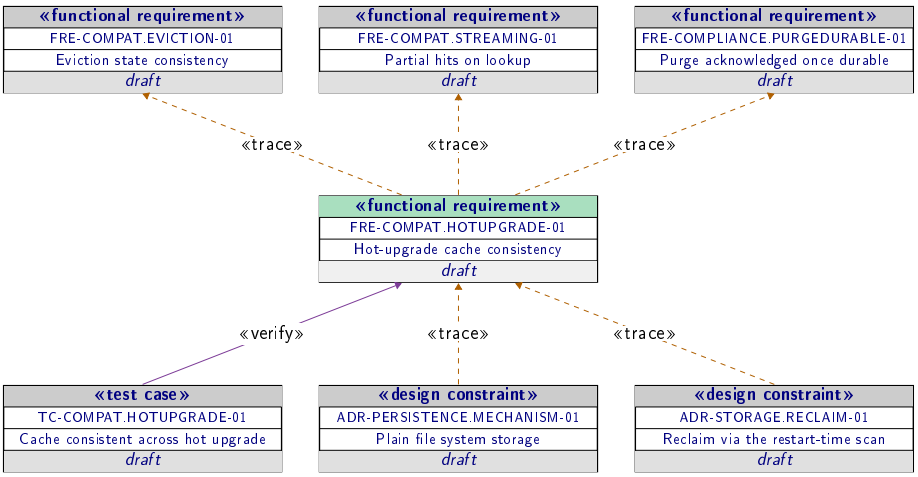


Figure 4.4: Hot-upgrade cache consistency (cont.)

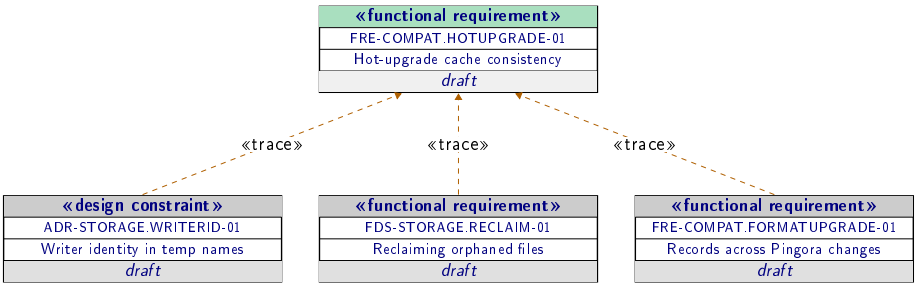


Figure 4.4: Hot-upgrade cache consistency (cont.)

Pingora supports *hot* (graceful, zero-downtime) upgrades: a new (*incoming*) executable takes over the listening sockets from the running (*outgoing*) one, which drains its already-accepted in-flight requests before exiting, so for a bounded window two processes run concurrently. Because this backend persists objects durably (HLR-PERSISTENCE.DURABLE-01), the persisted state outlives any single process — so both executables, configured for the same upstreams, access the very same cache during that window, a concern that would not arise if state were confined to one process's memory. Only objects already durably published are shared between the two processes — in-flight activity such as streaming partial hits (FRE-COMPAT.STREAMING-01) is not yet part of the cache, and is outside this requirement's scope.

While two processes share the same durable cache during a hot upgrade, the backend in each shall keep the shared cache consistent:

- **Outgoing process.** WHEN the outgoing process receives the Pingora upgrade signal, its backend shall make no further durable change to the shared cache: a store that completes afterwards shall not be durably published, and a purge or metadata update requested afterwards shall fail with an error rather than be reported as applied. It shall keep serving reads of objects already durably published, by either process, so that its remaining requests complete.
- **Incoming process.** Its backend is subject to the ordinary concurrent-writer and eviction-consistency guarantees this backend already provides for a single running instance (FRE-COMPAT.CONCURRENT-01, FRE-COMPAT.EVICTION-01) as though it were the sole writer — hot upgrade grants no exception to those — and none of its storage operations, the restart-time reclaim scan included, shall make an object unreadable through a file descriptor the outgoing process already holds open.

A purge refused by the draining process surfaces as a failed purge request, which its client can retry against the incoming process; reporting it as applied would break the purge-acknowledgement guarantee (FRE-COMPLIANCE.PURGEDURABLE-01). How the outgoing process's backend learns that the signal has arrived is left to the design: the server announces the transition to its own code through an execution-phase broadcast.[9]

[9] `Server::watch_execution_phase` subscribes to the phase broadcast, which announces `ExecutionPhase::GracefulUpgradeTransferringFds` when the upgrade signal (`SIGQUIT`) arrives (L279); both processes then accept connections for `CLOSE_TIMEOUT` (5 s) before the old one stops listening and drains.

This requirement derives from both parents: HLR-COMPAT.PINGORA-01 brings the Pingora hot-upgrade model, with its window of two processes, and HLR-PERSISTENCE.DURABLE-01 makes the cache state they share outlive either process.

Basis: Pingora graceful (zero-downtime) upgrade (HLR-COMPAT.PINGORA-01).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-EVICTION.STARTUP-01 via ⟨⟨*satisfy*⟩⟩ — *Realise the reconstruction procedure so the incoming process's eviction view is rebuilt from durably published objects only, keeping the shared cache consistent across the hot-upgrade window.* ✓ FDS-COMPAT.READONLY-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how the outgoing process's backend learns that the upgrade signal arrived, what it suppresses from then on (publication, purge, metadata update) and how it reports refusals, and how the incoming process's reconstruction is ordered relative to the outgoing process's last publications.*
Derives from	HLR-COMPAT.PINGORA-01 HLR-PERSISTENCE.DURABLE-01
Traces to	FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.EVICTION-01 FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.PURGEDURABLE-01
Traced from	FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.FORMATUPGRADE-01 ADR-PERSISTENCE.MECHANISM-01 ADR-STORAGE.WRITERID-01 ADR-STORAGE.RECLAIM-01 FDS-STORAGE.RECLAIM-01
Satisfied by	ADR-COMPAT.READONLY-01 FDS-EVICTION.STARTUP-01 FDS-COMPAT.READONLY-01
Verified by	TC-COMPAT.HOTUPGRADE-01

[FRE-COMPAT.STREAMFLAG-01]

Advertise streaming writes ⟨⟨*functional requirement*⟩⟩ `draft`

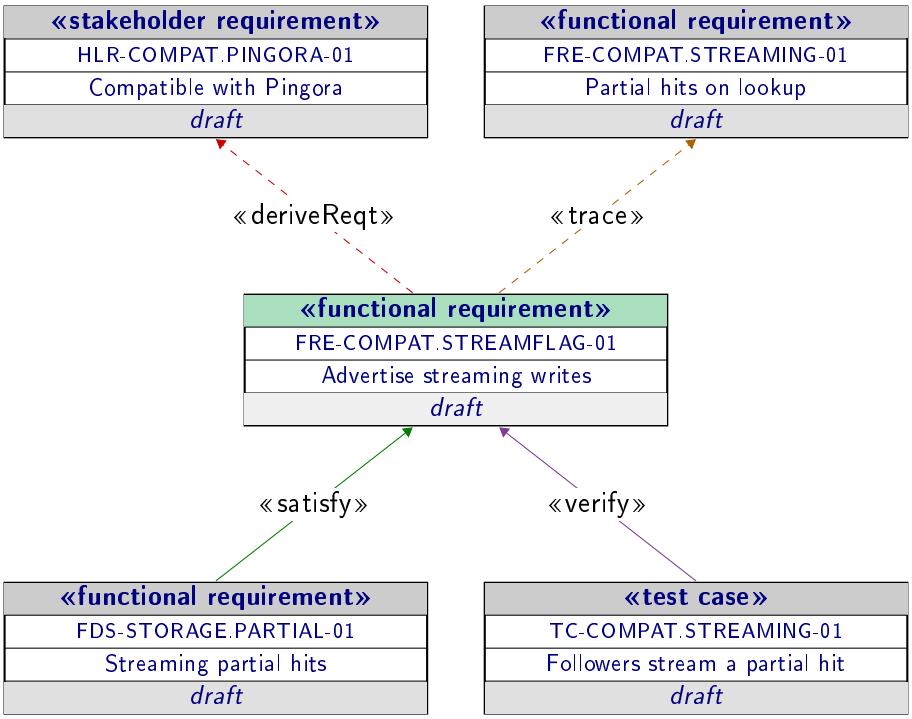


Figure 4.5: Advertise streaming writes

The backend shall report `true` from its implementation of the `Storage` trait's `support_streaming_partial_write` method.

The stampede protection in `pingora-cache` assumes a backend that can serve an object *while it is still being written*: when `Storage` advertises `support_streaming_partial_write`, `HttpCache` releases the cache lock as soon as the leader's miss handler opens — right after `get_miss_handler` rather than at `finish_miss_handler` — and every concurrent request for the key is then expected to be served by a *partial* read of the in-flight object. A backend without partial reads instead holds every follower, under the lock, for the leader's whole body; for a large or slow upstream response that turns stampede protection into a head-of-line delay of the full download for every concurrent client. Advertising the flag is what lets followers reach the partial hits of FRE-COMPAT.STREAMING-01 at all, so the flag is only truthful together with that requirement, FRE-COMPAT.STREAMDELIVERY-01, and FRE-COMPAT.STREAMEND-01.

Basis: the `Storage` trait's `support_streaming_partial_write` flag;[10] *`HttpCache` releasing the cache lock right after the miss handler opens, instead of waiting for the store to finish.*[11]

[10] `Storage::support_streaming_partial_write` defaults to `false`; a backend opts in by overriding it.

[11] `HttpCache::set_miss_handler` releases the lock immediately once

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPAT.PINGORA-01
Traces to	FRE-COMPAT.STREAMING-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPAT.STREAMING-01

[FRE-COMPAT.STREAMING-01]

Partial hits on lookup ⟨⟨*functional requirement*⟩⟩ `draft`

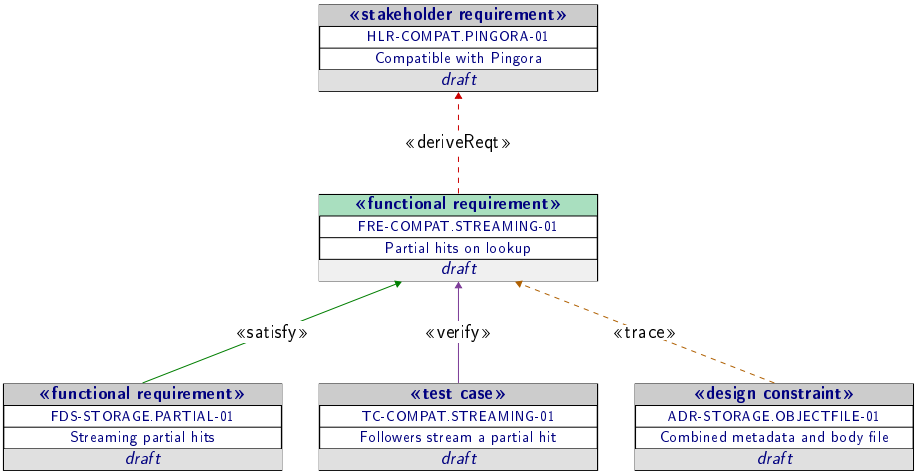


Figure 4.6: Partial hits on lookup

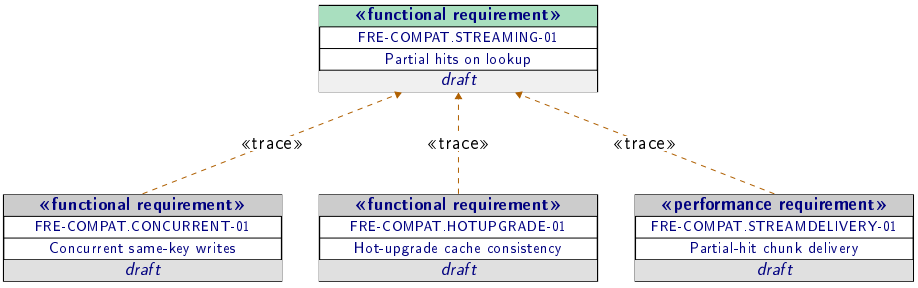


Figure 4.6: Partial hits on lookup (cont.)

`support_streaming_partial_write()` is true, rather than at `finish_miss_handler`.

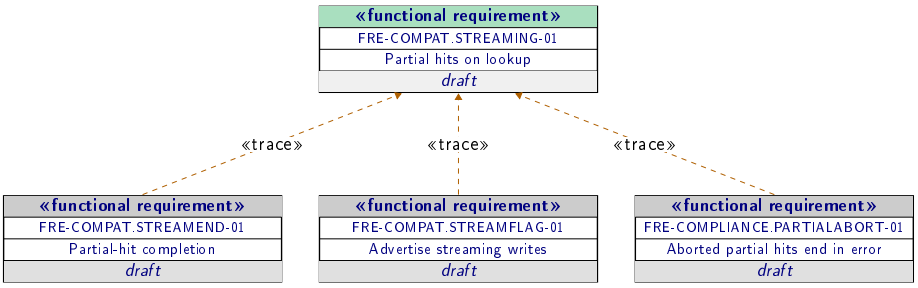


Figure 4.6: Partial hits on lookup (cont.)

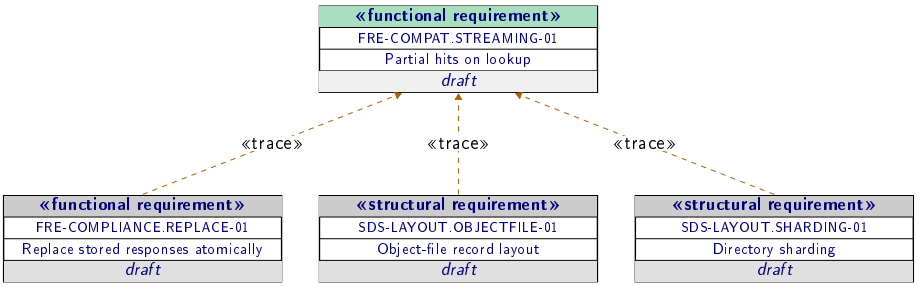


Figure 4.6: Partial hits on lookup (cont.)

WHILE one or more stores of a key that no invalidating purge has ended (FRE-COMPLIANCE.PURGE-01) are in flight — each from the opening of its miss handler until it completes or aborts — the backend shall answer a `lookup` of that key with a *partial hit* of the most recently opened of those stores: that store's object metadata together with a `HitHandler` that streams the body bytes the store writes.

The partial hit takes precedence over any published object of the same key, exactly as the stock `MemCache` prefers a partial read on lookup. `pingora-cache` permits several stores of one key to be in flight at once; resolving to the most recently opened one gives a follower the freshest response the cache is fetching. What the partial hit's reader delivers, and when, is stated in FRE-COMPAT.STREAMDELIVERY-01 and FRE-COMPAT.STREAMEND-01; what it owes when the store aborts, in FRE-COMPLIANCE.PARTIALABORT-01.

Basis: the stock `MemCache`'s unconditional preference for a partial read on lookup.[12]

View complete diagram.

[12] `MemCache::lookup` always prefers an in-flight write over a complete cached entry; per its own comment it has no preference among multiple in-flight writers, so the most-recently-opened tie-break above is this backend's own addition, not inherited framework behavior.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-STORAGE.PARTIAL-01 via ⟨⟨*satisfy*⟩⟩ — *Specify the in-flight write registry, the lookup order (in-flight write before the published record), and the most-recently-opened tie-break.*
Derives from	HLR-COMPAT.PINGORA-01
Traced from	FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.STREAMFLAG-01 FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 ADR-STORAGE.OBJECTFILE-01 FRE-COMPLIANCE.REPLACE-01 FRE-COMPLIANCE.PARTIALABORT-01 SDS-LAYOUT.SHARDING-01 SDS-LAYOUT.OBJECTFILE-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPAT.STREAMING-01

[FRE-COMPAT.STREAMDELIVERY-01]

Partial-hit chunk delivery ⟨⟨*performance requirement*⟩⟩ `draft`

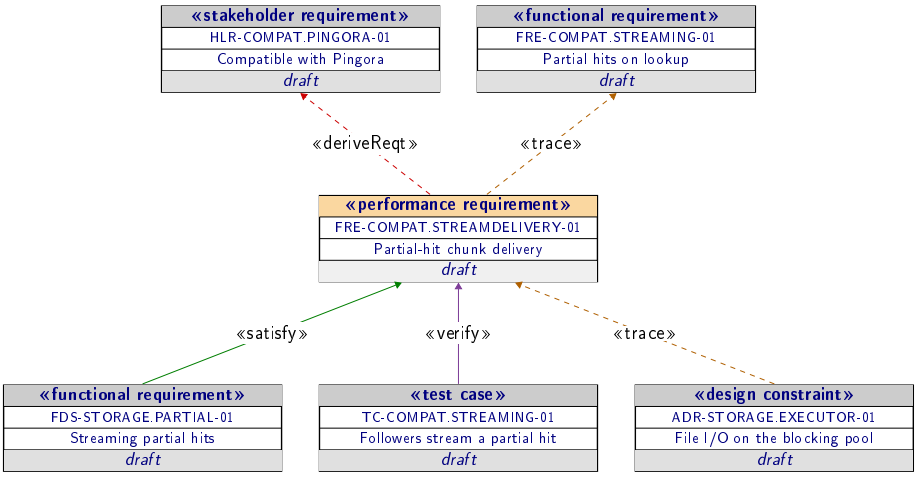


Figure 4.7: Partial-hit chunk delivery

WHILE a partial hit's reader is attached to an in-flight store, the reader shall deliver the body bytes that store had already written, followed by the chunks the store subsequently appends, with at least 99 % of those appended chunks delivered within 10 ms of their `write_body` call returning.

A chunk is delivered when the reader's `read_body` call returns it; the bound applies on a host that is not CPU-saturated. The follower thus streams the body as the leader writes it, without waiting for the store to finish. The bound is a percentile rather than a worst case so that a single scheduling delay does not violate it, while still excluding designs that poll the file or batch notifications, whose delays are typically an order of magnitude longer.

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-STORAGE.PARTIAL-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how a reader learns of each appended chunk so that it can deliver the chunk within the latency bound.*
Derives from	HLR-COMPAT.PINGORA-01
Traces to	FRE-COMPAT.STREAMING-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Traced from	ADR-STORAGE.EXECUTOR-01
Verified by	TC-COMPAT.STREAMING-01

[FRE-COMPAT.STREAMEND-01]

Partial-hit completion ⟨⟨*functional requirement*⟩⟩ `draft`

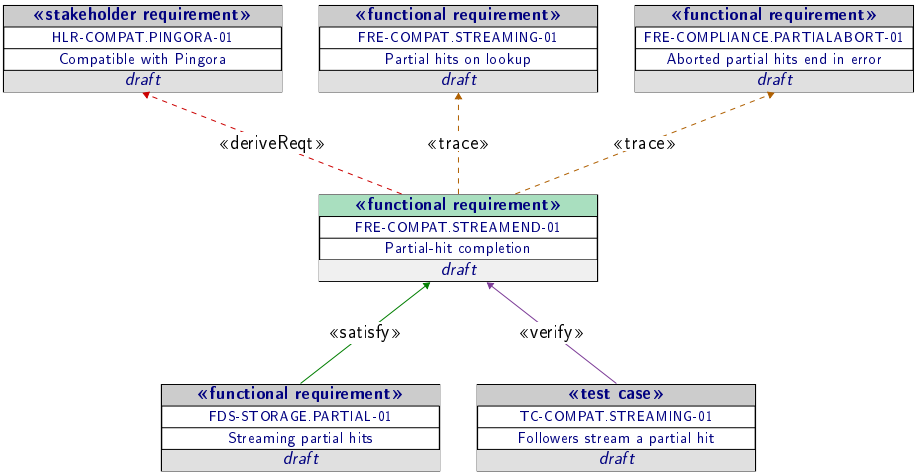


Figure 4.8: Partial-hit completion

WHEN the in-flight store a partial hit's reader is attached to completes, the reader shall deliver the store's remaining body bytes and then end the body normally, whether or not the object has meanwhile been durably published.

Completion is followed by the object's durable publication, which may happen while a follower is still reading, so a follower's normal end cannot rely on the in-flight store still existing when it gets there. When the store instead aborts — an upstream failure or a cancellation — the effect on an attached reader is governed by FRE-COMPLIANCE.PARTIALABORT-01.

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-STORAGE.PARTIAL-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how each reader delivers the remainder and ends normally on completion, and why its reader stays valid across the object's durable publication.*
Derives from	HLR-COMPAT.PINGORA-01
Traces to	FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.PARTIALABORT-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPAT.STREAMING-01

[FRE-COMPAT.FORMATUPGRADE-01]

Records across Pingora changes ⟨⟨*functional requirement*⟩⟩ `draft`

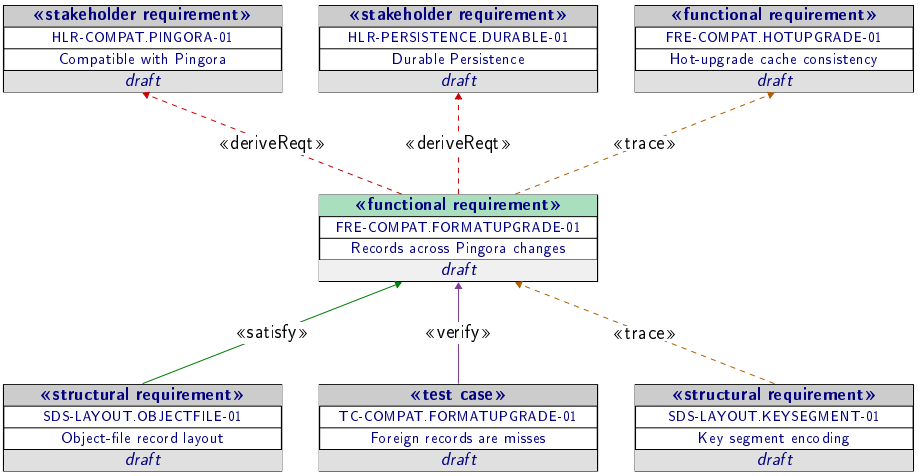


Figure 4.9: Records across Pingora changes

The metadata the backend persists is the serialisation, by `pingora-cache` itself, of each object's `CacheMeta`,[13] and that encoding is not the backend's to control. Its internal-metadata segment carries a version the framework decodes across its own releases,[14] and its header segment is compressed with a dictionary each process loads at startup.[15] Because persisted records outlive the process that wrote them (HLR-PERSISTENCE.DURABLE-01), a cache directory written by a process built against one `pingora-cache` release may be opened by a process built against another — most visibly across a hot upgrade that also changes the Pingora version (FRE-COMPAT.HOTUPGRADE-01). This requirement derives from both parents: HLR-COMPAT.PINGORA-01 fixes the framework whose encoding may change, and HLR-PERSISTENCE.DURABLE-01 makes records survive such a change.

[13] `CacheMeta::serialize` yields the internal-metadata and header byte segments the backend stores.

[14] `internal_meta::deserialize` decodes versions 0–2 and rejects any other (L190).

[15] `HEADER_SERDE` is built from the dictionary set through `set_compression_dict_path` or `set_compression_dict_content`, if any.

WHEN the backend reads a record written by a process that used a different `pingora-cache` release, the backend shall either return the object exactly as that record stored it or report a miss for it, and shall never return incorrectly decoded metadata.

The header compression dictionary is outside this requirement. `pingora-cache` offers no way to learn which dictionary a process loaded, nor which one a stored header segment was compressed with, so the backend cannot tell a record written under another dictionary from one written under its own. Every process sharing a cache directory is therefore expected to load the same dictionary, and a deployment that changes it deletes the cache directory or gives the new process a new one. This is a deployment constraint the deliverable documents, like the requirement that processes sharing a directory run on one host (ADR-STORAGE.WRITERID-01).

The backend meets this without tying its records to any Pingora release. Each record carries the backend's *own* format version (SDS-LAYOUT.OBJECTFILE-01), which the backend changes whenever records written before would no longer be read back exactly: a change to its own layout, or a `pingora-cache` release whose encoding of the metadata differs. A record of another version is reported as a miss. A `pingora-cache` release that leaves the encoding alone therefore keeps the cache warm; one that changes it starts the cache cold once.

Whether a release changed the encoding is a decision the backend's maintainers make, not an inference from a version number. The Pingora release numbering cannot settle it: 0.9.0 extended `InternalMetaV2` with new fields while keeping its version byte at 2, compatibly, so 0.8.1's metadata would have decoded under 0.9.0, yet a release could as well change the encoding under an unchanged version byte. TC-COMPAT.FORMATUPGRADE-01 holds the encoding the current format relies on as fixed bytes, and fails when the release in use reads or writes them differently, so no release is adopted without that decision. The guarantee is therefore verified for every release that test has run against. A release an integrator resolves beyond those is assumed, not verified, to keep the encoding, as a patch release of the 0.9 line should.

View complete diagram.

Required coverage	✓ SDS via ⟨⟨*satisfy*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩
Derives from	HLR-COMPAT.PINGORA-01 HLR-PERSISTENCE.DURABLE-01
Traces to	FRE-COMPAT.HOTUPGRADE-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Traced from	SDS-LAYOUT.KEYSEGMENT-01
Verified by	TC-COMPAT.FORMATUPGRADE-01

[FRE-COMPAT.WRITEFAIL-01]

Write failures fail the store ⟨⟨*functional requirement*⟩⟩ `draft`

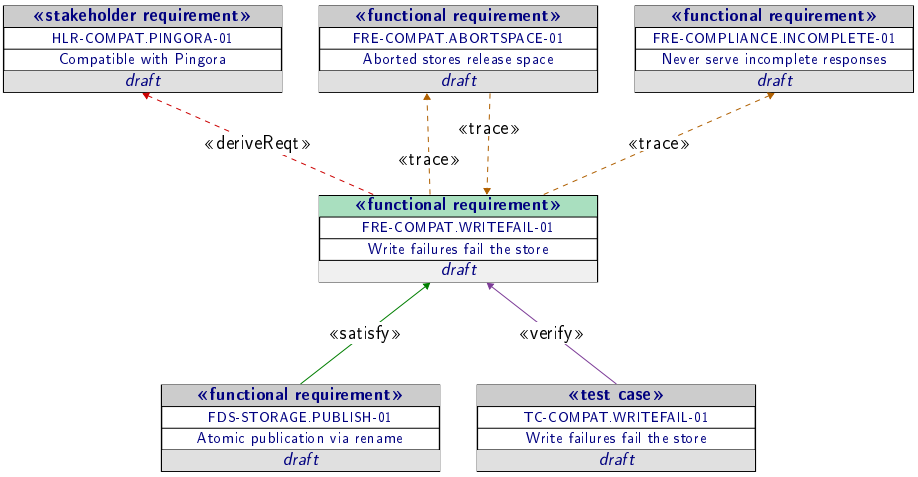


Figure 4.10: Write failures fail the store

IF writing a store's data to persistent storage fails — for example because the file system is full — THEN the backend shall return an error from the `write_body` or `finish` call during which the failure occurred.

The `HandleMiss` contract gives each call a `Result` for exactly this purpose.[16] A store that has failed this way is incomplete, so what lookups may then observe of it is governed by FRE-COMPLIANCE.INCOMPLETE-01, and the storage it consumed by FRE-COMPAT.ABORTSPACE-01.

View complete diagram.

Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how a failed write or sync during a store is detected and surfaced as an error from the call in progress, and that the store is then treated as aborted.* ✓ TC via ⟨⟨*verify*⟩⟩
Derives from	HLR-COMPAT.PINGORA-01
Traces to	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPAT.ABORTSPACE-01
Traced from	FRE-COMPAT.ABORTSPACE-01
Satisfied by	FDS-STORAGE.PUBLISH-01
Verified by	TC-COMPAT.WRITEFAIL-01

[16] `HandleMiss::write_body` returns `Result<()>`.

[FRE-COMPAT.ABORTSPACE-01]

Aborted stores release space ⟨⟨*functional requirement*⟩⟩ `draft`

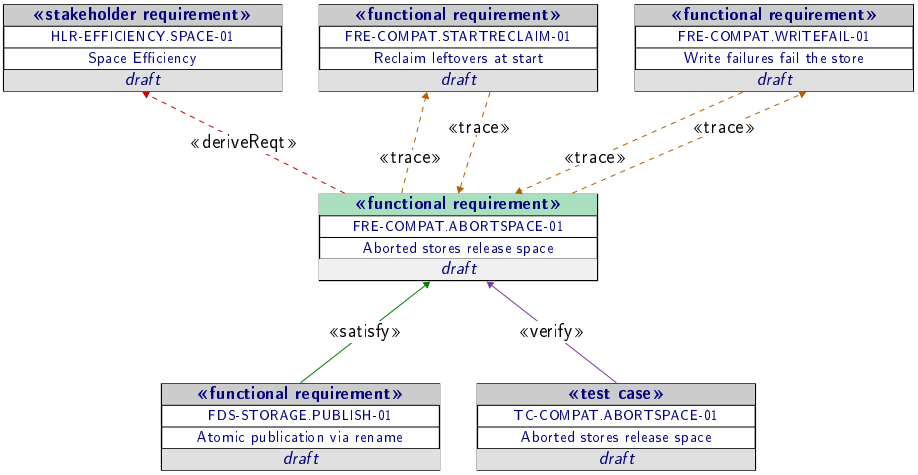


Figure 4.11: Aborted stores release space

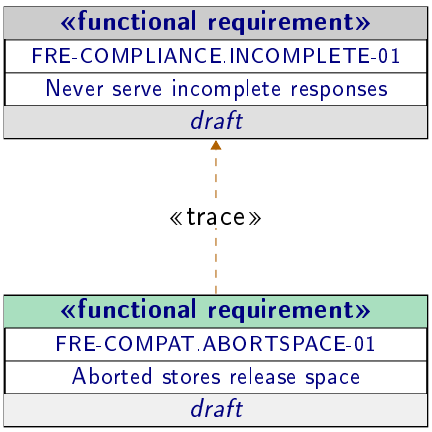


Figure 4.11: Aborted stores release space (cont.)

WHEN a store ends without being durably published — aborted by an upstream failure, a cancellation, or a write failure — the backend shall remove the store's temporary no later than when its miss handler is dropped, and shall free the storage that store consumed without waiting for a restart.

Space held by an unpublished store is not counted by the eviction manager, which admits an object only once its store completes, so a store that failed without releasing its space would shrink the disk available to the cache with every upstream failure until the next restart. Freeing a large file's storage can take a long time, so it may complete shortly after the drop, away from the thread that dropped the handler; a reader still streaming the store keeps the storage until it finishes (FDS-STORAGE.PARTIAL-01). The stock backend releases an unfinished write the same

way.[17] A process crash is different — no handler is dropped — and its leftovers are reclaimed as FRE-COMPAT.STARTRECLAIM-01 requires.

View complete diagram.

Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*satisfy*⟩⟩ — *Specify that an unpublished store's per-writer temporary is unlinked when the store ends, including on abort and on write failure, and how attached partial-hit readers are unaffected by that unlink.* ✓ TC via ⟨⟨*verify*⟩⟩
Derives from	HLR-EFFICIENCY.SPACE-01
Traces to	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPAT.WRITEFAIL-01 FRE-COMPAT.STARTRECLAIM-01
Traced from	FRE-COMPAT.WRITEFAIL-01 FRE-COMPAT.STARTRECLAIM-01
Satisfied by	FDS-STORAGE.PUBLISH-01
Verified by	TC-COMPAT.ABORTSPACE-01

[FRE-COMPAT.STARTRECLAIM-01]

Reclaim leftovers at start ⟨⟨*functional requirement*⟩⟩ `draft`

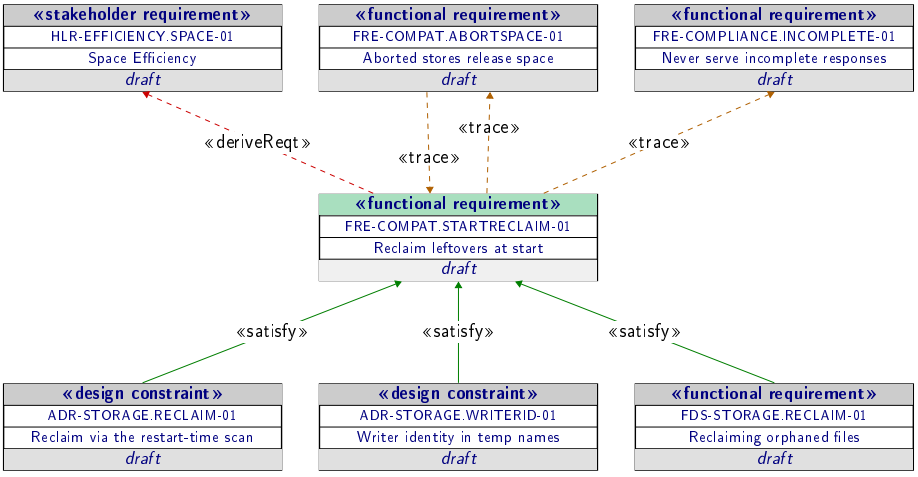


Figure 4.12: Reclaim leftovers at start

[17] `Drop for MemMissHandler` removes the handler's in-flight write.

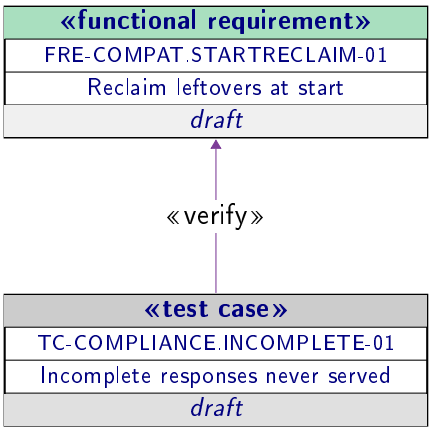


Figure 4.12: Reclaim leftovers at start (cont.)

WHEN a process of the backend starts, the backend shall release the storage held by unpublished stores whose writing process is no longer running.

Such a leftover is not reachable through a keyed `Storage` call again: a process crash drops no miss handler, so the release on abort (FRE-COMPAT.ABORTSPACE-01) never runs for its stores. Without a release at start, each crash would shrink the disk available to the cache for good. A store whose writer is still running — such as a hot-upgrade peer's — is not a leftover, and the incoming process leaves it untouched (FRE-COMPAT.HOTUPGRADE-01).

A `Vary` variant record whose primary key holds no stored object is not a leftover either. It is a published object, which the eviction manager counts against the cache's size budget and evicts like any other (FRE-COMPAT.EVICTION-01), and a lookup may still reach it: an integrator's cache key can carry a variance of its own, and no primary is then ever stored.

View complete diagram.

Required coverage	✓ FDS via ⟨⟨*satisfy*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩
Derives from	HLR-EFFICIENCY.SPACE-01
Traces to	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPAT.ABORTSPACE-01
Traced from	FRE-COMPAT.ABORTSPACE-01
Satisfied by	ADR-STORAGE.WRITERID-01 ADR-STORAGE.RECLAIM-01 FDS-STORAGE.RECLAIM-01
Verified by	TC-COMPLIANCE.INCOMPLETE-01

[FRE-COMPAT.EXPIRE-01]

Expire keeps the body ⟨⟨*functional requirement*⟩⟩ `draft`

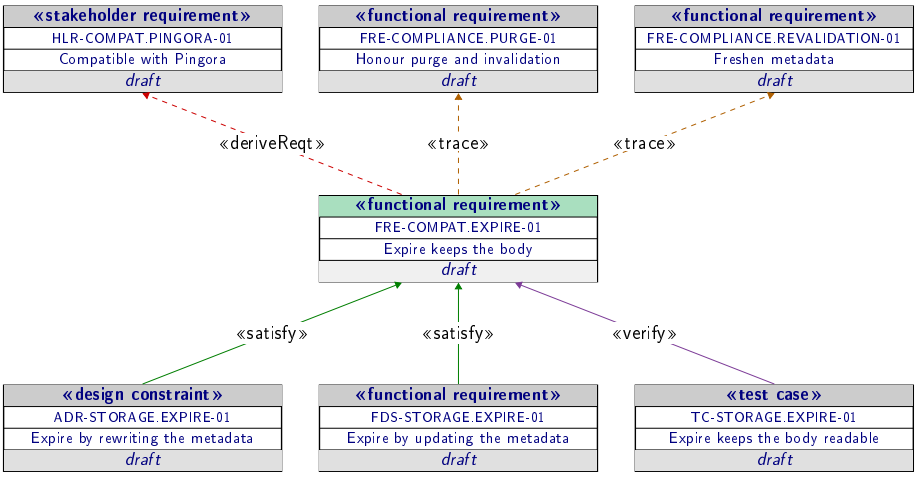


Figure 4.13: Expire keeps the body

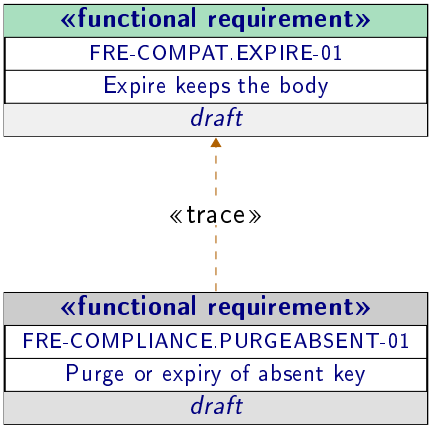


Figure 4.13: Expire keeps the body (cont.)

WHEN the framework asks the storage backend to expire a stored object, the backend shall leave that object in the stale state.

An object is in the *stale state* when a later lookup for its key returns it, with its stored body readable in full, and reports it as not fresh at any instant after the expiry. Deleting the object is therefore not the stale state, and neither is leaving it fresh.

The boundary instant itself is deliberately excluded, because the framework owns both halves of that comparison and places it there. `CacheMeta::expire_at(instant)`, which Pingora documents as marking an asset "stale as of *instant*",[18] lowers the

[18] `CacheMeta::expire_at` sets `fresh_until` to the earlier of its current value and *instant*.

stored freshness deadline to that instant, while `CacheMeta::is_fresh` compares inclusively.[19] An object expired at t is therefore reported fresh at exactly t and stale at every instant after it — a distinction below the one-second resolution RFC 9111 works at, and one no backend storing the deadline the framework derives can alter.

Pingora 0.9.0 added this second removal operation beside `purge`.[20] Where a purge deletes the object and costs the next request a full refetch, an expiry keeps it and costs only a conditional request the origin can answer with a 304, after which the retained body is served again. A proxy chooses between the two per request.[21]

The obligation is about the object's observable state, not about any particular field or file the backend rewrites to reach it. Two things it deliberately leaves alone. Whether a reader is served the retained body *while* that revalidation is in flight is decided by the object's own serve-stale windows, which an expiry does not touch. And a store already in flight for the same key is not cancelled: if it completes it publishes a fresh object. An invalidating purge, by contrast, ends such a store (FRE-COMPLIANCE.PURGE-01), since it may carry the content the purge removes; an expiry asks only for a revalidation, which a fresh object provides.

The trait supplies a default that deletes the object instead,[22] which does keep the object from being served as fresh, but by removing it rather than by reaching the stale state, so it does not satisfy this requirement. What an expiry of a key holding nothing reports follows FRE-COMPLIANCE.PURGEABSENT-01.

View complete diagram.

Required coverage	✓ ADR via ⟨⟨*satisfy*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩
Derives from	HLR-COMPAT.PINGORA-01
Traces to	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.REVALIDATION-01
Satisfied by	ADR-STORAGE.EXPIRE-01 FDS-STORAGE.EXPIRE-01
Traced from	FRE-COMPLIANCE.PURGEABSENT-01
Verified by	TC-STORAGE.EXPIRE-01

[19] `CacheMeta::is_fresh` returns `fresh_until >= time`.
[20] `Storage::expire`.
[21] `ProxyHttp::purge_action`.
[22] The default forwards to `Storage::purge`.

[ADR-PERSISTENCE.MECHANISM-01]

Plain file system storage ⟨⟨*design constraint*⟩⟩ `draft`

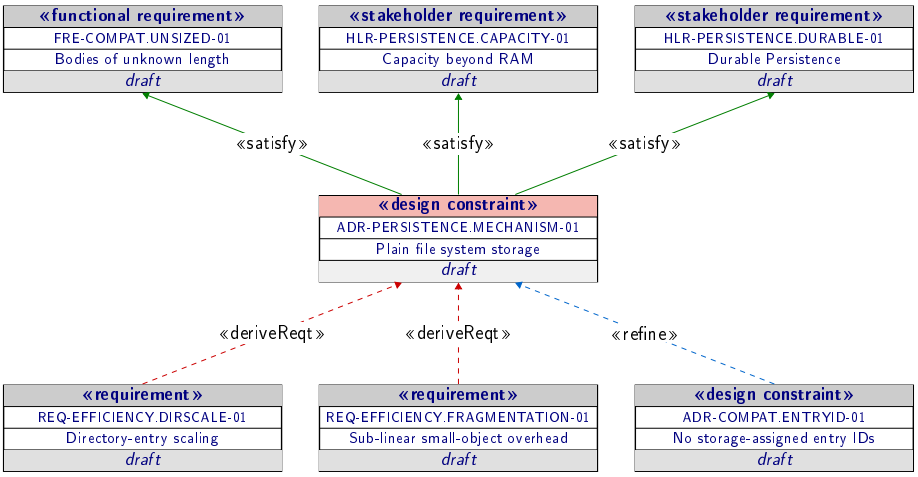


Figure 4.14: Plain file system storage

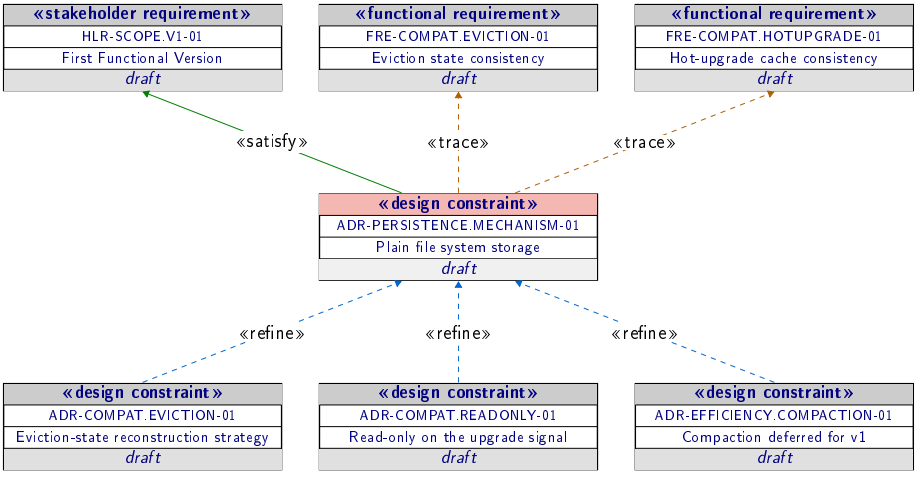


Figure 4.14: Plain file system storage (cont.)

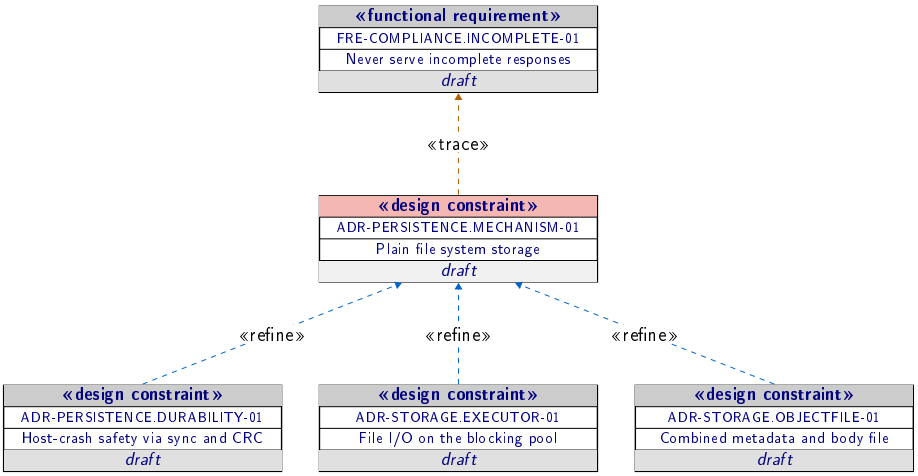


Figure 4.14: Plain file system storage (cont.)

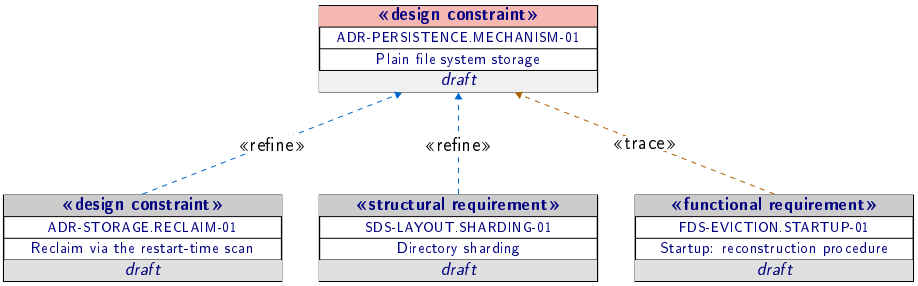


Figure 4.14: Plain file system storage (cont.)

Context: The cache must persist HTTP response headers and bodies durably (HLR-PERSISTENCE.DURABLE-01), this is a first version that favours the simplest adequate design (HLR-SCOPE.V1-01), and a body arrives as a stream whose total length is unknown when writing begins (FRE-COMPAT.UNSIZED-01). The choice is twofold: which storage substrate to build on, and whether to adopt an established storage engine or write the storage layer ourselves.

Decision: Persist cached objects on a plain file system — growable files in directories — rather than adopting an embeddable storage engine or building on a raw block device, a custom preallocated arena, or an embedded database. The kernel's file system already does an allocator's work: block allocation and free-space management, growing files in place, keeping *its own* structures intact across crashes (journaling), and giving every process on the host one coherent view of the same files. What it deliberately does not give, the backend keeps as its own duty:

- *Atomic publication.* A file being written when the process dies is not rolled back, so object publication must be made atomic and leftovers discarded (FRE-COMPLIANCE.INCOMPLETE-01, FRE-COMPAT.STARTRECLAIM-01, ADR-STORAGE.ATOMICPUBLISH-01).

- *Crash safety of its own writes.* A write can be reported as done while the bytes are still only in the operating system's cache, and syncing a file does

not persist the directory entry that names it, so a host crash can lose or reorder writes. Which writes the backend synchronises, and what it detects instead, is its own decision (ADR-PERSISTENCE.DURABILITY-01). The eviction manager's state is not among them: it is rebuilt from the object records themselves at every start (ADR-COMPAT.EVICTION-01).

- *Hot-upgrade coordination*. During a hot upgrade two processes see the same bytes but are not coordinated by the file system; making the outgoing process stop changing the shared cache, while leaving its open readers intact, is the backend's own protocol (FRE-COMPAT.HOTUPGRADE-01).

Every alternative below leaves those same coordination duties open — or solves them at the price of a poorer fit for streaming bodies and a larger integration surface — which is why the simplest substrate wins for a first version (HLR-SCOPE.V1-01). Appendix A examines each alternative in detail.

Alternatives considered:

- *Adopt Apache Traffic Server's object store*: the most proven raw-disk HTTP cache, but not an adoptable component — the store is built into Traffic Server's own process model and not published as a library,[23] so adopting it means carrying the server along.
- *Adopt Meta's CacheLib*, an embeddable C++ caching engine: proven allocation, admission, and eviction machinery,[24] and a body of unknown length fits as incrementally chained per-chunk allocations.[25] Its eviction machinery need not conflict with the one in `pingora-cache`: the `EvictionManager` `pingora-cache` drives is optional,[26] so a backend can leave it unset and let CacheLib manage capacity on its own (FRE-COMPAT.EVICTION-01). Two costs rule it out for a first version instead:
  - a C++ dependency, driven from Rust through an FFI boundary;
  - one active cache instance per path,[27] where a hot-upgrade window has two processes sharing one store (FRE-COMPAT.HOTUPGRADE-01).
- *Embedded database as the whole store* (libmdbx/LMDB, RocksDB, or pure-Rust engines such as `redb` and `fjall`): record packing and crash consistency come built in, and the pure-Rust engines remove the FFI cost as well, but their unit of storage is the record — a body streaming in with unknown length (FRE-COMPAT.UNSIZED-01) must be chunked into records by the application — and streaming partial hits (FRE-COMPAT.STREAMING-01) need the same in-memory coordination beside the store anyway. Cross-process use during a hot upgrade also varies by engine: RocksDB admits no multi-process writers,[28] so the incoming process could not write while the outgoing

[23] The cache implementation lives under `src/iocore/cache`, part of IOCore (the process's core I/O and event system), and is built as an internal library linked only into the `traffic_server` binary — there is no separately packaged cache library. https://github.com/apache/trafficserver/tree/10.1.4/src/iocore/cache

[24] https://cachelib.org/docs/Cache_Library_User_Guides/eviction_policy/#eviction-policy

[25] https://cachelib.org/docs/Cache_Library_User_Guides/chained_items/#chained-allocations

[26] `HttpCache::enable` takes it as an `Option`; without it, nothing is evicted from storage.

[27] https://cachelib.org/docs/Cache_Library_User_Guides/Cache_persistence/#common-failure-scenarios

[28] https://github.com/facebook/rocksdb/wiki/RocksDB-FAQ#basic-readwrite

one still holds the database open, whereas the LMDB family allows several processes, one write transaction at a time.[29]

- *Adopt an existing Rust disk cache* such as `cacache`, a key- and content-addressed cache with integrity-checked content:[30] proven, no FFI, and streaming writes are supported. Rejected for this version: content becomes visible only when a write commits, so streaming partial hits still need the backend's own in-flight coordination; content addressing separates each object's metadata from its body, so a purge or metadata update leaves content that some unreferenced-content collection must remove, which its documentation does not provide; and a cryptographic integrity hash is computed over every body.
- *Build on a raw block device or custom arena* (the substrate beneath Traffic Server's cache, CacheLib's flash engine, and Squid's Rock store): eliminates all per-object file-system overhead, and the systems built on it prove the model serves HTTP caching well. Choosing this substrate means building space management and an on-device directory from scratch. That combination is the largest bespoke surface of any alternative here, and the antithesis of a first version (HLR-SCOPE.V1-01).

Consequences:

+ Total cache size is bounded by disk capacity, not process RAM (HLR-PERSISTENCE.CAPACITY-01).
+ A growable file holds a body of unknown length with no size declared up front; bytes are appended as they stream in — no per-chunk allocation and chaining, no pre-declared record sizes.
+ No foreign engine to integrate: no FFI boundary, no second eviction policy beside the one in `pingora-cache`, no engine-specific recovery tooling; the custom surface is confined to layout, naming, and the coordination duties above.
– Ties the cache to file-system semantics: block-granular allocation with per-file overhead (REQ-EFFICIENCY.FRAGMENTATION-01) and per-directory entry scaling (REQ-EFFICIENCY.DIRSCALE-01) — stated as derived requirements that the detailed decisions must satisfy.
– An established engine would come with packing and compaction already built and tested; giving that up means the derived requirements above only set the efficiency targets — the detailed design still has to build the machinery that meets them.

View complete diagram.

[29] `https://github.com/erthink/libmdbx#gotchas`

[30] `cacache` crate documentation: "a Rust library for managing local key and content address caches"; a streaming `Writer`'s data is committed only by `commit()`.

Required coverage	✓ REQ-EFFICIENCY.FRAGMENTATION-01 via ⟨⟨*deriveReqt*⟩⟩ — *Bound the aggregate space overhead a plain file system imposes on objects smaller than one block — block rounding and per-file inode/directory-entry costs — so that it grows sub-linearly with the number of such objects.* ✓ REQ-EFFICIENCY.DIRSCALE-01 via ⟨⟨*deriveReqt*⟩⟩ — *Bound the number of entries that accumulate in any one directory, up to a stated cache capacity, so object lookup and maintenance stay fast as the number of cached objects grows.* ✓ ADR via ⟨⟨*refine*⟩⟩
Satisfies	HLR-PERSISTENCE.DURABLE-01 HLR-PERSISTENCE.CAPACITY-01 HLR-SCOPE.V1-01 FRE-COMPAT.UNSIZED-01
Traces to	FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.EVICTION-01 FRE-COMPLIANCE.INCOMPLETE-01
Derived from	REQ-EFFICIENCY.FRAGMENTATION-01 REQ-EFFICIENCY.DIRSCALE-01
Refined by	ADR-STORAGE.OBJECTFILE-01 ADR-PERSISTENCE.DURABILITY-01 ADR-STORAGE.EXECUTOR-01 ADR-STORAGE.RECLAIM-01 ADR-EFFICIENCY.COMPACTION-01 ADR-COMPAT.EVICTION-01 ADR-COMPAT.ENTRYID-01 ADR-COMPAT.READONLY-01 SDS-LAYOUT.SHARDING-01
Traced from	FDS-EVICTION.STARTUP-01

[REQ-EFFICIENCY.FRAGMENTATION-01]

Sub-linear small-object overhead ⟨⟨*requirement*⟩⟩ `draft`

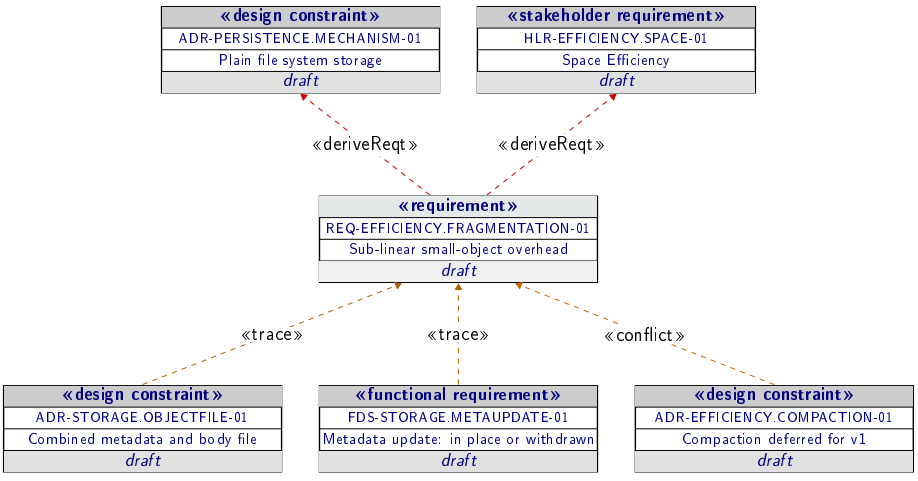


Figure 4.15: Sub-linear small-object overhead

For cached objects smaller than one file-system block, the aggregate space the on-disk layout consumes beyond those objects' own metadata and body bytes shall grow sub-linearly with the number of such objects.

A plain file system (ADR-PERSISTENCE.MECHANISM-01) allocates space in whole blocks (commonly 4 KiB), so a stored file's tail can waste up to nearly a full block. It also charges every file a fixed cost for its inode and directory entry, regardless of the file's size. Across many small cached objects — a busy HTTP cache may hold many — these costs grow in step with the object count and can waste a significant fraction of the disk. This requirement derives from both parents: the stakeholder space-efficiency goal (HLR-EFFICIENCY.SPACE-01) is what the waste works against, and the plain-file-system decision is what generates it. Verification is by analysis of the layout, confirmed by measuring disk use at growing object counts. The first functional version does not meet this requirement: ADR-EFFICIENCY.COMPACTION-01 defers the packing that would, and declares the conflict.

View complete diagram.

Required coverage	☐ ADR via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-EFFICIENCY.SPACE-01 ADR-PERSISTENCE.MECHANISM-01
Traced from	ADR-STORAGE.OBJECTFILE-01 FDS-STORAGE.METAUPDATE-01
Conflicts with	ADR-EFFICIENCY.COMPACTION-01

[REQ-EFFICIENCY.DIRSCALE-01]

Directory-entry scaling ⟨⟨*requirement*⟩⟩ `draft`

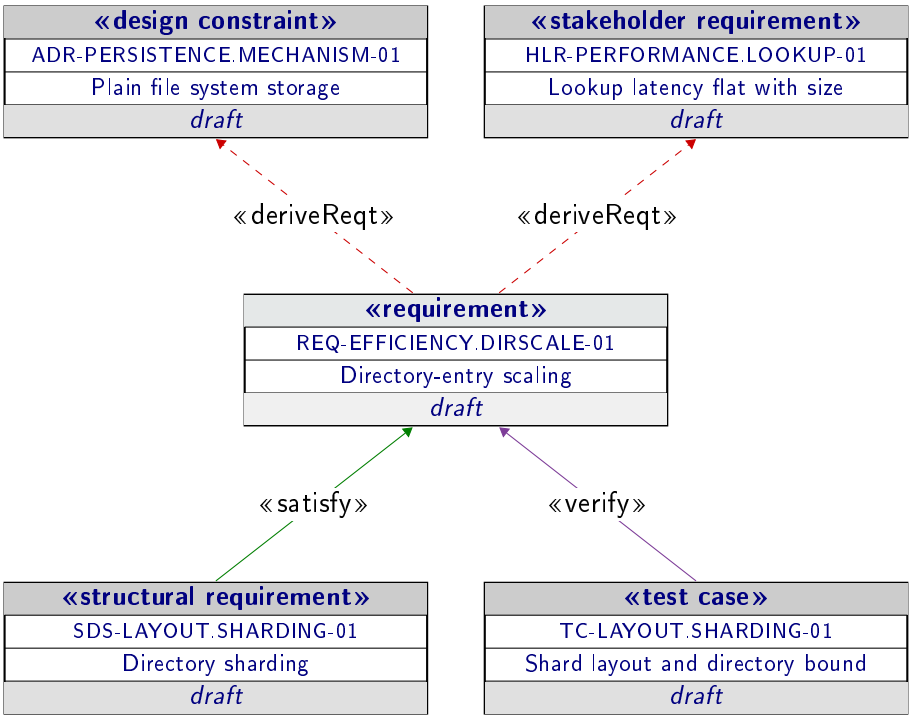


Figure 4.16: Directory-entry scaling

For a cache holding up to 10^8 objects whose cache-key hashes are uniformly distributed, the on-disk layout shall hold no more than 2,000 entries in any one directory.

This requirement derives from two parents. On a plain file system (ADR-PERSISTENCE.MECHANISM-01) every cached object is reached through a directory entry, and many file systems slow down on name lookup and insertion as a single directory's entry count grows; listings and housekeeping scans grow more expensive with it. Left unbounded, that growth would make lookups slower as the cache fills, which HLR-PERFORMANCE.LOOKUP-01 rules out. The capacity and the bound are a design target, deliberately beyond the 10^7 objects that requirement measures, not a limit on what the cache may hold: beyond 10^8 objects the bound no longer applies, but nothing else in this specification stops working. Verification is by analysis of the expected entry distribution for the layout, together with a test of the fan-out the analysis assumes.

View complete diagram.

Required coverage	✓ SDS-LAYOUT.SHARDING-01 via ⟨⟨*satisfy*⟩⟩ — *Lay out objects so that, for up to* 10^8 *uniformly hashed objects, no directory holds more than 2,000 entries.* ✓ TC via ⟨⟨*verify*⟩⟩
Derives from	ADR-PERSISTENCE.MECHANISM-01 HLR-PERFORMANCE.LOOKUP-01
Satisfied by	SDS-LAYOUT.SHARDING-01
Verified by	TC-LAYOUT.SHARDING-01

5. Detailed Refinement

[ADR-STORAGE.OBJECTFILE-01]

Combined metadata and body file ⟨⟨*design constraint*⟩⟩ `draft`

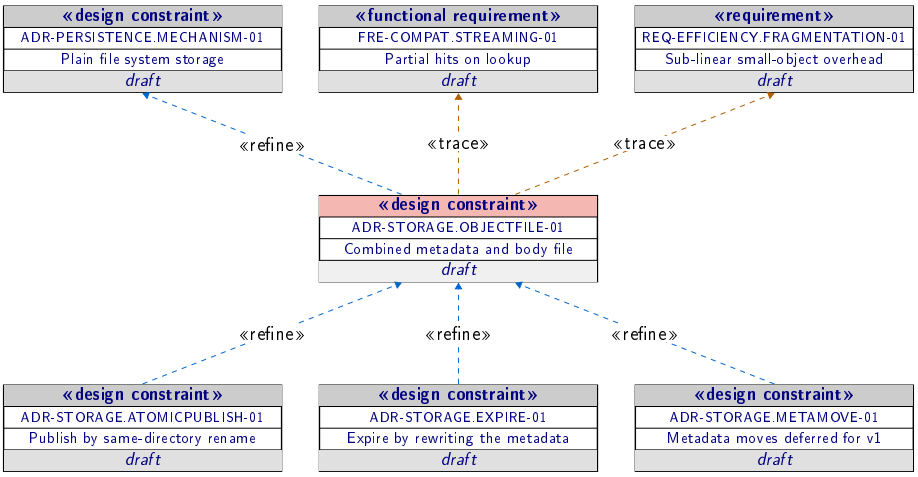


Figure 5.1: Combined metadata and body file

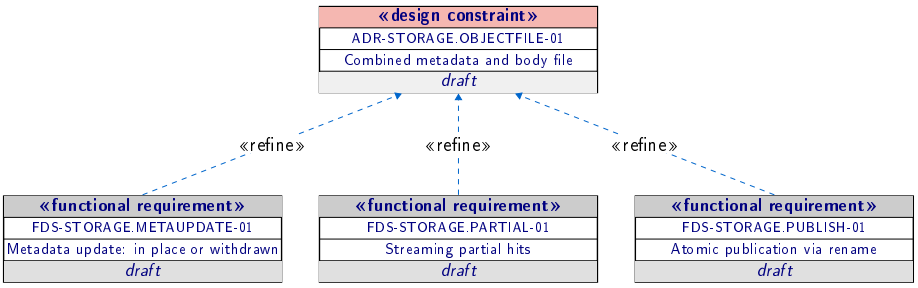


Figure 5.1: Combined metadata and body file (cont.)

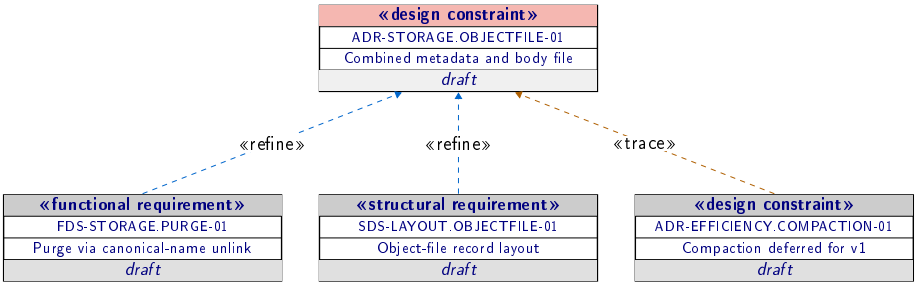


Figure 5.1: Combined metadata and body file (cont.)

Context: Each cached object has a small block of metadata and a body of arbitrary length, unknown when writing begins (FRE-COMPAT.UNSIZED-01), persisted on a plain file system (ADR-PERSISTENCE.MECHANISM-01). Block-granular allocation means a separate file holding only the small metadata would waste most of a block per object (REQ-EFFICIENCY.FRAGMENTATION-01).

Decision: Store each object's metadata and body together in a single growable file, in file order: the metadata written ahead of the body content, the body appended as its chunks arrive. A metadata update that later outgrows the space reserved ahead of the body withdraws the record, and the object is fetched again (FDS-STORAGE.METAUPDATE-01); moving that metadata after the body instead is deferred to a later version (ADR-STORAGE.METAMOVE-01).

Alternatives considered:

- *A separate file holding only the object's metadata*, body kept in its own file: keeps each region independently sized and simple to read in isolation, but the small, mostly-empty metadata file wastes most of a block per object regardless of the body's size (REQ-EFFICIENCY.FRAGMENTATION-01). Committing the two files as one atomic unit also loses the single-rename publication this design enables (FDS-STORAGE.PUBLISH-01): a plain rename is atomic per path, not across two, so publishing both together would need a per-object subdirectory renamed as a whole — an extra inode and directory entry per object, working against the same efficiency goal — or a separate commit marker. Nor does splitting the files cleanly avoid in-place metadata updates (FDS-STORAGE.METAUPDATE-01): replacing the metadata file wholesale on every revalidation avoids mutating it in place, but pays a fresh inode and directory entry each time, so a design that cares about that cost would still want to reserve headroom and update it in place — the same capacity accounting and mid-write-race handling this design already needs, just for a smaller file.

- *Metadata in the body file's extended attributes*: no second file, and a small attribute can live inside the inode itself, costing no extra block. Rejected: a header set has no size cap (FRE-COMPLIANCE.HEADERS-01), while attribute space is capped[1] — so large header sets could not be stored at all, and typical header segments overflow the in-inode space into a block anyway; attribute writes are also separate from the file's data, adding a second update channel to keep consistent with the body.

- *Metadata as a trailer after the body*: the metadata's size no longer has to be reserved ahead of a body of unknown length, and a growing update simply rewrites the tail. Rejected: every update changes the file's length while readers may be streaming the body's final bytes and validating the length they were promised, and every lookup must locate the trailer through a fixed footer before it can read anything; a header-first record confines updates to a fixed-position region instead.

- *A sidecar index* — one embedded key-value store holding every object's metadata, with bodies in their own files: small metadata records pack tightly, and

[1] Linux manual page: `xattr(7)`: "The VFS-imposed limits on attribute names and values are 255 bytes and 64 kB, respectively", and on ext2, ext3, and ext4 "the total bytes used by the names and values of all of a file's extended attributes must fit in a single filesystem block".

updates become transactional. Rejected: publishing an object then spans two stores, the body file and the index entry, which can no longer be made visible by a single rename and need an ordering and recovery protocol of their own; and it reintroduces the embedded engine, with its cross-process locking, that ADR-PERSISTENCE.MECHANISM-01 declined.

Consequences:

+ No block is wasted on a separate, mostly-empty metadata file.
+ One growable file naturally holds a body whose size is not known when writing begins, and a streaming partial reader can follow the same file the leader is still writing (FRE-COMPAT.STREAMING-01).
– Metadata and body share a single file, so reading the metadata requires opening the object file rather than a smaller dedicated one.
– The on-disk format must encode the metadata length or body offset so the two regions can be told apart on read.
– Metadata that outgrows the region ahead of the body costs a fetch of the whole object from the origin (ADR-STORAGE.METAMOVE-01).

View complete diagram.

Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify how object publication is made atomic under concurrent same-key writes and process crashes.* ✓ FDS-STORAGE.METAUPDATE-01 via ⟨⟨*refine*⟩⟩ — *Specify how a metadata update that grows past its record's reserved capacity is handled.* ✓ SDS-LAYOUT.OBJECTFILE-01 via ⟨⟨*refine*⟩⟩ — *Specify the concrete on-disk record format — header, key, metadata region, and body placement — that realises the combined metadata-and-body file.*
Refines	ADR-PERSISTENCE.MECHANISM-01
Traces to	FRE-COMPAT.STREAMING-01 REQ-EFFICIENCY.FRAGMENTATION-01
Refined by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PARTIAL-01 FDS-STORAGE.METAUPDATE-01 ADR-STORAGE.METAMOVE-01 ADR-STORAGE.EXPIRE-01 FDS-STORAGE.PURGE-01 SDS-LAYOUT.OBJECTFILE-01
Traced from	ADR-EFFICIENCY.COMPACTION-01

[ADR-PERSISTENCE.DURABILITY-01]

Host-crash safety via sync and CRC ⟨⟨*design constraint*⟩⟩ `draft`

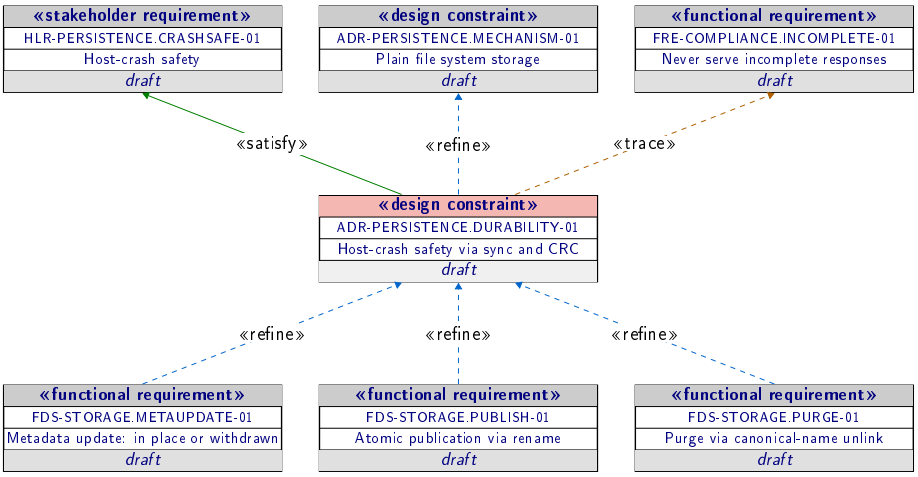


Figure 5.2: Host-crash safety via sync and CRC

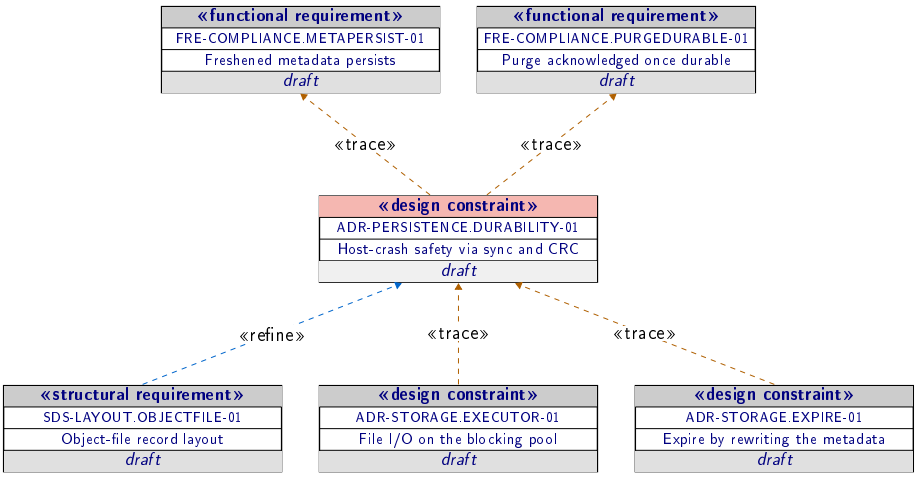


Figure 5.2: Host-crash safety via sync and CRC (cont.)

Context: After the host crashes or loses power, the cache may lose objects but must never serve content that differs from what was stored, nor an object whose purge it acknowledged (HLR-PERSISTENCE.CRASHSAFE-01); the same bar applies to incomplete stores (FRE-COMPLIANCE.INCOMPLETE-01), acknowledged purges (FRE-COMPLIANCE.PURGEDURABLE-01), and applied metadata updates (FRE-COMPLIANCE.METAPERSIST-01). A plain file system (ADR-PERSISTENCE.MECHANISM-01) gives none of this for free. A `write` returns while the bytes are still only in the page cache; syncing a file does not persist the

directory entry that names it;[2] and a rename can reach the journal before the renamed file's data does. ext4 guards only the *replace*-via-rename pattern against that, forcing the new file's data out before such a rename commits;[3] a first-time publication under a new name gets no such help. A synchronous flush costs roughly 1–5 ms on consumer SSDs and under 0.1 ms on drives with power-loss protection.[4]

Decision: Synchronise exactly the writes whose loss could make the cache serve wrong or purged content, and detect the rest.

- *Publication:* `fdatasync` a store's temporary — after its last body byte and its patched header — before renaming it to the canonical name. The directory is not synced after the rename, so a host crash may lose a just-published object (the key then keeps its previous record, or misses), but the name can never refer to data that is not on stable storage.
- *Invalidating purge:* `fsync` the shard directory after the `unlink`, or after finding nothing to unlink, before reporting the outcome. A purge made to evict is not synchronised: a host crash that undoes it brings back a valid object the cache chose to drop.
- *Metadata update, in place or a withdrawal:* not synced. After a host crash the region holds the old metadata, the new, or a torn mixture, and a withdrawn record may be back as it was; the metadata checksum rejects the mixture, so the object is either served with one complete version or reported as a miss.
- *Body checksum:* each record carries a CRC-32C of its body, computed while the body streams in and recorded at completion. Serving a hit does not check it. A build can leave it out (the Cargo feature `body-crc`, on by default; without it a record carries zero), and a build with the `body-crc-verify` feature can switch on, in its configuration, a read-time check that logs a mismatch once a read has delivered the whole body, and still ends the read normally.

With synchronised publication, on a storage stack that honours flush requests as HLR-PERSISTENCE.CRASHSAFE-01 assumes, a host crash cannot leave a published name over wrong bytes. Integrity at rest — media corruption, or a device that acknowledges a flush it has not made — is the storage layer's: a data-checksumming file system (ZFS, btrfs, bcachefs) or `dm-integrity` turns a corrupted block into a read error, which ends the read with an error in place of the corrupted bytes. The header and metadata checksums stay: they detect a torn in-place metadata update, and a lookup racing one, which no storage layer can.

Alternatives considered:

- *Process-crash safety only* (no synchronisation): cheapest, and sufficient for HLR-PERSISTENCE.DURABLE-01, since a process crash leaves the page cache intact. Rejected: after a host crash a first-time rename can commit ahead of its data, leaving a canonical name over a zero-filled or short body

[2] Linux manual page: `fsync(2)`: "Calling `fsync()` does not necessarily ensure that the entry in the directory containing the file has also reached disk. For that an explicit `fsync()` on a file descriptor for the directory is also needed."

[3] Linux kernel documentation: "ext4 General Information", `auto_da_alloc`: the replace-via-rename pattern is detected so that "the data blocks of the new file are forced to disk before the `rename()` operation is committed", avoiding the "zero-length" problem.

[4] Mark Callaghan, "Small Datum": "SSDs, power loss protection and fsync latency".

whose header still passes validation, and an unsynced purge `unlink` can be lost, bringing a purged object back — both forbidden by HLR-PERSISTENCE.CRASHSAFE-01.

- *Checksum only, without synchronisation*: detects a corrupted body, but only once a reader reaches its end, after the corrupted bytes have already gone to the client; and it does nothing for a lost purge. Rejected.
- *Check the body checksum on every whole-body read*, ending a read whose body does not match with an error instead of a clean end of body. Rejected: the check can only run after the last byte, and `pingora-cache` sends each chunk downstream as the backend returns it, so a client given a `Content-Length` has the whole corrupted body by then and the error can only cut the connection. On the storage stack HLR-PERSISTENCE.CRASHSAFE-01 assumes it never fires, yet it costs a CRC-32C pass per read, about as much CPU as the read itself when the body is in the page cache (13.3 against 12.3 microseconds per 64 KiB chunk, measured on an 8-core host).
- *Full host-crash durability*: additionally sync the directory after every rename and sync every in-place metadata update, so no completed store or update is ever lost. Rejected: it roughly doubles the synchronous cost of a store and adds one to every revalidation, to prevent only losses HLR-PERSISTENCE.CRASHSAFE-01 explicitly accepts — the kind of cost HLR-SCOPE.V1-01 asks not to pay without a requirement behind it.

Consequences:

+ A host crash can lose recently stored objects or recent metadata updates, but can neither corrupt what is served nor resurrect an object an invalidation removed; an eviction it undoes brings back a valid object.
+ Process crashes and orderly restarts lose nothing that had been published (HLR-PERSISTENCE.DURABLE-01).
– One `fdatasync` per completed store, paid when the store finishes, and one directory `fsync` per invalidating purge; evictions pay none.
+ Serving a hit computes no checksum over the body.
– The record header carries a body-checksum field, and every store computes a CRC-32C over its body, unless the build leaves it out.
– Serving a hit does not detect a body corrupted at rest. That takes a storage layer that verifies data as it reads it, or the optional read-time check, which only logs the mismatch after the bytes have been sent.
– The guarantees assume the storage stack honours flush requests. A device or virtualisation layer that acknowledges a flush it has not made can lose a purge, or leave a published name over wrong bytes, undetected.

View complete diagram.

Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify the fdatasync of the temporary before its rename, and how a failed sync is surfaced.* ✓ FDS-STORAGE.PURGE-01 via ⟨⟨*refine*⟩⟩ — *Specify the fsync of the shard directory before an invalidating purge is reported, and that a purge made to evict is not synchronised.* ✓ FDS-STORAGE.METAUPDATE-01 via ⟨⟨*refine*⟩⟩ — *Specify that a metadata update, written in place or withdrawing the record, is not synced.* ✓ SDS-LAYOUT.OBJECTFILE-01 via ⟨⟨*refine*⟩⟩ — *Add the body CRC-32C field and how it is computed and recorded, and specify the optional read-time check that logs a mismatch at the end of a whole-body read.*
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	HLR-PERSISTENCE.CRASHSAFE-01
Traces to	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.METAPERSIST-01
Refined by	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PURGE-01 SDS-LAYOUT.OBJECTFILE-01
Traced from	ADR-STORAGE.EXPIRE-01 ADR-STORAGE.EXECUTOR-01

[ADR-STORAGE.ATOMICPUBLISH-01]

Publish by same-directory rename ⟨⟨*design constraint*⟩⟩ `draft`

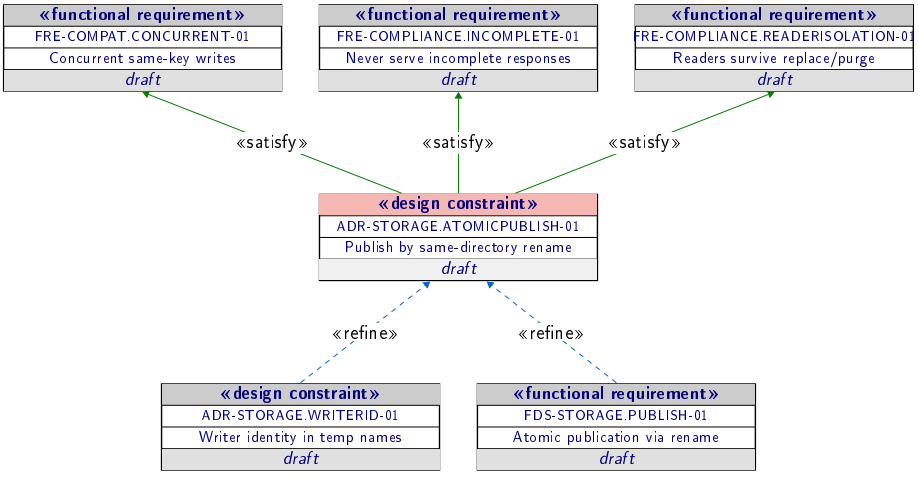


Figure 5.3: Publish by same-directory rename

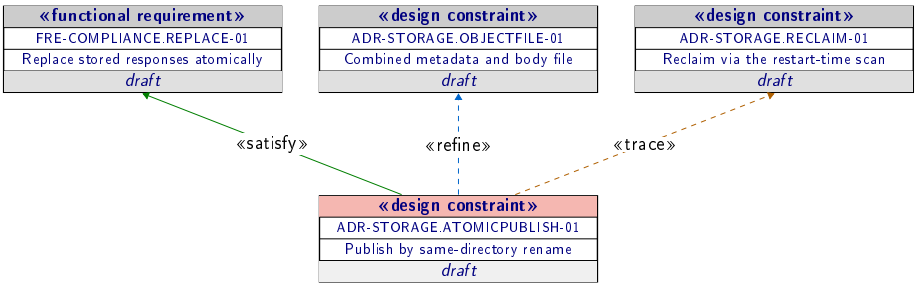


Figure 5.3: Publish by same-directory rename (cont.)

Context: A store's record grows over time (FRE-COMPAT.UNSIZED-01) in a single file (ADR-STORAGE.OBJECTFILE-01), yet every write must be durably published as a complete object or not at all, including when several same-key writers race (FRE-COMPAT.CONCURRENT-01), a store is abandoned half-way (FRE-COMPLIANCE.INCOMPLETE-01), or a new response replaces an old one that readers still hold (FRE-COMPLIANCE.REPLACE-01, FRE-COMPLIANCE.READERISOLATION-01). The question is how a completed record becomes visible by name — to a scan, a restart, or a peer process — in one indivisible step.

Decision: Write each store to its own temporary file in the *same directory* as the object's canonical name, and publish it with a single POSIX `rename` over that name once the store is complete. POSIX makes `rename` atomic between two paths on the same file system, replacing any existing target in the same step, while a descriptor already open on the old file keeps reading it. Keeping the temporary in the canonical name's directory makes that precondition structural rather than a deployment convention. The guarantee assumes POSIX `rename` semantics, matching the platform focus of `pingora` itself: Linux is its tier-1 environment, and Windows support is preliminary, community best-effort only.[5] Windows is out of scope for this guarantee.

Alternatives considered:

- *Write in place under the canonical name, with a "complete" flag in the header*: no temporary name and nothing to rename. Rejected: a peer process or a restart sees the partial record under the real name and must trust the flag; a replacement overwrites the old object in place while readers may still be streaming it; and a crash mid-write leaves a torn record where a complete one used to be.

- *Stage the temporary in a separate scratch directory*: simpler cleanup, since every leftover sits in one place. Rejected: `rename` is atomic only within one file system, and one attempted across devices fails outright (`EXDEV`); a copy-then-delete fallback would reopen exactly the incomplete-publication window this decision exists to close.

- *An unnamed temporary (`O_TMPFILE`) published with `linkat`*: the kernel discards an unlinked temporary when its last descriptor closes, so a crash leaves no leftovers at all.[6] Rejected for this version: it is Linux-specific and

[5] The `pingora` README states this directly: "Linux is our tier 1 environment and main focus.", and "Windows support is preliminary by community's best effort only."

[6] Linux manual page: `open(2)`, `O_TMPFILE`: "Anything written to the resulting file will be lost when the last file descriptor is closed, unless the file is given a name."

depends on file-system support, and a link never replaces an existing name,[7] so a replacement would still need a named intermediate and a rename. The leftovers it would avoid are already reclaimed by a scan the backend runs anyway (ADR-STORAGE.RECLAIM-01).

- *Windows* `MoveFileEx`: by default it refuses to overwrite an existing destination, and even with `MOVEFILE_REPLACE_EXISTING` the replace can fail with a sharing violation while another handle holds the destination open — a failure mode POSIX `rename` does not have. Out of scope with Windows.

Consequences:

+ Publication, replacement, and the concurrent-writer race all reduce to one atomic step: the last rename wins, and readers of the old object are undisturbed.
+ Atomicity follows from placement alone; no deployment setting can break it.
– A crash leaves named temporaries behind, which a later scan must reclaim (ADR-STORAGE.RECLAIM-01), and those temporaries share a directory with canonical names (SDS-LAYOUT.SHARDING-01).
– POSIX-only.

View complete diagram.

Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify the temporary's naming and placement, and the completion sequence that ends in the rename.*
Refines	ADR-STORAGE.OBJECTFILE-01
Satisfies	FRE-COMPAT.CONCURRENT-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.REPLACE-01 FRE-COMPLIANCE.READERISOLATION-01
Traces to	ADR-STORAGE.RECLAIM-01
Refined by	ADR-STORAGE.WRITERID-01 FDS-STORAGE.PUBLISH-01

[7] Linux manual page: `link(2)`: "If *newpath* exists, it will not be overwritten."

[ADR-STORAGE.WRITERID-01]

Writer identity in temp names ⟨⟨*design constraint*⟩⟩ `draft`

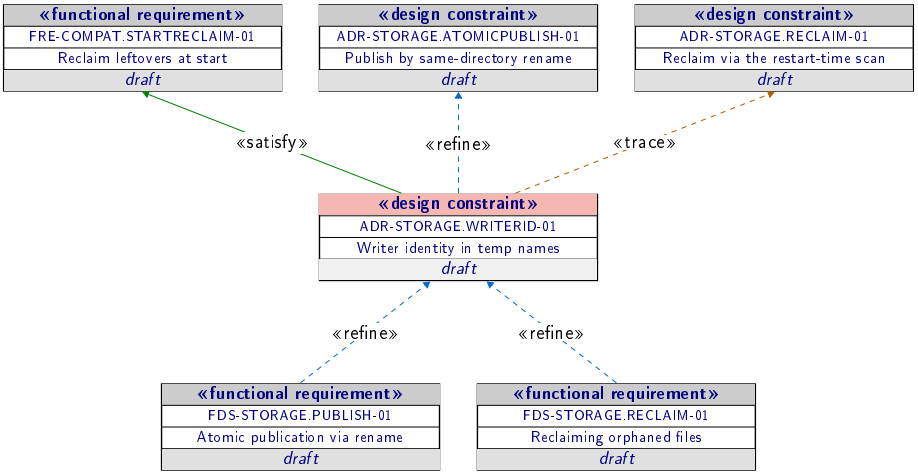


Figure 5.4: Writer identity in temp names

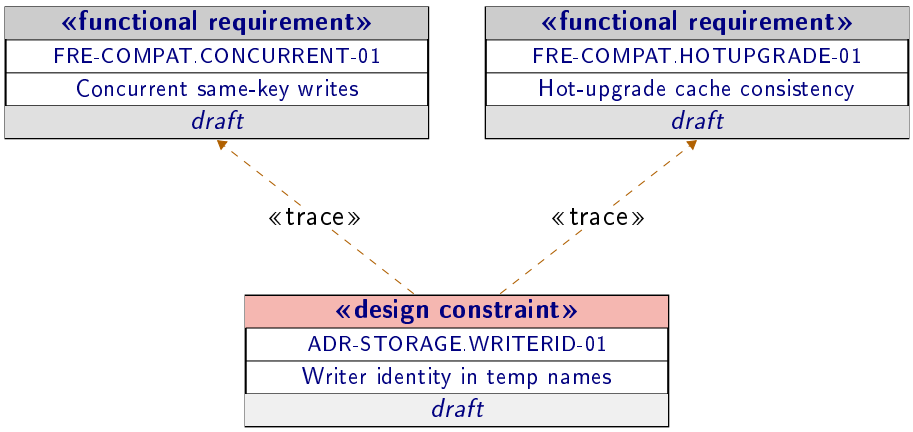


Figure 5.4: Writer identity in temp names (cont.)

Context: Each store writes a named temporary (ADR-STORAGE.ATOMICPUBLISH-01) that must never collide with another writer's, even across a restart, and a crash's leftover temporaries must be reclaimed by the first process start after their writer stopped (FRE-COMPAT.STARTRECLAIM-01) — without ever removing one a live writer, such as a hot-upgrade peer (FRE-COMPAT.HOTUPGRADE-01), still depends on. A reclaiming scan therefore has to tell, from a temporary's name alone, whether its writer could still be running. A process ID alone cannot: IDs are reused, and a cache process running as PID 1 in a container gets the same ID on every start.

Decision: Give each backend instance a *lease*: a file under the cache root, `writers/<pid>-<nonce>.lock`, locked with `flock` for as long as the instance

lives, where <nonce> is a random 64-bit value. Name each temporary after it: the object's hash, the lease's identity <pid>-<nonce>, the instance's own store counter, then the transient extension. A temporary's writer is considered possibly running exactly when its lease exists and its lock is held; the kernel releases a lock when its holder's descriptors close, a crash included.[8] The nonce makes the identity unique; the PID only tells an operator which process holds the lease. A lease is created under a staging name, locked, and only then renamed into place, so a scan never finds an unlocked lease under its final name while its writer lives; the restart-time scan removes the leases of writers that have stopped. A temporary is created exclusively, so a name collision is detected rather than silently reusing another writer's file. All processes sharing one cache directory must run on one host, with the directory on a local file system, so that each sees the others' locks; this is a deployment constraint the deliverable documents.

Alternatives considered:

- *PID only, tested with `kill(pid, 0)`*: the earliest design. Rejected: a reused PID makes a stopped writer look alive, and with PID 1 in containers every crash leftover looks alive forever and collides with the next process's temporaries once its counter restarts; `EPERM` also leaves liveness ambiguous.
- *The writer's process instance*: a token of the host's boot identifier, the PID, and the process's start time, which a scan compares with the process's `/proc` entry. The previous decision. Rejected: it depends on Linux's `/proc` and boot identifier, and every process sharing a directory must run in one PID namespace, which excludes containers on one host sharing a volume.
- *A lock on each temporary*: the writer locks its temporary for the life of the store and the scan tests that lock — no new on-disk structure. Rejected: a temporary exists unlocked between its creation and its lock, which only a minimum-age rule could cover, and every store pays for one more lock; a lease is locked once, before its instance creates any temporary.
- *A random name, reclaimed once older than a threshold*: rejected — it ties correctness to a timeout that a long, slow upstream body can always exceed.

Consequences:

+ A stopped writer's leftovers — a crash's, a killed process's, or an earlier boot's — are recognised at the first scan, and no reused PID can make a stopped writer look alive.
+ Two writers can never share a temporary name.
+ No dependency on `/proc`, a boot identifier, or a PID namespace: processes in separate PID namespaces on one host may share a cache directory.
– A new on-disk structure (HLR-SCOPE.V1-01): one empty lease per running instance, in a directory of its own, which the restart-time scan cleans up.
– A lease outlives its instance until the next restart-time scan.

[8] Linux manual page: `flock(2)`: a lock created by `flock` is associated with an open file description, and is released by an explicit unlock "or when all such file descriptors have been closed".

– The cache directory must be on a local file system. On NFS, Linux emulates `flock` with POSIX record locks, which never conflict within one process and are all released when the process closes any descriptor of the file: a scan would take its own process's lease for a stopped writer's.

– Temporary names grow longer.

View complete diagram.

Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify the temporary name format and exclusive creation, including what happens on a collision.* ✓ FDS-STORAGE.RECLAIM-01 via ⟨⟨*refine*⟩⟩ — *Specify the liveness test the scan applies to a temporary's writer (its lease's lock), the removal of stopped writers' leases, and the deployment constraint.*
Refines	ADR-STORAGE.ATOMICPUBLISH-01
Satisfies	FRE-COMPAT.STARTRECLAIM-01
Traces to	FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.HOTUPGRADE-01 ADR-STORAGE.RECLAIM-01
Refined by	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.RECLAIM-01

[FDS-STORAGE.PUBLISH-01]

Atomic publication via rename ⟨⟨*functional requirement*⟩⟩ `draft`

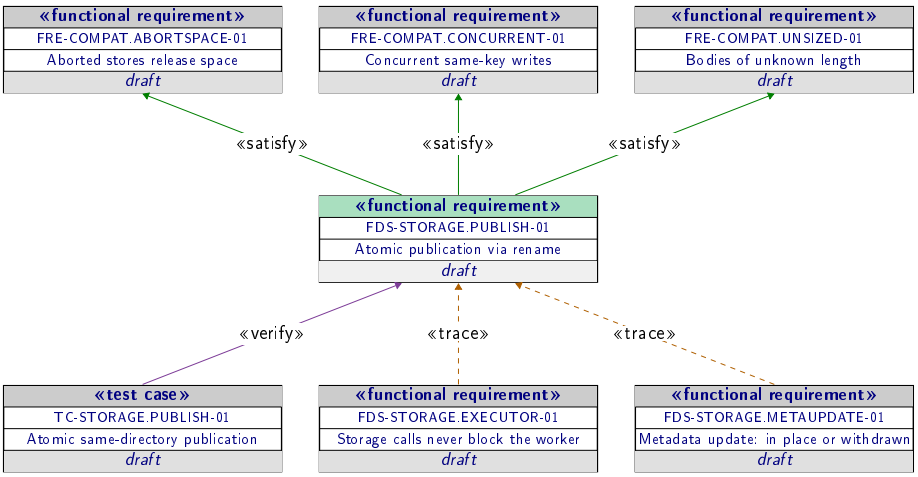


Figure 5.5: Atomic publication via rename

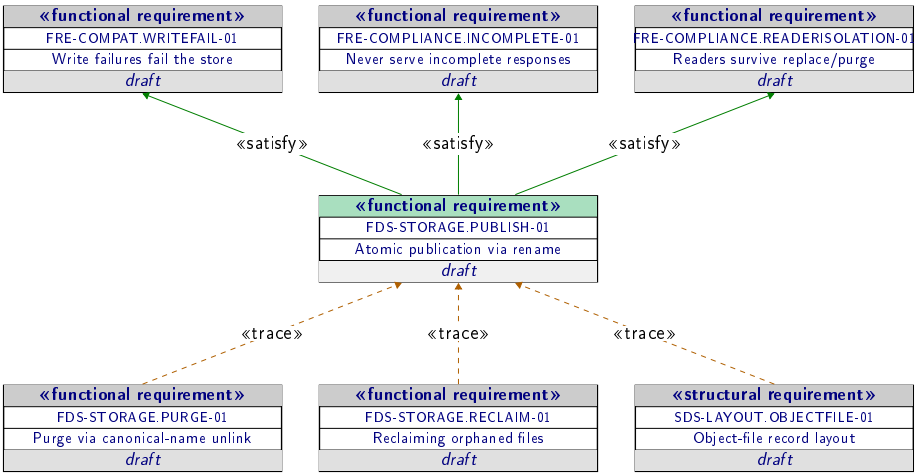


Figure 5.5: Atomic publication via rename (cont.)

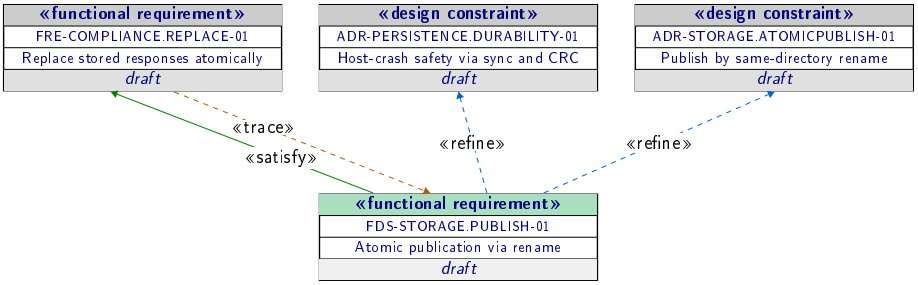


Figure 5.5: Atomic publication via rename (cont.)

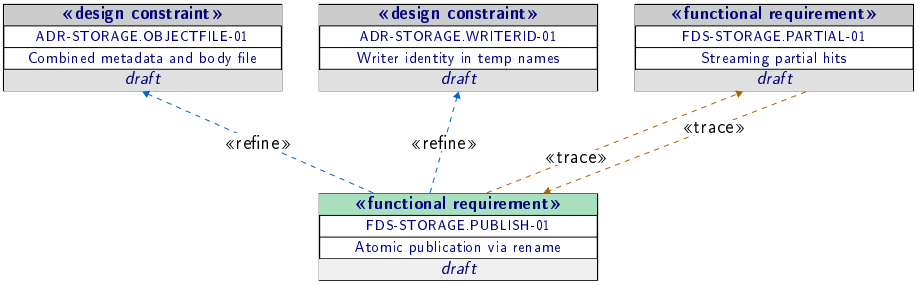


Figure 5.5: Atomic publication via rename (cont.)

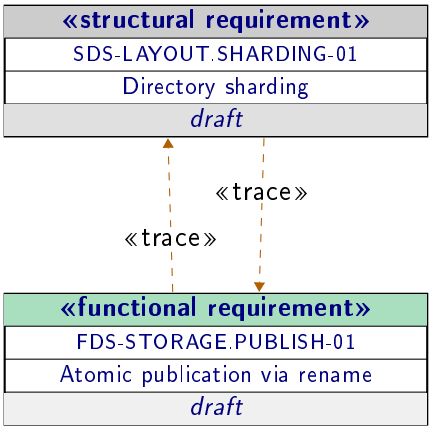


Figure 5.5: Atomic publication via rename (cont.)

The writer's lease. When the backend is constructed, it shall create its lease (ADR-STORAGE.WRITERID-01) under the cache root as `writers/<pid>-<nonce>.lock.new`, exclusively, take an exclusive `flock` on it, and rename it to `writers/<pid>-<nonce>.lock`, choosing a new nonce if the rename finds the file gone. It holds the lock, with the file open, for as long as it lives.

Opening a store. When a store begins, the backend shall create the store's *per-writer temporary* exclusively — failing rather than opening an existing file — in the same directory as the object's canonical name (SDS-LAYOUT.SHARDING-01), so the final rename is atomic by construction (ADR-STORAGE.ATOMICPUBLISH-01). The canonical name is the object's name as SDS-LAYOUT.SHARDING-01 defines it: the hex-encoded combined hash of the cache key,[9] or, for a key with a user tag, a hash of that hash and the tag. The temporary's name appends the writer identity ADR-STORAGE.WRITERID-01 decides, the instance's own write counter, and the transient extension: `<hash>.<pid>-<nonce>.<counter>.tmp`. Here `<pid>-<nonce>` names the lease of the backend instance writing it — its process ID in decimal and a random 64-bit value in sixteen hexadecimal digits — and `<counter>`, in decimal, is incremented for every store the instance opens. Each instance's identity keeps its names apart from every other instance's, and the counter keeps the instance's own writers of one key apart — the cache lock is optional, and its age timeout lets a new writer start while a previous one may still be writing (FRE-COMPAT.CONCURRENT-01) — just as the stock `MemCache` keys its in-flight writes by a per-writer identifier.[10] If exclusive creation reports that the name exists, the backend shall advance the counter and retry.

The backend shall then write the record header, key, and metadata region (SDS-LAYOUT.OBJECTFILE-01), and append each body chunk as `write_body` delivers it, folding it into a running CRC-32C unless the build leaves out the `body-crc` feature (ADR-PERSISTENCE.DURABILITY-01). While the temporary grows it is also the read source for streaming partial hits, as FDS-STORAGE.PARTIAL-01 specifies.

[9] `CacheHashKey::combined`.

[10] `MemCache::get_miss_handler` assigns each new write its own incrementing id and inserts it into the per-key map alongside any other writes already in flight.

Write-through. The backend shall write each chunk to the temporary before its `write_body` call returns, holding no body bytes beyond the chunk being written (FRE-COMPAT.UNSIZED-01). Only then does the valid length advance, so a partial-hit reader, which sees only bytes already in the file, is woken as soon as the chunk is there (FDS-STORAGE.PARTIAL-01).

Publishing. WHEN a store completes, `finish` shall, in order: patch `body_len` and `body_crc`, recompute `header_crc`, synchronise the temporary's data to stable storage (ADR-PERSISTENCE.DURABILITY-01), and rename the temporary over the canonical name, dropping the suffix, while holding the key's publication lock (FDS-STORAGE.METAUPDATE-01). The shard directory is not synchronised afterwards. The record the rename replaces is held open across it and closed last, once the lock is released: closing it frees its storage, which can take a long time for a large record. `finish` shall report the record size — the published file's full length — as the created size,[11] which `HttpCache` admits to the eviction manager (FRE-COMPAT.EVICTION-01). The rename is the object's *durable publication* — the point at which it becomes visible by name to a scan, a restart, or a peer process — not the first moment it can be read, and an incomplete store therefore never becomes visible under the canonical name (FRE-COMPLIANCE.INCOMPLETE-01). Once the backend is read-only (FDS-COMPAT.READONLY-01), `finish` instead removes the temporary rather than renaming it, while still reporting success, so the draining process's request completes normally. WHEN an invalidating purge has ended the store (FDS-STORAGE.PURGE-01), `finish` shall likewise remove the temporary rather than rename it, deciding under the publication lock the purge marked the store under, and shall report an error, so that `HttpCache` admits nothing to its eviction manager and lets the requests waiting on its cache lock retry.[12]

Failure and abort. IF a write, the synchronisation, or the rename fails during a store, THEN the `write_body` or `finish` call in progress shall return that error (FRE-COMPAT.WRITEFAIL-01), and the store is treated as aborted from then on. WHEN a store ends without being published — after such an error, after an upstream failure or cancellation in which `HttpCache` drops the miss handler, once the backend is read-only, or once a purge has ended the store — the backend shall unlink the temporary no later than when the miss handler is dropped (FRE-COMPAT.ABORTSPACE-01). Its own handle on the temporary stays open across the unlink, so the unlink removes only the name, and it closes that handle on the blocking pool (FDS-STORAGE.EXECUTOR-01): the close frees the storage, which can take a long time for a large temporary. A partial-hit reader still holding the temporary open is unaffected by the unlink; how its stream ends is specified by FDS-STORAGE.PARTIAL-01.

Isolation and replacement. Same-key writers stay isolated under this scheme: each has its own temporary, so no writer's bytes are ever interleaved with another's (FRE-COMPAT.CONCURRENT-01), and a temporary abandoned by a crash is identifiable by its extension, never served or promoted, and is reclaimed by the restart-time scan (FDS-STORAGE.RECLAIM-01, FRE-COMPAT.STARTRECLAIM-01). A replacement store (FRE-COMPLIANCE.REPLACE-01) is just another per-writer temporary racing to publish under the same canonical name; convergence on a single winner falls out of last-rename-wins, and a reader that already opened the outgoing object keeps reading it to completion undisturbed,

[11] `HandleMiss::finish` returns a `MissFinishType`.

[12] `HttpCache::finish_miss_handler` releases its cache lock as a transient error when `finish` fails, and returns before admitting the entry (L1105).

since a rename never invalidates a file descriptor already open on the old name. A peer process observes an object only at its durable publication.

Figure 5.6 drills into the `get_miss_handler` / `write_body` / `finish` calls that Figure 2.3 shows as opaque calls to `Storage`, detailing what this backend does inside each one.

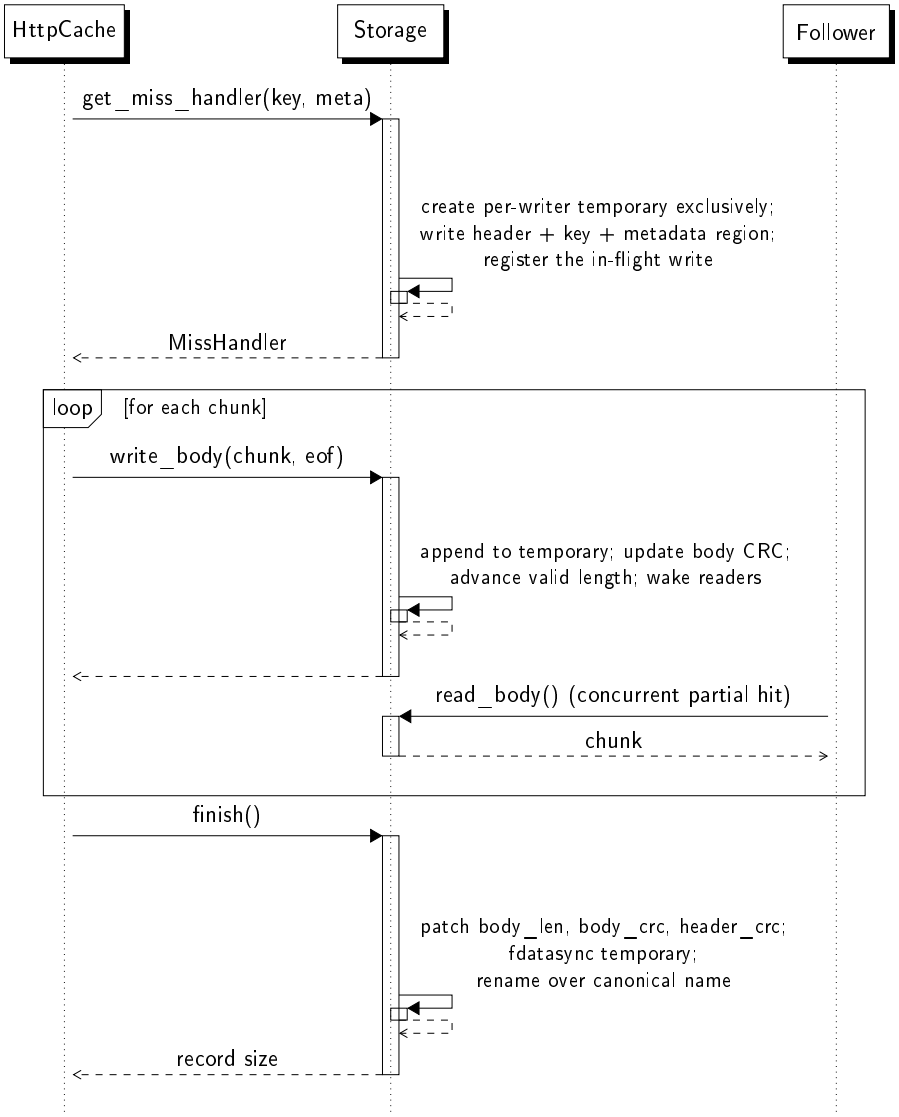


Figure 5.6: Publication sequence, drilling into the calls Figure 2.3 shows opaque: `get_miss_handler` creates the per-writer temporary, writes its header, key, and metadata region, and registers the write for partial hits (FDS-STORAGE.PARTIAL-01); each `write_body` call appends a chunk and advances the valid length a concurrent `Follower` streams from; and `finish` patches the header, synchronises the temporary, and renames it over the canonical name — the object's durable publication — returning the record size.

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.OBJECTFILE-01 ADR-STORAGE.ATOMICPUBLISH-01 ADR-STORAGE.WRITERID-01 ADR-PERSISTENCE.DURABILITY-01
Satisfies	FRE-COMPAT.CONCURRENT-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.REPLACE-01 FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPAT.WRITEFAIL-01 FRE-COMPAT.ABORTSPACE-01 FRE-COMPAT.UNSIZED-01
Traces to	FDS-STORAGE.PARTIAL-01 SDS-LAYOUT.SHARDING-01
Traced from	FDS-STORAGE.PARTIAL-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PURGE-01 FDS-STORAGE.EXECUTOR-01 FDS-STORAGE.RECLAIM-01 FRE-COMPLIANCE.REPLACE-01 SDS-LAYOUT.SHARDING-01 SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-STORAGE.PUBLISH-01
Satisfied by	IMP-STORAGE.PUBLISH-01 (durable-cache/src/miss.rs:55)

[FDS-STORAGE.PARTIAL-01]

Streaming partial hits ⟨⟨*functional requirement*⟩⟩ `draft`

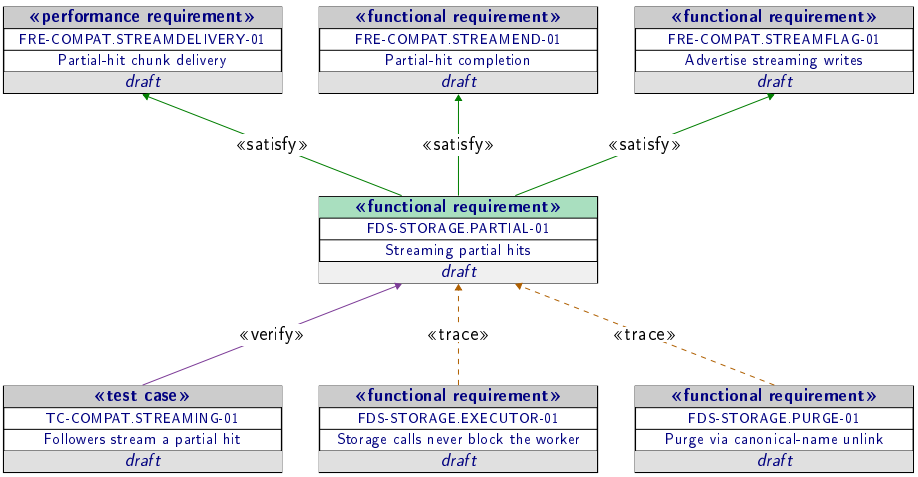


Figure 5.7: Streaming partial hits

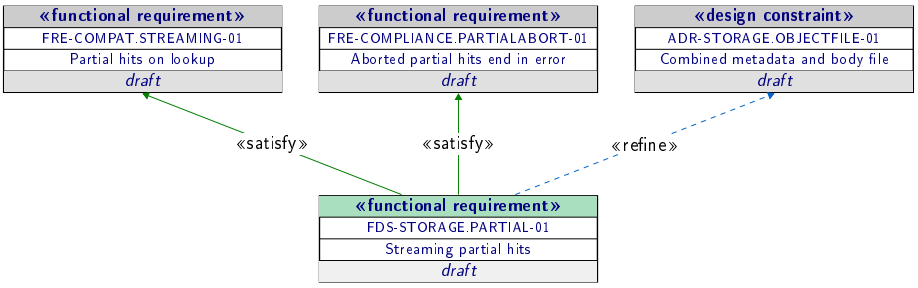


Figure 5.7: Streaming partial hits (cont.)

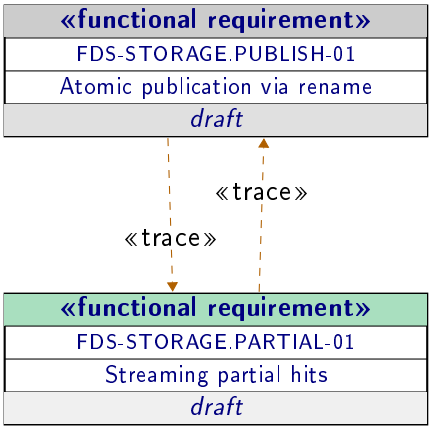


Figure 5.7: Streaming partial hits (cont.)

In-flight registry. The file system does not signal appends, so partial hits need coordination beside the file. The backend shall keep, in process memory, a registry of the stores in flight, keyed by the object's combined hash; for each store it holds the temporary's path, the object's `CacheMeta`, the valid body length written so far, a notification channel, and the order in which the store opened. A store enters the registry when `get_miss_handler` opens its temporary (FDS-STORAGE.PUBLISH-01), and leaves it when the store completes — published, or removed once the backend is read-only or an invalidating purge has ended it — or aborts. An invalidating purge marks the key's stores in flight as ended (FDS-STORAGE.PURGE-01); they stay in the registry until they complete or abort. The backend's `support_streaming_partial_write` shall return `true` (FRE-COMPAT.STREAMFLAG-01).

Lookup order. WHEN `lookup` is called, the backend shall first consult the registry. If one or more stores of the key that no invalidating purge has ended are in flight, it shall return a partial hit backed by the most recently opened of them; otherwise it shall look up the published record as SDS-LAYOUT.OBJECTFILE-01 specifies. The request writing a store reads its body back through a lookup that names the store by its streaming write tag;[13] that lookup shall be answered with

[13] `Storage::lookup_streaming_write`, which `HttpCache::set_miss_handler` calls with the tag of the store it has just opened.

the store it names, or with nothing, and with the store even once a purge has ended it.

Partial-hit readers. A partial hit's reader opens its own descriptor on the store's temporary and delivers the bytes up to the valid length, then waits on the store's notification channel. WHEN `write_body` appends a chunk, the backend shall advance the valid length and wake every reader of that store before `write_body` returns, so that each reader can deliver the chunk within the latency bound FRE-COMPAT.STREAMDELIVERY-01 sets. WHEN the store completes — whether it is published, or removed once the backend is read-only or a purge has ended it — each reader shall deliver the remaining bytes and end normally (FRE-COMPAT.STREAMEND-01); its descriptor stays valid across the rename and across the removal. WHEN the store aborts, each reader shall end its stream with an error, never a clean end of body (FRE-COMPLIANCE.PARTIALABORT-01). A partial hit does not support seeking and, like the stock backend's, counts no eviction access.[14]

Scope. The registry belongs to one process. A peer process — such as the other side of a hot upgrade — observes an object only at its durable publication (ADR-STORAGE.ATOMICPUBLISH-01).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.OBJECTFILE-01
Satisfies	FRE-COMPAT.STREAMFLAG-01 FRE-COMPAT.STREAMING-01 FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 FRE-COMPLIANCE.PARTIALABORT-01
Traces to	FDS-STORAGE.PUBLISH-01
Traced from	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 FDS-STORAGE.EXECUTOR-01
Verified by	TC-COMPAT.STREAMING-01
Satisfied by	IMP-STORAGE.PARTIAL-01 (durable-cache/src/lookup.rs:40)

[14] `should_count_access` returns `false` for the stock backend's partial reads.

[FDS-STORAGE.METAUPDATE-01]

Metadata update: in place or withdrawn ⟨⟨*functional requirement*⟩⟩ `draft`

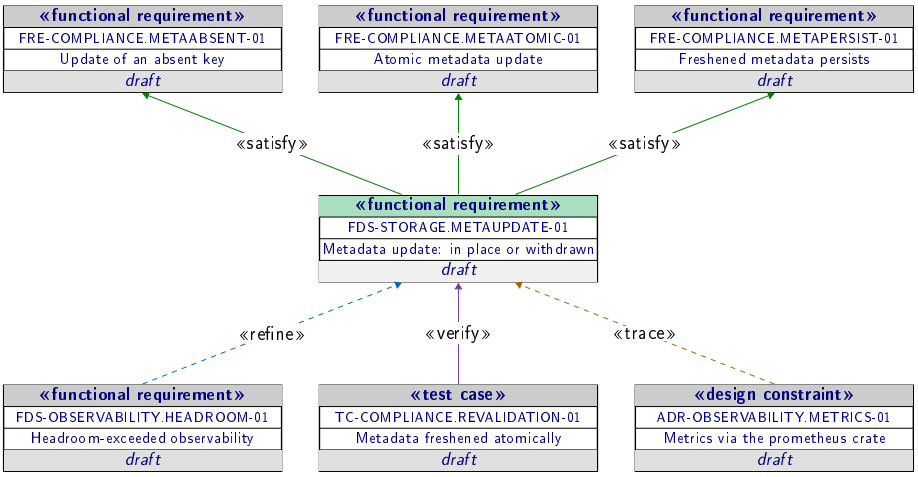


Figure 5.8: Metadata update: in place or withdrawn

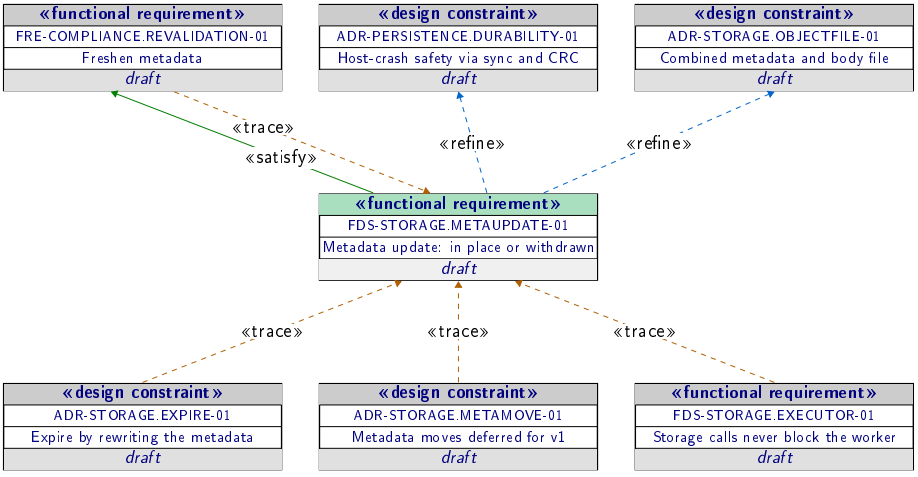


Figure 5.8: Metadata update: in place or withdrawn (cont.)

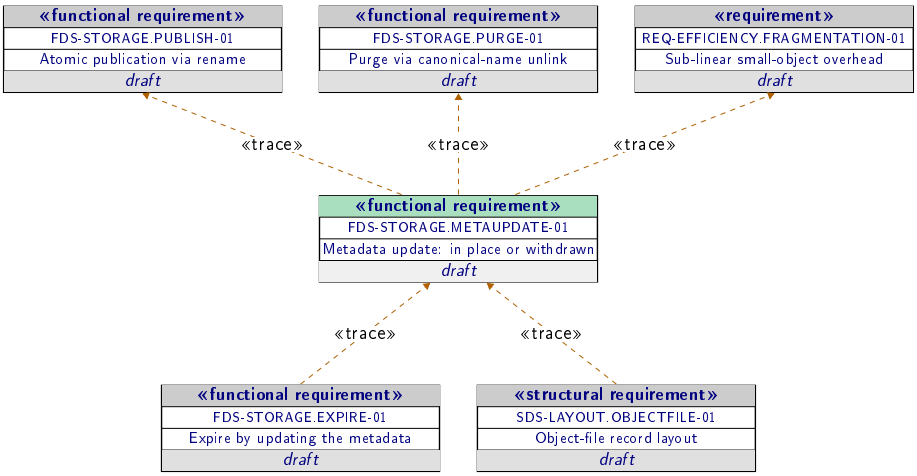


Figure 5.8: Metadata update: in place or withdrawn (cont.)

Each object's metadata sits in a region of its record, right after the key and ahead of the body (ADR-STORAGE.OBJECTFILE-01), whose capacity the record's header holds (SDS-LAYOUT.OBJECTFILE-01). Unlike the body, whose length is unknown until the stream ends (FRE-COMPAT.UNSIZED-01), the metadata's exact size is already known when the record is created — `CacheMeta` is fully serialised at that point — so the region reserves exactly that size plus a *configurable* amount of growth headroom beyond it — 512 bytes by default (SDS-LAYOUT.OBJECTFILE-01) — to anticipate a later metadata update. How much headroom to reserve is deployment-tunable, trading reserved-but-possibly-unused space against how often a later update fits in place; a deployment configured with zero headroom simply withdraws the record, as described below, on every metadata change that grows it, rather than needing a separate mechanism for it.

A metadata update (FRE-COMPLIANCE.REVALIDATION-01) whose new segments still fit the region shall be applied in place: the segments are rewritten within the region and the record's recorded lengths and checksums updated, without touching the body and without synchronising to stable storage (ADR-PERSISTENCE.DURABILITY-01).

The backend shall open and read the stored record, decide which of the two paths applies, and write the new metadata in place or withdraw the record, as described below, all while holding the key's *publication lock*. Every change that replaces or removes the key's record — a store's rename, a purge's or the reclaim scan's unlink — holds that lock too, so the update works on the record the key holds once the lock is taken. A record opened before the lock could lose its name in between, and the update would change a file no lookup reads any more, yet report success. Reading outside that lock would let one update observe another's region write half-done. An update that read a torn region could not distinguish a racing writer from a damaged record, and would report a perfectly healthy object as absent (FRE-COMPLIANCE.METAABSENT-01) rather than updating it. A *reader* re-reads instead, and takes the lock only for a last read of a record whose checksums kept failing, which then finds it whole (SDS-LAYOUT.OBJECTFILE-01).

Before either path, still holding the lock, the backend shall check that the record holds the response the update freshens. `pingora-cache` gives the new metadata

the creation time of the response its 304 validated,[15] so a stored response created at another time has replaced that one since the lookup that began the revalidation. `pingora-cache` permits that once a writer outlives its cache lock,[16] or when no cache lock is configured. The update shall then not be applied: `update_meta` reports `false`, and the newer response stands whole (FRE-COMPLIANCE.REPLACE-01). Applied, it would pair the validated response's header fields with the newer response's body, which RFC 9111 Section 4.3.4 rules out: a 304 whose strong validators no stored response shares must not update any.

When an update's new segments no longer fit the region's capacity — whether because the growth is unusually large or because no headroom was configured — the backend shall withdraw the record instead, so that the next request fetches the object again: it rewrites only the record's header, marked withdrawn (SDS-LAYOUT.OBJECTFILE-01), and reports the update not applied. Every lookup then misses, and Pingora fetches the object from the origin and stores it, through a store whose region is sized for the new metadata and whose rename replaces the file (FDS-STORAGE.PUBLISH-01). The response that triggered the update is still served, from the record its hit handler holds open. Until the store replaces it, the withdrawn file keeps its length, so the eviction manager, which the backend never calls (FRE-COMPAT.EVICTION-01), keeps counting the space it takes; an eviction or a purge removes it as it removes any record, and the restart procedure reclaims it (FDS-STORAGE.RECLAIM-01). The body is neither read nor moved. A withdrawal whose header write fails reports an error and leaves the record as it was. A withdrawal is not synchronised to stable storage either (ADR-PERSISTENCE.DURABILITY-01): after a host crash the record is withdrawn, as it was, or torn, which a reader rejects. Moving outgrown metadata after the body instead, which would spare the fetch, is deferred to a later version (ADR-STORAGE.METAMOVE-01).

`update_meta` reports `true` once the update is written in place, and so visible to lookups, and `false` when it is not applied: a newer response has replaced the one it validated, or the record was withdrawn. The publication lock is a per-key lock in process memory, which a store's final rename (FDS-STORAGE.PUBLISH-01), a purge's unlink (FDS-STORAGE.PURGE-01), the reclaim scan's unlink of a canonical record (FDS-STORAGE.RECLAIM-01), and the restart procedure's admission checks (FDS-EVICTION.STARTUP-01) also take, so none can land between a confirmation and the action that depends on it; a lookup takes it only for the last read described above. Across processes, only the hot-upgrade window before the outgoing backend becomes read-only (FDS-COMPAT.READONLY-01) lets two backends write, and the residual race there is accepted. Once the backend is read-only, `update_meta` fails with an error on either path (FRE-COMPAT.HOTUPGRADE-01). With a non-zero configured headroom, the common freshening after a 304 (`Date`- and `Age`-scale changes) is small enough to take the in-place path; growth beyond whatever headroom is configured always withdraws the record.

A stale hit's revalidation succeeds when the origin returns a 304 (FRE-COMPLIANCE.REVALIDATION-01). `HttpProxy`'s `revalidate_or_stale` merges the 304 into the cached header and calls into `HttpCache`'s metadata-update handling, which resolves to one of the two execution paths above.

[15] `HttpCache::revalidate_cache_meta` copies the stale response's `created` into the new metadata.

[16] `CacheLock::lock` hands a new write permit to a waiter once the holder's lock has passed its age timeout.

When a CacheLock is configured (it is an optional collaborator), HttpCache already holds the key's write lock by this point: cache_lookup takes it as soon as the hit is first found stale, well before the upstream request even starts, so that other requests for the same key queue behind this one leader. The lock is released only after the metadata-update call returns, whether it succeeds or fails.[17]

update_meta cannot manipulate the lock.[18] It shall make the update, or the withdrawal, visible to lookups before returning. HttpCache releases the lock as soon as this call returns (Figure 5.9), so every lookup after it, the followers queued behind the lock included, must find the update, or miss. On either path the change is complete when it is visible.

Lookups do not wait for the lock: a fresh hit never takes it, and a stale response may be served while a background subrequest revalidates it.[19] A concurrent lookup can therefore read the object while either path runs. The in-place path writes into the region a lookup may be reading, and a withdrawal into the header; a lookup that catches either mid-write is protected by the read-side validation and retry that SDS-LAYOUT.OBJECTFILE-01 specifies, so it observes the pre-update metadata, the post-update metadata, or a miss once the record is withdrawn, never a mixture. The body is untouched on either path.

A withdrawal costs one header write, whatever the body's size, and then one fetch of the whole object from the origin. A herd of revalidations withdraws the record once: the first update that outgrows the region withdraws it, and the next ones find no object stored, which they report as an update of an absent key does (FRE-COMPLIANCE.METAABSENT-01). The configurable headroom above keeps withdrawals rare, and they are observable, as FDS-OBSERVABILITY.HEADROOM-01 specifies.

Figure 5.9 shows the whole sequence, using the same HttpProxy, HttpCache, CacheLock, and Storage structure as the cache-miss sequence in the System Context chapter (Figure 2.3).

[17] HttpCache::cache_lookup takes the lock when storage returns nothing; the stale path does so in cache_found. HttpProxy::revalidate_or_stale merges the response (revalidate_merge_header) and calls revalidate_cache_meta; HttpCache::revalidate_cache_meta calls update_meta and releases the lock regardless of the result (L1230).

[18] Its signature is (key, meta, trace) -> Result<bool>: no reference to the lock is passed in.

[19] Under stale-while-revalidate, the lock writer in HttpProxy::proxy_cache hands its write lock to a background subrequest and serves the stale object, and a request finding the key locked serves stale too (L233) instead of waiting.

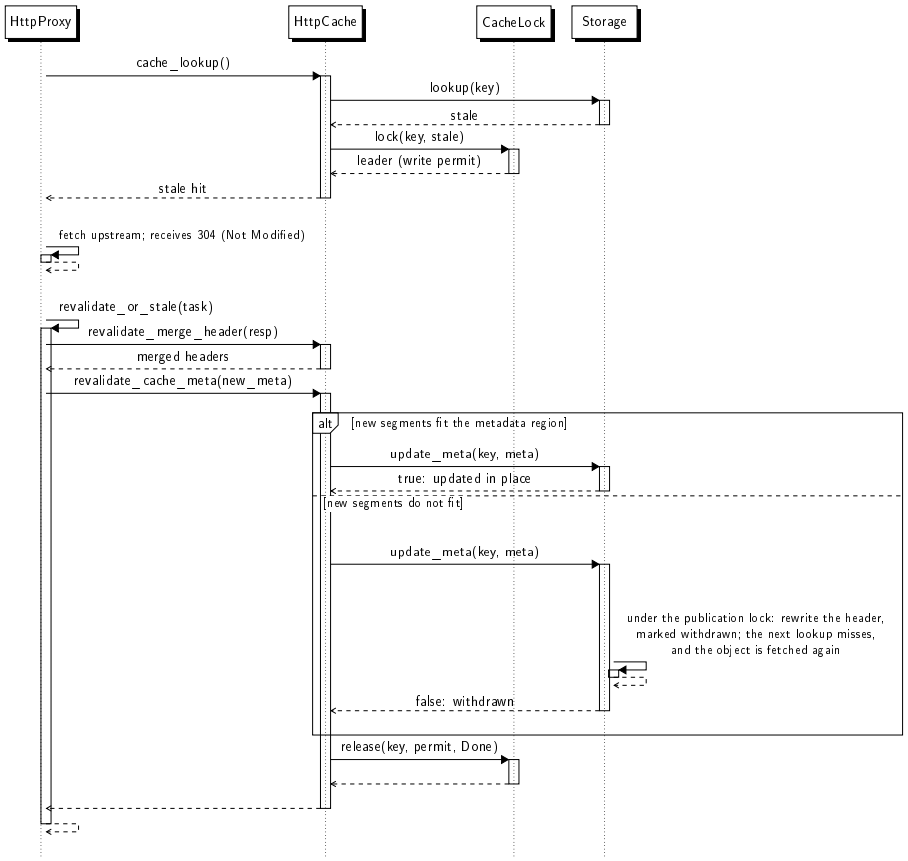


Figure 5.9: Metadata-update sequence, in the same `HttpProxy`, `HttpCache`, `CacheLock`, and `Storage` structure as Figure 2.3. The write lock is already held when `revalidate_or_stale` begins and is released only after `update_meta` returns, whichever path it took. On either path a header write, made last, commits the change, and the body stays where it is.

View complete diagram.

Required coverage	✓ TC-COMPLIANCE.REVALIDATION-01 via ⟨⟨*verify*⟩⟩ — *Extend this TC to also verify FDS-STORAGE.METAUPDATE-01: that an update written in place is visible to lookups when update_meta returns; that metadata outgrowing its region withdraws the record, which lookups then miss while a client already reading it gets its whole body; that the next store replaces a withdrawn record; that a withdrawal whose header write fails leaves the record as it was; that an update applies only to the response it validated; and that zero configured headroom withdraws the record on any growth.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.OBJECTFILE-01 ADR-PERSISTENCE.DURABILITY-01
Satisfies	FRE-COMPLIANCE.REVALIDATION-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.METAABSENT-01
Traces to	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 REQ-EFFICIENCY.FRAGMENTATION-01
Refined by	FDS-OBSERVABILITY.HEADROOM-01
Traced from	ADR-STORAGE.METAMOVE-01 ADR-OBSERVABILITY.METRICS-01 ADR-STORAGE.EXPIRE-01 FDS-STORAGE.EXPIRE-01 FDS-STORAGE.EXECUTOR-01 FRE-COMPLIANCE.REVALIDATION-01 SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01
Satisfied by	IMP-STORAGE.METAUPDATE-01 (durable-cache/src/metaupdate.rs:30)

[FDS-OBSERVABILITY.HEADROOM-01]

Headroom-exceeded observability ⟨⟨*functional requirement*⟩⟩ `draft`

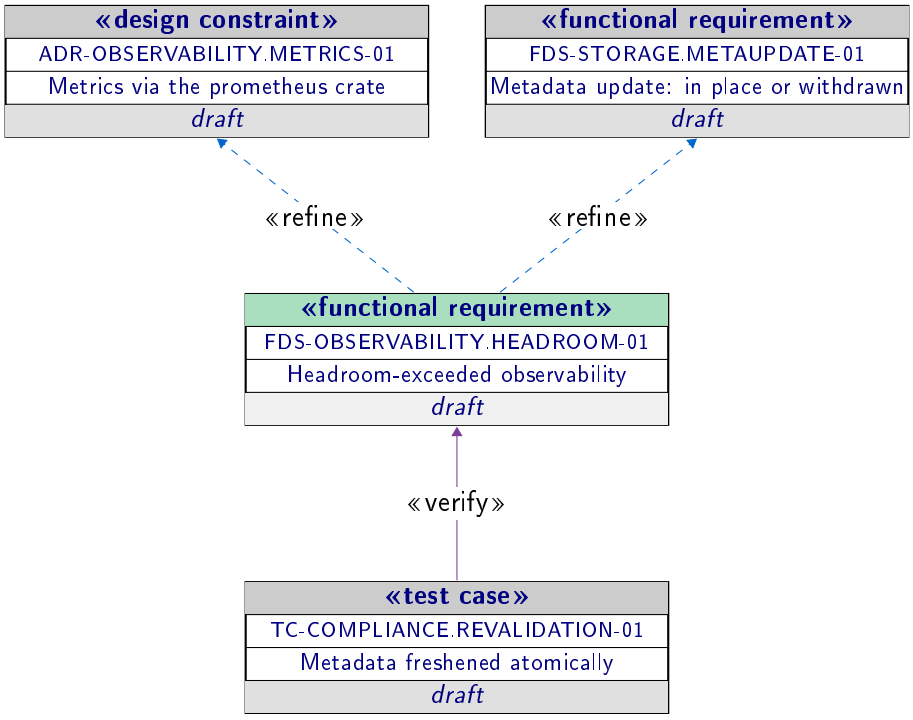


Figure 5.10: Headroom-exceeded observability

WHEN a metadata update finds that its new metadata outgrew the record's reserved headroom, so that the update withdraws the record (FDS-STORAGE.METAUPDATE-01), the backend shall log a warning and, when its `metrics` feature is enabled (it is off by default, ADR-OBSERVABILITY.METRICS-01), increase a counter:

- The warning, emitted through the `log` facade `pingora-cache` already depends on,[20] names the record by its path, whose file name is the key's hex-encoded combined hash[21] and states the headroom shortfall: how many more bytes of headroom would have kept the update in place.
- The counter `pingora_durable_meta_headroom_exceeded_total`, registered in the `prometheus` default registry (ADR-OBSERVABILITY.METRICS-01) without any per-key label, so how often it happens is visible without per-key cardinality, is increased by one. The backend registers it, at zero, when it is built: a counter that appeared only with its first event could not be told from one that is not served at all, and `rate()` and `increase()` would miss the event that created it.

[20] `log` is a dependency of `pingora-cache`; its own `warn!("revalidate_cache_meta failed...")` is precedent for logging a metadata-update outcome.

[21] `CacheHashKey::combined`: the only human-inspectable identity `Storage::update_meta`'s `CacheKey` exposes is a hash, not the original request URL.

Both are emitted once the record is withdrawn, so a withdrawal whose header write fails, and so did not happen, is neither logged nor counted. With the `metrics` feature disabled, the counter is not registered and the warning is the only record of each case. A count that keeps rising tells an operator to raise the configured headroom.

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-OBSERVABILITY.METRICS-01 FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01
Satisfied by	IMP-OBSERVABILITY.HEADROOM-01 (durable-cache/src/metaupdate.rs:31)

[ADR-STORAGE.METAMOVE-01]

Metadata moves deferred for v1 ⟨⟨*design constraint*⟩⟩ `draft`

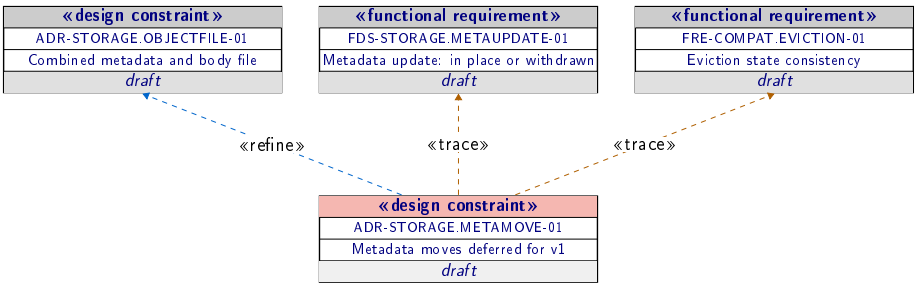


Figure 5.11: Metadata moves deferred for v1

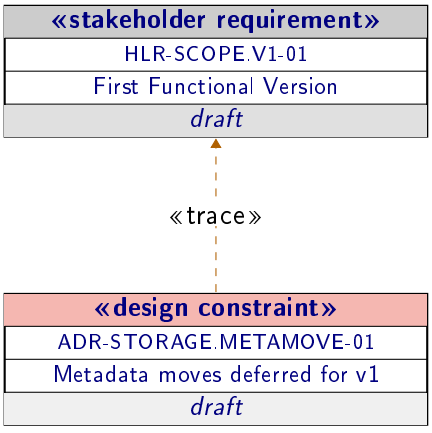


Figure 5.11: Metadata moves deferred for v1 (cont.)

Context: A metadata update whose new metadata outgrows the region reserved ahead of the body cannot be written in place (FDS-STORAGE.METAUPDATE-01). The backend then withdraws the record: the response at hand is still served, the next request fetches the object from the origin, and its store, sized for the new metadata, replaces the file. That costs a fetch of the whole body, and, should the origin fail before it, the stale copy that could have been served instead. The configurable headroom keeps it rare, and every case is observable (FDS-OBSERVABILITY.HEADROOM-01), so an operator can raise the headroom where it is not.

Decision: Do not move outgrown metadata for this version. The first alternative below would spare the fetch, but it changes the record format and the reader's rules, and it needs the backend to tell the eviction manager how much each move grows a record, which it cannot do today: the manager is the framework's, and the backend holds it only during its restart procedure (FRE-COMPAT.EVICTION-01). HLR-SCOPE.V1-01 prefers the simplest adequate design, so the fetch is accepted for this version, and moving the metadata is to be revisited in a later one.

Alternatives considered:

- *Move the metadata after the body*: write the new metadata into a slot appended after the end of the record, with headroom, then the header that points to it. The body is never read or moved, and the old metadata stays where it was, so a lookup that read the old header reads it whole. It needs two more header fields, saying where the metadata and the body lie; a reader that accepts a file longer than its record, since a move writes its slot before its header; and a report to the eviction manager of each move's growth, through `EvictionManager::increment_weight`, purging the victims it returns. Deferred: those changes, for a case the headroom makes rare.
- *Store the object again from its own body*, through a store of the key opened with the new metadata and a larger region. Rejected: it copies a whole body to change its metadata, and lookups follow the copy while it runs, so a purge, an expiry, or a newer store that lands meanwhile is not seen until it ends.
- *Withdraw the record* (chosen): as the context describes.

Consequences:

+ The record format keeps the metadata in one place, and a reader checks one exact length.
+ The eviction manager stays in step without the backend calling it: a withdrawn record keeps its file, and the space the manager counts for it, until a store replaces it.
– An update that outgrows the headroom costs a miss for the next request and a fetch of the whole object from the origin.
– Between the withdrawal and that fetch, no stale copy is left to serve should the origin fail.

View complete diagram.

Required coverage	*terminal* — Deferred: metadata moves are not implemented for this version, so nothing further refines, satisfies, or verifies this decision; revisit in a later version.
Refines	ADR-STORAGE.OBJECTFILE-01
Traces to	HLR-SCOPE.V1-01 FDS-STORAGE.METAUPDATE-01 FRE-COMPAT.EVICTION-01

[ADR-OBSERVABILITY.METRICS-01]

Metrics via the prometheus crate ⟨⟨*design constraint*⟩⟩ `draft`

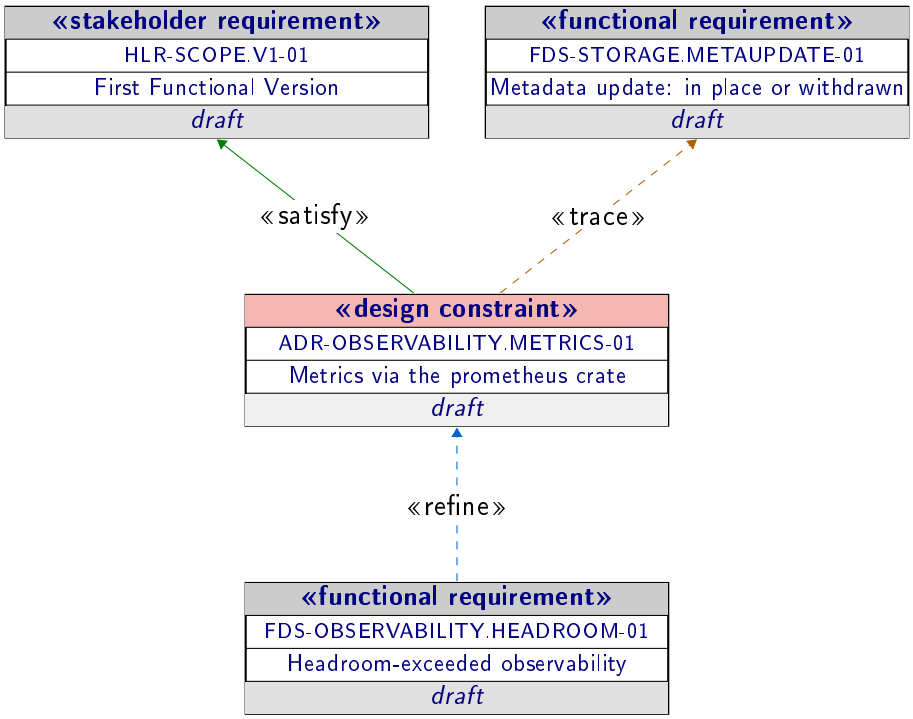


Figure 5.12: Metrics via the prometheus crate

Context: The backend accepts some costs rather than engineering them away but makes them observable — for example the fetch from the origin that a metadata update causes when it outgrows its reserved headroom (FDS-STORAGE.METAUPDATE-01). Aggregate counters need a metrics library, and the choice decides where an operator sees them. Pingora ships an HTTP application that exports the `prometheus` crate's default registry,[22] so registering there is what puts the backend's counters in front of an operator who already scrapes Pingora.

Where that application lives changed in Pingora 0.9.0, and with it the cost of this decision. Under 0.8.1 the exporter was the `prometheus_http_app` module of `pingora-core` itself, and `prometheus` was an unconditional dependency

[22] `PrometheusHttpApp` serves `prometheus::gather()`, the crate's default registry.

of pingora-core, so every build that used pingora-cache[23] already contained the crate and no feature of this backend's could remove it. In 0.9.0 the exporter moved to a separate pingora-prometheus crate and prometheus is no longer a pingora-core dependency at all. A build of this backend that registers metrics therefore now pulls prometheus in on its own account. The exporter crate re-exports prometheus for exactly this use,[24] and no Pingora crate has a feature flag for metrics that the backend's own choice could follow.

Decision: Register the backend's metrics with the prometheus crate's default registry, through the prometheus crate that pingora-prometheus re-exports, so they appear wherever an integrator exposes the Pingora metrics — but only when the backend's metrics Cargo feature is enabled, which it is not by default. The feature gates both the registration and the dependency: with it disabled the backend neither registers metrics nor links pingora-prometheus or prometheus at all. Going through the exporter's re-export makes the registry the exporter's by construction: the default registry is a static of the prometheus crate, and the backend cannot resolve to a different version of it than the exporter does. Enabling the feature is the same decision as running that exporter, since counters nothing exports are of no use. With the feature disabled, the backend registers no metrics and its warning logs remain the only record of the events the metrics would have counted.

Alternatives considered:

- *Register unconditionally, with no feature*: the simplest option. Rejected: it leaves an integrator no way to keep the backend's names out of a registry it manages itself, and — since Pingora 0.9.0 made prometheus optional — it would also force the dependency on builds that do not want it.
- *Gate the metrics behind a feature enabled by default*: this decision's previous revision. Rejected: every integrator who does not run the pingora-prometheus exporter would carry a dependency and counters nothing reads, while the events the counters measure are logged regardless.
- *Follow a Pingora feature flag*: not available — none of the Pingora 0.9.0 crates has a metrics feature, and Cargo cannot enable a feature because the integrator depends on some other crate.
- *Depend on prometheus directly*: the previous revision did. Rejected: it registers into the exporter's registry only while both crates happen to resolve to the same prometheus version, which nothing checks.
- *The metrics facade crate*, leaving the exporter to the integrator: rejected — metrics recorded through it do not reach the prometheus default registry that the Pingora exporter serves without an additional bridge, and it would replace one optional dependency with a different one plus that bridge.
- *Logs only, no counters*: rejected — a warning per event shows individual occurrences, but not the aggregate volume the design needs visible without per-key cardinality.

Consequences:

[23] pingora-core is an unconditional dependency of pingora-cache.

[24] "Re-export of the prometheus crate": "Use this re-export to ensure your metrics are registered in the same" registry the exporter serves; pub use prometheus.

+ With the feature on, metrics appear alongside the Pingora metrics, in the registry the `pingora-prometheus` exporter serves, by construction.

+ By default the backend links neither `pingora-prometheus` nor `prometheus`, and registers nothing.

– The counters exist only where an integrator enables the feature; elsewhere the costs they measure are visible in the warning logs alone.

– With the feature on, the deliverable depends on `pingora-prometheus`, which brings the exporter's code along with the re-export, even for an integrator who serves the registry another way.

– The metrics live in a process-global registry, so their names must not collide with those of the integrator or of Pingora.

– Two build configurations to keep compiling and tested.

View complete diagram.

Required coverage	✓ FDS-OBSERVABILITY.HEADROOM-01 via ⟨⟨*refine*⟩⟩ — *Specify each metric emitted when new metadata outgrows its record's headroom — name, type, unit, and when it changes — the accompanying warning log, and what remains when the metrics feature is disabled.*
Satisfies	HLR-SCOPE.V1-01
Traces to	FDS-STORAGE.METAUPDATE-01
Refined by	FDS-OBSERVABILITY.HEADROOM-01

[ADR-STORAGE.EXPIRE-01]

Expire by rewriting the metadata ⟨⟨*design constraint*⟩⟩ `draft`

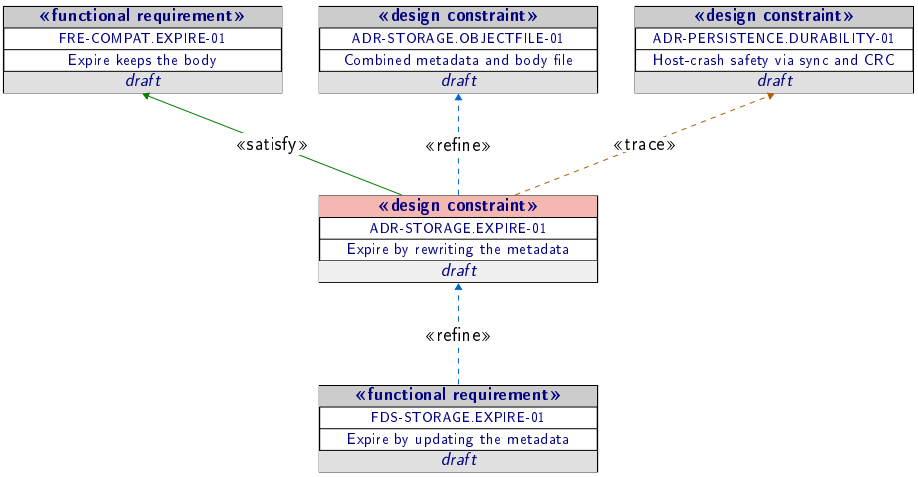


Figure 5.13: Expire by rewriting the metadata

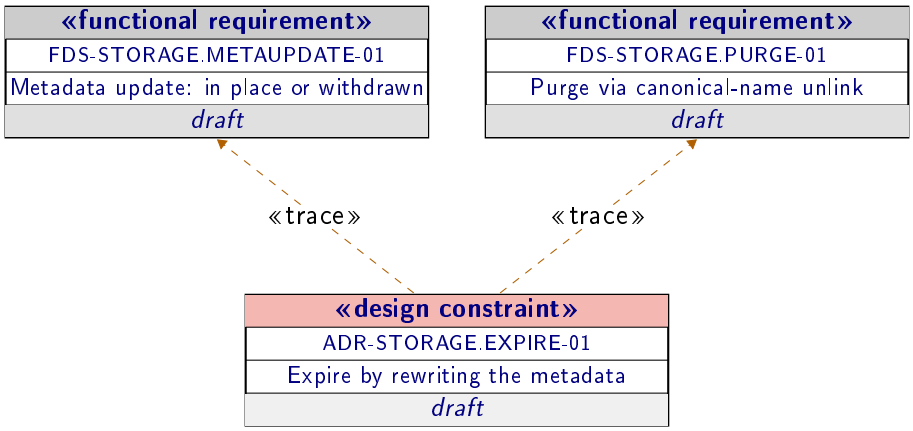


Figure 5.13: Expire by rewriting the metadata (cont.)

Context: FRE-COMPAT.EXPIRE-01 asks that an expired object stop being served as fresh while its body stays available for a conditional revalidation. `Storage::expire` supplies a default that deletes the object instead, which meets the first half and discards the second. Pingora is explicit that how an expiry is recorded is the backend's to choose, and that rewriting the stored freshness and applying an expiry at read time both satisfy the trait's contract.[25] The framework also supplies the transformation itself: `CacheMeta::expire_at`[26] produces the expired metadata from the stored metadata, without touching the body.

What makes the choice cheap here is that this backend already has the machinery. An object's metadata sits in a region with configured growth headroom, and FDS-STORAGE.METAUPDATE-01 already specifies how to replace it in place when the new segments fit the region, without reading or copying the body. An expiry's new segments differ from the stored ones only in the internal-metadata segment's freshness deadline — the header segment is untouched — yet they need not be the same length. `pingora-cache` serialises that segment with MessagePack, which writes an integer in as few bytes as its value allows,[27] and a deadline's nanoseconds are one such integer. A deadline taken from an `Expires` header falls on a whole second,[28] so its nanoseconds take one byte, where those of an arbitrary instant nearly always take five: expired at the current instant, such an object's metadata grows by four bytes.

Decision: Implement `expire` natively, by deriving the expired metadata with `expire_at` at the start of the current second and writing it in place, as FDS-STORAGE.METAUPDATE-01 writes an update that fits, rather than inheriting the deleting default. `expire_at` only ever lowers the deadline, so its seconds take no more bytes than before, and a deadline on a whole second has nanoseconds of zero, which take the fewest: the expired metadata is never longer than the stored metadata, so it fits the region whatever headroom is left. The stored body is left

[25] `Storage::expire`: "How that is recorded is up to storage: rewriting the stored freshness and applying it on read both satisfy the contract."

[26] `CacheMeta::expire_at`.

[27] One byte from 0 to 127, and two, three, five or nine above (MessagePack specification: int format family); the serialiser `pingora-cache` uses picks the shortest form that holds the value.

[28] `calculate_expires_header_time` returns the header's date, which has no fraction of a second.

untouched, so the 304 that follows reuses it.

Alternatives considered:

- *Inherit the default and delete*: the cheapest to specify and to build. Rejected — it turns every expiry into a full refetch of a body the cache is still holding, which is the cost an expiring purge exists to avoid; a durable backend, whose whole argument is that bodies are large and worth keeping (HLR-PERSISTENCE.DURABLE-01), is the last place to discard one.
- *Apply the expiry at read time from a side marker* — a tombstone file, or a flag in the record header that `lookup` consults: rejected — it adds a second on-disk thing per key for the restart scan to classify and reclaim (FDS-STORAGE.RECLAIM-01), and a header flag would have to be reconciled against the freshness the metadata segment itself carries, leaving two sources of truth for one question.
- *Expire at the current instant, and fall back when the expired metadata does not fit the region*: rejected — the fallback costs more than the four bytes it is for. Withdrawing the record, as a metadata update does (FDS-STORAGE.METAUPDATE-01), would report the entry still stored while no lookup could reuse its body; removing it, as the default does, reports a purge, which must then be as durable as one (FRE-COMPLIANCE.PURGEDURABLE-01).
- *Keep four bytes free in every region for the expiry*: a store adds them, and an update in place leaves them free. Rejected — three writers would keep an invariant that the start of the second keeps within the expiry alone.

Consequences:

+ An expiry costs one metadata region's worth of writes, never a body copy, and the retained body answers the revalidating 304.
+ No new on-disk structure, no new reclaim class, and no new lock: the operation reuses the publication lock and the in-place path FDS-STORAGE.METAUPDATE-01 already specifies.
– An expiry inherits that path's crash semantics: after a host crash an in-place expiry reads back as the old metadata, the new metadata, or a miss (ADR-PERSISTENCE.DURABILITY-01), so an acknowledged expiry can be lost. This is weaker than the purge-acknowledgement guarantee FRE-COMPLIANCE.PURGEDURABLE-01 makes, and is why an expiry is not a substitute for a purge when the object must provably stop being served.
+ An expiry never removes an object, whatever headroom its region has left.
– An expired object is stale from the start of the second in which it was expired, so its serve-stale windows, measured from its deadline, close up to a second earlier than they would from the instant itself. HTTP caching works in whole seconds,[29] and Pingora itself backdates the deadline of a response with no freshness lifetime by a second.[30]

[29] `CacheMeta::is_fresh`: "HTTP cache time resolution is second".
[30] `freshness_ttl_to_time`.

View complete diagram.

Required coverage	✓ FDS-STORAGE.EXPIRE-01 via ⟨⟨*refine*⟩⟩ — *Specify how a target is resolved, how the expired metadata is derived and written in place, why it always fits, which outcome each case reports, and how the operation behaves once the backend is read-only.*
Satisfies	FRE-COMPAT.EXPIRE-01
Refines	ADR-STORAGE.OBJECTFILE-01
Traces to	FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PURGE-01 ADR-PERSISTENCE.DURABILITY-01
Refined by	FDS-STORAGE.EXPIRE-01

[FDS-STORAGE.PURGE-01]

Purge via canonical-name unlink ⟨⟨*functional requirement*⟩⟩ `draft`

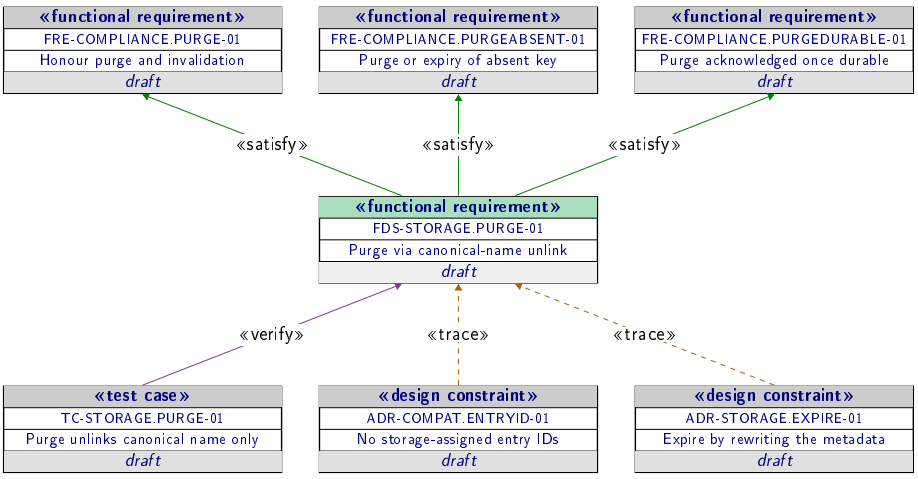


Figure 5.14: Purge via canonical-name unlink

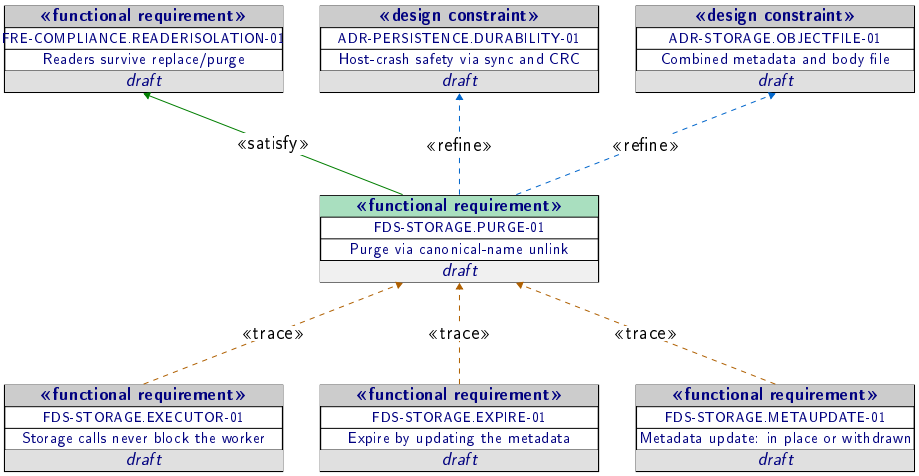


Figure 5.14: Purge via canonical-name unlink (cont.)

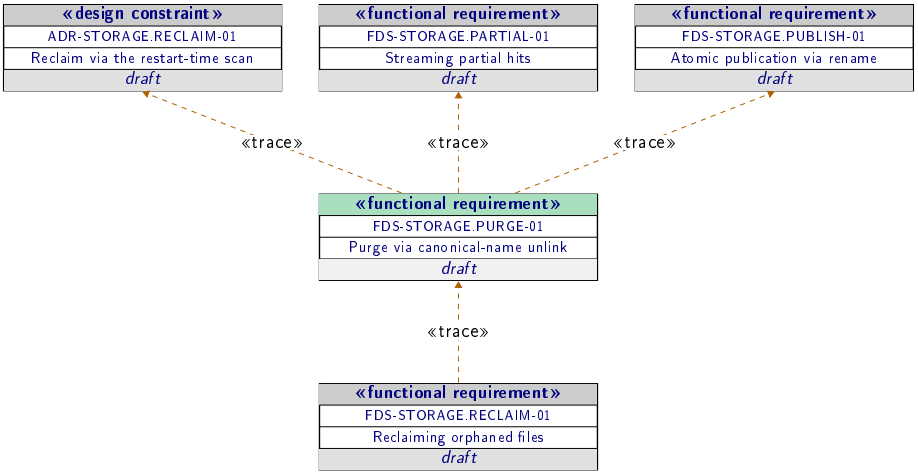


Figure 5.14: Purge via canonical-name unlink (cont.)

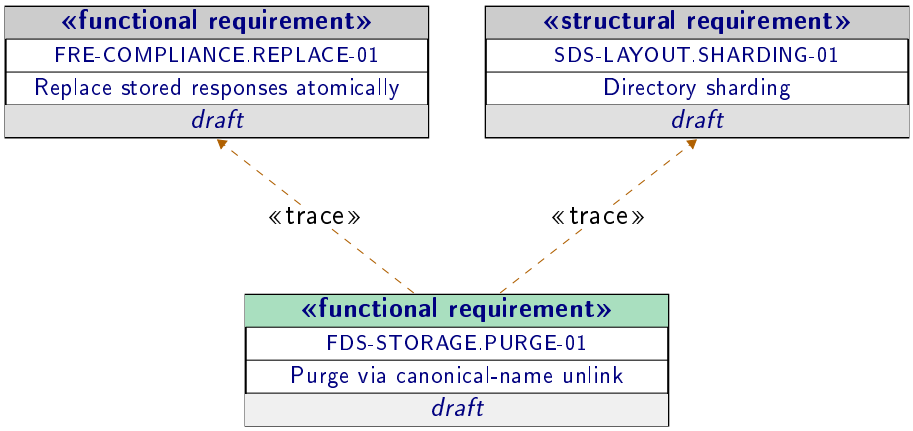


Figure 5.14: Purge via canonical-name unlink (cont.)

A purge is keyed directly off the presented `PurgeTarget`, with no lookup and no `CacheLock` (Figure 2.5). The target names the entry to remove in one of two ways: `Active`, carrying a `CompactCacheKey`, asks for whichever entry is currently active for that logical key — how `HttpCache` issues an invalidation — while `Exact`, carrying a `CacheEntryKey`, names one entry completely, which is how eviction victims arrive.[31] Because this backend assigns no entry identifiers (ADR-COMPAT.ENTRYID-01), the two carry the same information whenever the backend can act at all, and it shall resolve either — an `Active` target, or an `Exact` target whose `CacheEntryKey` is `KeyOnly` — to the canonical path of exactly that key, computed as SDS-LAYOUT.SHARDING-01 specifies, from the hex encoding of the key's combined hash, just as the stock backend does[32] — and shall never touch any other file. An `Exact` target carrying an `Identified` entry key names a generation this backend never created, so the backend shall report it as not found and change nothing, as the stock backend does.[33]

For the purge path `pingora-proxy` itself drives, that key is always a primary: `HttpCache::purge` panics unless the request is still in its initial `CacheKey` phase,[34] so the key it purges is `to_compact()` of the key set for the request, before any lookup could narrow it to a variant slot, and its combined hash is `key.primary` itself[35] (FRE-COMPLIANCE.PURGE-01). That is an observation about this one caller, not an assumption the backend relies on: the public `HttpCache::spawn_async_purge`[36] can present a varianced key, and resolving by the combined hash then purges exactly that variant rather than its primary.

A purge addressed to a primary never touches a `Vary` variant, which lives at an

[31] `PurgeTarget`; `purge_impl` passes `Active` and the eviction loop passes `Exact`.

[32] `MemCache::purge` resolves the entry by `target.key().combined()`.

[33] `MemCache::purge` returns `PurgeOutcome::NotFound` for an identified exact target, "because it cannot safely match identified entries".

[34] `HttpCache::purge` matches on `self.phase` and panics for any phase other than `CacheKey` (L1844); only `cache_vary_lookup` ever sets a variance on the key, and it runs only from the `CacheKey` or `Stale` phase of an ordinary lookup — a call sequence a purge request never enters (`HttpProxy::proxy_purge` calls `purge()` directly, with no preceding lookup).

[35] `CacheHashKey::combined_bin` returns `primary_bin()` unchanged when `variance_bin()` is `None`.

[36] `HttpCache::spawn_async_purge` accepts any enabled phase and purges `key.to_compact()`, which carries the variance once a `Vary` lookup has set one.

unrelated shard path the primary key cannot derive (FRE-COMPLIANCE.PURGE-01), nor a per-writer temporary for the same key, which is a distinct directory entry identified by its own writer-identity and `.tmp` suffix (FDS-STORAGE.PUBLISH-01) that a purge addressed to the bare canonical name never names or opens.

The purge operation itself is an `unlink` of that canonical path, performed while holding the key's publication lock (FDS-STORAGE.METAUPDATE-01). The backend answers with a `PurgeOutcome` rather than a boolean.[37] When nothing exists at that path, and no store of the key is in flight for an invalidation to end (below), the backend shall report `NotFound`, not an error — `unlink`'s own `ENOENT` is already exactly that signal, so idempotency and the absent-key case (FRE-COMPLIANCE.PURGEABSENT-01) need no separate bookkeeping. IF the `unlink` fails for any other reason, THEN the purge shall fail with an error, leaving the record in place. When the path existed, or an invalidation ended a store of the key in flight, the backend shall report `Purged`, carrying no entry identifier, since the backend assigns none.[38]

An invalidation shall be answered only once it is durable (FRE-COMPLIANCE.PURGEDURABLE-01): after releasing the lock, the backend shall synchronise the shard directory to stable storage (ADR-PERSISTENCE.DURABILITY-01), and only then report the outcome. It does so also when the path did not exist: a purge made to evict may have unlinked the record just before without synchronising, and a crash undoing that would otherwise bring the object back after the invalidation reported it absent. A shard directory that does not exist has nothing to synchronise. IF that synchronisation fails, THEN the purge shall fail with an error, since its answer might not survive a host crash. A purge made to evict (`PurgeType::Eviction`), as `HttpCache` purges the victims its eviction manager returns and as the restart scan purges its own (FDS-EVICTION.STARTUP-01), shall not synchronise: an eviction a host crash undoes brings back a valid object, which the restart scan admits, and evicts again if the cache is over capacity. The record the `unlink` removes is held open across it and closed last, after any synchronisation and outside the lock: closing it frees its storage, which can take a long time for a large record. Once the backend is read-only (FDS-COMPAT.READONLY-01), every purge shall fail with an error without touching the file (FRE-COMPAT.HOTUPGRADE-01).

Racing writers. A same-key store in flight publishes by renaming its own per-writer temporary over the canonical path at completion, under the key's publication lock (FDS-STORAGE.PUBLISH-01). An invalidation shall end every store of the key in flight, under the same lock (FRE-COMPLIANCE.PURGE-01): it marks them ended in the in-flight registry, so that later lookups pass over them (FDS-STORAGE.PARTIAL-01), and each one's `finish`, deciding under the lock, removes its temporary instead of renaming it and reports an error (FDS-STORAGE.PUBLISH-01). A store that published before the invalidation took the lock has its record removed by the `unlink`, so no store that began before an invalidation lands after it; a store that begins after it is not affected. When the key holds no record but a store in flight, the invalidation ends the store and reports `Purged(None)`, as the stock backend reports a purge that removed only a store in flight.[39] The purge never names or opens a writer's temporary: the writer removes it, as it does after any store that

[37] `PurgeOutcome`.

[38] Returning `Purged(None)` is what makes `eviction.remove` address the same key-only entry the backend admitted; an identifier the backend never issued would leave the entry tracked.

[39] `MemCache::purge` reports `Purged` when it removed a store in flight, a stored response, or both.

ends unpublished. The stock backend's purge instead removes the store's entry from its in-flight map, and the writer then panics on the `unwrap()`s[40] in its own `finish` — tolerable only because that struct is explicitly documented as "for testing only, not for production use."[41] Here the writer's `finish` completes, and reports the purge.

A purge made to evict leaves stores in flight alone: a racing store's `rename` and the eviction's `unlink`, each made under the lock, land in either order, and whichever is last determines the outcome — the last-writer-wins convergence FRE-COMPLIANCE.REPLACE-01 requires among racing stores. A metadata update or expiry is no such racer: it opens the record under the same publication lock, so it acts on the record before the purge removes it, or finds the key absent after (FDS-STORAGE.METAUPDATE-01).

Safe under a racing reader. A reader already streaming the canonical file holds it open from before the purge arrives (Figure 6.6); POSIX `unlink` removes only the directory entry, and "if the name was the last link to a file but any processes still have the file open, the file will remain in existence until the last file descriptor referring to it is closed."[42] The reader therefore runs to the end of the body it already opened undisturbed, and the space it reads from is not reused for anything else while that descriptor remains open, satisfying FRE-COMPLIANCE.READERISOLATION-01 without any reference-counting of the backend's own: the kernel already does it.

Figure 5.15 drills into the `purge(target, Invalidation)` call Figure 2.5 shows opaque to `Storage`. Variant records left behind once their primary is gone are outside this operation's own reach (FRE-COMPLIANCE.PURGE-01); eviction removes them like any other record (ADR-STORAGE.RECLAIM-01).

[40] `MemMissHandler::finish` unwraps its own lookup in `self.temp` at L273 and L275; a concurrent `purge` racing ahead of it removes that same entry (L403).

[41] `MemCache`'s own doc comment: "For testing only, not for production use."

[42] Linux manual page: `unlink(2)`.

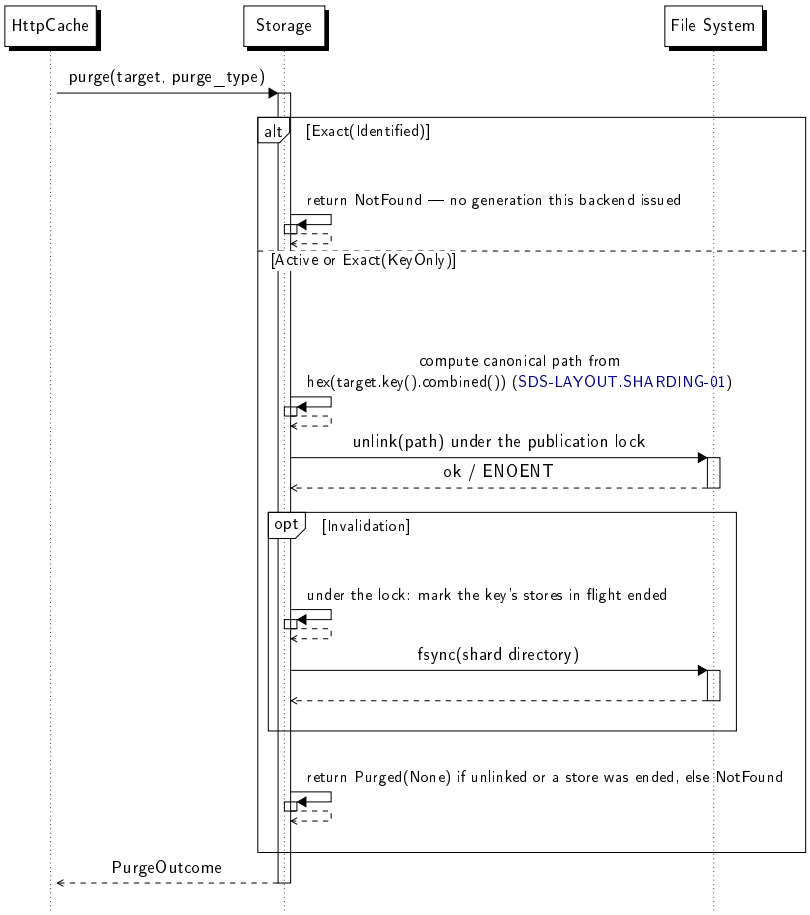


Figure 5.15: Purge, drilling into the call Figure 2.5 shows opaque to `Storage`. A target naming a generation identifier is rejected unseen, since this backend issues none (ADR-COMPAT.ENTRYID-01); otherwise the canonical path is computed directly from the target key's combined hash, never scanned or looked up, and a single `unlink` either removes it or reports `ENOENT`, which the backend folds into `NotFound`. An invalidation also marks the key's stores in flight ended, under the lock, and then synchronises the shard directory before it reports; a purge made to evict does neither — the *opt* fragment's other branch. No per-writer temporary, and no file of any other key, is named or touched by this call.

View complete diagram.

Required coverage	✓ TC-STORAGE.PURGE-01 via ⟨⟨*verify*⟩⟩ — *Verify that purge unlinks only the canonical path derived from the presented target key's combined hash (a primary or a variant), for an Active target and a key-only Exact target alike; that an identified Exact target is reported not-found with nothing unlinked; that purge is idempotent and reports not-found for an absent key; that an invalidation ends a same-key store in flight, which then publishes nothing while the request writing it and a reader already streaming it still read its body, and reports a key holding only such a store purged, while a purge made to evict leaves the store to land; that it never names a writer's temporary nor disturbs a concurrent reader's already-open descriptor; that an invalidation synchronises the directory before it reports, also when it found nothing, while a purge made to evict does not; that a removal is reported carrying no entry identifier; that a purge whose unlink fails reports an error and leaves the record in place; and that it fails once the backend is read-only.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.OBJECTFILE-01 ADR-PERSISTENCE.DURABILITY-01
Satisfies	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.READERISOLATION-01
Traces to	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PARTIAL-01 FRE-COMPLIANCE.REPLACE-01 SDS-LAYOUT.SHARDING-01 ADR-STORAGE.RECLAIM-01
Traced from	FDS-STORAGE.METAUPDATE-01 ADR-STORAGE.EXPIRE-01 FDS-STORAGE.EXPIRE-01 FDS-STORAGE.EXECUTOR-01 FDS-STORAGE.RECLAIM-01 ADR-COMPAT.ENTRYID-01
Verified by	TC-STORAGE.PURGE-01
Satisfied by	IMP-STORAGE.PURGE-01 (durable-cache/src/purge.rs:39)

[FDS-STORAGE.EXPIRE-01]

Expire by updating the metadata ⟨⟨*functional requirement*⟩⟩ `draft`

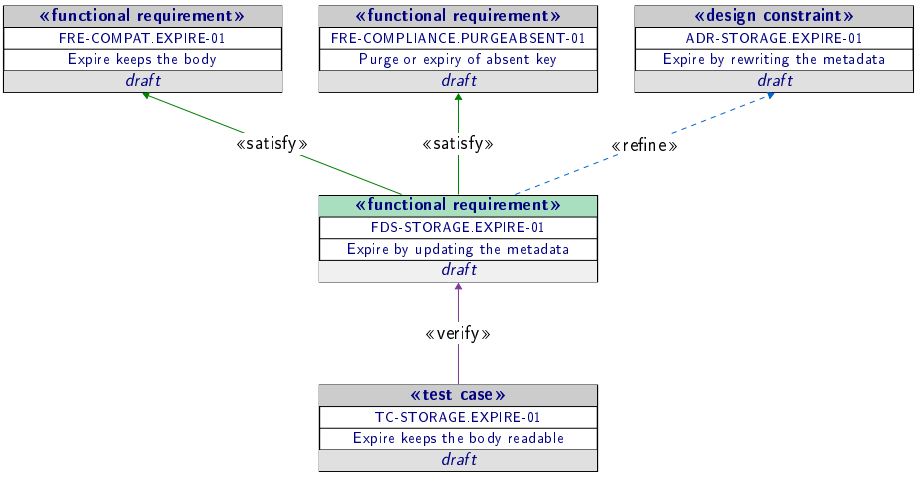


Figure 5.16: Expire by updating the metadata

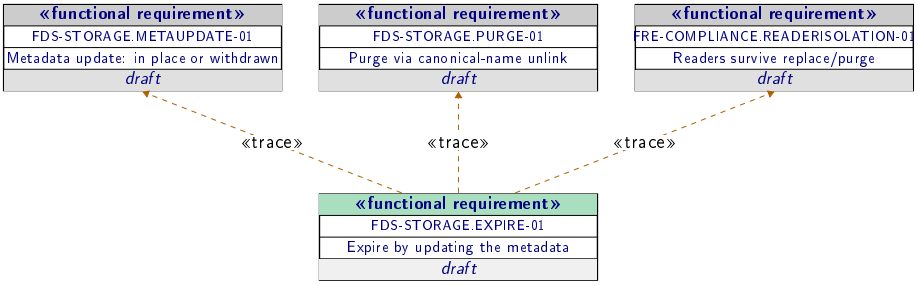


Figure 5.16: Expire by updating the metadata (cont.)

An expiry is keyed off a `PurgeTarget` exactly as a purge is, and the backend shall resolve it exactly as FDS-STORAGE.PURGE-01 resolves one: an `Active` target, or an `Exact` target whose `CacheEntryKey` is `KeyOnly`, to the canonical path of that key; an `Exact` target carrying an `Identified` entry key to `NotFound`, since this backend issues no entry identifiers (ADR-COMPAT.ENTRYID-01). No other file is named or opened.

The operation. Holding the key's publication lock (FDS-STORAGE.METAUPDATE-01), the backend shall open the record, read its stored metadata, derive the expired metadata from it by applying `CacheMeta::expire_at` at the start of the current second, and write the result in place, as a metadata update that fits is written (ADR-STORAGE.EXPIRE-01), without reading or writing the body. The result always fits the record's region, whatever headroom it has left: `expire_at` changes only the freshness deadline in the internal-metadata segment, and only ever lowers it, so the deadline's seconds take no more bytes than before and its nanoseconds, zero, take the fewest; and the backend writes the header

segment back as stored, byte for byte. The expired metadata is therefore never longer than the stored metadata.

Taking the lock for the whole read-derive-write sequence is what keeps two concurrent expiries, or an expiry and a revalidation's `update_meta`, from clobbering one another: neither can read metadata the other is between reading and rewriting. This is why the backend derives the expired metadata from what is on disk rather than from anything the caller holds — the caller of an expiry holds no `CacheMeta` at all.

Outcomes. The backend shall report `PurgeOutcome::Expired` once the expired metadata is written, and so visible to lookups; `NotFound` when no record exists at the canonical path — the absent-key case (FRE-COMPLIANCE.PURGEABSENT-01), which is not an error — and an error when the record exists but cannot be read, decoded, or written. An expiry never reports `Purged`. `Expired` tells `HttpCache` the entry is still stored, so it stays in the eviction manager's accounting, which is correct: the record still occupies its full length on disk.[43] A record that fails the read-time validation SDS-LAYOUT.OBJECTFILE-01 specifies — damaged, truncated, of another format version, or withdrawn — is reported `NotFound`, matching what a lookup of that same record would report, and is left as it is for FDS-STORAGE.RECLAIM-01.

Races. The record is opened under the lock, so a store or purge that lands before the expiry takes the lock is seen: the expiry acts on the record the key then holds, or reports `NotFound`. A same-key store already in flight is unaffected: it publishes by renaming over the canonical name when it completes, replacing the expired record with a fresh object, which is the behaviour FRE-COMPAT.EXPIRE-01 states; an invalidating purge, unlike an expiry, ends such a store (FDS-STORAGE.PURGE-01). A reader already streaming the record keeps its open descriptor and is undisturbed, since the expiry does not touch the body (FRE-COMPLIANCE.READERISOLATION-01).

Once the backend is read-only (FDS-COMPAT.READONLY-01), every expiry shall fail with an error without changing anything on disk (FRE-COMPAT.HOTUPGRADE-01).

[43] `purge_impl` makes no eviction call for an `Expired` outcome.

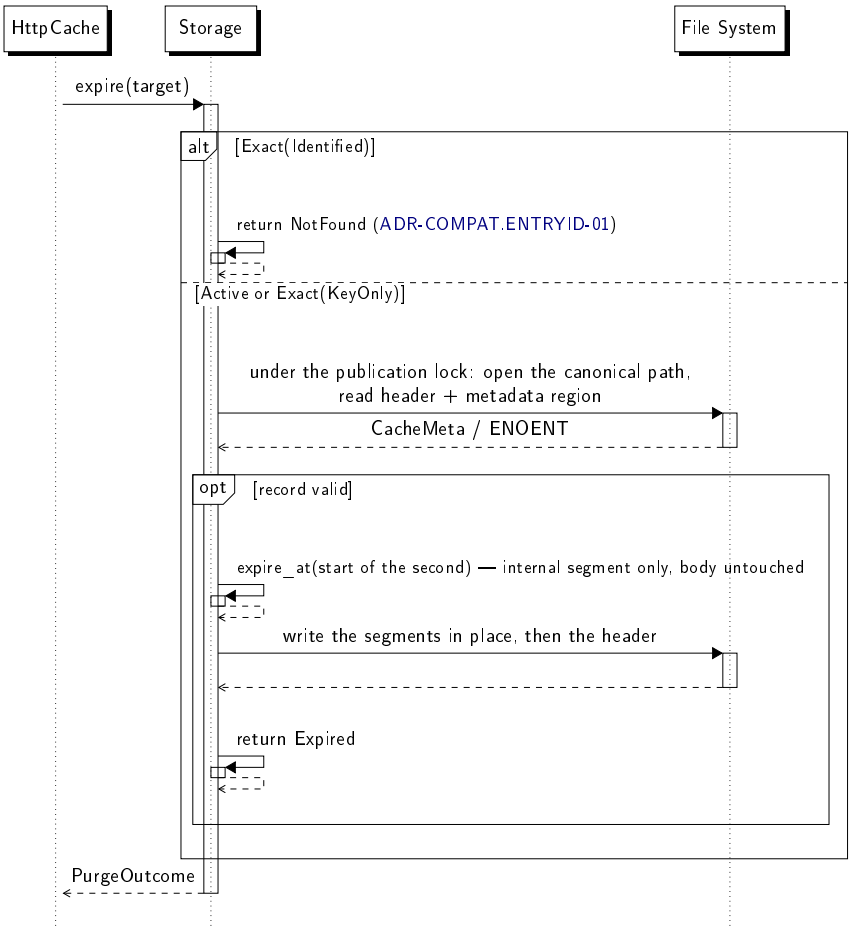


Figure 5.17: Expiry, the second removal operation beside Figure 5.15's purge. The target is resolved the same way, but instead of unlinking the canonical name the backend derives expired metadata from what that record already stores and writes it back in place, as FDS-STORAGE.METAUPDATE-01 writes an update that fits. The body is never read or copied, and the record survives, so the 304 that follows the revalidation serves the very bytes the cache already held. The read, the derivation, and the write are one critical section under the key's publication lock, so a concurrent expiry or metadata update cannot clobber it. The *opt* fragment's other branch — no record at the path, or one that fails validation — reports `NotFound`.

View complete diagram.

Required coverage	✓ TC-STORAGE.EXPIRE-01 via ⟨⟨*verify*⟩⟩ — *Verify that an expiry stops the object being served as a fresh hit while leaving its body byte-identical and readable; that the stored serve-stale windows are not closed by it; that it is idempotent; that it reports not-found for an absent key, for an identified Exact target, and for a record that fails the read-time validation, a withdrawn one included, which it leaves untouched; that it writes its metadata in place, with headroom configured and without, also for a deadline on a whole second; that a concurrent store still replaces the expired record; and that it fails once the backend is read-only.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.EXPIRE-01
Satisfies	FRE-COMPAT.EXPIRE-01 FRE-COMPLIANCE.PURGEABSENT-01
Traces to	FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PURGE-01 FRE-COMPLIANCE.READERISOLATION-01
Verified by	TC-STORAGE.EXPIRE-01
Satisfied by	IMP-STORAGE.EXPIRE-01 (durable-cache/src/expire.rs:33)

[ADR-STORAGE.EXECUTOR-01]

File I/O on the blocking pool ⟨⟨*design constraint*⟩⟩ `draft`

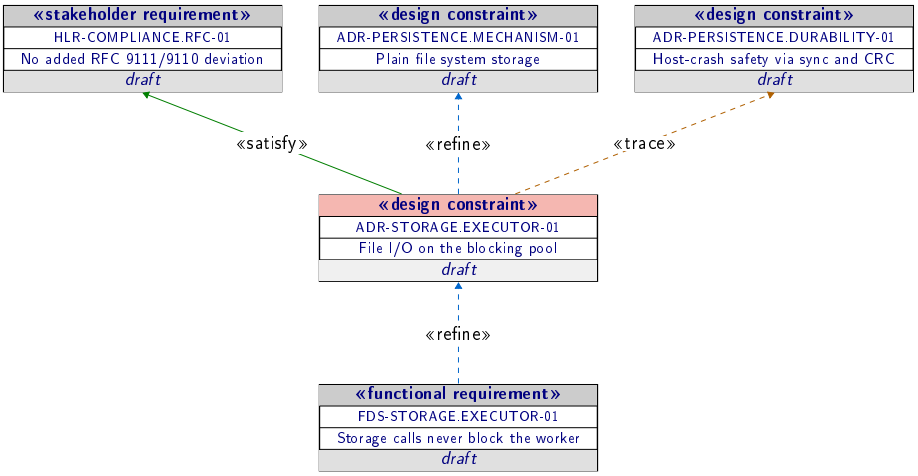


Figure 5.18: File I/O on the blocking pool

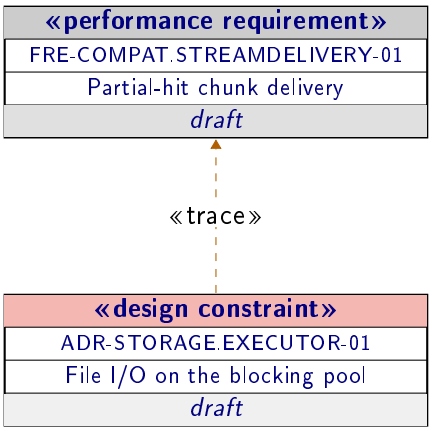


Figure 5.18: File I/O on the blocking pool (cont.)

Context: Every operation this backend performs is an ordinary blocking syscall — `open`, `pread`, `pwrite`, `fdatasync`, `rename`, `unlink` — while every `Storage` method is an `async fn` that Pingora polls on a service worker. Pingora gives each service *one* worker thread by default,[44] and the deliverable cannot assume an integrator raised it. A syscall run inline therefore stalls not just the request that issued it but every other connection that worker is driving.

The cost is not hypothetical, and not confined to slow devices. A cache whose whole purpose is to hold more than fits in RAM (HLR-PERSISTENCE.CAPACITY-01) serves reads that miss the page cache as a matter of course, and a store's `fdatasync` (ADR-PERSISTENCE.DURABILITY-01) costs 1–5 ms on a consumer SSD by this specification's own figures. Blocking a sole worker for that long reorders what the proxy does next: measured against the stock `MemCache` of `pingora-cache` on one worker, an inline backend lost an upstream `103 Early Hints` on 30 of 40 requests where `MemCache` lost none, because the proxy was late to poll its upstream task channel and the interim response was batched together with the final one. That is observable HTTP behaviour differing from the reference backend, which HLR-COMPLIANCE.RFC-01 rules out.

Decision: Run each storage operation's blocking work on Tokio's blocking pool — `spawn_blocking` — rather than on the worker that polls the `async fn`. The handoff is per *operation*, not per syscall: one for a lookup, one for a store's opening, one per body chunk, one for `finish`, one for a purge, a metadata update, or an expiry. The synchronous code that does the work is unchanged; only where it runs is.

Alternatives considered:

- *Leave the syscalls inline and document a minimum thread count*: no code to write, and an integrator who sets `ServerConf::threads` high enough is unaffected. Rejected: it makes correctness against HLR-COMPLIANCE.RFC-01 a deployment setting, and the default — one thread — is the failing case, so the out-of-the-box configuration would be the broken one.
- *`tokio::task::block_in_place`*: hands the worker's other tasks to a replacement thread without moving the work. Rejected: it panics on a current-

[44] `ServerConf::threads` defaults to 1 (L236).

thread runtime, so it would make the backend depend on which runtime flavour the integrator's Pingora build uses.

- *`tokio::fs`*: the same blocking pool with an async surface. Rejected: it offloads one syscall at a time, so a store would pay several handoffs where one suffices, and its API does not cover the positional reads and writes (`pread`/`pwrite`) this layout depends on.
- *`io_uring`*: genuinely asynchronous file I/O, no pool and no handoff. Rejected for this version: a new dependency, a Linux version floor above what Pingora itself requires, and a second I/O path to keep correct — machinery HLR-SCOPE.V1-01 asks not to build when a pool that ships with the runtime already closes the gap.

Consequences:

+ A storage operation never holds a service worker, so a slow device delays the request that touched it and nothing else.
+ Correct on the default Pingora configuration, with no requirement on the integrator.
– One thread handoff per storage operation, which is charged against every lookup and every body chunk whether or not the data was in the page cache.
– The blocking pool is shared with everything else in the process that uses it, and its size is the runtime's to decide, not this backend's.
– The restart scan is the one path this does not cover: FDS-EVICTION.STARTUP-01's `recover()` borrows the eviction manager for the length of the call, so its work cannot be sent to another thread. It bounds the stall by yielding instead.

View complete diagram.

Required coverage	✓ FDS-STORAGE.EXECUTOR-01 via ⟨⟨*refine*⟩⟩ — *Specify which work moves to the blocking pool, what stays on the worker, and how the restart scan bounds the stall it cannot move.*
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	HLR-COMPLIANCE.RFC-01
Traces to	ADR-PERSISTENCE.DURABILITY-01 FRE-COMPAT.STREAMDELIVERY-01
Refined by	FDS-STORAGE.EXECUTOR-01

[FDS-STORAGE.EXECUTOR-01]

Storage calls never block the worker ⟨⟨*functional requirement*⟩⟩ `draft`

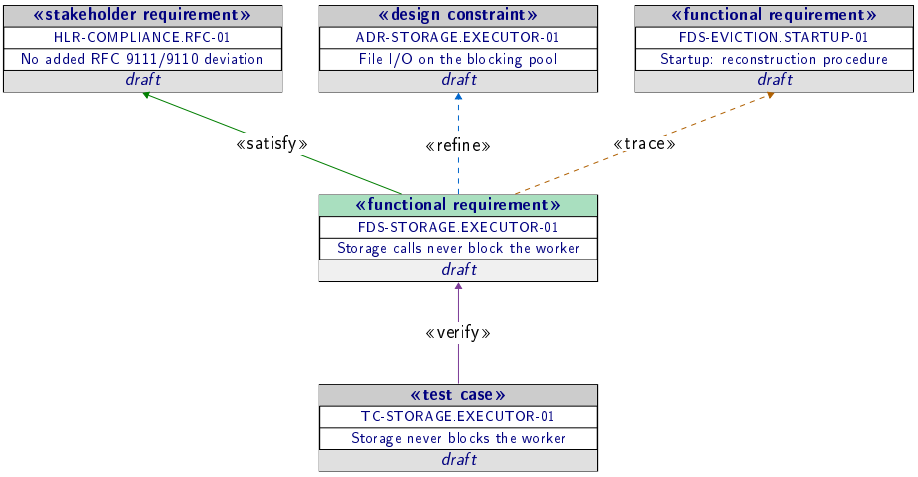


Figure 5.19: Storage calls never block the worker

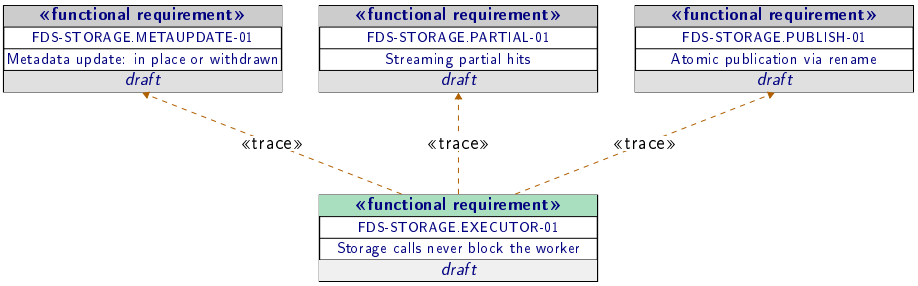


Figure 5.19: Storage calls never block the worker (cont.)

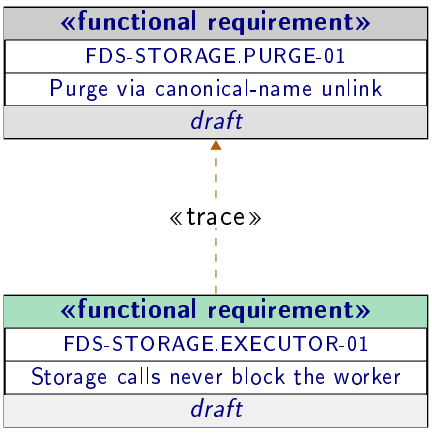


Figure 5.19: Storage calls never block the worker (cont.)

WHILE a `Storage` call, a `HandleHit` read, or a `HandleMiss` write is in progress, the backend shall leave the executor thread that polled it free to drive other tasks (ADR-STORAGE.EXECUTOR-01).

What moves. Each of the following runs as one unit on the blocking pool, so an operation costs a single handoff rather than one per syscall:

- `lookup` — opening the record, validating it, and deserialising its metadata (SDS-LAYOUT.OBJECTFILE-01), for a published record and for a partial hit's temporary alike (FDS-STORAGE.PARTIAL-01);
- `get_miss_handler` — creating the per-writer temporary and writing the record's fixed part (FDS-STORAGE.PUBLISH-01);
- `write_body` — writing a chunk to the temporary;
- `finish` — patching the header, the `fdatasync`, and the rename (ADR-PERSISTENCE.DURABILITY-01), then dropping the miss handler, whose close frees the store's file if the file has lost its name meanwhile;
- `read_body` — each positional read, on both the published and the partial-hit path;
- `purge` — the unlink and the shard-directory `fsync` (FDS-STORAGE.PURGE-01);
- `update_meta` and `expire` — reading the record and writing it back in place, or withdrawing or removing it (FDS-STORAGE.METAUPDATE-01, FDS-STORAGE.EXPIRE-01).

What stays. Work that touches no file stays on the worker, because a handoff would cost more than it saves: hashing the cache key, serialising and deserialising `CacheMeta`, folding a chunk into the running CRC-32C, and every consultation of the in-memory structures — the in-flight registry, the shard-directory bitset, and the publication lock's stripe table. A call therefore does its CPU work, hands the file work over, and resumes: the borrowed `CacheKey` and `CacheMeta` never have to cross the boundary, since only owned bytes do.

A partial hit's wait for the next chunk also stays on the worker: it is a notification, not a syscall (FDS-STORAGE.PARTIAL-01), and blocking a pool thread on it would hold that thread for as long as the store takes.

Two syscalls stay deliberately. The first is the `unlink` that removes an aborted store's temporary when its miss handler is dropped. A destructor cannot await, and the temporary's name must be gone when the drop returns (FRE-COMPAT.ABORTSPACE-01). The unlink is cheap: the handler still has the file open, so it removes only the name. The storage is freed when that handle is closed, which can take a long time for a large file, so the handler gives the handle to the pool to close; should the process exit first, the kernel closes it, and nothing is left behind.

The second is an `fstat` when a hit handler is dropped holding the last descriptor of the file it read. Closing that descriptor frees the file if the file has lost its last name: a record replaced, purged or evicted while the hit read it, or a partial hit's temporary removed with its store. The `fstat` tells: a file that still has a name is closed in place, since its close frees nothing, and one that has none is given to the pool to close, without waiting. Handing every hit's close to the pool would cost the

worker more than the close itself. The `fstat` reads the open file's inode, which is in memory, as the cache directory is on a local file system.

The publication lock. The per-key lock and the read-only gate (FDS-STORAGE.METAUPDATE-01, FDS-COMPAT.READONLY-01) are taken and released entirely within one blocking unit, so neither is ever held across an `await` — the same rule as before, now enforced by the shape of the code rather than only by convention.

The restart scan. FDS-EVICTION.STARTUP-01's `recover()` borrows the eviction manager it admits into for the length of the call, so its admissions cannot be sent to another thread. It shall instead yield to the executor between directory entries, and between the records its ghost-entry check visits, so that a scan of a large tree never holds the worker for longer than one record's header, key, and metadata region take to read. What the scan removes — a record or temporary it reclaims, an eviction victim — it shall remove on the blocking pool, since freeing a large file can take far longer than that read.

View complete diagram.

Required coverage	✓ TC-STORAGE.EXECUTOR-01 via ⟨⟨*verify*⟩⟩ — *Verify that no storage operation holds the executor thread for its duration, by measuring what a task co-resident on a single-worker runtime observes while cache traffic runs.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.EXECUTOR-01
Satisfies	HLR-COMPLIANCE.RFC-01
Traces to	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PARTIAL-01 FDS-STORAGE.PURGE-01 FDS-STORAGE.METAUPDATE-01 FDS-EVICTION.STARTUP-01
Verified by	TC-STORAGE.EXECUTOR-01
Satisfied by	IMP-STORAGE.EXECUTOR-01 (durable-cache/src/blocking.rs:32)

[ADR-STORAGE.RECLAIM-01]

Reclaim via the restart-time scan ⟨⟨*design constraint*⟩⟩ `draft`

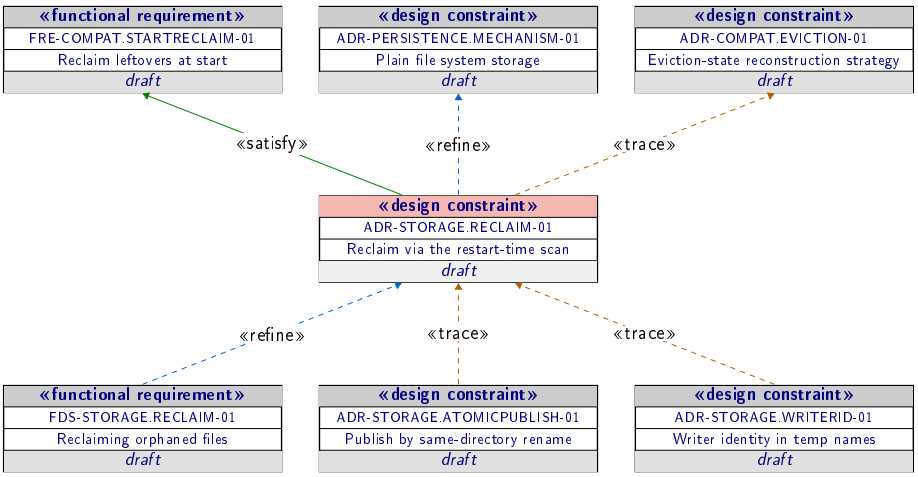


Figure 5.20: Reclaim via the restart-time scan

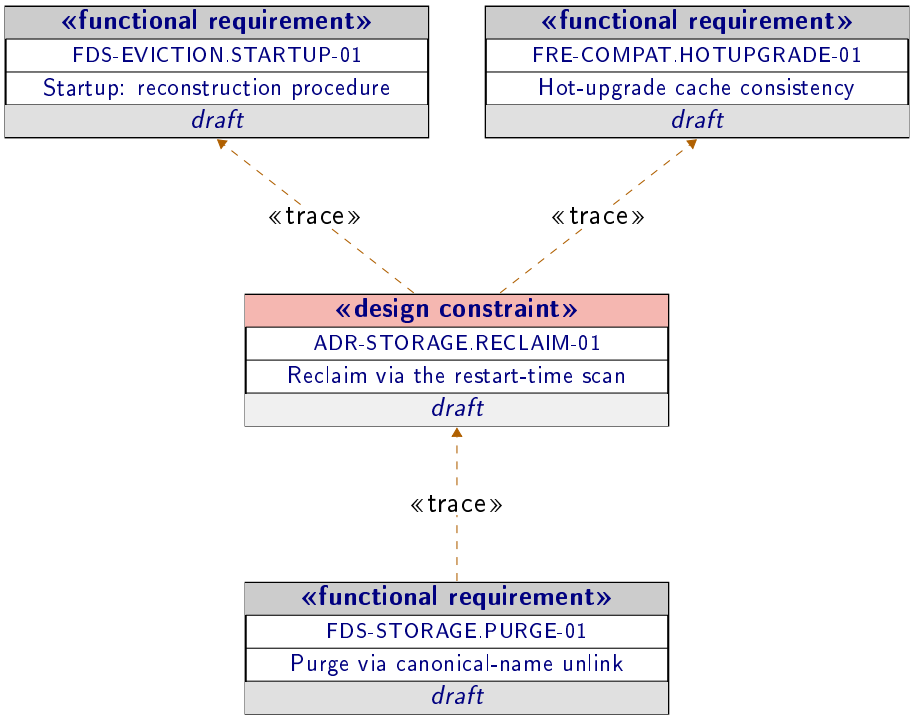


Figure 5.20: Reclaim via the restart-time scan (cont.)

Context: One kind of file is left behind by otherwise-correct operation of this design and never revisited by any keyed `Storage` call again: a per-writer tem-

porary a crash orphans before its rename (FDS-STORAGE.PUBLISH-01, FRE-COMPAT.STARTRECLAIM-01). It is not discoverable through `lookup`, `purge`, or `update_meta`: every `Storage` call pingora-cache itself makes is keyed to one specific object, never an enumeration of what the tree actually holds. The one place this specification already walks the whole tree is Eviction Recovery's own reconstruction scan (FDS-EVICTION.STARTUP-01), run once at every process start and explicitly precedented by ADR-COMPAT.EVICTION-01 for a structurally similar problem: no snapshot, no delta log, no distinction between a clean shutdown and a crash, because only a fresh scan can be trusted regardless of how the previous run ended.

A purge of a primary leaves records behind too: its `Vary` variants, whose keys FDS-STORAGE.PURGE-01 establishes the backend is neither handed nor able to derive at the moment the primary is purged (FRE-COMPLIANCE.PURGE-01). Unlike a temporary, such a variant is a published object, which the eviction manager counts against the size budget and evicts like any other (FRE-COMPAT.EVICTION-01).

Decision: Reclaim crash-orphaned temporaries as part of that same restart-time scan, rather than by any separate mechanism — together with two kinds of canonical record the scan meets anyway: one that fails validation, which lookups report as a miss but deliberately leave in place (SDS-LAYOUT.OBJECTFILE-01), and one a metadata update withdrew (FDS-STORAGE.METAUPDATE-01). While Eviction Recovery walks the tree to reconstruct the eviction manager, the same walk also classifies and removes them, at the cost of no second full-tree traversal and no new trigger. Leave `Vary` variant records to eviction, whatever their primary holds: the scan admits them like any other record.

Alternatives considered:

- *Reclaim, in the same scan, the variants no lookup can reach*: those whose primary is absent, fails validation, or carries a `provenance()` later than the variant's own `created()`, which is the test `pingora-cache` itself applies.[45] Rejected: that test settles reachability only when every variance comes from the `Vary` handling in `pingora-cache` itself, which consults the primary first. An integrator's `cache_key_callback` can set a variance itself;[46] its responses are then stored in variant slots with no primary ever written, and lookups serve them, so the scan would delete every such object at each start. A variant no lookup can reach costs only space the size budget already counts, and eviction removes it like any object no request reaches.

- *Reclaim a purge's variants synchronously, inside `purge` itself*: rejected on structural grounds, not just simplicity — FDS-STORAGE.PURGE-01 shows that the proxy's purge path only ever gives `purge` the primary's `CompactCacheKey`, and `pingora-cache` hands the backend neither a variant's own key nor any index from primary to variant. At that moment, without an index of the backend's own (the next alternative), there is no key to purge a variant *by*.

[45] `HttpCache::cache_vary_lookup` sets `valid_after` to the primary's `meta.provenance()` (L1576); the following `cache_lookup` on the variant key discards any result whose own `meta.created()` is earlier (L1518).

[46] `CacheKey::set_variance_key` is public. `HttpProxy::proxy_cache` looks a key that already carries a variance up as a variant slot, and `update_variance` keeps that variance when `cache_vary_filter` returns the same one.

- *Record each primary's variants, and reclaim them inside `purge`*: when a variant is stored, append its hash to a small list kept beside its primary, so that purging the primary can unlink the listed variants at once. Rejected: a new on-disk structure per varied primary, which must stay consistent while variants of one primary are stored and replaced concurrently, across processes, and through crashes — and the restart-time scan must exist anyway, for crash-orphaned temporaries and for eviction reconstruction. The space a variant whose primary is gone holds is already counted against the size budget, so leaving it to eviction costs no correctness.
- *A separate periodic background thread*, scanning the tree on its own timer independently of restart: rejected as machinery HLR-SCOPE.V1-01's simplest-adequate-design preference does not ask for — a timer, an interval to tune, and a cancellation path for shutdown or hot-upgrade drain — when the restart-time scan is already mandatory for eviction-state correctness (FRE-COMPAT.EVICTION-01) even with reclaim entirely absent, and needs no new machinery to extend.
- *Reclaim a crash-orphaned temporary opportunistically*, the moment some other component happens to be looking at its directory: rejected for lack of such a moment — FDS-LAYOUT.STARTUP-01's shard-directory cache only ever tracks which second-level directories *exist*, learned by listing the first-level directories once at construction; it never lists a shard's files, and no ordinary store operation is ever already enumerating a shard's files.

Consequences:

+ No new subsystem: no timer, no scheduler, no second tree walk, no shutdown-drain interaction to design — extends a scan the backend must run regardless.
+ One trigger, one code path, matching ADR-COMPAT.EVICTION-01's own precedent of never distinguishing a clean restart from a crash restart.
+ Compatible with a hot upgrade by construction: FRE-COMPAT.HOTUPGRADE-01 already anticipates background reclamation running on the incoming side while the outgoing process drains, under the same open-file safety argument FDS-STORAGE.PURGE-01 gives for purge — unlinking a file leaves an already-open reader on it undisturbed.
– Variant records a purge of their primary left behind stay on disk, across restarts too. Their space is counted against the size budget, and capacity-driven eviction removes them like any unrequested object once the cache nears that budget; a cache that never nears it keeps them, as leftover files in HLR-EFFICIENCY.SPACE-01's sense. This specification accepts that as a v1 cost, matching the trade-offs ADR-EFFICIENCY.COMPACTION-01 and ADR-COMPAT.EVICTION-01 already record.
– A record withdrawn or damaged while a process is up is only reclaimed at that process's *next* restart, unless a store of its key replaces it first; a deployment wanting a tighter bound must restart or hot-upgrade more often. This specification does not mandate a schedule for that. To make what accumulates visible, each scan logs the number and bytes of the files it reclaimed, by class, and also reports them as metrics when the metrics feature is enabled (ADR-OBSERVABILITY.METRICS-01).

View complete diagram.

Required coverage	✓ FDS-STORAGE.RECLAIM-01 via ⟨⟨*refine*⟩⟩ — *Specify the concrete detection and removal procedure for each class of file the scan reclaims, riding Eviction Recovery's existing enumeration, and the per-class count and bytes of reclaimed files each scan reports.*
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	FRE-COMPAT.STARTRECLAIM-01
Traces to	ADR-COMPAT.EVICTION-01 FDS-EVICTION.STARTUP-01 FRE-COMPAT.HOTUPGRADE-01
Traced from	ADR-STORAGE.ATOMICPUBLISH-01 ADR-STORAGE.WRITERID-01 FDS-STORAGE.PURGE-01
Refined by	FDS-STORAGE.RECLAIM-01

[FDS-STORAGE.RECLAIM-01]

Reclaiming orphaned files ⟨⟨*functional requirement*⟩⟩ `draft`

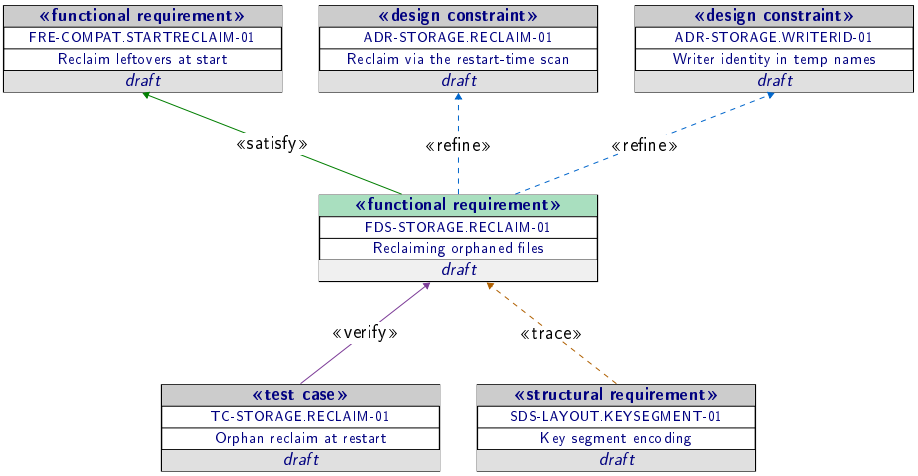


Figure 5.21: Reclaiming orphaned files

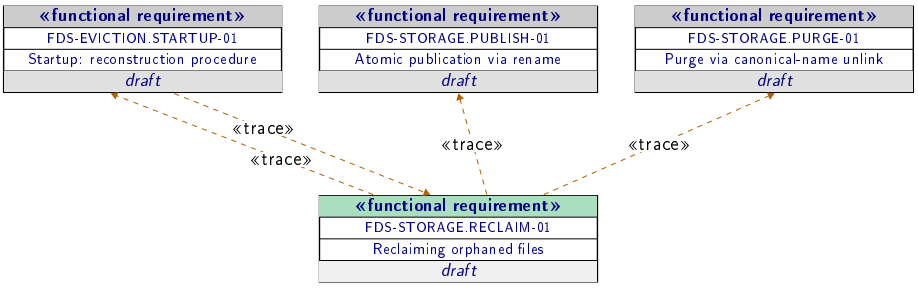


Figure 5.21: Reclaiming orphaned files (cont.)

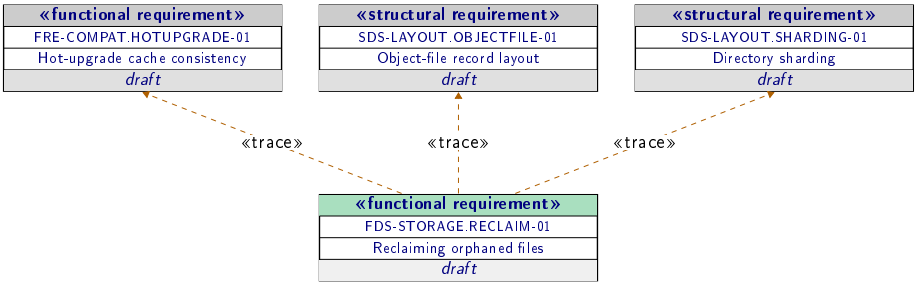


Figure 5.21: Reclaiming orphaned files (cont.)

Eviction Recovery's scan (FDS-EVICTION.STARTUP-01) already visits every file in the tree once, reading each canonical record's header, key, and internal metadata to `admit` it. This procedure (ADR-STORAGE.RECLAIM-01) rides that same pass: for each directory entry it classifies by name and, for a canonical record, by validation, then either admits it as before or reclaims it — no second walk of the tree, and no entry visited twice. Three classes are reclaimed: crash-orphaned temporaries, damaged, foreign or misplaced canonical records, and withdrawn records.

Crash-orphaned temporaries. An entry with the `.tmp` extension carries its writer's lease identity in its name (FDS-STORAGE.PUBLISH-01). The procedure shall treat the writer as possibly running — and leave the temporary untouched — exactly when the writer's lease exists and the procedure cannot take its lock, or cannot tell (ADR-STORAGE.WRITERID-01). The process's own writers satisfy this test, since serving may already have begun while the scan runs: a lock held through one open file description refuses another, even in the same process. So does a live hot-upgrade peer. Every other temporary belongs to a writer that has stopped, and the procedure shall unlink it. A temporary whose name has another form is left alone. The test requires every process sharing the cache directory to run on this host, with the directory on a local file system, a deployment constraint the deliverable documents.

Stopped writers' leases. After the walk, the procedure shall remove every lease, and every lease left under its staging name, whose lock it can take, holding the lock across the removal; a lease whose lock is held belongs to a live writer, or to one being created, and is left. Since a lease gets its final name only once it is locked, a live writer's lease is never removed. A temporary whose lease is missing counts as its writer's having stopped, so the order of the two removals does not matter.

Damaged or foreign canonical records. A canonical record that fails the read-time validation SDS-LAYOUT.OBJECTFILE-01 specifies — including one of another format version — cannot be admitted, since its key or freshness deadline cannot be trusted or decoded. Lookups report such records as misses but leave them in place, so the procedure shall reclaim them. A record whose checksums mismatch on every read may instead be an in-place metadata update caught between its two writes: before treating it as damaged, the procedure reads it once more holding the key's publication lock, as a lookup does (SDS-LAYOUT.OBJECTFILE-01), and admits it if that read finds it whole.

Misplaced records. A canonical record found anywhere but at the path its stored key names (SDS-LAYOUT.SHARDING-01) — put there by hand, or by a defect — is never served: a lookup of its key reads the other path, and a lookup of the key this path belongs to finds a key mismatch (SDS-LAYOUT.OBJECTFILE-01). The procedure shall reclaim it, counted as damaged.

Withdrawn records. A canonical record that a metadata update withdrew (FDS-STORAGE.METAUPDATE-01) holds no object: lookups report it as a miss and leave it in place until a store of its key replaces it. The procedure shall reclaim it, under a class of its own, so that its count is not mistaken for damage.

Variants. A canonical record whose own stored key carries a variance (`Some(variance)` in its `CompactCacheKey`) is a secondary slot (SDS-LAYOUT.OBJECTFILE-01). The procedure shall admit it like any other valid record, whatever its sibling primary holds (ADR-STORAGE.RECLAIM-01). It never opens that primary: a lookup can reach a variant for which no primary was ever stored, and a variant no lookup can reach is evicted like any other record.

Entries it cannot read. Nothing is known of a directory the procedure cannot list, or of an entry it cannot read, so it can be neither admitted nor reclaimed. Unless the directory or entry has vanished meanwhile, the procedure shall skip it and name it in a warning in the log.

Reclaiming safely under live traffic. This scan may run while the process serves (FDS-EVICTION.STARTUP-01), and while an outgoing hot-upgrade peer, its backend already read-only, completes its last requests (FDS-COMPAT.READONLY-01). A live store can publish a fresh record at a path the procedure has just classified as reclaimable, at any moment — its upstream fetch may have begun long before. The procedure shall therefore never unlink a canonical record by path alone: it notes the file's identity (device and inode) when it opens the entry to classify it, and unlinks it only while holding the key's publication lock (FDS-STORAGE.METAUPDATE-01), after confirming that the path still refers to that same file; otherwise it skips the entry. Because the read-only peer publishes nothing, the lock — which this process's stores and purges also take — covers every writer. A temporary needs no such check: nothing is ever renamed onto a temporary's name. One race remains harmless by construction: a file the peer or a reader already holds open is unaffected by the unlink, for the same POSIX reason FDS-STORAGE.PURGE-01 relies on.

Reporting. When the scan completes, or stops because the backend is read-only or its server is shutting down (FDS-EVICTION.STARTUP-01), the procedure shall log one summary line, saying which, and giving the number of directories and entries it skipped and the number and bytes of the files it reclaimed in each class — `temporary`, `damaged`, and `withdrawn` — and, when the backend's `metrics` feature is enabled (it is off by default, ADR-OBSERVABILITY.METRICS-01), add the same figures to two counters in the `prometheus` default registry, each labelled

by class:

- `pingora_durable_reclaimed_files_total`, the number of files;
- `pingora_durable_reclaimed_bytes_total`, their bytes.

The backend registers both when it is built, with every class's series at zero, so that each is served from the start rather than from the first reclaim of its class (as FDS-OBSERVABILITY.HEADROOM-01 does for its counter).

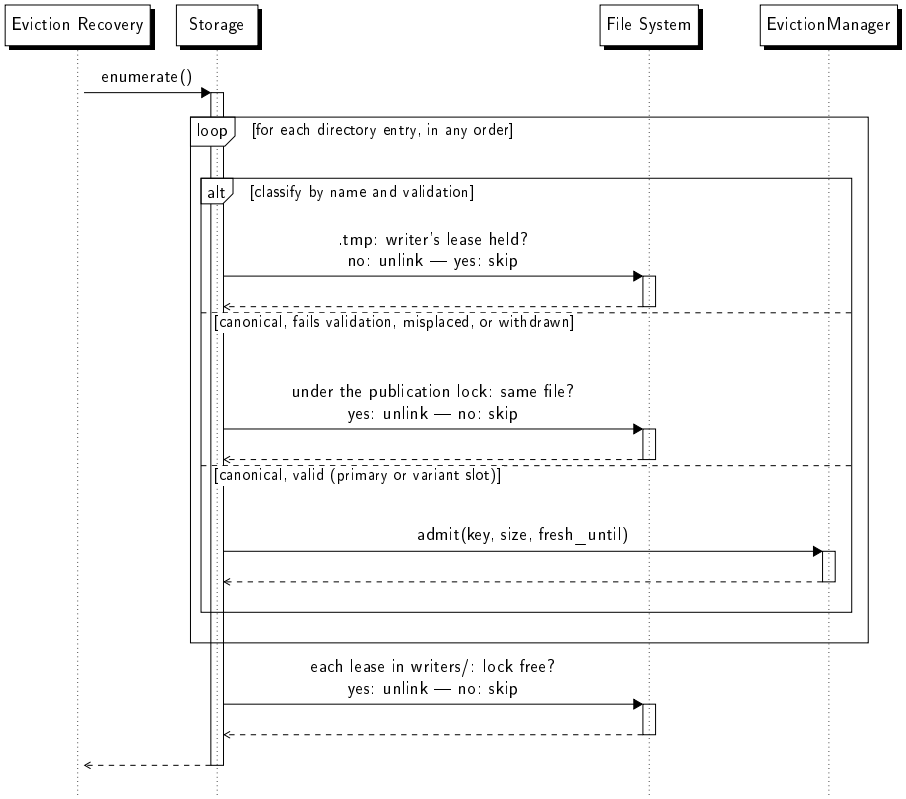


Figure 5.22: Reclaim, riding the same `enumerate()` pass Figure 6.10 shows as one opaque call: each entry is classified once; a crash-orphaned temporary or a damaged, foreign, misplaced, or withdrawn record is unlinked — a canonical record only after confirming, under its publication lock, that the path still names the file classified — and never admitted; every other record, variants included, is admitted exactly as FDS-EVICTION.STARTUP-01 already specifies. After the walk, the leases of stopped writers are removed.

View complete diagram.

Required coverage	✓ TC-STORAGE.RECLAIM-01 via ⟨⟨*verify*⟩⟩ — *Verify that a temporary whose writer is not running (its lease missing, or present but unlocked) is reclaimed, and one whose writer is running is left; that a stopped writer's lease is removed and a running writer's kept; that a damaged, foreign-format, misplaced, or withdrawn record is reclaimed; that a variant is admitted whatever its primary holds; that a record replaced by a live store between classification and unlink survives; that a directory or entry it cannot list or read is skipped, named in a warning, and counted; and that the log summary, and the reclaim counters when the metrics feature is enabled, report each class.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.RECLAIM-01 ADR-STORAGE.WRITERID-01
Satisfies	FRE-COMPAT.STARTRECLAIM-01
Traces to	FDS-EVICTION.STARTUP-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 SDS-LAYOUT.OBJECTFILE-01 SDS-LAYOUT.SHARDING-01 FRE-COMPAT.HOTUPGRADE-01
Traced from	SDS-LAYOUT.KEYSEGMENT-01 FDS-EVICTION.STARTUP-01
Verified by	TC-STORAGE.RECLAIM-01
Satisfied by	IMP-STORAGE.RECLAIM-01 (durable-cache/src/reclaim.rs:19)

[ADR-EFFICIENCY.COMPACTION-01]

Compaction deferred for v1 ⟨⟨*design constraint*⟩⟩ `draft`

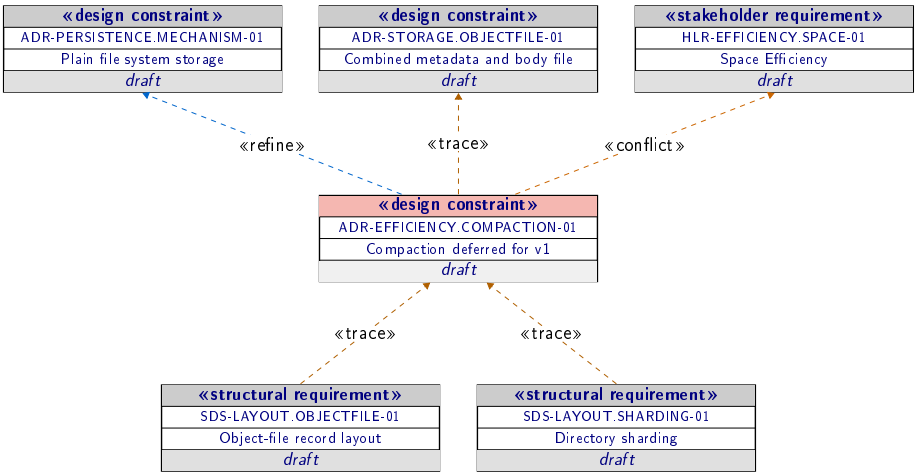


Figure 5.23: Compaction deferred for v1

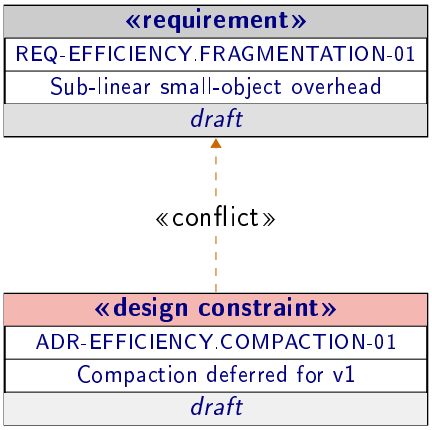


Figure 5.23: Compaction deferred for v1 (cont.)

Context: Each object is stored as a self-contained record — its metadata followed by its body (ADR-STORAGE.OBJECTFILE-01). Giving each record its own file still pays block rounding and the per-file overhead once per object: block-granular allocation may waste almost a full block on a small object's tail, a cost that does not shrink with the object's size and works directly against the stakeholder space-efficiency goal (HLR-EFFICIENCY.SPACE-01, REQ-EFFICIENCY.FRAGMENTATION-01).

Decision: Do not implement object compaction for this version. Closing this waste requires real, multi-part machinery — a background trigger and threshold policy, an aggregate file format and its own internal index, a lookup path that checks two locations instead of one, relocation on revalidation, and reclaiming space after a purge — against which HLR-SCOPE.V1-01 prefers the simplest adequate design, even at the cost of peak space efficiency. This decision therefore conflicts with HLR-EFFICIENCY.SPACE-01 and with the derived REQ-EFFICIENCY.FRAGMENTATION-01, which no other decision satisfies for this version: the waste described above is accepted, not eliminated, for as long as it stands. The resolution, for both conflicts, is to revisit compaction in a later version, once HLR-SCOPE.V1-01's scope constraint no longer applies.

Alternatives considered:

- *Implement compaction now*: a deferred background step that moves a completed object's record, back-to-back with others, into a shared *aggregate* file once its wasted space passes a configurable threshold. It would still require choosing the aggregate backend — a custom format reusing ADR-STORAGE.OBJECTFILE-01's record layout, or an embedded engine such as libmdbx or SQLite. Rejected: real, multi-part complexity — the aggregate format and index, the two-location lookup, relocation and reclaim handling, and the backend choice itself — for a version HLR-SCOPE.V1-01 asks to keep simple, even though it would fully close REQ-EFFICIENCY.FRAGMENTATION-01.
- *Do not implement compaction for this version* (chosen): every object keeps its own file; the block-rounding and per-file waste on small objects is accepted rather than eliminated.

Consequences:

+ No new subsystem: no aggregate format, no index, no background scheduler, no second lookup location, no purge-hole reclamation — matches HLR-SCOPE.V1-01's preference for the simplest adequate design.

+ Nothing beyond what ADR-STORAGE.OBJECTFILE-01 already specifies needs building or trusting.

– Block-granular allocation and per-file overhead keep scaling with the number of small cached objects, unresolved for this version.

– REQ-EFFICIENCY.FRAGMENTATION-01's obligation is not satisfied by this decision.

View complete diagram.

Required coverage	*terminal* — Deferred: compaction is not implemented for this version, so nothing further refines, satisfies, or verifies this decision; revisit if HLR-SCOPE.V1-01's scope changes.
Refines	ADR-PERSISTENCE.MECHANISM-01
Conflicts with	HLR-EFFICIENCY.SPACE-01 REQ-EFFICIENCY.FRAGMENTATION-01
Traces to	ADR-STORAGE.OBJECTFILE-01
Traced from	SDS-LAYOUT.SHARDING-01 SDS-LAYOUT.OBJECTFILE-01

[ADR-COMPAT.EVICTION-01]

Eviction-state reconstruction strategy ⟨⟨*design constraint*⟩⟩ `draft`

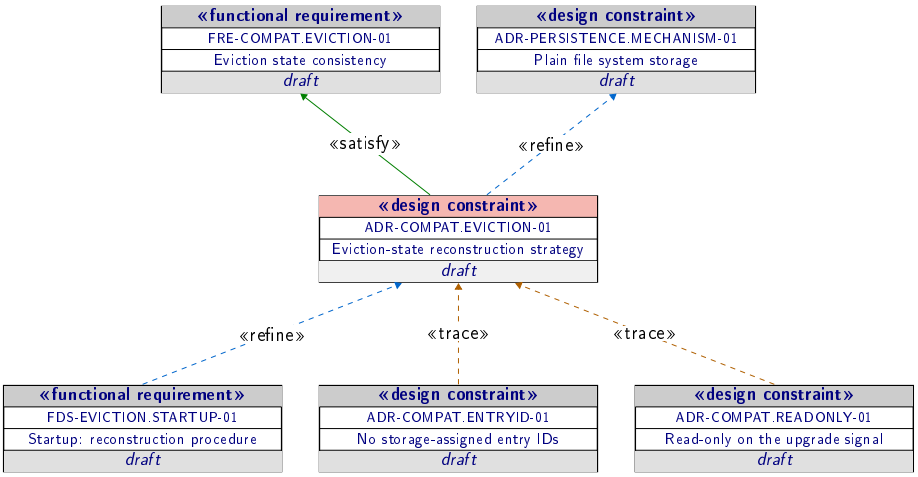


Figure 5.24: Eviction-state reconstruction strategy

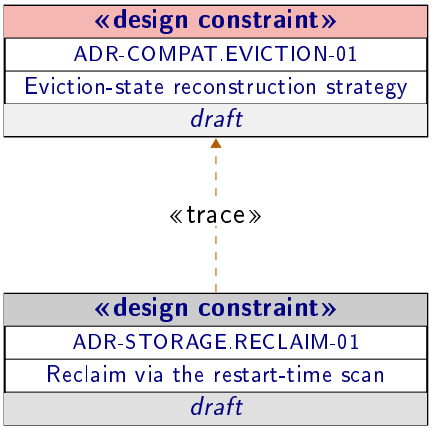


Figure 5.24: Eviction-state reconstruction strategy (cont.)

Context: FRE-COMPAT.EVICTION-01 requires the `pingora-cache` eviction manager's view to stay consistent with the objects persisted across a restart, but the manager is driven by `HttpCache`, not by the `Storage` backend, so the backend cannot keep it synchronised on its own — reconstructing it from the persisted set is one way to close that gap. Doing so can trade startup latency against the length of any inconsistency window, reliance on a custom `EvictionManager` versus the stock one, and how far the result departs from a pure drop-in `Storage` backend (HLR-COMPAT.PINGORA-01); the alternatives below weigh those trade-offs.

Decision: Adopt *file-system reconstruction as the sole mechanism, on every restart*: enumerate the persisted entries (ADR-PERSISTENCE.MECHANISM-01) and `admit`[47] each one into the manager, purging any victim an `admit` returns. No snapshot is taken, no delta log is maintained, and no distinction is made between a clean shutdown and a crash: reconstruction always follows the same procedure, and it admits every object with its record size, the same size a completed store reports (FRE-COMPAT.EVICTION-01).

Whether the integrator begins serving before reconstruction completes (accepting inaccurate capacity accounting until it does) or waits for it (accepting startup latency proportional to the number of stored objects) is an integration choice this decision leaves open. The window of inaccuracy therefore ends exactly when the procedure completes; no fixed time bound is claimed for it. Running alongside live traffic creates three races, each closed by a rule the procedure follows:

- *A stale size.* Live traffic may replace an object between the scan reading its record and admitting it. The scan therefore re-checks the record immediately before admitting, and uses the size it finds then — or skips the entry if the object is gone, since the live store already admitted its replacement.
- *A ghost entry.* A live eviction drops its victim from the manager at once, and removes the victim's record from storage only afterwards, in a task of its own,[48] so the scan may admit the victim again in between, and its entry would outlive the record. After enumeration, the procedure re-checks every

[47] `EvictionManager::admit`.

[48] `HttpCache::finish_miss_handler` admits the completed store (L1118), then spawns a task that purges each victim the manager returned (L1128).

object it admitted and removes the manager's entry for any that no longer exists. A purge leaves no such entry: `HttpCache` removes a purged object's entry itself, after the purge.[49]

- *An early victim.* An admission during the scan may return, as a victim, an object live traffic has just read. It is purged like any other victim: capacity accounting stays exact, and the object is simply fetched again when next requested.

Alternatives considered:

- *Snapshot on clean shutdown, reconstruction only on crash*: on a clean shutdown, snapshot the manager with `save`;[50] at startup, if the snapshot file is present, `load`[51] it and delete it durably before serving resumes, so its presence means exactly "a durable, unconsumed snapshot exists" and its absence means "reconstruct." That still leaves real machinery: a shutdown-side lifecycle hook in Eviction Recovery (FDS-EVICTION.STARTUP-01) — today there is only one, at process start — a rule that a draining process never saves (its snapshot would miss what the incoming process stores meanwhile), tied to the read-only call ADR-COMPAT.READONLY-01 introduces, and a startup branch choosing between loading and scanning. HLR-SCOPE.V1-01 prefers the simplest adequate design even at the cost of peak space or I/O efficiency; a recurring per-restart scan is exactly such an efficiency cost, and the crash path already has to build the scan/admit/reconcile machinery regardless (FDS-EVICTION.STARTUP-01). Rejected: a second lifecycle hook, a save rule coupled to the read-only switch, and a second startup path are more machinery than the restart-time saving justifies.
- *Save/load with log replay*: snapshot the manager and keep an append-only delta log between snapshots; replay the log at startup instead of scanning the file system. Rejected: adds delta-log maintenance and save-time reset machinery to the steady-state hot path, which the adopted approach avoids entirely.
- *Lazy admission*: no scan at all; admit an object the first time a lookup finds it. Rejected: an object nobody requests again is never admitted, so it is never counted against the size budget and never evicted — exactly the gap FRE-COMPAT.EVICTION-01 exists to close.

Consequences:

+ Simplest possible mechanism: no snapshot, no delta log, no validity marker, no startup branch — matches HLR-SCOPE.V1-01's preference for the simplest adequate design.
+ One behaviour regardless of how the previous run ended: nothing to misclassify a restart as clean when it was not, or vice versa.
+ Reuses the stock `EvictionManager` only through `admit`; no custom manager, and no `save`/`load` pairing to keep synchronised.

[49] `L1896`, once the backend has reported the object purged.
[50] `EvictionManager::save`.
[51] `EvictionManager::load`.

– Every restart pays a full file-system scan; starting to serve before it completes leaves capacity accounting inaccurate until it does, and waiting for it to finish instead trades that window for startup latency proportional to the number of stored objects — unlike the snapshot alternative's clean-restart case, which could avoid both costs at once.

– Admission order follows the scan, not past use, so the recency order the manager chose victims by before the restart is lost: right after a restart, a frequently used object can be evicted ahead of one nobody requests (accepted for this version, FRE-COMPAT.EVICTION-01).

– Reconstruction is carried out by a separate component, Eviction Recovery, additional to the `Storage` trait implementation (FDS-EVICTION.STARTUP-01), so the deliverable is more than a pure drop-in `Storage` backend (HLR-COMPAT.PINGORA-01).

View complete diagram.

Required coverage	✓ FDS-EVICTION.STARTUP-01 via ⟨⟨*refine*⟩⟩ — *Specify the concrete reconstruction procedure, and the API surface that orchestrates it, realising this strategy.*
Satisfies	FRE-COMPAT.EVICTION-01
Refines	ADR-PERSISTENCE.MECHANISM-01
Traced from	ADR-STORAGE.RECLAIM-01 ADR-COMPAT.ENTRYID-01 ADR-COMPAT.READONLY-01
Refined by	FDS-EVICTION.STARTUP-01

[ADR-COMPAT.ENTRYID-01]

No storage-assigned entry IDs ⟨⟨*design constraint*⟩⟩ `draft`

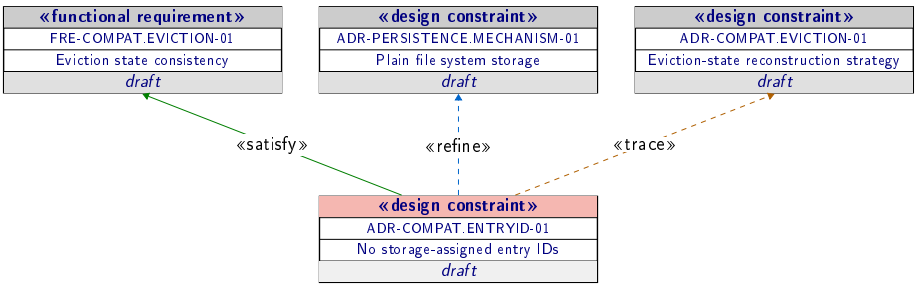


Figure 5.25: No storage-assigned entry IDs

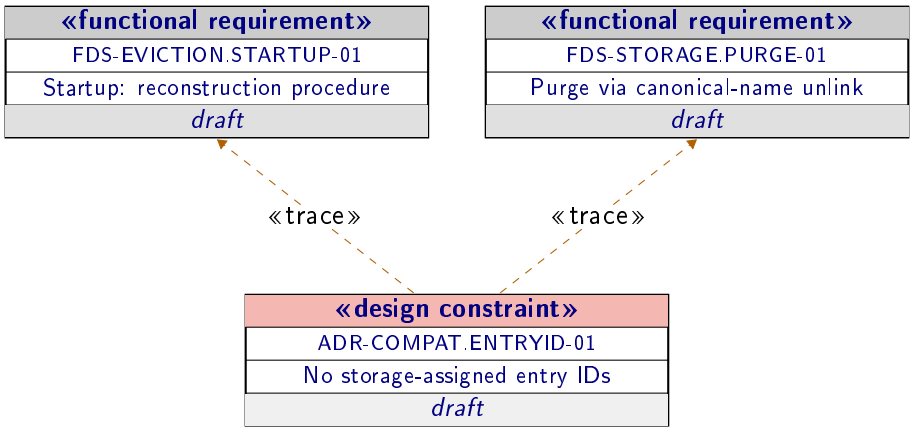


Figure 5.25: No storage-assigned entry IDs (cont.)

Context: Pingora 0.9.0 lets a backend give the eviction manager an identity finer than the cache key. `HandleHit` and `HandleMiss` may each return a `CacheEntryId`,[52] which `HttpCache` pairs with the `CompactCacheKey` into the `CacheEntryKey` it admits, accesses, and removes. The facility exists for a backend that keeps several *generations* of one key at once, so that eviction accounts for the exact stored object rather than for whichever one currently answers the key. The choice is all-or-nothing and must be consistent across both handles.

This backend has no such generations to distinguish. One key names one canonical path (SDS-LAYOUT.SHARDING-01) and a store replaces whatever is there by renaming over it (FDS-STORAGE.PUBLISH-01); a per-writer temporary is not a readable generation, and the identifier is required to survive `finish`, so a temporary's identity could not serve as one.

Decision: Return `None` from both `entry_id` methods and keep eviction keyed on the `CompactCacheKey` alone. Every entry this backend admits is therefore a `CacheEntryKey::KeyOnly`, and every purge outcome it reports is `Purged(None)` (FDS-STORAGE.PURGE-01). An `Exact` purge target carrying an `Identified` entry key names a generation this backend never issued, so it is reported as not found rather than resolved to the key's canonical path.

Alternatives considered:

- *Assign an identifier per published record* (its inode number, or a counter recorded in the record header): rejected — it buys accuracy only for a backend that retains generations, and this one does not. Here it would make accounting worse: a store that replaces a key would be admitted as a second entry, while the manager, which storage never tells of the replacement, would keep the first until it aged out, counting space the rename already freed. It would also cost an identity that must be read back on every hit, and survive a restart scan.

- *Retain generations and identify them*, publishing each store under its own name and letting eviction remove them individually: rejected — it is a different storage design, not an addition to this one, and FRE-COMPLIANCE.

[52] `HandleHit::entry_id` and `HandleMiss::entry_id`, both defaulting to `None`.

REPLACE-01's last-writer-wins convergence over a single canonical name is what several other decisions here already rest on.

Consequences:

+ The restart scan and live traffic address the same entry without having to agree on an identifier the scan would have to recover from disk (FDS-EVICTION.STARTUP-01).

+ The backend's eviction accounting is exactly what it was under Pingora 0.8.1, where the manager took the cache key directly.

– An eviction victim arrives as an `Exact` target and a purge request as an `Active` one, yet both resolve identically here, so the distinction the framework draws is one this backend can only degenerate.

– Adopting generations later is a change to this decision and to the record layout, not an incremental addition.

– An eviction purge can remove a newer object stored under the victim's key. `HttpCache` purges victims in a spawned task after `admit` chooses them;[53] a store of that key completing in between is what the purge removes. The eviction manager then tracks an entry with no record behind it until it is evicted again, and the next lookup misses. It is rare, since a victim is the least recently used key, and costs one refetch; the stock `MemCache`, which assigns no identifiers either, behaves the same.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Satisfies	FRE-COMPAT.EVICTION-01
Refines	ADR-PERSISTENCE.MECHANISM-01
Traces to	ADR-COMPAT.EVICTION-01 FDS-STORAGE.PURGE-01 FDS-EVICTION.STARTUP-01
Satisfied by	IMP-COMPAT.ENTRYID-01 (durable-cache/src/lookup.rs:543) IMP-COMPAT.ENTRYID-01 (durable-cache/src/miss.rs:410) IMP-COMPAT.ENTRYID-01 (durable-cache/src/purge.rs:40)

[53] The miss handler's `finish` path spawns the purges of the victims `admit` returned.

[ADR-COMPAT.READONLY-01]

Read-only on the upgrade signal ⟨⟨*design constraint*⟩⟩ `draft`

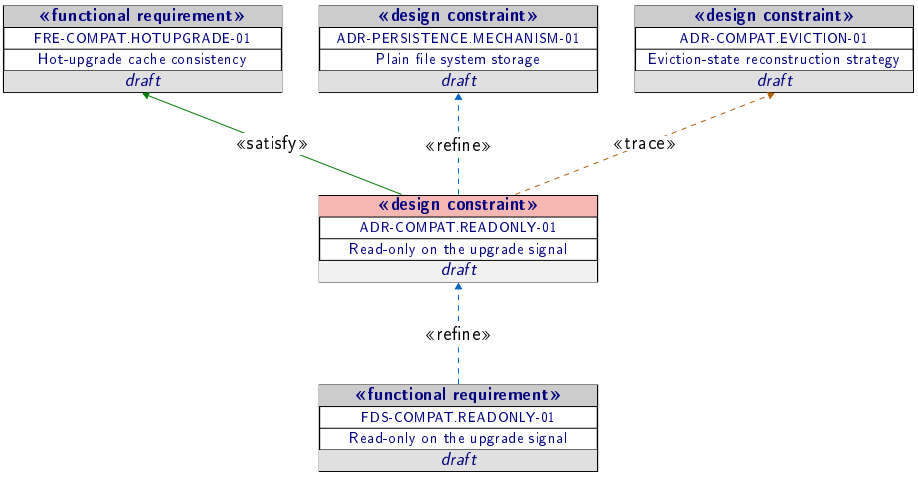


Figure 5.26: Read-only on the upgrade signal

Context: Once the outgoing process receives the Pingora upgrade signal, its backend must stop changing the shared cache — no publication, and purges and metadata updates refused — while still serving reads (FRE-COMPAT.HOTUPGRADE-01); meanwhile the incoming process must reach a consistent eviction view (FRE-COMPAT.EVICTION-01). The `Storage` trait receives no such signal. Pingora does announce it to the process's own code: the server broadcasts `ExecutionPhase::GracefulUpgradeTransferringFds` when `SIGQUIT` arrives, before it sends the listening sockets to the new process, and both processes then accept connections for `CLOSE_TIMEOUT` (5 s) before the old one stops listening and drains its sessions for up to `EXIT_TIMEOUT` (300 s).[54]

Decision: The backend exposes an operation that makes it *read-only*, which the integrator invokes when its process's server broadcasts `GracefulUpgradeTransferringFds`. The two words are kept apart: the *process* drains — it finishes the sessions it has accepted — while its *backend* is read-only, reading the shared cache but no longer changing it. From that call on, the backend publishes nothing (a store that completes has its temporary removed instead of renamed), fails purges and metadata updates with an error, and keeps serving lookups and reads. The incoming process starts its restart procedure only once it holds the listening sockets *and* the upgrade's close timeout has elapsed: the outgoing process's read-only call — delivered asynchronously, shortly after its phase broadcast — has taken effect by then, so every object it published is already on disk before the incoming scan begins.

Alternatives considered:

- *No read-only switch: two independent writers for the whole window.* Rejected: objects the outgoing process publishes after the incoming scan are unknown to

[54] `Server::watch_execution_phase`; the phase is sent at `L279`, before the sockets are sent (`L290`); `CLOSE_TIMEOUT` and `EXIT_TIMEOUT`.

the incoming eviction manager until the next restart, violating FRE-COMPAT.EVICTION-01, and the two managers together can let disk use exceed the budget.

- *An on-disk generation marker*: each backend records a generation at start and checks, before every publication or purge, that no newer one exists — no integrator hook needed. Rejected: a new on-disk structure (HLR-SCOPE.V1-01), a file-system check on every write, and a window between the check and the rename that the marker cannot close.
- *The incoming process defers all writes until the outgoing one exits.* Rejected: it runs uncached for up to `EXIT_TIMEOUT` — five minutes of full origin load after every upgrade.

Consequences:

+ The incoming process is the sole writer for its whole life after the close timeout, so the single-instance guarantees apply unchanged.
+ The integration is one call from a Pingora hook the server already provides.
– An integrator that does not wire the call gets the two-writer behaviour rejected above; the deliverable documents the call as required for hot upgrades.
– Purges sent to the outgoing process once its backend is read-only fail and must be retried; responses it stores from then on are served but not cached.
– The incoming process's restart procedure starts at least 5 s after it takes over, lengthening the window of inaccurate accounting ADR-COMPAT.EVICTION-01 already accepts.

View complete diagram.

Required coverage	✓ FDS-COMPAT.READONLY-01 via ⟨⟨*refine*⟩⟩ — *Specify the read-only operation and its integration with the execution-phase broadcast, the behaviour of each Storage call after it, and the incoming process's start condition for its restart procedure.*
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	FRE-COMPAT.HOTUPGRADE-01
Traces to	ADR-COMPAT.EVICTION-01
Refined by	FDS-COMPAT.READONLY-01

[FRE-COMPLIANCE.HEADERS-01]

Faithful header round-trip ⟨⟨*functional requirement*⟩⟩ `draft`

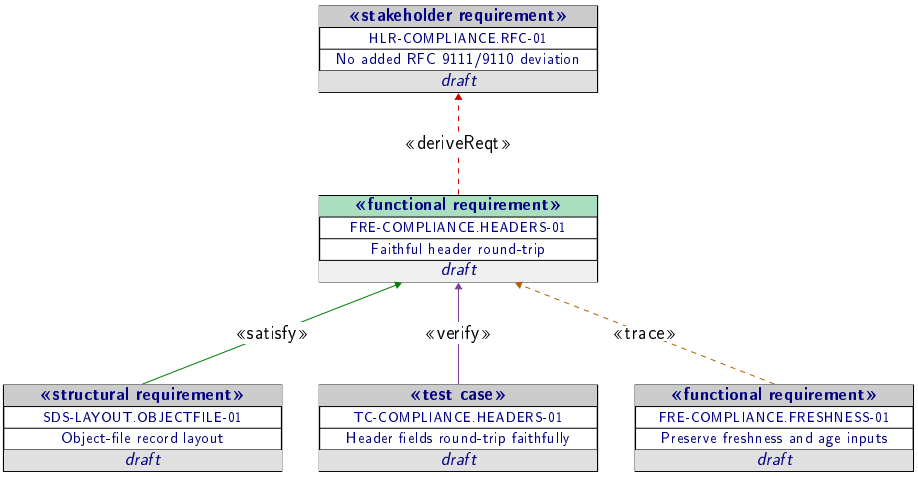


Figure 5.27: Faithful header round-trip

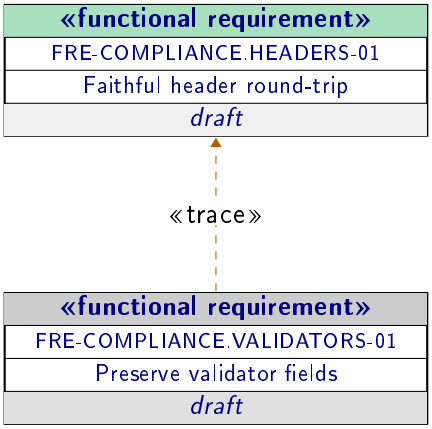


Figure 5.27: Faithful header round-trip (cont.)

WHEN `pingora-cache` looks up a stored object, including after a process restart, the storage backend shall return the complete response head it was handed for that object — the status code, the protocol version, and every response header field with its name and value preserved byte for byte and same-named fields in their relative order.

"Every header field" covers any head `pingora-cache` hands over: fields `pingora-cache` does not itself recognise, opaque non-ASCII octets in a value, repeated field names (including `Set-Cookie`-style fields that must not be list-combined), empty field values, and a header set large enough to exceed any fixed-size metadata region the on-disk format reserves. Returning the head

unchanged keeps the status, freshness, validation, and `Cache-Control` inputs `pingora-cache` relies on from being dropped or altered by storage.

Basis: RFC 9111 Section 3.1 (storing header fields); RFC 9110 Section 5.2 (field lines), RFC 9110 Section 5.3 (field order), RFC 9110 Section 6.2 (control data), RFC 9110 Section 6.3 (header fields).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traced from	FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.VALIDATORS-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.HEADERS-01

[FRE-COMPLIANCE.TRAILERS-01]

Trailer fields kept distinct ⟨⟨*functional requirement*⟩⟩ `draft`

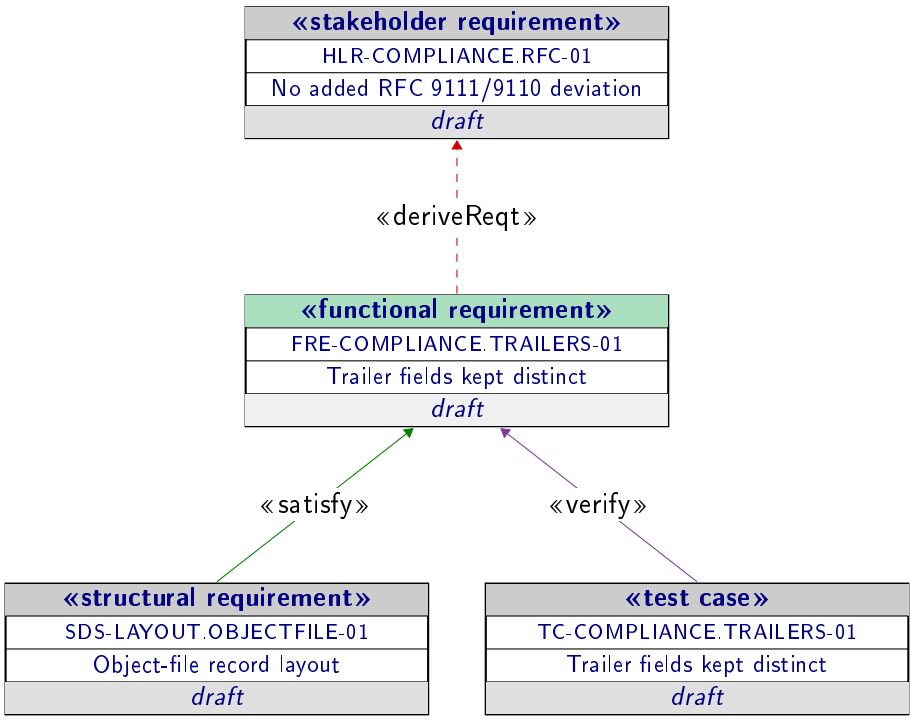


Figure 5.28: Trailer fields kept distinct

The storage backend shall return no field it received as a trailer field to `pingora-cache` as a header field.

The backend may persist trailer fields separately from the header fields or discard them; either meets the requirement. In the targeted `pingora-cache` version the

Storage interface offers no trailer channel: the metadata handed to the backend carries header fields only, and the miss handler carries body bytes only, so the framework can never present a trailer to the backend and the interface itself discharges this obligation. The requirement still constrains the backend's own on-disk format — which offers no way to fold trailer-originated fields into the stored header set — and any future framework version that does carry trailers.

Because no trailer can reach the backend through the targeted interface, no test can exercise this obligation; it is verified by *inspection* of the Storage interface and of the on-disk record format, recorded in TC-COMPLIANCE.TRAILERS-01.

Basis: RFC 9111 Section 3.1 ("Caches MUST NOT combine trailer fields with header fields"); RFC 9110 Section 6.5 (trailer fields).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.TRAILERS-01

[FRE-COMPLIANCE.BODY-01]

Faithful round-trip of the body ⟨⟨*functional requirement*⟩⟩ `draft`

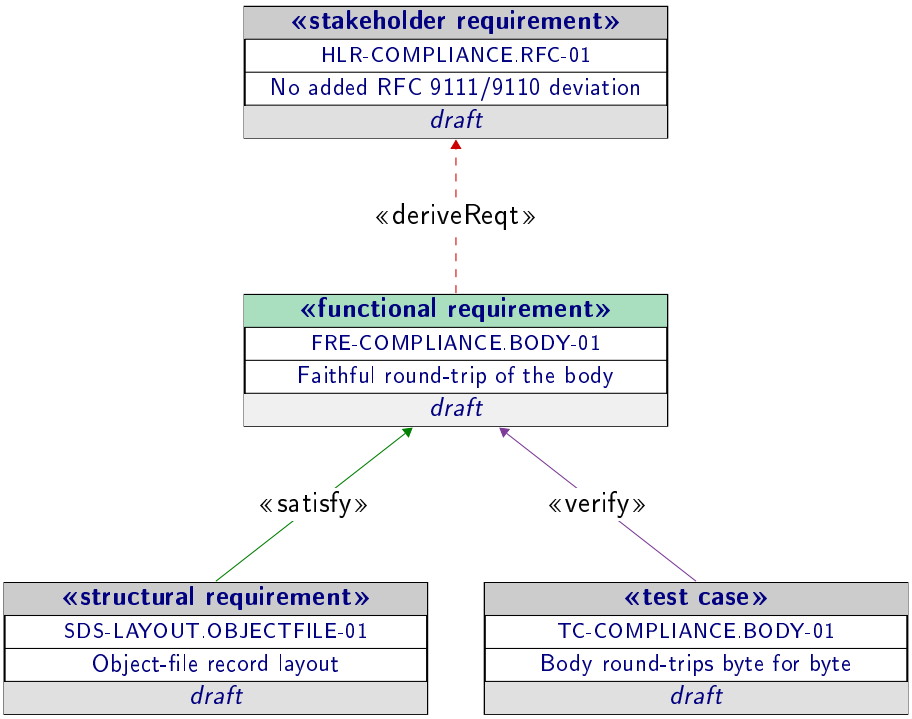


Figure 5.29: Faithful round-trip of the body

The storage backend shall deliver, on every read of a stored object's body — from its start, or from a position sought through the hit handler[55] — exactly the corresponding bytes of the body written for that object, in order, with no truncation, padding, re-encoding, or other transformation.

The requirement applies to bodies of every size: empty bodies, bodies at the backend's chunk- and block-size boundaries, and bodies spanning many chunks. Exact reads from a sought position matter because `pingora-cache` builds range responses from a sought reader, so those responses then carry exactly the stored bytes for the requested range.

Basis: RFC 9110 Section 8.1 (representation data); RFC 9111 Section 3 (storing responses); RFC 9110 Section 14.2 (Range).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.BODY-01

[FRE-COMPLIANCE.FRESHNESS-01]

Preserve freshness and age inputs ⟨⟨*functional requirement*⟩⟩ `draft`

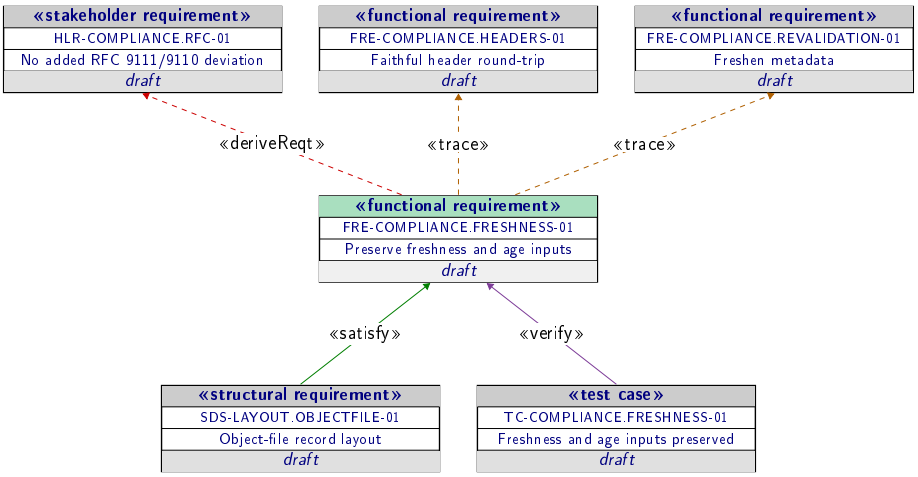


Figure 5.30: Preserve freshness and age inputs

Freshness lifetime and current age that `pingora-cache` evaluates from a retrieved object, at a given evaluation instant, shall match those it evaluates at that same instant from the object as it was handed to the backend — including across a restart, and after a metadata update (FRE-COMPLIANCE.REVALIDATION-01), when the updated object is the one handed over.

[55] `HitHandler::can_seek` and `HitHandler::seek`.

Two kinds of input feed that evaluation. The response header fields — `Date`, `Expires`, `Age`, and the `Cache-Control` `max-age`/`s-maxage` directives — are already required to round-trip byte for byte by FRE-COMPLIANCE.HEADERS-01, boundary and degenerate values included; interpreting them is up to `pingora-cache`. What this requirement adds is the other kind: the created, updated, and fresh-until instants `pingora-cache` keeps in the object's internal metadata, which the backend round-trips as part of the metadata it persists, never substituting timestamps of its own. Boundary values — an `Age` of 0, delta-seconds at the overflow cap (2147483648), `max-age=0`, an already-past or syntactically invalid `Expires` — are where a mismatch in evaluation would surface, so this requirement's test exercises them.

Basis: RFC 9111 Section 4.2 (freshness), RFC 9111 Section 4.2.3 (calculating age), RFC 9111 Section 5.1 (Age), RFC 9111 Section 5.2.2 (response directives), RFC 9111 Section 5.3 (Expires).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.REVALIDATION-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.FRESHNESS-01

[FRE-COMPLIANCE.VALIDATORS-01]

Preserve validator fields ⟨⟨*functional requirement*⟩⟩ `draft`

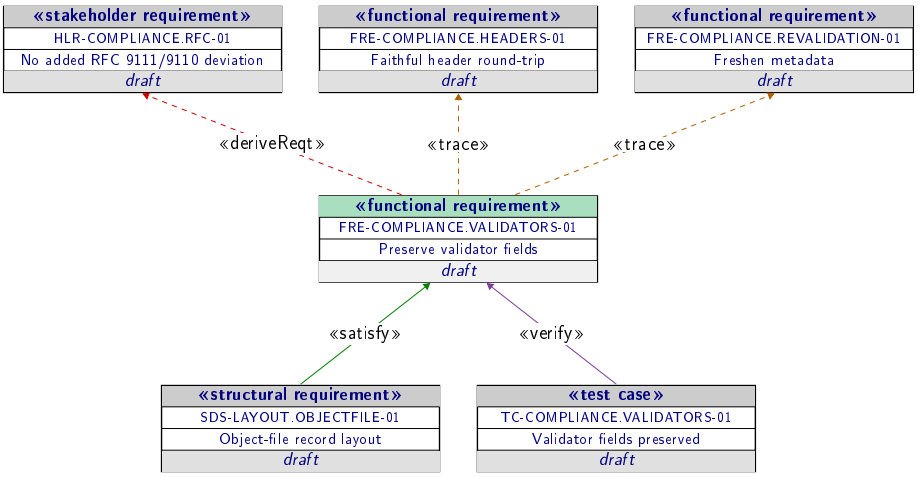


Figure 5.31: Preserve validator fields

The storage backend shall return the `ETag` and `Last-Modified` validator fields byte for byte in each form conditional revalidation depends on: a strong entity tag,

a weak one (`W/`), an empty entity tag (`ETag: ""`), and both validators on one response.

FRE-COMPLIANCE.HEADERS-01 already requires every header field to round-trip byte for byte. This requirement names the validator forms whose silent normalisation would specifically break revalidation — the fields `pingora-cache` uses to issue `If-None-Match` / `If-Modified-Since` and to match a `304 (Not Modified)` to the stored representation — so that each is verified explicitly. Validators changed by a metadata update are governed by FRE-COMPLIANCE.REVALIDATION-01.

Basis: RFC 9110 Section 8.8.2 (Last-Modified), RFC 9110 Section 8.8.3 (ETag); RFC 9111 Section 4.3 (validation).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.REVALIDATION-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.VALIDATORS-01

[FRE-COMPLIANCE.REVALIDATION-01]

Freshen metadata ⟨⟨*functional requirement*⟩⟩ `draft`

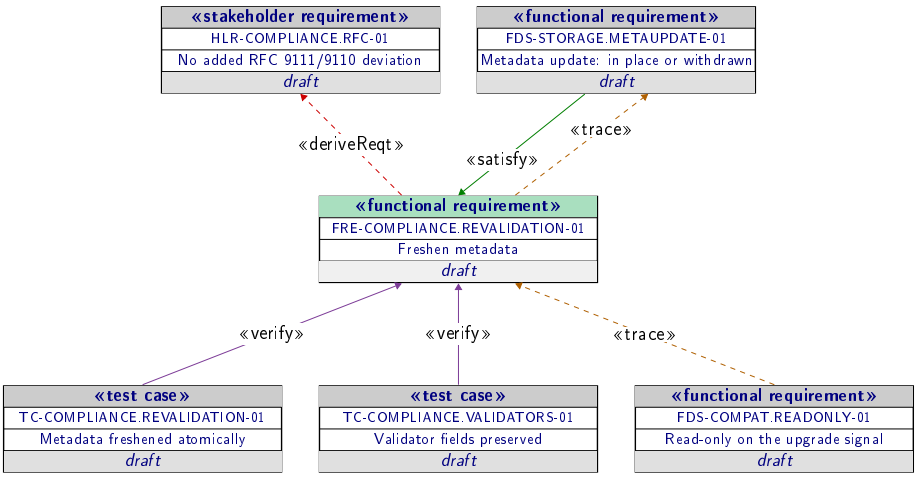


Figure 5.32: Freshen metadata

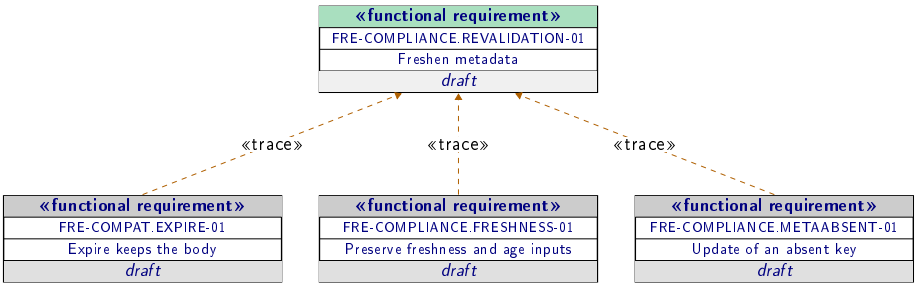


Figure 5.32: Freshen metadata (cont.)

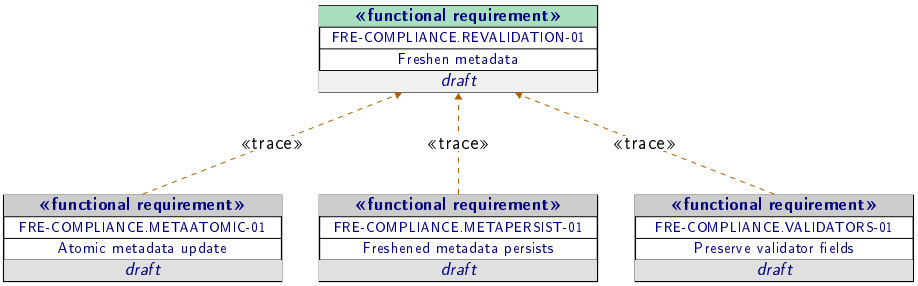


Figure 5.32: Freshen metadata (cont.)

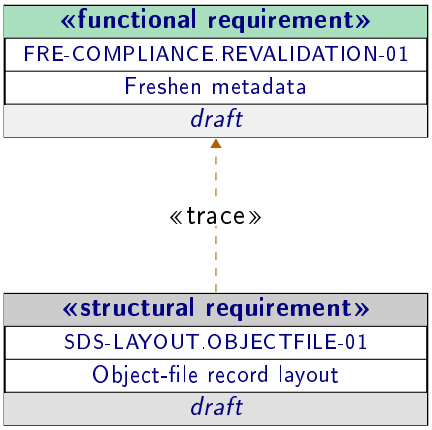


Figure 5.32: Freshen metadata (cont.)

WHEN `update_meta` is called with newer metadata for the object a key holds, the storage backend shall replace that object's metadata and header fields with the newer set, keeping the object's body unchanged and retrievable under the same key, or, if the newer set exceeds the space the backend reserved for the object's metadata, stop holding the object, so that the next request for it fetches it again.

`pingora-cache` freshens a stored response this way after a successful validation — a `304 (Not Modified)` response — through the backend's `update_meta` operation.[56] A newer header set may be larger than the one it replaces, and no

[56] `Storage::update_meta`.

longer fit the space the on-disk layout reserved ahead of the body (FDS-STORAGE.METAUPDATE-01). For this version, the backend then stops holding the object rather than keep it (ADR-STORAGE.METAMOVE-01). RFC 9111 Section 4.3.4 binds the responses a cache keeps, but nothing obliges a cache to keep one, and RFC 9111 Section 4.4 counts removing a stored response as invalidating it. Either way, the object is never served truncated, nor with a mixture of the two metadata sets. The newer metadata belongs to the object that was validated: a key whose object a newer one has replaced since then keeps the newer object, whole, and the update is not applied (FDS-STORAGE.METAUPDATE-01). What a concurrent lookup may observe is stated in FRE-COMPLIANCE.METAATOMIC-01, what survives a restart or crash in FRE-COMPLIANCE.METAPERSIST-01, and what an update of a key with no stored object does in FRE-COMPLIANCE.METAABSENT-01. An update requested after the backend's process has received the Pingora upgrade signal is refused instead (FRE-COMPAT.HOTUPGRADE-01).

Basis: RFC 9111 Section 3.2 (updating stored header fields), RFC 9111 Section 4.3.4 (freshening stored responses upon validation).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FDS-STORAGE.METAUPDATE-01
Traced from	FRE-COMPAT.EXPIRE-01 FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.VALIDATORS-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.METAABSENT-01 SDS-LAYOUT.OBJECTFILE-01 FDS-COMPAT.READONLY-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.VALIDATORS-01 TC-COMPLIANCE.REVALIDATION-01

[FRE-COMPLIANCE.METAATOMIC-01]

Atomic metadata update ⟨⟨*functional requirement*⟩⟩ `draft`

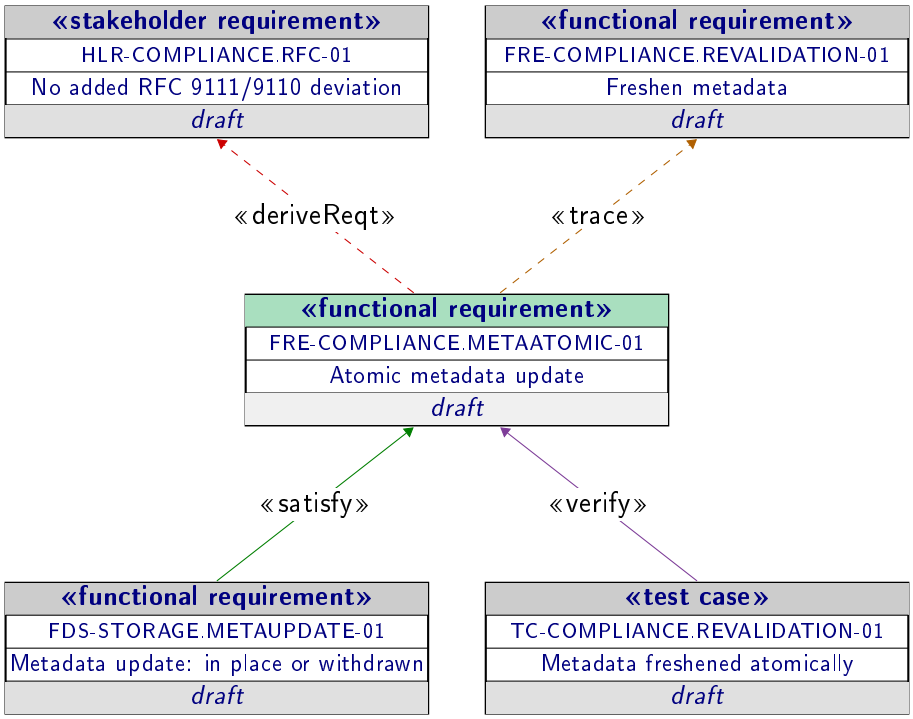


Figure 5.33: Atomic metadata update

WHILE a metadata update of an object is in progress, the storage backend shall answer every lookup of that object with either its pre-update or its post-update metadata, whole, together with the object's unchanged body.

A lookup therefore never observes a partially updated mixture of the two metadata sets, nor a body that no longer matches the metadata it is served with; a mixture could pair freshness inputs and validators that never belonged together. The update itself is stated in FRE-COMPLIANCE.REVALIDATION-01.

Basis: RFC 9111 Section 3.2 (updating stored header fields).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.REVALIDATION-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01

[FRE-COMPLIANCE.METAPERSIST-01]

Freshened metadata persists ⟨⟨*functional requirement*⟩⟩ `draft`

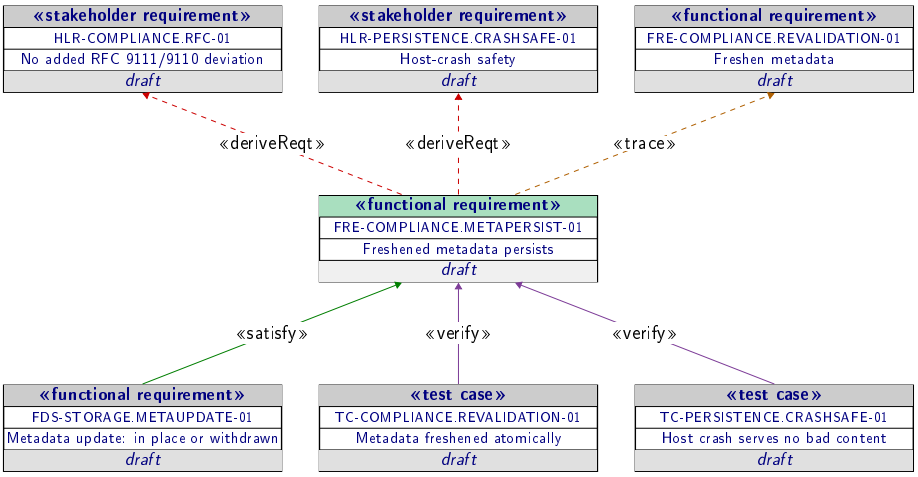


Figure 5.34: Freshened metadata persists

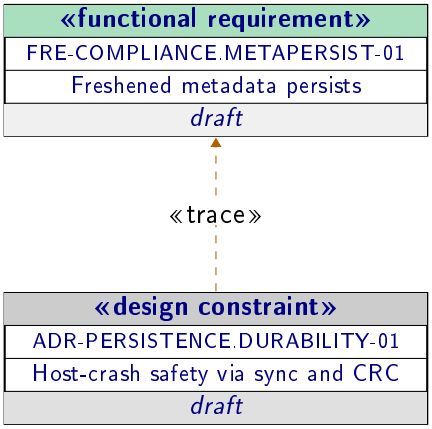


Figure 5.34: Freshened metadata persists (cont.)

WHEN a process of the backend looks up an object whose metadata update an earlier process reported as applied, the storage backend shall return the updated metadata — or, only if a host crash or power loss intervened, either the updated or the superseded metadata, never a mixture of the two.

Superseded metadata only makes the object due for revalidation sooner, whereas a mixture could pair freshness inputs and validators that never belonged together. The requirement holds until a later store, update, or purge of the key changes the object again.

This requirement derives from two parents: HLR-COMPLIANCE.RFC-01 for freshening semantics, which an update lost on an orderly restart would undo, and

HLR-PERSISTENCE.CRASHSAFE-01 for what a host crash may and may not do to an applied update.

Basis: RFC 9111 Section 4.3.4 (freshening stored responses upon validation).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01 HLR-PERSISTENCE.CRASHSAFE-01
Traces to	FRE-COMPLIANCE.REVALIDATION-01
Traced from	ADR-PERSISTENCE.DURABILITY-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01 TC-PERSISTENCE.CRASHSAFE-01

[FRE-COMPLIANCE.METAABSENT-01]

Update of an absent key ⟨⟨*functional requirement*⟩⟩ `draft`

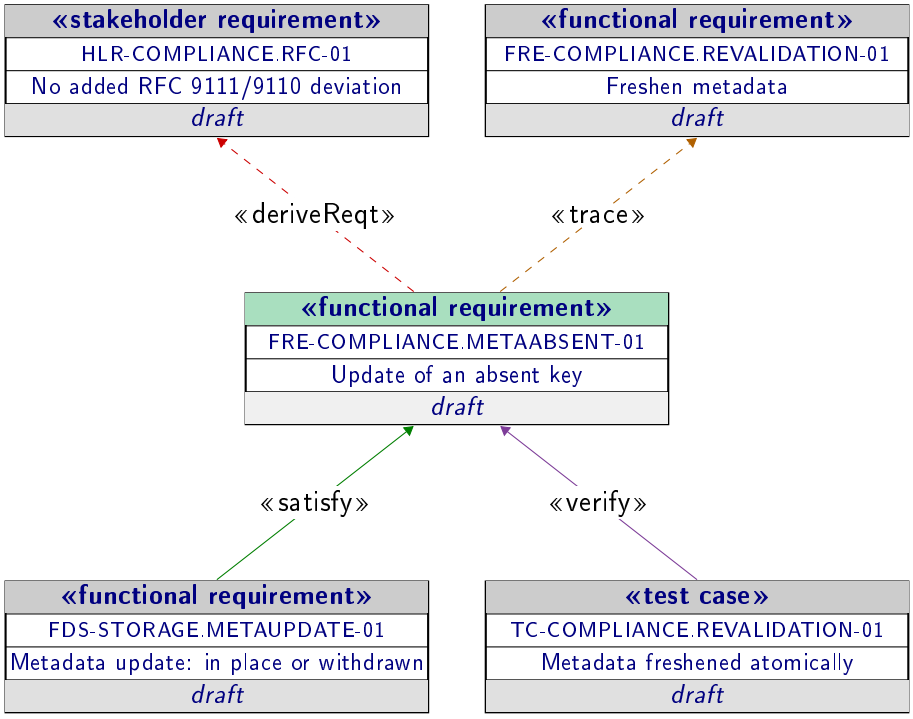


Figure 5.35: Update of an absent key

IF `update_meta` is called for a key that holds no stored object, THEN the storage backend shall report an error distinguishable from an applied update, leaving the key without a stored object.

Materialising a bodyless object from the metadata alone would let a later lookup serve a response with no representation. This deliberately differs from the stock backend, which panics in that case.[57]

Basis: RFC 9111 Section 3.2 (updating stored header fields).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.REVALIDATION-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01

[FRE-COMPLIANCE.REPLACE-01]

Replace stored responses atomically ⟨⟨*functional requirement*⟩⟩ `draft`

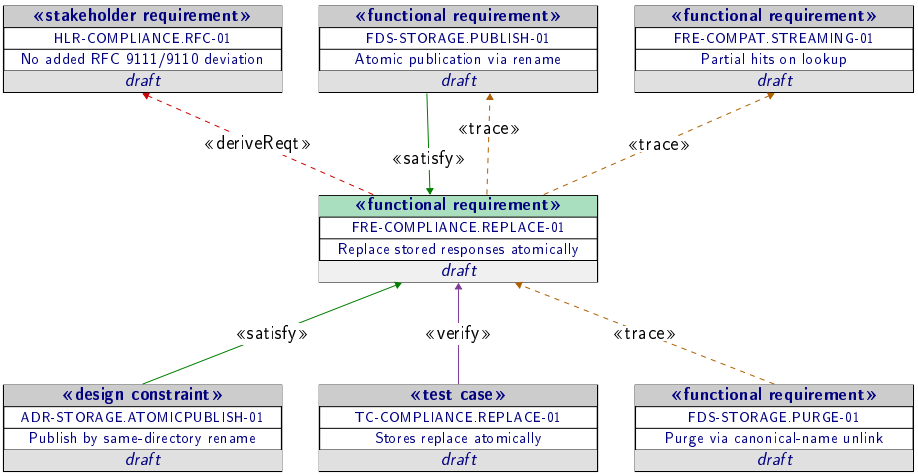


Figure 5.36: Replace stored responses atomically

[57] `MemCache::update_meta` calls `panic!("no meta found")` when the key holds no object.

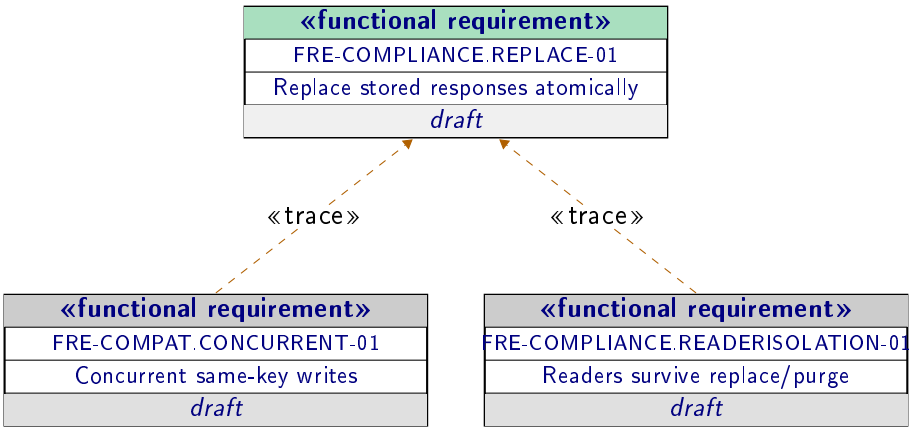


Figure 5.36: Replace stored responses atomically (cont.)

The storage backend shall answer every lookup it serves from a published object with the object most recently published under the looked-up key, whole — never a mixture of two objects, nor the metadata of one paired with the body of another.

`pingora-cache` stores a new response under a key that already holds one when a full response to a validation request replaces the stored response it could not freshen, or when an expired object is simply re-fetched and re-admitted. The old object therefore stays the answer, whole, until the new store is published, and the new object is the answer, whole, from then on. Until then a lookup within the writing process may instead be served a partial hit of the in-flight new object (FRE-COMPAT.STREAMING-01); a lookup that cannot see the in-flight write — such as one from a peer process — gets the old object. When several stores of the same key race — which `pingora-cache` permits (FRE-COMPAT.CONCURRENT-01) — the last to publish is the single winner. What a reader already streaming the old object receives is stated in FRE-COMPLIANCE.READERISOLATION-01.

Basis: RFC 9111 Section 4.3.3 (handling a validation response — a full response replaces the stored one); RFC 9111 Section 3 (storing responses).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FDS-STORAGE.PUBLISH-01 FRE-COMPAT.STREAMING-01
Traced from	FRE-COMPAT.CONCURRENT-01 FDS-STORAGE.PURGE-01 FRE-COMPLIANCE.READERISOLATION-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01
Verified by	TC-COMPLIANCE.REPLACE-01

[FRE-COMPLIANCE.READERISOLATION-01]

Readers survive replace/purge ⟨⟨*functional requirement*⟩⟩ `draft`

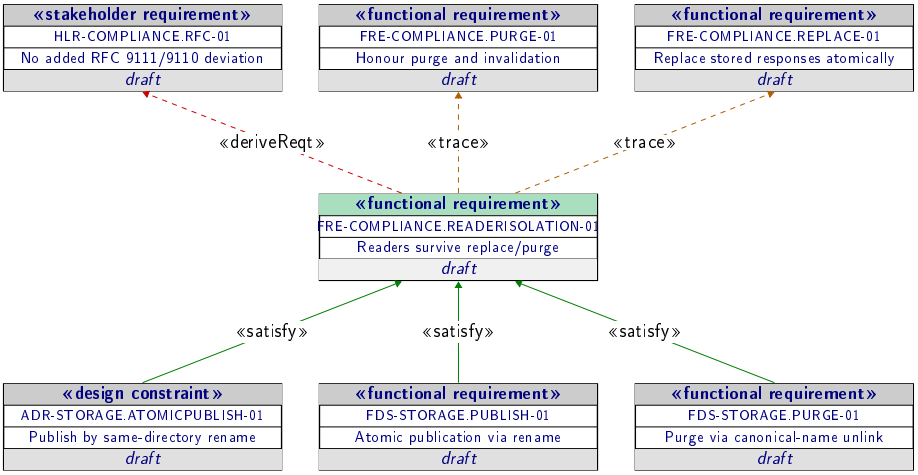


Figure 5.37: Readers survive replace/purge

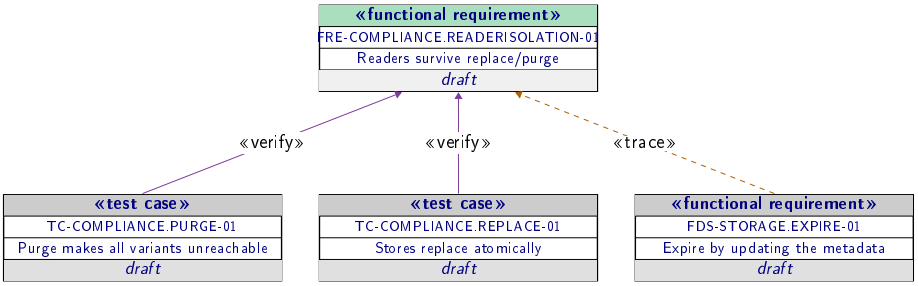


Figure 5.37: Readers survive replace/purge (cont.)

WHEN the object a reader is streaming is replaced or purged, that reader shall deliver only that object's own bytes, ending either at that object's normal end of body or with an error.

A reader never crosses into the bytes of the replacing object, of another object, or of a torn record. Delivering only the object's own bytes implies that the storage the reader reads from is not reused for anything else while the reader still references it; the design decides how (FDS-STORAGE.PUBLISH-01, FDS-STORAGE.PURGE-01).

Basis: RFC 9111 Section 4.4 (invalidating stored responses); RFC 9111 Section 3 (storing responses).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.REPLACE-01 FRE-COMPLIANCE.PURGE-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01
Traced from	FDS-STORAGE.EXPIRE-01
Verified by	TC-COMPLIANCE.PURGE-01 TC-COMPLIANCE.REPLACE-01

[FRE-COMPLIANCE.VARY-01]

Store and select Vary variants ⟨⟨*functional requirement*⟩⟩ `draft`

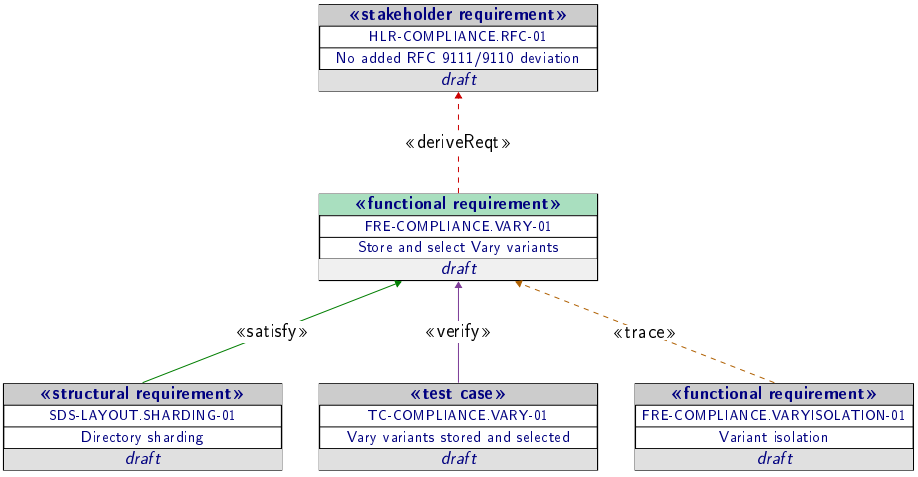


Figure 5.38: Store and select Vary variants

WHEN `pingora-cache` looks up a key, the storage backend shall return the object stored under exactly that key — primary slot or variant slot — or report a miss when none is stored there.

The backend thus holds several response variants for the same primary cache key, each under the variance-augmented cache key `pingora-cache` presents, and never serves one slot's object in place of another's, so that `Vary`-based content negotiation is honoured. The variance is derived from the request fields named by the response's `Vary` header and folded into the cache key *before* the backend is called; the backend never sees the request itself. Variants persist across a restart like any other stored object (HLR-PERSISTENCE.DURABLE-01); how mutating one variant affects the others is stated in FRE-COMPLIANCE.VARYISOLATION-01.

Basis: RFC 9111 Section 4.1 (calculating cache keys with Vary); RFC 9110 Section 12.5.5 (Vary).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traced from	FRE-COMPLIANCE.VARYISOLATION-01
Satisfied by	SDS-LAYOUT.SHARDING-01
Verified by	TC-COMPLIANCE.VARY-01

[FRE-COMPLIANCE.VARYISOLATION-01]

Variant isolation ⟨⟨*functional requirement*⟩⟩ `draft`

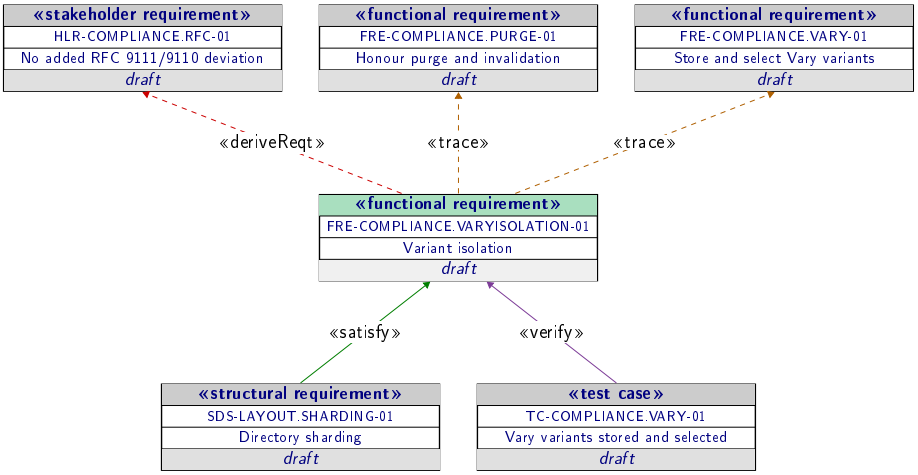


Figure 5.39: Variant isolation

WHEN one variant of a primary key is stored, replaced, freshened, or purged, the storage backend shall leave every other variant of that primary key unchanged.

A purge addressed to the *primary* slot is the deliberate exception: it makes every variant unreachable (FRE-COMPLIANCE.PURGE-01).

Basis: RFC 9111 Section 4.1 (calculating cache keys with Vary).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.VARY-01 FRE-COMPLIANCE.PURGE-01
Satisfied by	SDS-LAYOUT.SHARDING-01
Verified by	TC-COMPLIANCE.VARY-01

[FRE-COMPLIANCE.PURGE-01]

Honour purge and invalidation ⟨⟨*functional requirement*⟩⟩ `draft`

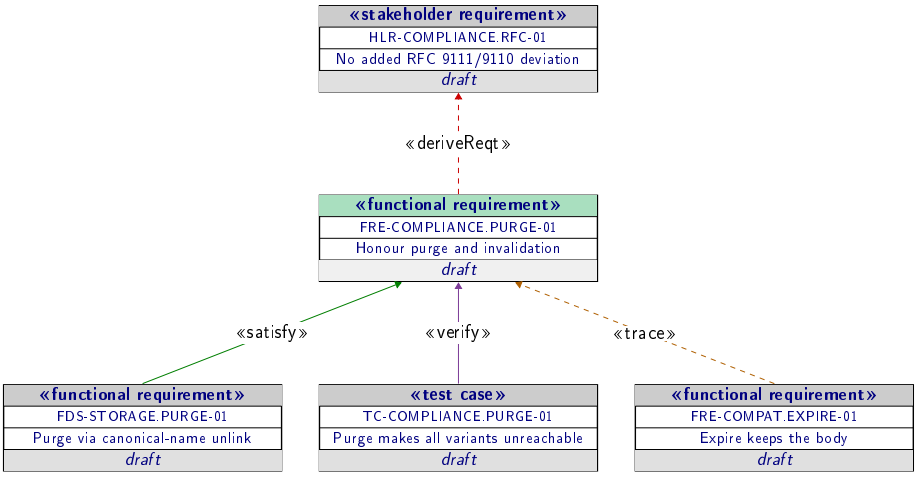


Figure 5.40: Honour purge and invalidation

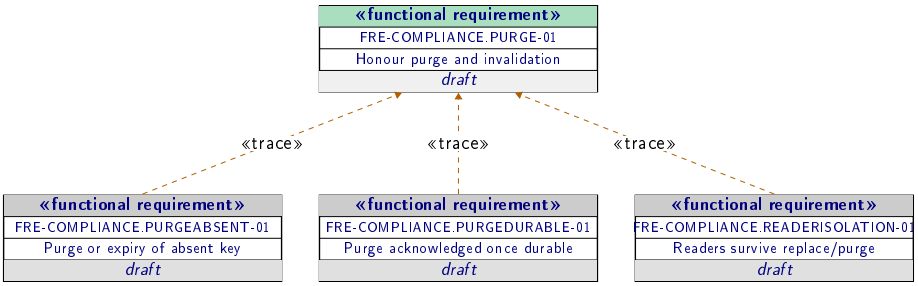


Figure 5.40: Honour purge and invalidation (cont.)

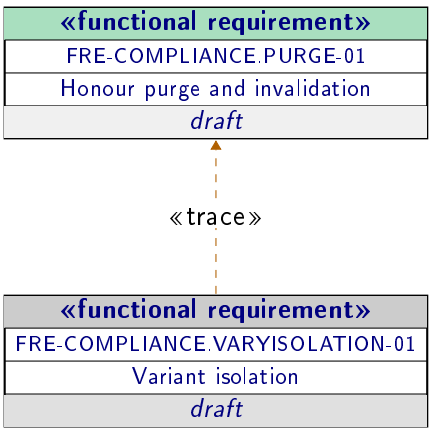


Figure 5.40: Honour purge and invalidation (cont.)

WHEN the storage backend applies an invalidating purge of a key, it shall serve neither the response stored under that key, nor any store of that key in flight when the purge arrives, nor — for a primary key — any of its `Vary` variants on any later lookup, until a store that begins after the purge publishes a new object under that key.

The purge request path `pingora-proxy` drives addresses a purge to the *primary* slot of an asset, and the variant lookups in `pingora-cache` consult the primary slot first; removing the primary slot therefore acts as the tombstone of the whole variant set, and the backend is not handed — and cannot derive — the keys of the individual variants. Such orphaned variant records stay in storage, counted against the cache's size budget like any other object, until eviction removes them (FRE-COMPAT.EVICTION-01). A same-key store in flight when the purge arrives may carry the very content the purge removes, if it began before the change that prompted the purge reached the origin, so the purge ends it as well: later lookups pass over it, and it publishes nothing when it completes. The request writing it still reads its own body back, and a reader already streaming it completes, as one streaming a purged record does (FRE-COMPLIANCE.READERISOLATION-01). The `pingora-cache` contract would let such a store land,[58] but its in-memory backend removes a key's stores in flight together with its stored response,[59] and an invalidation is the case where landing is wrong. A purge made to evict leaves stores in flight alone: it is not an invalidation. `pingora-cache` itself invalidates only on an explicit purge request from the proxy application — it does not invalidate on unsafe request methods, leaving that RFC 9111 duty to the application.

A purge of a key holding nothing is stated in FRE-COMPLIANCE.PURGEABSENT-01, when a purge may be acknowledged in FRE-COMPLIANCE.PURGEDURABLE-01, and what a reader streaming the purged object receives in FRE-COMPLIANCE.READERISOLATION-01.

Basis: RFC 9111 Section 4.4 (invalidating stored responses); RFC 9110 Section 9.2.1 (safe methods).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traced from	FRE-COMPAT.EXPIRE-01 FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPLIANCE.VARYISOLATION-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01
Satisfied by	FDS-STORAGE.PURGE-01
Verified by	TC-COMPLIANCE.PURGE-01

[58] `Storage::expire`'s doc calls a store that commits after an expiry "the same race `Storage::purge` has", to be "resolved by whatever serializes writes for that key".

[59] `MemCache::purge` removes the key from `self.temp`, its stores in flight, as well as from `self.cached`.

[FRE-COMPLIANCE.PURGEABSENT-01]

Purge or expiry of absent key ⟨⟨*functional requirement*⟩⟩ `draft`

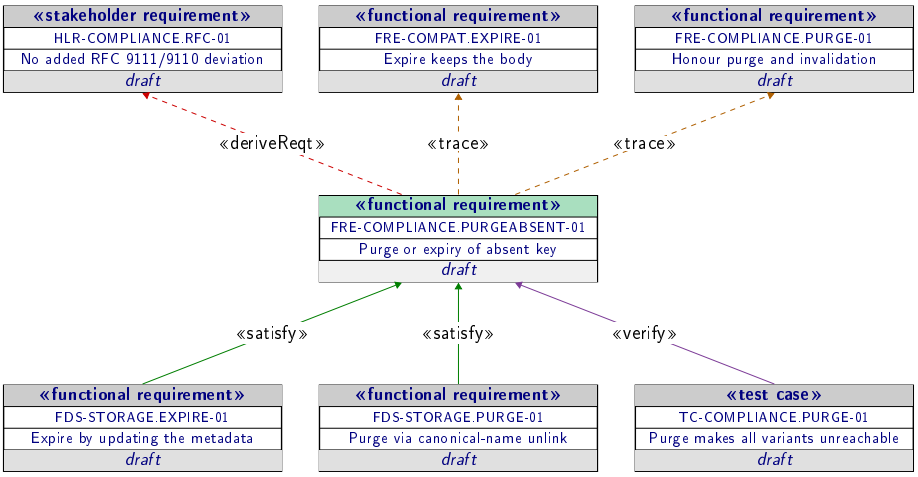


Figure 5.41: Purge or expiry of absent key

IF a purge or an expiry addresses a key under which no object is stored, THEN the storage backend shall report that nothing was found, with no error and no change to any other stored object.

The case covers a key never stored and a key purged or expired a second time, so both operations are idempotent. FRE-COMPAT.EXPIRE-01 states what an expiry does when the key *does* hold an object.

Basis: RFC 9111 Section 4.4 (invalidating stored responses).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.PURGE-01 FRE-COMPAT.EXPIRE-01
Satisfied by	FDS-STORAGE.PURGE-01 FDS-STORAGE.EXPIRE-01
Verified by	TC-COMPLIANCE.PURGE-01

[FRE-COMPLIANCE.PURGEDURABLE-01]

Purge acknowledged once durable ⟨⟨*functional requirement*⟩⟩ `draft`

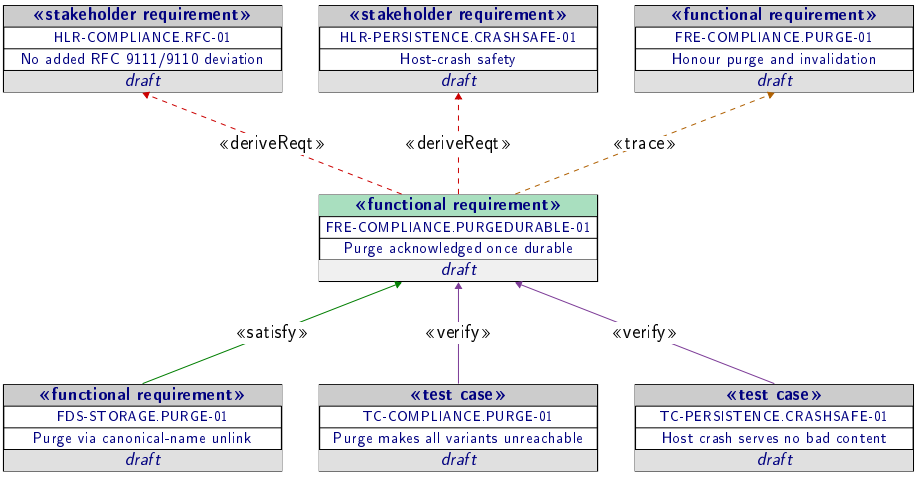


Figure 5.42: Purge acknowledged once durable

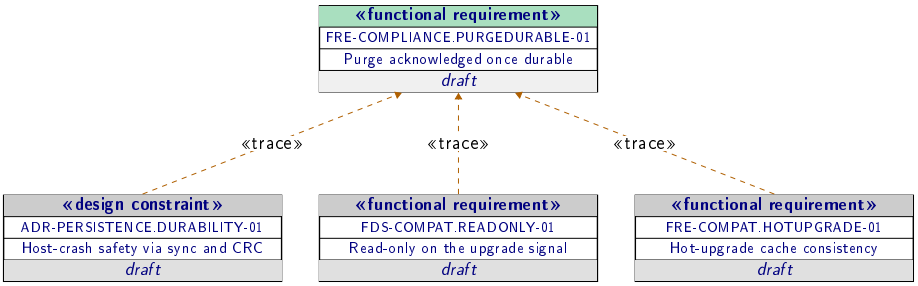


Figure 5.42: Purge acknowledged once durable (cont.)

The storage backend shall report the outcome of an invalidating purge — an object removed, or none found — only once the key's absence will survive a crash or restart of the backend's process or host.

An invalidation the backend will not make durable — as after its process has received the Pingora upgrade signal (FRE-COMPAT.HOTUPGRADE-01) — therefore fails with an error rather than being reported as having removed something or nothing. An invalidation that found nothing is no exception: the key's record may have been removed just before by a purge made to evict, which nothing has made durable, and a crash undoing that removal would bring the object back after the invalidation reported it absent.

A purge made to evict (`PurgeType::Eviction`) is outside this requirement. No one asked for the object's removal: the cache chose to drop it for space, and an evicted object back after a crash is still a valid response, which the cache may serve or evict again. It is not a purge in HLR-PERSISTENCE.CRASHSAFE-01's sense.

This requirement derives from two parents: HLR-COMPLIANCE.RFC-01 for invalidation semantics, which a purged object reappearing would undo, and HLR-

PERSISTENCE.CRASHSAFE-01 for the durability of an acknowledged purge across a host crash.

Basis: RFC 9111 Section 4.4 (invalidating stored responses).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01 HLR-PERSISTENCE.CRASHSAFE-01
Traces to	FRE-COMPLIANCE.PURGE-01
Traced from	FRE-COMPAT.HOTUPGRADE-01 ADR-PERSISTENCE.DURABILITY-01 FDS-COMPAT.READONLY-01
Satisfied by	FDS-STORAGE.PURGE-01
Verified by	TC-COMPLIANCE.PURGE-01 TC-PERSISTENCE.CRASHSAFE-01

[FRE-COMPLIANCE.INCOMPLETE-01]

Never serve incomplete responses ⟨⟨*functional requirement*⟩⟩ `draft`

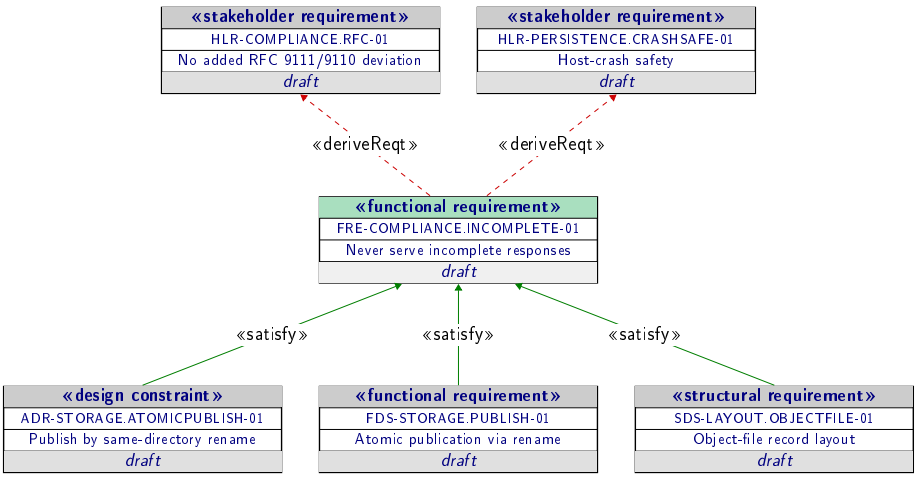


Figure 5.43: Never serve incomplete responses

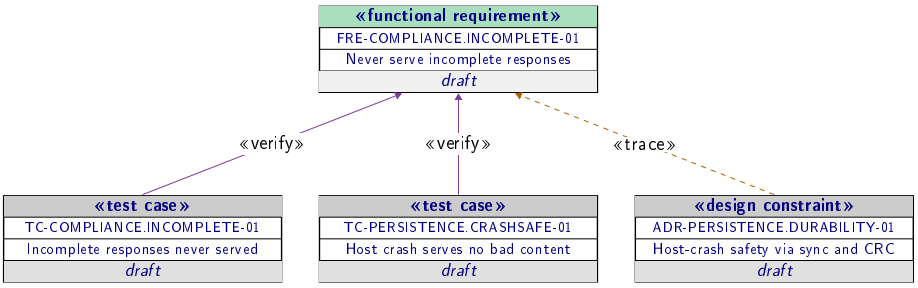


Figure 5.43: Never serve incomplete responses (cont.)

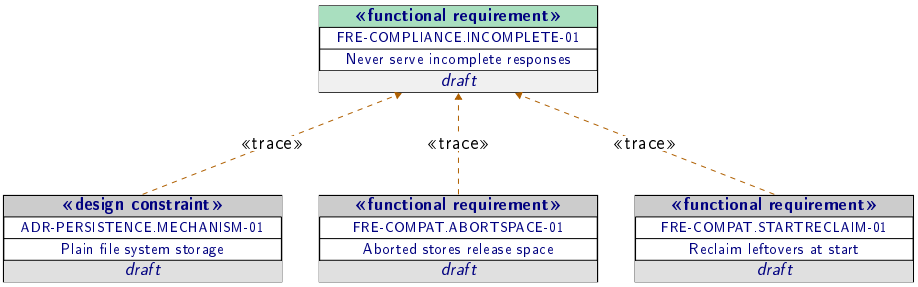


Figure 5.43: Never serve incomplete responses (cont.)

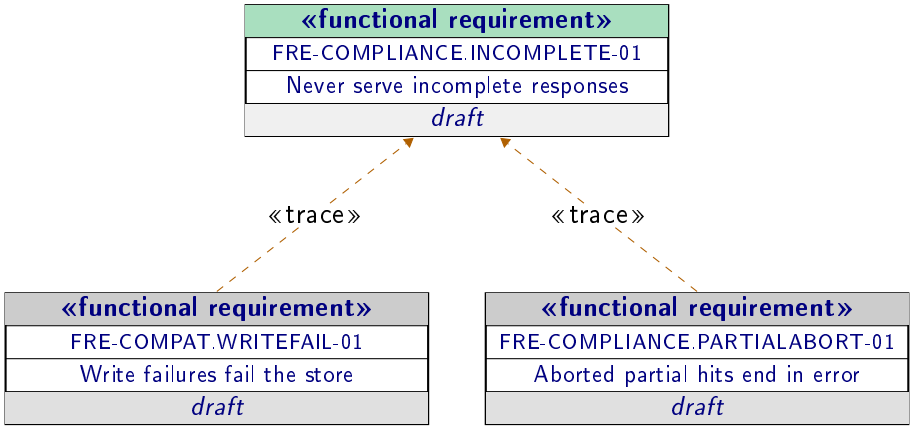


Figure 5.43: Never serve incomplete responses (cont.)

IF a store does not complete — it aborts, or its process or host crashes first — or a host crash occurs before a completed store's content has reached stable storage, THEN the storage backend shall serve that store's object on no later lookup, in the writing process or any later one.

A lookup that would have returned the object reports a miss instead, or — when the key already held an object — returns that earlier object (FRE-COMPLIANCE.REPLACE-01); truncated, torn, or otherwise corrupted content is never served as complete, and nothing such a store leaves on disk is promoted by a later process. The `Storage` interface gives `pingora-cache` no way to consume an "incomplete" record,

so a miss is the only compliant outcome. The backend does not combine partial content from 206 (`Partial Content`) or range responses, so it stores only complete representations and leaves any range handling to `pingora-cache`. What a reader already following the store receives when it aborts is stated in FRE-COMPLIANCE.PARTIALABORT-01, and the release of the storage such a store consumed in FRE-COMPAT.ABORTSPACE-01 and FRE-COMPAT.STARTRECLAIM-01.

This requirement derives from two parents: HLR-COMPLIANCE.RFC-01 for the RFC 9111 rule on storing incomplete responses, and HLR-PERSISTENCE.CRASHSAFE-01 for extending it to content a host crash leaves incomplete.

Basis: RFC 9111 Section 3.3 (storing incomplete responses), RFC 9111 Section 3.4 (combining partial content); RFC 9110 Section 14.2 (Range), RFC 9110 Section 6.1 (framing and completeness).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01 HLR-PERSISTENCE.CRASHSAFE-01
Traced from	FRE-COMPAT.WRITEFAIL-01 FRE-COMPAT.ABORTSPACE-01 FRE-COMPAT.STARTRECLAIM-01 ADR-PERSISTENCE.MECHANISM-01 ADR-PERSISTENCE.DURABILITY-01 FRE-COMPLIANCE.PARTIALABORT-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01 SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.INCOMPLETE-01 TC-PERSISTENCE.CRASHSAFE-01

[FRE-COMPLIANCE.PARTIALABORT-01]

Aborted partial hits end in error ⟨⟨*functional requirement*⟩⟩ `draft`

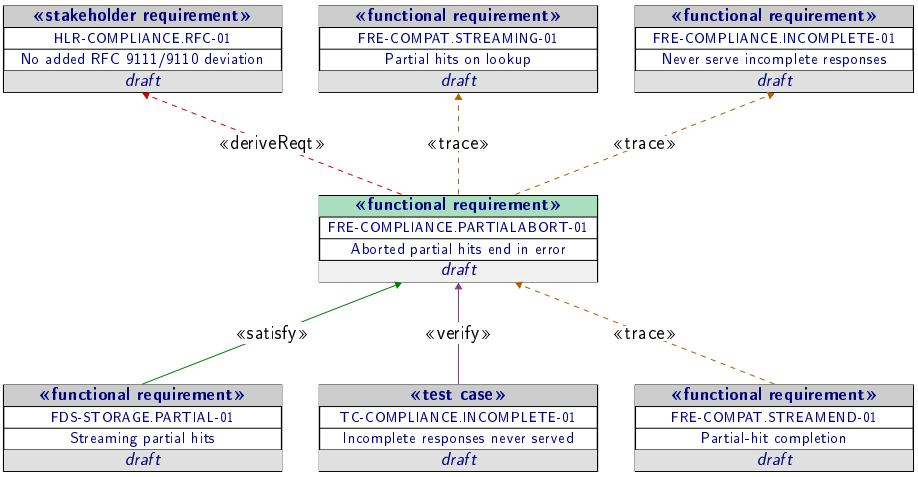


Figure 5.44: Aborted partial hits end in error

IF the in-flight store a partial hit's reader is attached to aborts, THEN that reader shall end its stream with an error, never with a normal end of body.

A truncated representation is thus never delivered to a follower as if it were complete. The abort may be an upstream failure, a cancellation, or a write failure (FRE-COMPAT.WRITEFAIL-01); the partial hit itself is stated in FRE-COMPAT.STREAMING-01, and its normal end on completion in FRE-COMPAT.STREAMEND-01.

Basis: RFC 9111 Section 3.3 (storing incomplete responses); RFC 9110 Section 6.1 (framing and completeness).

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩
Derives from	HLR-COMPLIANCE.RFC-01
Traces to	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPAT.STREAMING-01
Traced from	FRE-COMPAT.STREAMEND-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPLIANCE.INCOMPLETE-01

6. Structural and Functional Design

[SDS-LAYOUT.SHARDING-01]

Directory sharding ⟨⟨*structural requirement*⟩⟩ `draft`

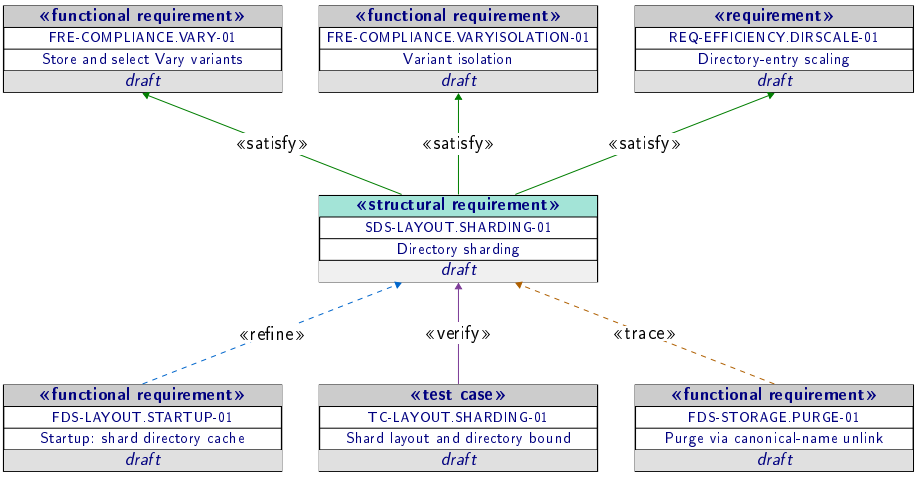


Figure 6.1: Directory sharding

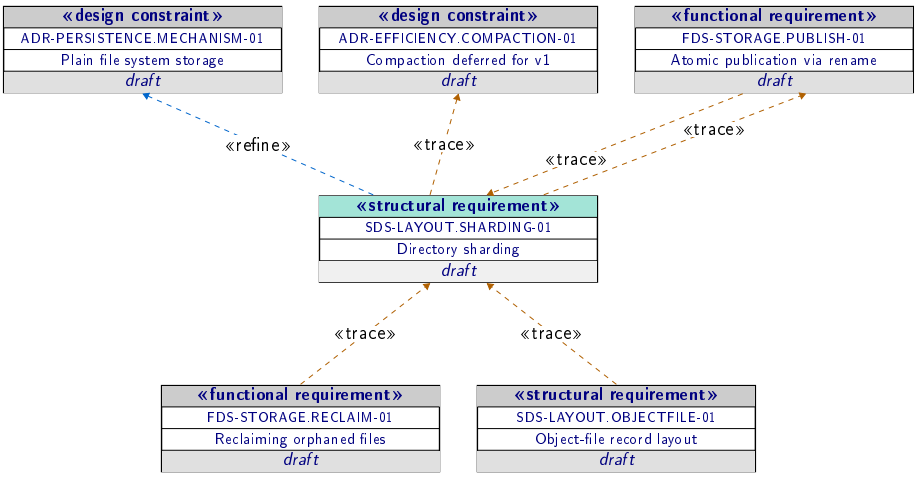


Figure 6.1: Directory sharding (cont.)

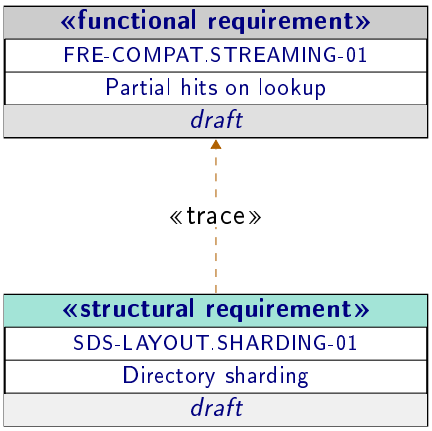


Figure 6.1: Directory sharding (cont.)

Because no directory may hold more than 2,000 entries for a cache of up to 10^8 objects (REQ-EFFICIENCY.DIRSCALE-01), each cache object is stored in a file named by the hexadecimal *combined hash*[1] of the object's cache key (or, for a key with a user tag, by the hash described under *User tags* below), spread across a sharded directory tree of two levels rather than left in one flat directory: the first-level directory is named after the first and second characters of the hash, and the second-level directory after the third and fourth characters, with the object file residing inside that second-level directory — 256 first-level directories, each holding up to 256 second-level directories, for 65,536 leaf directories in total. Splitting the hash this way, rather than keying a single level by all four characters, keeps every directory in the tree — including the first-level ones — at the same small entry count; a single level with the same 65,536-bucket fan-out would instead concentrate all 65,536 buckets as entries in one top-level directory, itself the kind of oversized directory this scheme exists to avoid. The combined hash and the user tag are both available from the full `CacheKey` presented on lookup and store and from the `CompactCacheKey` presented on purge, so every `Storage` operation can derive an object's file — and its shard — from the key form it receives alone, with no index lookup or directory scan needed.

User tags. A key's `user_tag` is part of an object's identity, as the `pingora-cache` eviction managers count it: they key their entries by the whole `CompactCacheKey`, whose `Hash` and `Eq` cover the tag,[2] while the combined hash leaves the tag out.[3] A key with a non-empty tag is therefore named by the Blake2b-128 hash — the hash `pingora-cache` names keys by — of its combined hash followed by the tag, in hexadecimal; a key with an empty tag keeps its combined hash as its name. Either way the name is 32 hexadecimal characters, and keys that differ only in their tag are stored, evicted, and purged as separate objects.

For example, an object whose cache key hashes to `a1b2c3d4e5f6...` is stored at the path `a1/b2/a1b2c3d4e5f6...`: the first-level shard `a1` comes from characters one and two, and the second-level shard `b2` from characters three and four.

[1] `CacheHashKey::combined`.
[2] `CompactCacheKey` derives both over the primary hash, the variance, and the tag.
[3] `CacheKey::new`: "Only the `primary` key will be hashed to produce the primary cache hash."

Figure 6.2 illustrates the layout.

While a store is in flight, the object is written in that same second-level directory as a per-writer temporary, named per FDS-STORAGE.PUBLISH-01 (e.g. `a1/b2/a1b2c3d4e5f6....4711-6fd5a44b00000001.17.tmp` for store 17 of the backend instance holding the lease `writers/4711-6fd5a44b00000001.lock`); the concluding atomic rename to the canonical name drops the suffix. While it grows, that temporary is also the read source for same-process streaming partial hits (FRE-COMPAT.STREAMING-01). Every object resides in this sharded tree for its whole lifetime, since compaction into aggregate files is not implemented for this version (ADR-EFFICIENCY.COMPACTION-01).

Meeting the bound. The fan-out is fixed, so a leaf directory's entry count grows with the cache rather than being capped: with N objects hashed uniformly, each leaf holds $N/65{,}536$ objects on average, with a standard deviation of about $\sqrt{N/65{,}536}$. At the 10^8-object capacity of REQ-EFFICIENCY.DIRSCALE-01 that is about 1,526 entries per leaf ($\sigma \approx 39$); the 2,000-entry bound sits more than twelve standard deviations above the mean, leaving room for the transient per-writer temporaries that share a leaf with the canonical names. First-level directories hold at most 256 entries each. Beyond that capacity the per-leaf count keeps growing with the cache; the bound is a design target at the stated size, not a guarantee at every size.

Trade-offs. Spreading objects this way keeps per-directory counts small and lookups fast up to that capacity, at the cost of intermediate directories that must exist before an object's file can be created inside them — FDS-LAYOUT.STARTUP-01 keeps that existence check cheap via a cache built at construction, and a shard directory, once created, is never removed — and object access that traverses two path components instead of one. The spread is only as uniform as the cache key's hash distribution; a skewed key space can leave some shards much larger than others.

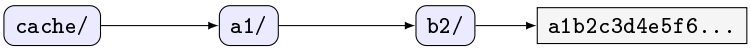


Figure 6.2: Sharded path for a cache key whose combined hash is `a1b2c3d4e5f6....`

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	REQ-EFFICIENCY.DIRSCALE-01 FRE-COMPLIANCE.VARY-01 FRE-COMPLIANCE.VARYISOLATION-01
Traces to	FDS-STORAGE.PUBLISH-01 ADR-EFFICIENCY.COMPACTION-01 FRE-COMPAT.STREAMING-01
Traced from	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 FDS-STORAGE.RECLAIM-01 SDS-LAYOUT.OBJECTFILE-01
Refined by	FDS-LAYOUT.STARTUP-01
Verified by	TC-LAYOUT.SHARDING-01
Satisfied by	IMP-LAYOUT.SHARDING-01 (durable-cache/src/sharding.rs:20)

[FDS-LAYOUT.STARTUP-01]

Startup: shard directory cache ⟨⟨*functional requirement*⟩⟩ `draft`

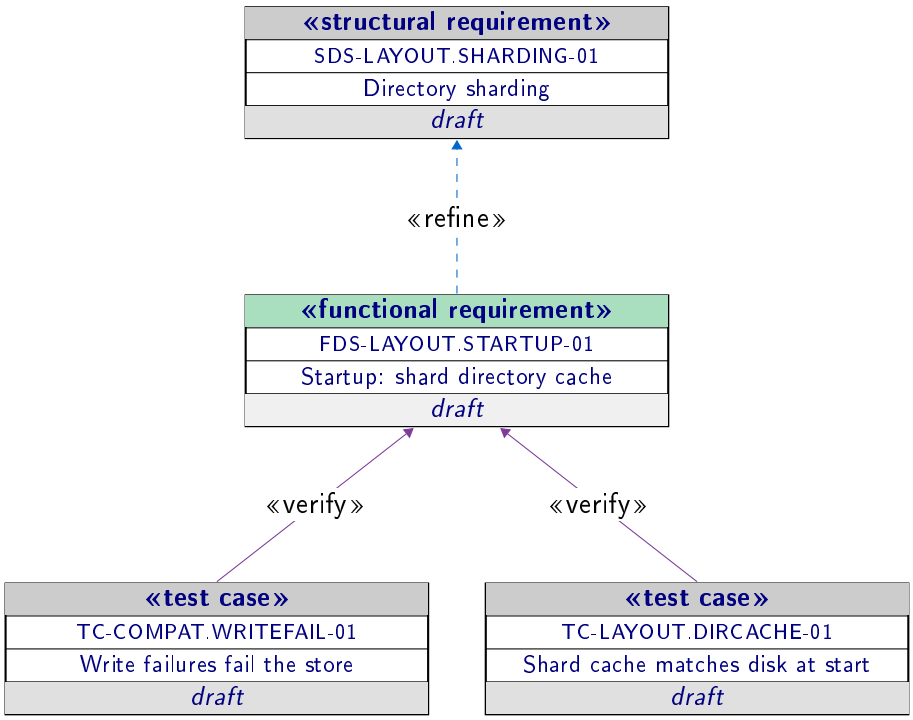


Figure 6.3: Startup: shard directory cache

Two lifecycle moments, one cache. The sharded tree of SDS-LAYOUT.

SHARDING-01 needs every shard directory to exist before an object's file can be created inside it. Checking that on every store, one file-system call at a time, is unnecessary: the tree is small and finite, so its existence can be established once, in memory, at construction, and consulted cheaply thereafter for the rest of the process's life, without depending on any state surviving a restart.

At construction. Before the backend is handed to `HttpCache` as its `Storage` implementation, it shall ensure every first-level directory exists, creating any that are missing, and then, for each first-level directory, list its existing second-level subdirectories and record in an in-memory cache those named as SDS-LAYOUT.SHARDING-01 names shards — exactly two lowercase hexadecimal digits — since anything else in the tree is not the backend's. This work is proportional only to the number of first- and second-level directories actually present — at most 256 directory creations and 256 directory listings — never to the number of objects the cache holds, unlike FDS-EVICTION.STARTUP-01's per-object scan. Because the cache is rebuilt this way at every start, it never depends on anything surviving a restart or a hot upgrade (FRE-COMPAT.HOTUPGRADE-01).

On each store. A store already knows which shard its object belongs to (SDS-LAYOUT.SHARDING-01); the first-level directory is guaranteed to exist by construction, so only the second-level directory needs a decision. The backend shall consult the in-memory cache before creating an object's file: if the shard is already known, it proceeds directly; if not, it creates the second-level directory — tolerating a concurrent store that reaches the same new shard first — and adds it to the cache before proceeding. Because SDS-LAYOUT.SHARDING-01 never removes a shard directory once created, the cache is purely additive: entries are only ever added, so none ever needs to be invalidated or re-checked against the file system. Something outside the backend may still remove a directory: IF creating a store's temporary fails because a shard directory the cache records as existing is missing, THEN the backend shall create that directory again and retry the creation once, and return a second failure as the store's error (FRE-COMPAT.WRITEFAIL-01).

Concurrency. `Storage` is called from multiple threads at once (FRE-COMPAT.CONCURRENT-01), so the cache shall be safe for concurrent reads and inserts. Inserting an already-present shard shall be a harmless no-op — two stores racing to create the same new shard both succeed, and both insertions land safely. The whole second-level namespace is small enough to track exactly, without approximation: a bitset over all 65,536 possible shards (SDS-LAYOUT.SHARDING-01) needs only 8 KiB.

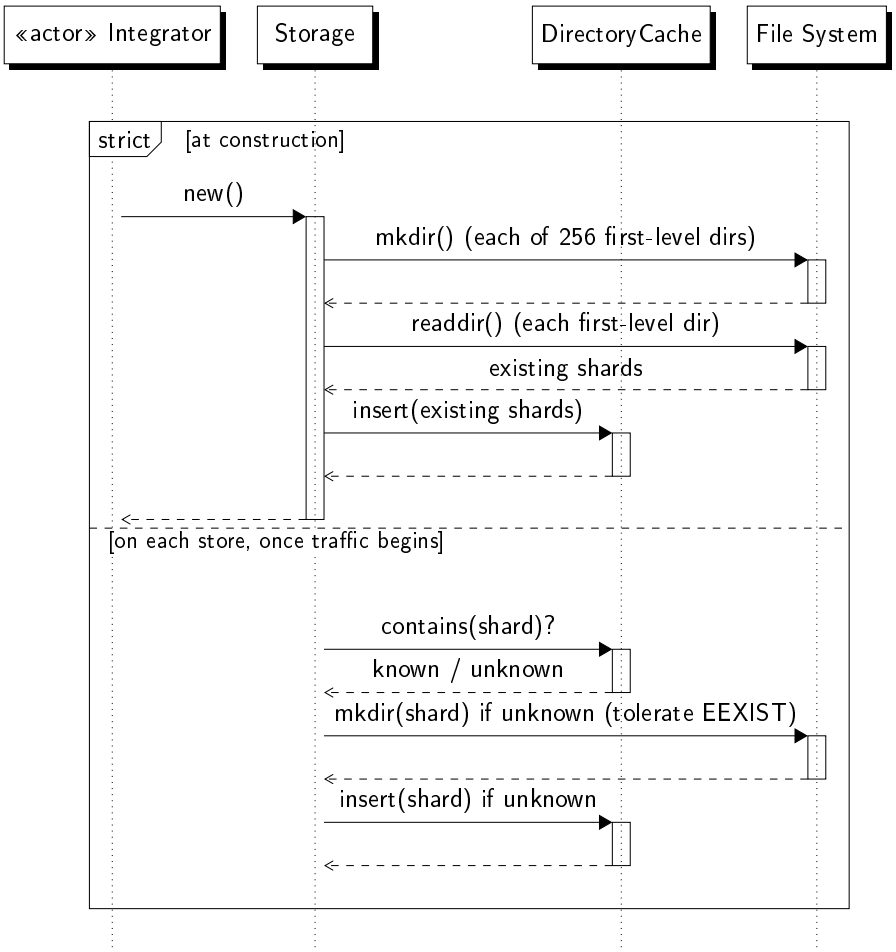


Figure 6.4: Shard-directory cache: built once at construction by scanning the existing tree, then consulted (and grown) on each store. The *strict* fragment marks its two regions as strictly sequential — construction always completes before the backend is handed to `HttpCache`.

View complete diagram.

Required coverage	✓ TC-LAYOUT.DIRCACHE-01 via ⟨⟨*verify*⟩⟩ — *Verify the cache is rebuilt accurately from disk at every construction, and that concurrent first-time stores into the same shard never fail or double-create.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	SDS-LAYOUT.SHARDING-01
Verified by	TC-COMPAT.WRITEFAIL-01 TC-LAYOUT.DIRCACHE-01
Satisfied by	IMP-LAYOUT.STARTUP-01 (durable-cache/src/dircache.rs:37)

[SDS-LAYOUT.OBJECTFILE-01]

Object-file record layout ⟨⟨*structural requirement*⟩⟩ `draft`

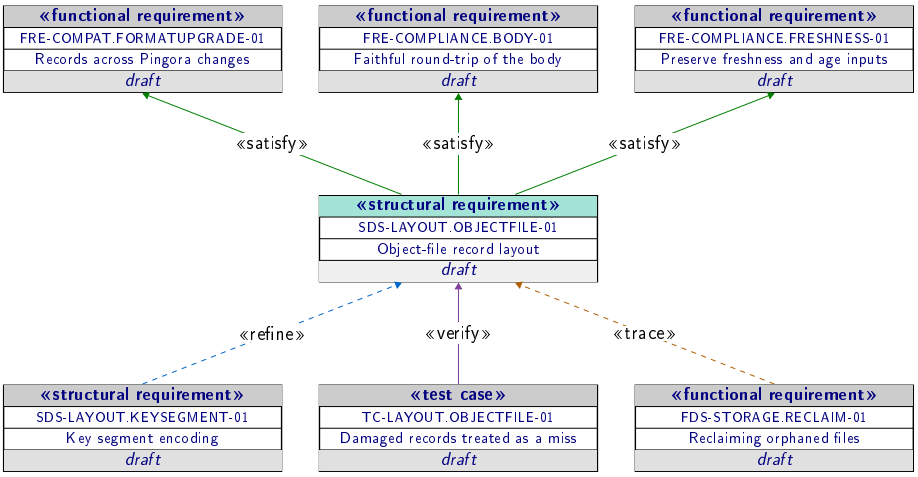


Figure 6.5: Object-file record layout

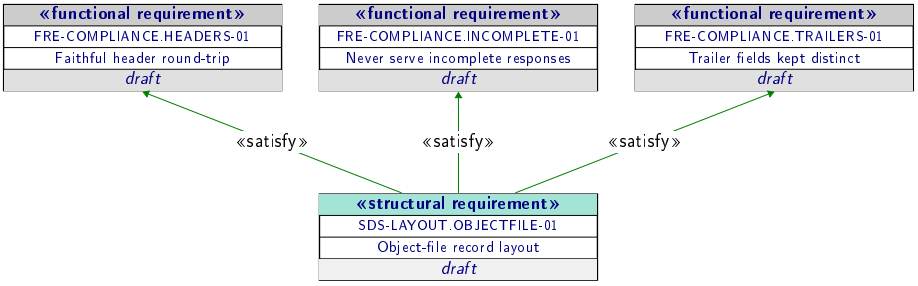


Figure 6.5: Object-file record layout (cont.)

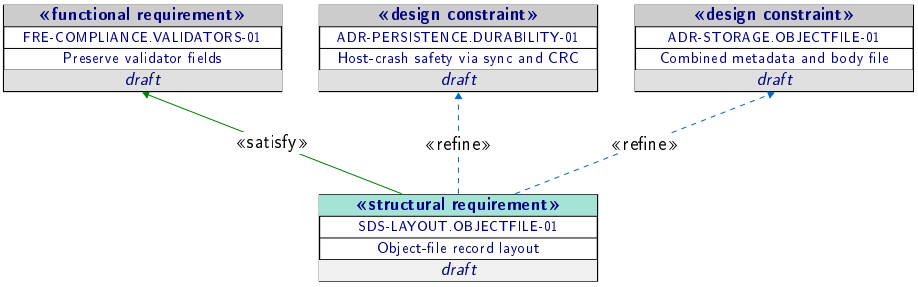


Figure 6.5: Object-file record layout (cont.)

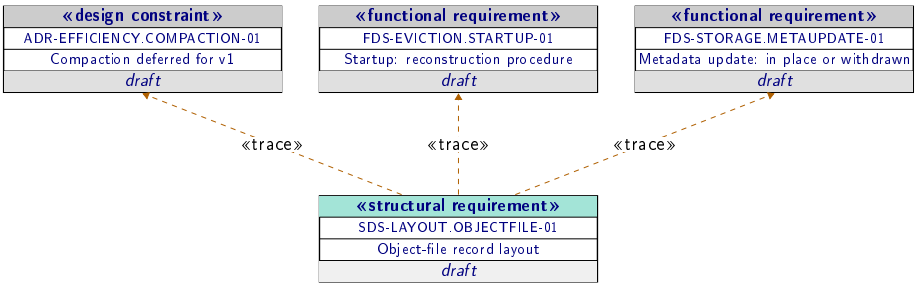


Figure 6.5: Object-file record layout (cont.)

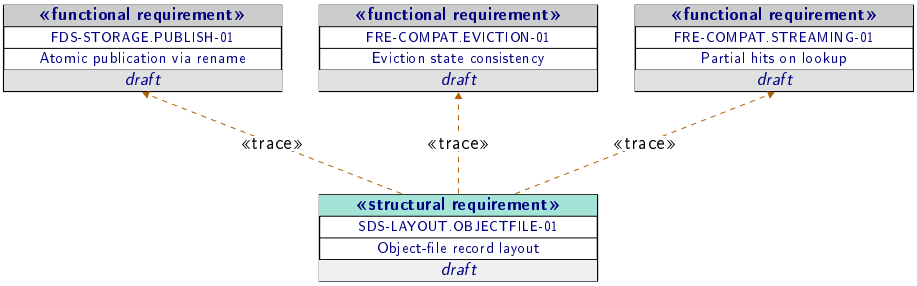


Figure 6.5: Object-file record layout (cont.)

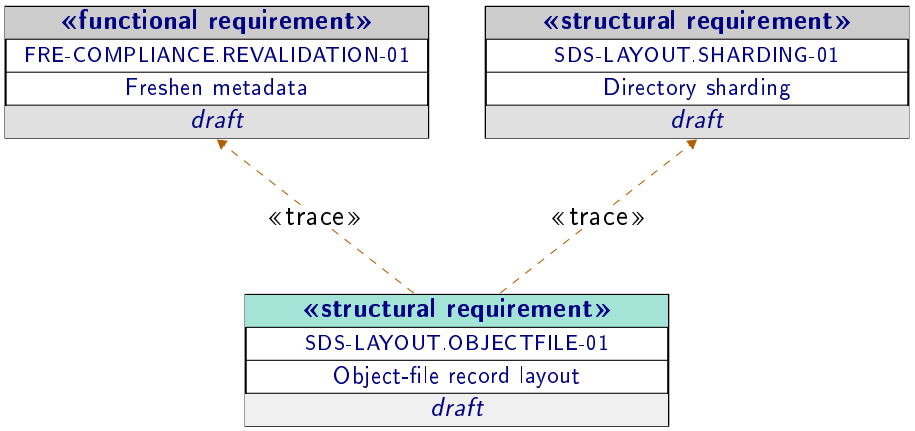


Figure 6.5: Object-file record layout (cont.)

Each cache object is persisted as one self-describing *record* — the internal layout of the single file of ADR-STORAGE.OBJECTFILE-01. The record has four parts, in file order (Figure 6.7a): a fixed-size *record header*, the object's serialised `CompactCacheKey`[4], a *metadata region* of recorded capacity holding the object's two metadata segments plus free headroom for their growth, and the body bytes.

Metadata exactly as `pingora-cache` hands it over. A `CacheMeta` serialises to two opaque byte segments — the *internal metadata* (the framework's created/updated instants, freshness deadline, and variance hash,

[4] `struct CompactCacheKey`.

MessagePack-encoded) and the *header metadata* (status, protocol version, and header fields in the framework's own header encoding) — and is rebuilt by `CacheMeta::deserialize(internal, header)`; the stock `MemCache` keeps precisely this pair per object. The record stores the same two segments byte for byte and never parses or edits them, so the round-trip fidelity duties on header fields and freshness inputs (FRE-COMPLIANCE.HEADERS-01, FRE-COMPLIANCE.FRESHNESS-01) are met by construction: lookup returns what store received.

Record header: one aligned block. The record header is a single 44-byte block at offset 0, written and read as one unit: every field is a fixed-width little-endian integer at a naturally aligned offset (the u64 at offset 8, every other field a u32 at a multiple of four), so the block is saved by writing an in-memory `#[repr(C)]` struct's bytes verbatim and read back by reading the block straight into that struct — no field-by-field parsing, and byte swapping only on a big-endian host, as each field is read or written. No byte is spent on alignment filler: `version` and `key_len` are each a u32 rather than the u16 their own range would need, so they close the gap that natural alignment would otherwise leave before `body_len` and `internal_len` respectively, instead of leaving it as unnamed padding. A future format revision that needs more header fields is free to define a different layout under a new `version`; nothing here is reserved against that in advance. Figure 6.7b shows the byte placement; the fields are:

- `magic` (u32, offset 0) and `version` (u32, offset 4): identify the record format and its revision, so a scan can recognise records and a later revision can evolve the layout. `magic` is the four bytes `PDCR` (the little-endian value `0x52434450`); this layout is `version` 1. The version is the backend's own: it changes whenever records written before would no longer be read back exactly — a change to this layout, or a `pingora-cache` release whose metadata encoding differs (FRE-COMPAT.FORMATUPGRADE-01) — and a reader treats a record of any other version as a miss. Version 0 marks a record *withdrawn* because its metadata outgrew the region (below), and is never a format version.

- `body_len` (u64, offset 8): the body's exact byte length, patched at store completion.

- `meta_capacity` (u32, offset 16): the byte size of the metadata region as allocated in *this* record.

- `key_len` (u32, offset 20): the length of the serialised `CompactCacheKey`, which is not constant across records: only `primary` (16 bytes) is fixed-size,[5] while `variance`[6] is present or absent per key and `user_tag`[7] is an arbitrary-length string; `key_len` is what lets a reader locate the metadata region without assuming a size. The key segment's encoding is SDS-LAYOUT.KEYSEGMENT-01's.

- `internal_len` (u32, offset 24) and `header_len` (u32, offset 28): the current lengths of the two metadata segments.

[5] `primary: HashBinary` is a `[u8; 16]`.

[6] `variance: Option<Box<HashBinary>>`: absent for a key with no `Vary` variant, 16 bytes when present.

[7] `user_tag: Box<str>`, "the len should be small to keep memory usage bounded" per its own source comment — guidance, not a fixed width.

- `meta_crc` (u32, offset 32): CRC-32C over the key and the two metadata segments.
- `body_crc` (u32, offset 36): CRC-32C over the body's `body_len` bytes, recorded at store completion (ADR-PERSISTENCE.DURABILITY-01); zero in a build without the `body-crc` feature. A reader never checks a stored zero.
- `header_crc` (u32, offset 40): CRC-32C over header bytes 0–39.

Every checksum in the record uses CRC-32C (the Castagnoli polynomial).

Segment placement. The key segment starts immediately after the header, at offset 44, and the metadata region immediately after the key, with no padding: every segment is read into its own buffer, never interpreted in place, so none needs alignment. The region holds the internal and header segments back-to-back, followed by free headroom. Its capacity is fixed when the record is created — the combined size of the segments actually presented (there is no fixed cap, so an arbitrarily large header set simply yields a larger region, FRE-COMPLIANCE.HEADERS-01) plus the configured growth headroom (FDS-STORAGE.METAUPDATE-01), a deployment setting defaulting to 512 bytes. The capacity is *recorded* in `meta_capacity`, so a reader computes the body offset (44 + `key_len` + `meta_capacity`) from the record alone and records written under different configurations coexist. The body follows, to the end of the file.

Why the key is stored. The file is *named* by the combined hash (SDS-LAYOUT.SHARDING-01), which cannot be inverted back to the key, while eviction admissions and purges are keyed by the `CompactCacheKey`.[8] The key segment is what lets the startup scan of FDS-EVICTION.STARTUP-01 recover `admit(key, size, fresh_until)` for every persisted entry — the key from the key segment, the size as the record's file length (the record size FRE-COMPAT.EVICTION-01 defines), the freshness deadline from the deserialised metadata — reading only the header, key, and metadata region, never the body (FRE-COMPAT.EVICTION-01). It also lets any reader confirm a record matches the key it looked up, catching a wrong-shard read caused by a software defect or on-disk corruption, and lets the restart scan reclaim a record found anywhere but at its own key's path (FDS-STORAGE.RECLAIM-01).

A weaker collision guard than it looks. That confirmation is not a defence against a genuine hash collision for every key: a key with neither a variance nor a user tag has `CompactCacheKey`'s `primary` field exactly equal to the combined hash that named its file (SDS-LAYOUT.SHARDING-01), so two different keys colliding there would carry identical stored keys too and go undetected. Only a varianced key gets a real check, because its stored `primary` and `variance` survive as the separate values they were before `combined_bin` folded them together (SDS-LAYOUT.SHARDING-01) — a collision in that folding step is independent of a collision in either value alone. No comparable cache pays for a stronger guarantee across the board: the stock `MemCache` trusts the combined hash as its map key with no comparison at all,[9] and Traffic Server's on-disk record stores only hashes of the key, never the

[8] `EvictionManager::admit` takes a `CacheEntryKey` and `Storage::purge` a `PurgeTarget`; both are thin wrappers that carry a `CompactCacheKey` and, for a backend that assigns them, an entry identifier this backend does not (ADR-COMPAT.ENTRYID-01). The key each needs is therefore the same `CompactCacheKey`.

[9] `MemCache::lookup` looks an object up by `key.combined()` alone — whatever is stored under that hash is returned unconditionally.

original URL, so its own read-time check is exactly as hash-bound as this one.[10] Keying storage on the full, unhashed request string instead — as Caddy's Souin-based cache module does[11] — would remove the question entirely, at the cost of storing, and comparing on every lookup, a full request string that is typically far longer than the 16-byte hashes this record carries.

Write, publication, and integrity. The header, key, and metadata region are written when the store's miss handler opens[12] — the `CacheMeta` is available then; body chunks are appended as they stream in, each folded into a running CRC-32C kept in memory unless the build leaves out the `body-crc` feature. While the per-writer temporary grows (SDS-LAYOUT.SHARDING-01), the valid length lives in the in-memory in-flight registry that backs streaming partial hits (FDS-STORAGE.PARTIAL-01); the on-disk header is *not* the source of truth for readers of an in-flight write. At completion the backend patches `body_len` and `body_crc`, recomputes `header_crc`, synchronises the file's data to stable storage, and renames the temporary to the canonical name (FDS-STORAGE.PUBLISH-01, ADR-PERSISTENCE.DURABILITY-01).

A reader accepts a published record only if, in this order: the magic and `version` match this layout's; `header_crc` verifies; the metadata segments fit within `meta_capacity`; the computed body offset lies within the file; `meta_crc` verifies (with the re-reads described under in-place updates below); the stored key equals the looked-up key; and the file's length is *exactly* the computed body offset plus `body_len`. The two bounds come before the reader sizes any read by a length the header carries, so no header, however forged, makes it allocate or read beyond the file. Any failure — a truncated file, a torn or foreign record — makes the lookup report a miss, so a damaged record is never served (FRE-COMPLIANCE.INCOMPLETE-01). The reader does not remove the file: a later store of the same key replaces it, and the restart-time scan reclaims it (FDS-STORAGE.RECLAIM-01), so a lookup never unlinks a path whose content may have changed since it opened it. A reader does not check the body against `body_crc`: integrity at rest is the storage layer's (ADR-PERSISTENCE.DURABILITY-01). A build with the `body-crc-verify` feature can switch a check on in its configuration: a reader that streams the whole body from its start then computes CRC-32C over the bytes it delivers and, at the end, compares it with `body_crc`; on a mismatch it logs an error naming the file and still ends the stream normally, since every byte has already been delivered. A published record supports seeking: its hit handler reports `can_seek`, and a seeked read delivers exactly the requested slice (FRE-COMPLIANCE.BODY-01) and is never checked, since the checksum covers only a whole body. Figure 6.6 shows the read path this validation belongs to.

[10] `first_key` and `key` are both a `CryptoHash`, never the request URL itself. Apache Traffic Server source (release 10.1.4): `src/iocore/cache/P_CacheDoc.h`, lines 45–49.

[11] Souin source (release v1.7.8): `context/key.go`, line 147 builds the storage key by concatenating the method, scheme, host, path, query, and varied header values verbatim, never folding them into a fixed-size hash.

[12] `Storage::get_miss_handler` hands the backend the `CacheMeta` directly, before any body bytes arrive.

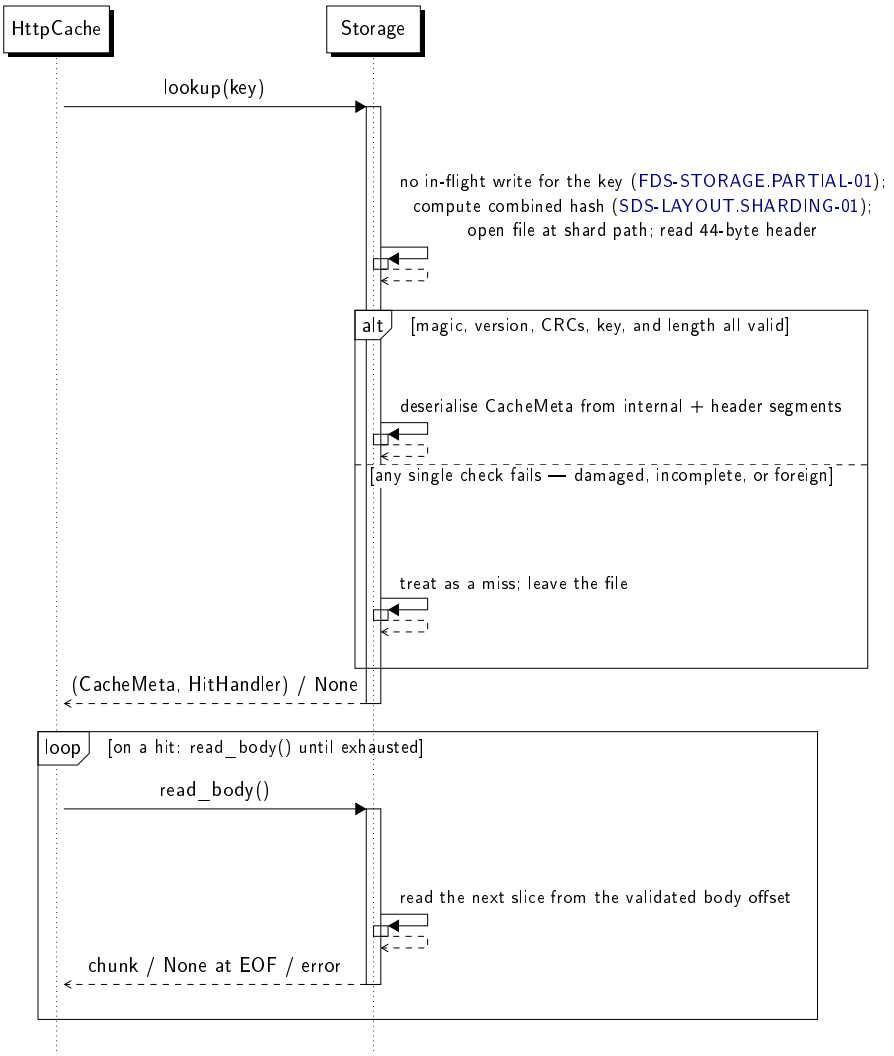


Figure 6.6: Lookup/read sequence for a published record (an in-flight write of the key is served first, as FDS-STORAGE.PARTIAL-01 specifies): the reader recomputes the file's own path from the key, opens it, and validates the record before trusting it. Any single validation failure — a bad magic or version, a CRC mismatch, a different stored key, or a length that is not exactly the computed body offset plus `body_len` — is indistinguishable from any other: the lookup reports a miss and leaves the file for a later store or the restart-time scan (FRE-COMPLIANCE.INCOMPLETE-01, verified by TC-LAYOUT.OBJECTFILE-01). On a hit, the already-validated file stays open and every `read_body` call simply advances through it; unlike Figure 5.6's concurrent `Follower`, this reader never races a body append, since the file it opened was already published at open time (an in-place metadata update may still race the metadata read, handled by the re-reads described below).

Metadata updates in place. A metadata update (FRE-COMPLIANCE.REVALIDATION-01) whose new segments fit `meta_capacity` — the reserved

headroom decided in FDS-STORAGE.METAUPDATE-01 — is applied in place: the segments are rewritten in the region, then `internal_len`, `header_len`, and the CRCs are updated — the body is never touched — and nothing is synchronised to stable storage (ADR-PERSISTENCE.DURABILITY-01). A reader that catches the update mid-write sees a CRC mismatch and re-reads the header and metadata region, up to three more times, yielding to other tasks between attempts. If all four reads mismatch — the updater may have been stopped between its two writes — the lookup reads once more holding the key's publication lock (FDS-STORAGE.METAUPDATE-01), which every metadata update holds from its first write to its last, so that this read finds the record whole. Only if that read mismatches too does the lookup report a miss and leave the file, as for any damaged record; the restart scan reads such a record once more in the same way before it reclaims it (FDS-STORAGE.RECLAIM-01). A reader therefore observes the pre- or post-update set, never a mixture, and an update in progress never turns a hit into a miss. This assumes nothing about the write itself being atomic — no POSIX filesystem or disk interface guarantees that a multi-byte write lands as one indivisible unit, even for a single nominal sector or page,[13] the metadata region can exceed any one write unit once the header set is large enough (there is no fixed cap on it), and the update spans two separate byte ranges — the metadata region and the header block's own length fields — that no single write covers together regardless. Treating this as a live, not theoretical, risk matches established practice: PostgreSQL's `full_page_writes` exists because "a page write that is in process during an operating system crash might be only partially completed, leading to an on-disk page that contains a mix of old and new data,"[14] and MySQL's InnoDB doublewrite buffer exists for the identical reason, recovering a good copy when a crash lands "in the middle of a page write."[15] A cheaper-looking alternative — marking the record *writing* before the update and *done* after, so only the rare writer pays a check instead of every reader — does not remove the requirement: without an explicit ordering barrier between the status write and the payload write, the file system is free to persist them out of order — `fsync` only flushes "modified in-core data… to the disk device," so anything not yet flushed simply sits reorderable in the page cache until it is,[16] and a crash could therefore leave a durable *done* status covering a payload that never actually landed. Enforcing that ordering costs at least one synchronous flush per write — PostgreSQL's own documentation calls exactly this delay "a major component of the total transaction time" for short transactions, the reason its asynchronous-commit mode exists at all,[17] typically millisecond-scale on hardware without power-loss protection[18] — far more than the microsecond-scale check it would replace. Nor is the read-side check adding a separate cost to shift in the first place: `CacheMeta::deserialize` already needs the whole internal and header byte range in memory to decode it, so the CRC recompute is one more pass over bytes already resident from the read that just happened, not a second read or a meaningfully separate computation.

[13] SQLite documentation: "Atomic Commit In SQLite": "SQLite has traditionally assumed that a sector write is not atomic."

[14] PostgreSQL 18 documentation: `full_page_writes`.

[15] MySQL 8.0 Reference Manual: "Doublewrite Buffer".

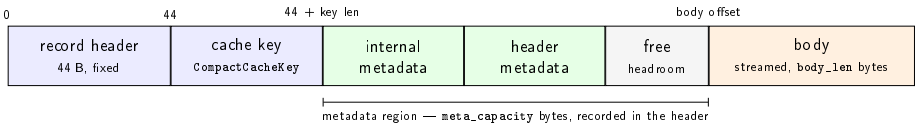
[16] Linux manual page: `fsync(2)`.

[17] PostgreSQL documentation: "Asynchronous Commit".

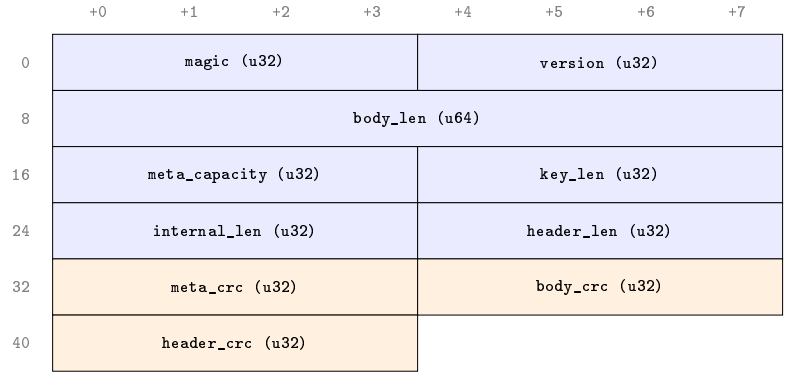
[18] Mark Callaghan, "Small Datum": "SSDs, power loss protection and fsync latency": consumer SSDs measured at roughly 1–5 ms per `fsync`, versus under 0.1 ms for enterprise drives with power-loss protection.

Metadata that outgrows the region. When the new metadata no longer fits `meta_capacity`, the record is *withdrawn* instead (FDS-STORAGE.METAUPDATE-01): only its header is rewritten, with `version` 0 and `header_crc` resealed. Every reader then treats the key as holding no object, so the next request fetches it again, and its store, sized for the new metadata, replaces the file; until then the file keeps its length, and the space the eviction manager counts for it. The body is neither read nor moved, and a reader that catches the header write midway sees a checksum or version mismatch and ends with the record whole or a miss, never a mixture. The reserved headroom is what makes the common freshening after a 304 (`Date`- and `Age`-scale changes) an in-place write rather than a withdrawal and a fetch from the origin. Moving outgrown metadata after the body instead, which would spare that fetch, is deferred to a later version (ADR-STORAGE.METAMOVE-01).

Self-delimiting for future reuse. The record is self-delimiting — header, key length, capacity, and body length define its exact extent. Object compaction into shared aggregate files is not implemented for this version (ADR-EFFICIENCY.COMPACTION-01); this property is what would let a future version move a completed record verbatim, back-to-back with others, into an aggregate without redesigning it, and let such an aggregate be re-scanned sequentially — for recovery or startup enumeration — without a separate index.



(a) The record's segments in file order.



(b) The 44-byte record header: fixed-width little-endian fields at naturally aligned offsets (row offsets left, byte positions above), so the block round-trips between disk and a `#[repr(C)]` struct without repacking.

Figure 6.7: Object-file record layout: the on-disk structure of one cache object. Its self-delimiting fields are also what would let a future version reuse it, unchanged, inside an aggregate file (ADR-EFFICIENCY.COMPACTION-01).

Basis: `MemCache`'s per-object metadata pair `BinaryMeta = (Vec<u8>, Vec<u8>)` and its `TempObject` (in-memory tracking of an in-flight write's valid length); `CacheMeta::serialize` / `deserialize(internal, header)`;

internal-metadata fields per `InternalMetaV2`.

View complete diagram.

Required coverage	✓ SDS-LAYOUT.KEYSEGMENT-01 via ⟨⟨*refine*⟩⟩ — *Specify the key segment's byte encoding — its fields' order and sizes, how the presence of a variance hash is marked, and how the tag is delimited — so that each key has exactly one encoding.* ✓ TC-LAYOUT.OBJECTFILE-01 via ⟨⟨*verify*⟩⟩ — *Verify that a reader accepts a record only when its magic, version, header and metadata CRCs, and stored key all validate, its metadata segments fit its capacity, its body offset lies within the file, and the file's length is exactly the computed body offset plus body_len; that any single violation is a miss and leaves the file in place; that a body whose checksum mismatches is read to a normal end, and is logged when the read-time check is on; and that a metadata read racing an in-place update returns one complete version.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-STORAGE.OBJECTFILE-01 ADR-PERSISTENCE.DURABILITY-01
Satisfies	FRE-COMPAT.FORMATUPGRADE-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.VALIDATORS-01 FRE-COMPLIANCE.BODY-01 FRE-COMPLIANCE.TRAILERS-01
Traces to	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.METAUPDATE-01 ADR-EFFICIENCY.COMPACTION-01 FRE-COMPLIANCE.REVALIDATION-01 FRE-COMPAT.EVICTION-01 FRE-COMPAT.STREAMING-01 FDS-EVICTION.STARTUP-01 SDS-LAYOUT.SHARDING-01
Traced from	FDS-STORAGE.RECLAIM-01
Refined by	SDS-LAYOUT.KEYSEGMENT-01
Verified by	TC-LAYOUT.OBJECTFILE-01
Satisfied by	IMP-LAYOUT.OBJECTFILE-01 (durable-cache/src/format.rs:18) IMP-STORAGE.LOOKUP-01 (durable-cache/src/lookup.rs:37) – – the read

[SDS-LAYOUT.KEYSEGMENT-01]

Key segment encoding ⟨⟨*structural requirement*⟩⟩ `draft`

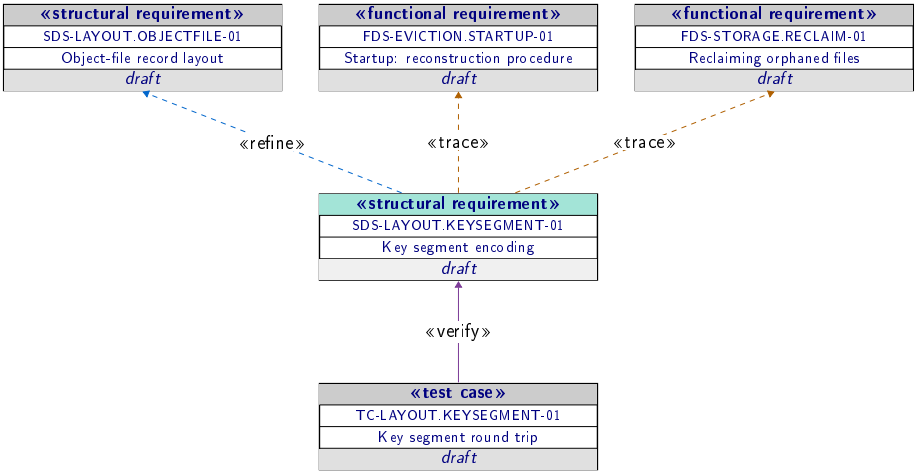


Figure 6.8: Key segment encoding

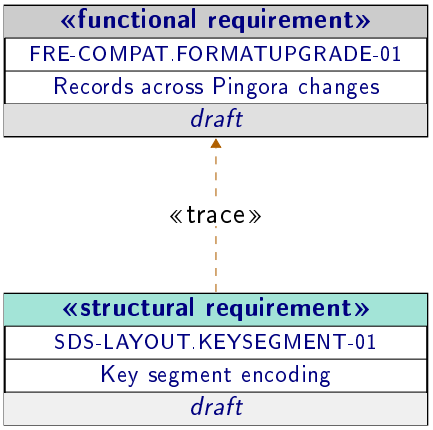


Figure 6.8: Key segment encoding (cont.)

The key segment of a record (SDS-LAYOUT.OBJECTFILE-01) holds the object's `CompactCacheKey`[19] as these fields, in this order, with no padding between them:

- `primary` (16 bytes): the key's primary hash.
- The *variance flag* (1 byte): 1 when a variance hash follows, 0 when none does.
- `variance` (16 bytes): the key's variance hash, present exactly when the variance flag is 1.

[19] `CompactCacheKey`.

- `user_tag`: the tag's UTF-8 bytes, filling the rest of the segment.

Both hashes are `HashBinary`[20] values. The tag carries no length of its own: the header's `key_len` bounds the segment, so whatever follows the fixed fields is the tag. Each key therefore has exactly one encoding, which is what lets a reader compare a stored key with the key it looked up byte for byte.

Only the restart-time scan decodes a stored key, to rebuild the `CompactCacheKey` it admits to the eviction manager (FDS-EVICTION.STARTUP-01). IF a key segment is shorter than the fields its variance flag announces, carries a variance flag other than 0 or 1, or holds a tag that is not valid UTF-8, THEN the restart-time scan shall reclaim the record as damaged (FDS-STORAGE.RECLAIM-01).

The field sizes are part of the record format: a `pingora-cache` release that changed the size of `HashBinary` would change what is stored, and so would require a new format version (FRE-COMPAT.FORMATUPGRADE-01).

View complete diagram.

Required coverage	✓ TC-LAYOUT.KEYSEGMENT-01 via ⟨⟨*verify*⟩⟩ — *Verify that keys with and without a variance hash, and with an empty and a non-empty tag, encode to the specified layout and decode back to themselves; that a segment too short for its fields, a variance flag other than 0 or 1, and a tag that is not UTF-8 are each rejected; and that the restart-time scan reclaims a record whose key does not decode.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	SDS-LAYOUT.OBJECTFILE-01
Traces to	FDS-EVICTION.STARTUP-01 FDS-STORAGE.RECLAIM-01 FRE-COMPAT.FORMATUPGRADE-01
Verified by	TC-LAYOUT.KEYSEGMENT-01
Satisfied by	IMP-LAYOUT.KEYSEGMENT-01 (durable-cache/src/key_codec.rs:15)

[20] `HashBinary` is a `[u8; 16]`.

[FDS-EVICTION.STARTUP-01]

Startup: reconstruction procedure ⟨⟨*functional requirement*⟩⟩ `draft`

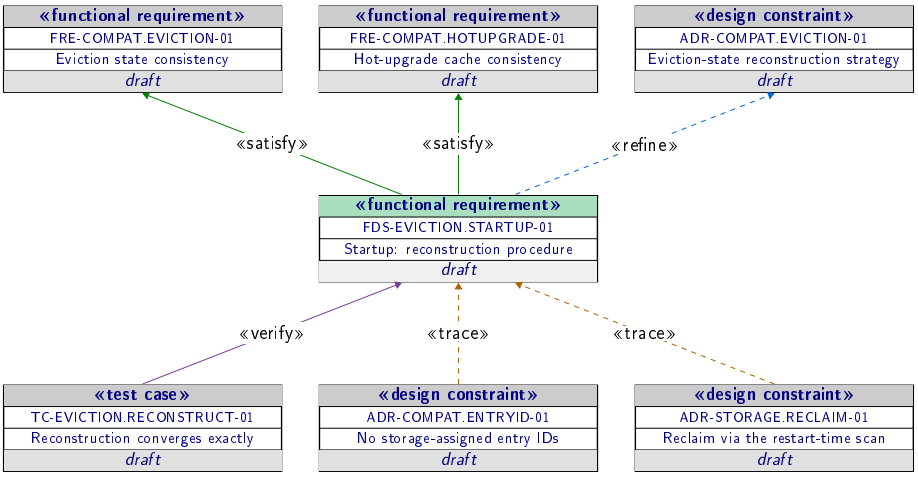


Figure 6.9: Startup: reconstruction procedure

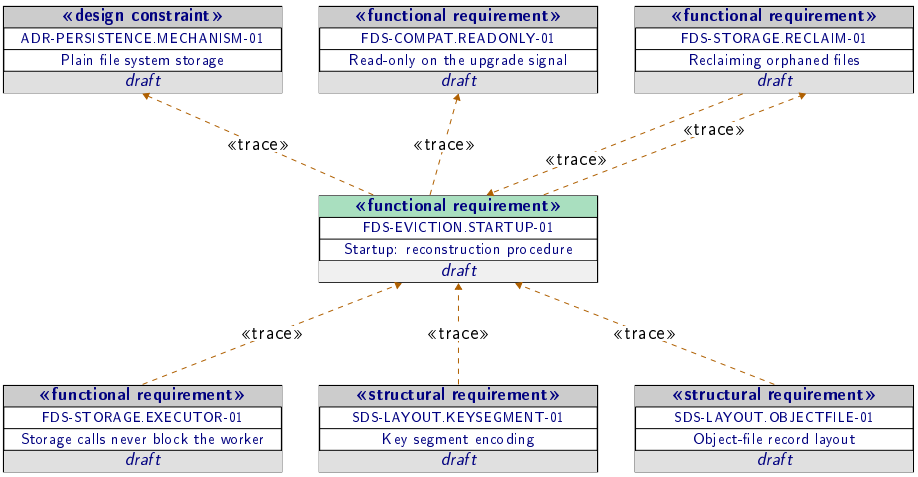


Figure 6.9: Startup: reconstruction procedure (cont.)

A dedicated API, not raw primitives. The `Storage` trait has no startup lifecycle hook, so this reconstruction is orchestrated by *Eviction Recovery*: a distinct API surface in the same crate as the `Storage` implementation, additional to it rather than folded into it, and — unlike `CacheLock` or the `EvictionManager` itself — never invoked *per request*: it is called once per process start, possibly after serving has already begun. Exposing only primitives instead (enumerate persisted entries, an `admit` passthrough) would force every integrator to reimplement the same scan-and-reconcile sequencing below, and doing so from outside the crate would mean exposing the private on-disk object-record format this specification

keeps internal everywhere else (SDS-LAYOUT.OBJECTFILE-01); folding it into a custom `EvictionManager` was already ruled out by ADR-COMPAT.EVICTION-01, which adopted the stock manager specifically to avoid building one. The cost is a second public API surface beyond `Storage`, with its own contract to document and version — but a single one, safe to call unconditionally whether a first launch, a post-crash restart, or the new side of a hot upgrade (FRE-COMPAT.HOTUPGRADE-01): with no shutdown hook and no snapshot, there is nothing to call at the wrong time or to poison during a drain.

The procedure. At process start, Eviction Recovery's `recover()` shall reconstruct the eviction manager from the published records, per ADR-COMPAT.EVICTION-01. On the incoming side of a hot upgrade the integrator calls it only once its server holds the listening sockets and the upgrade's close timeout has elapsed (FDS-COMPAT.READONLY-01). `recover()` enumerates the tree (ADR-PERSISTENCE.MECHANISM-01), classifying each entry as FDS-STORAGE.RECLAIM-01 specifies, and admits every record it keeps with `admit(entry key, size, fresh_until)`, where the entry key is the record's own `CompactCacheKey` carried in a `CacheEntryKey` with no entry identifier (ADR-COMPAT.ENTRYID-01) — the same shape `HttpCache` builds for a live admission, so the two paths address one entry rather than two — and `size` is the record size — the file's length — the same size a completed store reports (FDS-STORAGE.PUBLISH-01). Each `admit` may return eviction victims — the manager deducts their sizes immediately and expects the caller to remove them from storage — so the procedure shall purge every returned victim from `Storage` as an eviction, naming it by the exact entry the manager returned (`PurgeTarget::Exact`), exactly as `HttpCache` does after a live admission; a store found over its budget at startup is thereby brought back under it. Once the backend is read-only (FDS-COMPAT.READONLY-01), the procedure shall stop: it reads no further entry, and skips its remaining admissions and ghost-entry checks, since the eviction manager they would update is discarded when the process exits. It shall stop the same way, run by the hot-upgrade helper's recovery service, once the server starts shutting down (FDS-COMPAT.READONLY-01). `recover()` returns once enumeration and the reconciliation below have completed; that return is the backend reporting its restart procedure complete (FRE-COMPAT.EVICTION-01).

Running alongside live traffic. Whether the integrator starts serving before `recover()` returns is its own choice (ADR-COMPAT.EVICTION-01). The manager keys entries by `CacheEntryKey`, which for this backend carries the `CompactCacheKey` and nothing more, so the scan and live traffic admitting the same object update one entry rather than duplicating it. The procedure shall close the races the decision identifies, each by acting while holding the key's publication lock (FDS-STORAGE.METAUPDATE-01), which a live store's rename also takes before that store is admitted:

- *Stale size.* Immediately before admitting a record, the procedure re-checks its path under the lock. If the path is gone or now refers to a different file than the one read, it skips the record — live traffic removed it, or published and will admit its replacement; otherwise it admits the record with the file's current length while still holding the lock.
- *Ghost entries.* After enumeration, for every key it admitted, the procedure checks under the lock whether the canonical path still exists, and calls the manager's `remove` — which takes the borrowed `CacheEntryKeyRef` form of

the same identity — for any that does not. A key live traffic republishes after that check is admitted by the live path afterwards.

An object live traffic has just read may still come back from an `admit` as a victim; it is purged like any other, and accounting stays exact.

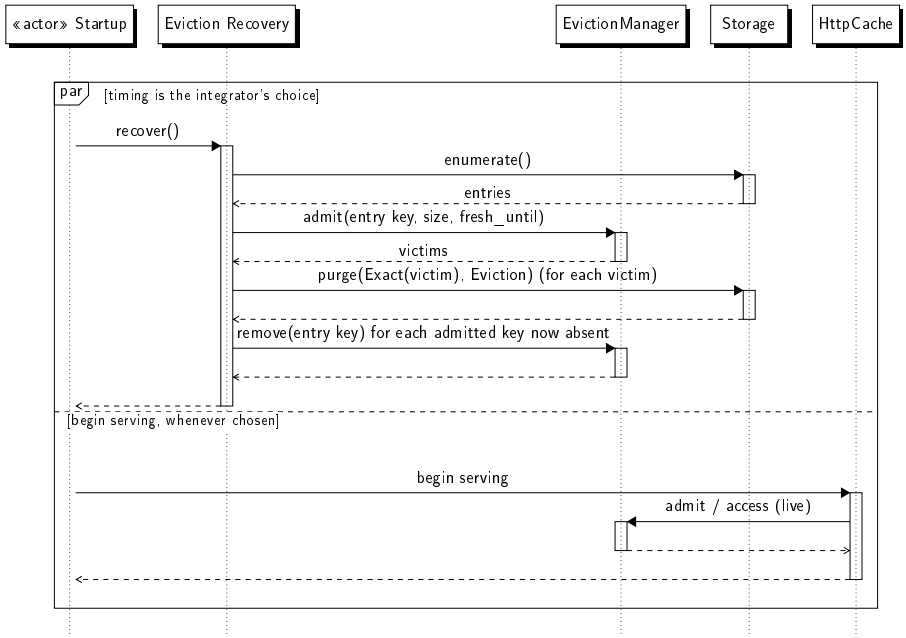


Figure 6.10: Startup reconstruction, carried out by Eviction Recovery (`Storage` and the `EvictionManager` never call each other). `Startup` — the integrator's own code — independently calls `recover()` and, on its own schedule, tells `HttpCache` to begin serving; if serving starts before reconstruction finishes, live `HttpCache` traffic touches the `EvictionManager` in parallel with the scan.

View complete diagram.

Required coverage	✓ TC-EVICTION.RECONSTRUCT-01 via ⟨⟨*verify*⟩⟩ — *Verify that when recover() returns the manager holds exactly the published set with record sizes, including when stores, replacements, and purges run during the scan, and that no snapshot or delta log is read or written at any point.* ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-COMPAT.EVICTION-01
Traces to	ADR-PERSISTENCE.MECHANISM-01 FDS-COMPAT.READONLY-01 FDS-STORAGE.RECLAIM-01
Satisfies	FRE-COMPAT.EVICTION-01 FRE-COMPAT.HOTUPGRADE-01
Traced from	FDS-STORAGE.EXECUTOR-01 ADR-STORAGE.RECLAIM-01 FDS-STORAGE.RECLAIM-01 ADR-COMPAT.ENTRYID-01 SDS-LAYOUT.OBJECTFILE-01 SDS-LAYOUT.KEYSEGMENT-01
Verified by	TC-EVICTION.RECONSTRUCT-01
Satisfied by	IMP-EVICTION.STARTUP-01 (durable-cache/src/recovery.rs:30)

[FDS-COMPAT.READONLY-01]

Read-only on the upgrade signal ⟨⟨*functional requirement*⟩⟩ `draft`

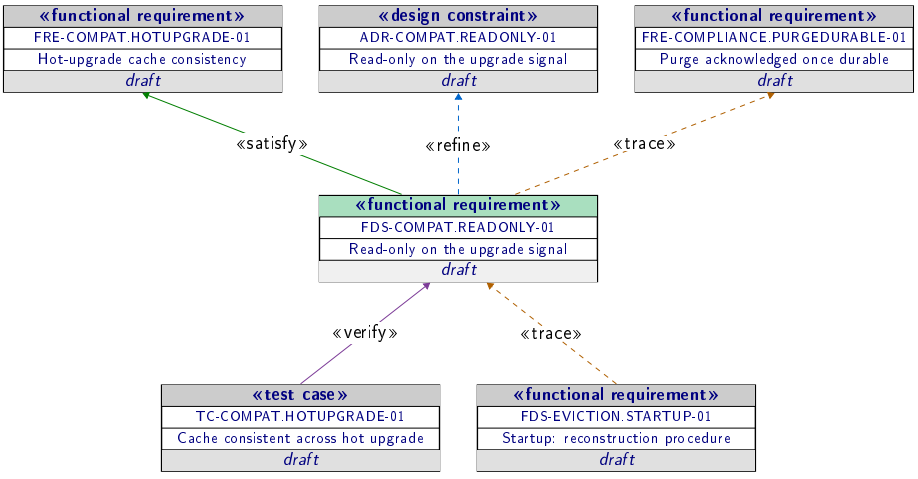


Figure 6.11: Read-only on the upgrade signal

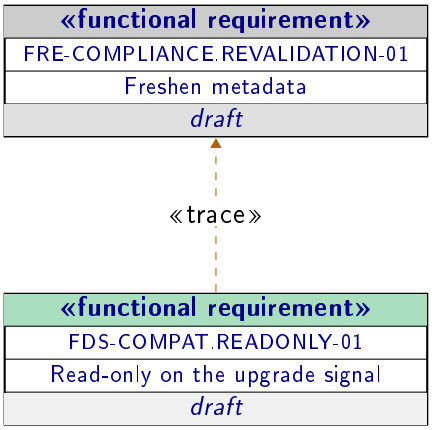


Figure 6.11: Read-only on the upgrade signal (cont.)

The read-only operation. The backend shall expose a `set_read_only` operation, callable from any thread and idempotent, which the integrator calls when its server's execution-phase broadcast announces `GracefulUpgradeTransferringFds`.[21] A graceful terminate, which announces `GracefulTerminate` instead, does not call it: no incoming process will take over the cache, so stores that complete while sessions drain should still be published.

The read-only gate. The backend shall keep a process-wide *read-only* flag behind a read–write gate. Every durable change — a store's final rename (FDS-STORAGE.PUBLISH-01), a purge's unlink (FDS-STORAGE.PURGE-01), either path of a metadata update (FDS-STORAGE.METAUPDATE-01) or of an expiry (FDS-STORAGE.EXPIRE-01), and a reclaim unlink (FDS-STORAGE.RECLAIM-01) — holds the gate shared from the moment it checks the flag until the change is done. So does each of the restart scan's admission checks (FDS-EVICTION.STARTUP-01), which changes no file but takes a key's publication lock (FDS-STORAGE.METAUPDATE-01): every taker of that lock enters the gate first. `set_read_only` takes the gate exclusively to set the flag, so when it returns, nothing that entered the gate before it is still running, and nothing can enter afterwards.

Once read-only. The process keeps draining its sessions; only its backend's changes to the shared cache stop. From then on the backend shall:

- keep opening stores and accepting their body writes, since failing to open a store would fail the client request that triggered it,[22] and partial-hit readers inside this process (FDS-STORAGE.PARTIAL-01) keep streaming;
- in `finish`, remove the temporary instead of renaming it, while reporting the store created with its record size, so the request completes normally — the entry this adds to the process's own eviction manager is discarded when the process exits;
- fail `purge` (of either purge type and either target kind), `expire`, and `update_meta` with an error, changing nothing on disk;

[21] `Server::watch_execution_phase` subscribes to the broadcast, which sends that phase when the upgrade signal arrives (L279).

[22] The proxy propagates the error from `set_miss_handler` with ?.

- stop the restart scan, skipping any reclaim unlink and any admission check still pending — the eviction manager the checks would update is discarded when the process exits;
- serve `lookup` and body reads unchanged.

Ordering with the incoming process. In the incoming process, Eviction Recovery's `recover()` (FDS-EVICTION.STARTUP-01) shall start only after the server has taken over the listening sockets and a further `CLOSE_TIMEOUT` (5 s) has elapsed, as ADR-COMPAT.READONLY-01 decides. That wait is the one the outgoing process takes after handing over its listening sockets; since 0.9.0 the outgoing process skips it entirely when it had no sockets to send, which is not the hot-upgrade case this ordering is about. The takeover is complete once `Server::bootstrap` returns, or — when bootstrap runs as a service — once that service reports ready;[23] the server's `Running` phase is not a substitute, since the main loop announces it without waiting for a bootstrap service. The deliverable shall therefore run `recover()` as a background service that waits the close timeout before starting and, when bootstrap runs as a service, depends on that service; a direct `bootstrap` call takes the sockets over before any service starts. In a process started without the Pingora upgrade flag, which takes the sockets over from none, the service shall not wait. It shall provide that service and the `set_read_only` subscription together, as a helper that adds both to the integrator's server and takes the bootstrap service's handle when there is one, and document that hot upgrades require it. Once the server starts shutting down, the eviction manager the service fills is about to be discarded, so the service shall start no scan, and stop a running one at its next entry, as it stops for a read-only backend.

View complete diagram.

Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩
Refines	ADR-COMPAT.READONLY-01
Satisfies	FRE-COMPAT.HOTUPGRADE-01
Traces to	FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.REVALIDATION-01
Traced from	FDS-EVICTION.STARTUP-01
Verified by	TC-COMPAT.HOTUPGRADE-01
Satisfied by	IMP-COMPAT.READONLY-01 (durable-cache/src/hotupgrade.rs:41)

[23] `Server::bootstrap` "will try to get all its listening sockets in order to take them over"; `bootstrap_as_a_service` runs the same work as a service, and a service started with dependencies waits for them to report ready (L735).

7. Test Case Specifications

We assume the compliance tests will be driven by `http-tests/cache-tests`, an existing, actively maintained test suite for HTTP caches (Appendix B details it fully). It ships its own controllable origin server, so the "mocked upstream" needed for the test scenarios does not need to be built: each test case addresses the origin with a unique ID and receives back exactly the headers, timing, and body that scenario requires.

What remains to be built is the *test subject*: a minimal Pingora proxy that enables `pingora-cache` with a single upstream peer pointed at the cache-tests origin and the file-system backend as its only `Storage` implementation. This proxy is a disposable test fixture, not a project deliverable — it exists solely to host the storage backend under test for the duration of a compliance run and ships nothing.[1] TC-COMPLIANCE.RFC-01 runs the cache-tests CLI against that proxy twice — once with the stock `MemCache` as its `Storage` and once with the file-system backend — and compares the two result sets test by test. Because `pingora-cache` does not itself claim full RFC 9111 conformance, the stock run is the reference: a test that fails on both runs is a limitation of `pingora-cache`, not of this backend, which is exactly what HLR-COMPLIANCE.RFC-01 requires to be distinguished (Appendix B).

[TC-COMPLIANCE.RFC-01]

No regression against MemCache ⟨⟨*test case*⟩⟩ `draft`

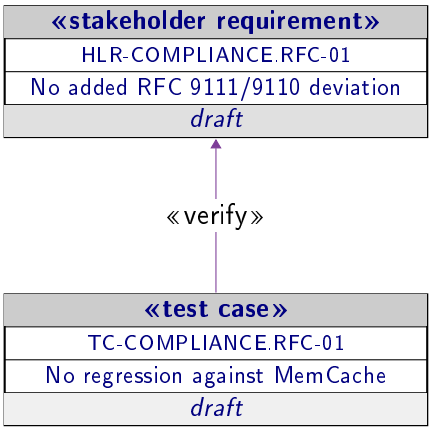


Figure 7.1: No regression against MemCache

Objective: Show that the file-system backend adds no RFC 9111/9110 deviation to the proxy's observable caching behaviour, by comparing it with the same proxy using the stock `MemCache`.

Setup: One proxy build whose `Storage` implementation is selectable at start — the stock `MemCache` or the file-system backend — with its upstream peer pointed at the `cache-tests` origin server, and all other configuration identical. `cache-tests`

[1] Running it also introduces a Node.js/npm dependency into the test or CI environment, since the cache-tests CLI client is JavaScript-based.

supplies its own corpus of per-test-case responses and expected outcomes; Appendix B (§ B.3) lists which `cache-tests` files exercise each `FRE-COMPLIANCE.*` requirement, and which requirements it does not reach.

Steps:

1. Start the proxy with `MemCache`; run the `cache-tests` CLI against it, exporting the results as JSON.
2. Start the proxy with the file-system backend on an empty cache directory; run the CLI again the same way.
3. Compare the two result sets test by test, ranking outcomes pass, then sub-optimal, then fail. Re-run against both backends every test whose outcome differs, to separate a reproducible difference from a timing-dependent one.

Expected result: No test has a reproducibly lower-ranked outcome with the file-system backend than with `MemCache`. Tests that fail on both runs are recorded as `pingora-cache` limitations, outside this requirement. Observation-only results that differ are listed for review. (Restart scenarios are outside this test — `cache-tests` cannot restart the process under test — and are covered by the per-requirement test cases.)

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	HLR-COMPLIANCE.RFC-01
Satisfied by	IMP-TC.COMPLIANCE.RFC-01 (xtask/src/main.rs:181) – – runs the

[TC-COMPLIANCE.HEADERS-01]

Header fields round-trip faithfully ⟨⟨*test case*⟩⟩ `draft`

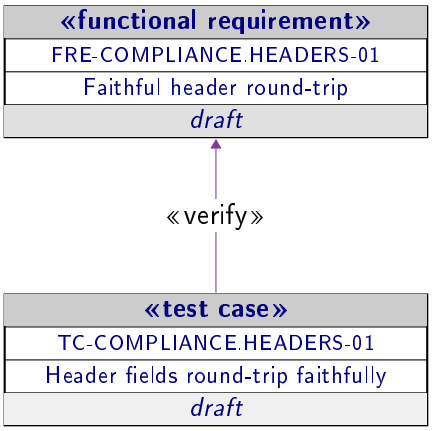


Figure 7.2: Header fields round-trip faithfully

Objective: A stored response returns the exact response head it was given — status code, version, and every header field.

Setup: The backend driven directly through the `Storage` interface by a test harness; a "restart" constructs a new backend instance over the same cache directory.

Steps:

1. Store a response whose head exercises each shape fidelity must not depend on: repeated field names in a deliberate order, an unrecognised field, an empty-valued field, a field value carrying opaque non-ASCII octets (obs-text), a `Set-Cookie`-style field repeated across lines, a non-200 status code, header sets of eight consecutive serialised sizes stored with no metadata headroom, so that each metadata region ends exactly where the body begins (SDS-LAYOUT.OBJECTFILE-01), and a header set many times the default headroom.

2. Look the object up; then restart the process and look it up again.

Expected result: Both lookups return the status code and version unchanged and every field with its name and value intact byte for byte, the relative order of same-named fields preserved; nothing is dropped, added, reordered, or normalised, regardless of the size of the header set. Where the metadata region is exactly full, the body that follows it is intact.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.HEADERS-01
Satisfied by	IMP-TC.COMPLIANCE.HEADERS-01 (durable-cache/tests/compat_headers.rs:24)

[TC-COMPLIANCE.TRAILERS-01]

Trailer fields kept distinct ⟨⟨*test case*⟩⟩ `draft`

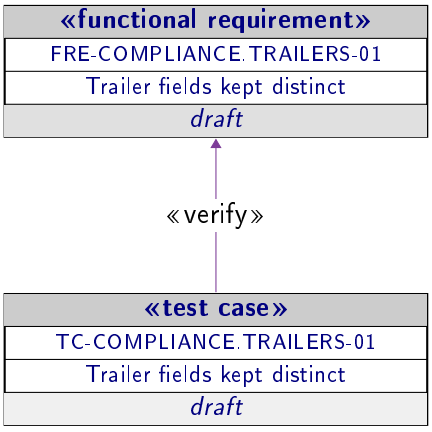


Figure 7.3: Trailer fields kept distinct

Objective: Show, by inspection, that trailer fields are never returned as header fields. No test can present a trailer through the targeted interface (FRE-COMPLIANCE.TRAILERS-01), so this specification records an inspection procedure rather than a test.

Steps (inspection):

1. Inspect the `Storage` and `HandleMiss` signatures of the targeted `pingora-cache` version: confirm that a store receives header metadata only as a `CacheMeta` and body bytes only through `write_body`, with no trailer parameter.
2. Inspect the on-disk record format (SDS-LAYOUT.OBJECTFILE-01) and its implementation: confirm that the stored header segment is written only from the `CacheMeta` handed to the store or metadata update, and that no other section could supply fields to it.

Expected result: The interface offers no trailer channel, and the record format has no path by which a trailer-originated field could enter the stored header segment. Repeat the inspection whenever the targeted `pingora-cache` version changes.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.TRAILERS-01
Satisfied by	IMP-TC.COMPLIANCE.TRAILERS-01 (durable-cache/tests/compat_trailers.rs:28)

[TC-COMPLIANCE.BODY-01]

Body round-trips byte for byte ⟨⟨*test case*⟩⟩ `draft`

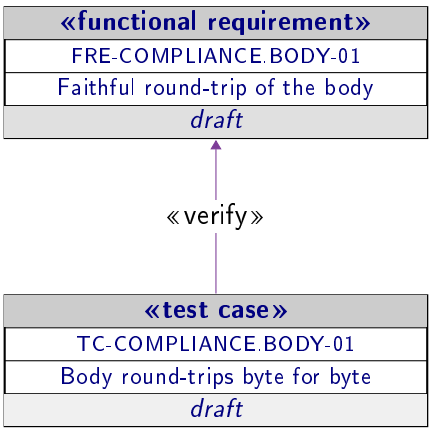


Figure 7.4: Body round-trips byte for byte

Objective: The served body equals the stored body exactly, for full and for seeked (partial) reads.

Setup: The backend driven directly through the Storage interface by a test harness; a "restart" constructs a new backend instance over the same cache directory.

Steps:

1. Store responses with random binary bodies at boundary sizes: empty, one byte, exactly at and one byte either side of the backend's write-chunk and file-system block sizes, and a large multi-chunk body.
2. Retrieve each, including after a restart, and compare to the bytes written.
3. Where the hit handler advertises seek support, read sub-ranges of the large body — the start, an interior slice spanning a chunk boundary, and the tail — and compare each against the corresponding slice of the stored bytes.

Expected result: Each full read is identical byte for byte and in order — no truncation, padding, or re-encoding; each seeked read delivers exactly the requested slice, so range responses built from the reader carry precisely the stored bytes.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.BODY-01
Satisfied by	IMP-TC.COMPLIANCE.BODY-01 (durable-cache/tests/compat_body.rs:17)

[TC-COMPLIANCE.FRESHNESS-01]

Freshness and age inputs preserved ⟨⟨*test case*⟩⟩ `draft`

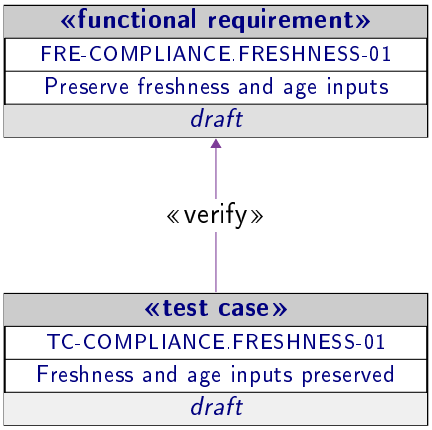


Figure 7.5: Freshness and age inputs preserved

Objective: Freshness and age computed from a retrieved object match those computed before it was stored.

Setup: The backend driven directly through the Storage interface by a test harness; a "restart" constructs a new backend instance over the same cache directory,

with freshness and age evaluated by the functions of `pingora-cache` itself over the returned `CacheMeta`.

Steps:

1. Store responses exercising `Date`, `Expires`, `Age`, and `max-age`/`s-maxage`, including boundary and degenerate values: `Age: 0`, delta-seconds at the overflow cap (2147483648), `max-age=0`, an already-past `Expires`, and a syntactically invalid `Expires`.
2. Evaluate freshness lifetime and current age before storing and after retrieving (including across a restart), using the same clock reference.
3. Apply a metadata update carrying new freshness fields (FRE-COMPLIANCE.REVALIDATION-01) and re-evaluate from a fresh lookup.

Expected result: The pre-store and post-retrieve evaluations agree, including the internal created/updated instants of `pingora-cache`; boundary and invalid values round-trip as opaque bytes, un-normalised; after the update, evaluation reflects the updated field set and the new updated-at instant.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.FRESHNESS-01
Satisfied by	IMP-TC.COMPLIANCE.FRESHNESS-01 (durable-cache/tests/compat_freshness.rs:28)

[TC-COMPLIANCE.VALIDATORS-01]

Validator fields preserved ⟨⟨*test case*⟩⟩ `draft`

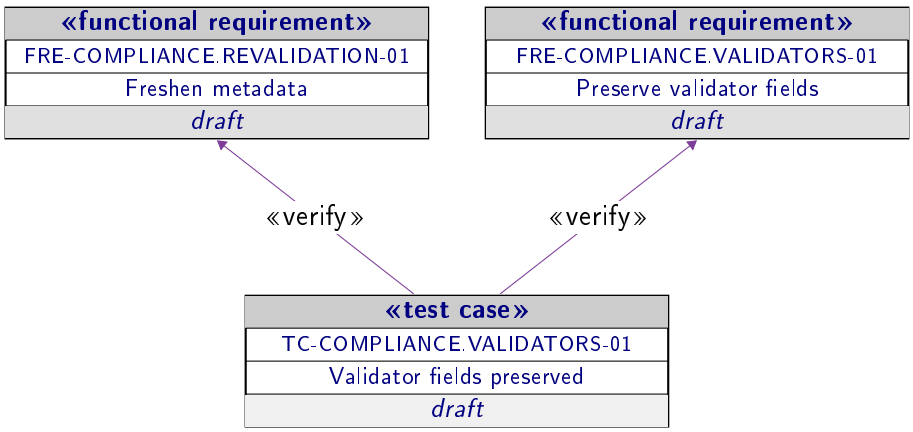


Figure 7.6: Validator fields preserved

Objective: `ETag` and `Last-Modified` survive storage unchanged, including the weak indicator.

Setup: The backend driven directly through the `Storage` interface by a test harness; a "restart" constructs a new backend instance over the same cache directory.

Steps:

1. Store responses with a strong `ETag`, a weak (`W/`) `ETag`, an empty entity tag (`ETag: ""`), a `Last-Modified` date, and one response carrying both validators.
2. Retrieve and compare the validator fields to those stored, including after a restart.
3. Apply a metadata update that changes the validators (FRE-COMPLIANCE.REVALIDATION-01); retrieve and compare again.

Expected result: The validators are returned byte-exactly, with the `W/` marker retained and the empty tag intact, so conditional revalidation can be issued and matched; after the update, every lookup returns the updated validators.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.VALIDATORS-01 FRE-COMPLIANCE.REVALIDATION-01
Satisfied by	IMP-TC.COMPLIANCE.VALIDATORS-01 (durable-cache/tests/compat_validators.rs:21)

[TC-COMPLIANCE.REVALIDATION-01]

Metadata freshened atomically ⟨⟨*test case*⟩⟩ `draft`

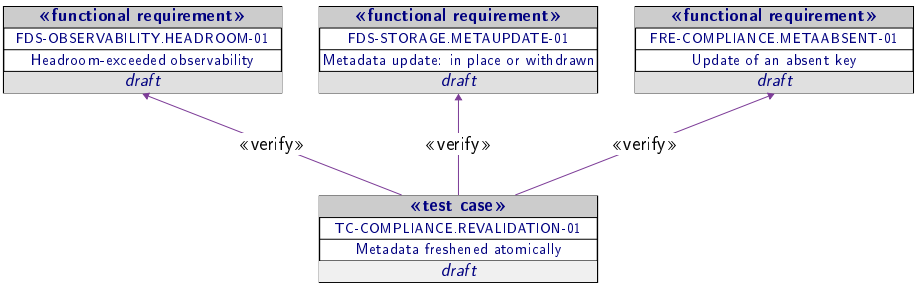


Figure 7.7: Metadata freshened atomically

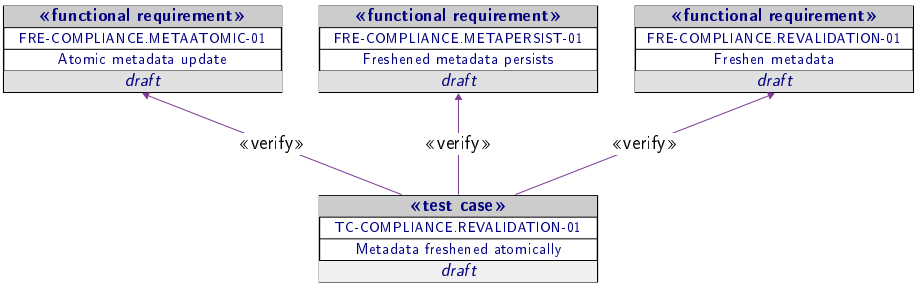


Figure 7.7: Metadata freshened atomically (cont.)

Objective: A metadata update after a 304 replaces metadata while leaving the body intact, applies only to the object it validated, is observed atomically and is visible to lookups when `update_meta` returns; an update that outgrows the record's reserved space withdraws the object, observably, so that the next request fetches it again.

Setup: The backend driven directly through the `Storage` interface, with a test hook able to make a withdrawal's header write fail, and log and metric capture.

Steps:

1. Store an object, then apply a metadata update with a newer header set; repeat with an update whose header set is large enough that it no longer fits the space the layout reserved ahead of the body (FDS-STORAGE.METAUPDATE-01), while a client reads the object through a hit opened before that update.
2. Run lookups concurrently with each update.
3. As each `update_meta` call returns, immediately look the object up, before any other task runs, and inspect the record on disk.
4. Restart the process and look the object up again.
5. Apply a metadata update to a key with no stored object.
6. Configure zero reserved headroom and repeat the small-update case from step 1.
7. After an update has withdrawn a record, store the object again, as the next request's fetch does, and look it up.
8. Apply sixteen updates that each outgrow the reserved space, all at once. With a store of the key from the origin in flight, apply an update that outgrows the record, then let the store finish.
9. Make the backend read-only (FDS-COMPAT.READONLY-01) and apply an update that fits and one that does not.
10. Make the header write of a withdrawal fail; then apply the update again.
11. Read the counter `pingora_durable_meta_headroom_exceeded_total` once the backend is built. Then, for every withdrawal above, and for the one whose header write failed, read the captured log and the counter.

12. In a build without the backend's `metrics` feature, apply one update that withdraws the record; read the captured log and the registry.

13. Store an object, then a newer object of the same key, created later; then apply the update a 304 for the first object brings: new header fields under the first object's creation time.

Expected result: The body is unchanged and still retrievable under the same key; every concurrent lookup sees either the pre-update or post-update metadata, never a mixture, and never a body mismatched to its metadata; when each call that fits returns, a lookup answers with the update and the record on disk holds it; after the restart, lookups return the updated metadata, never the superseded set; the missing-key update reports an error and materialises no object; the update that outgrows the reserved space reports `false` and withdraws the record in place, the file keeping its length: the client already reading it gets the whole body, and every later lookup misses, after the restart too; with zero headroom even the small update withdraws the record; the store made after a withdrawal replaces the withdrawn file, with a region sized for its own header set, and lookups hit it; of the sixteen updates, one withdraws the record and the others report that the key holds no object; the update made during the store from the origin withdraws the record, and that store then lands and replaces it; the updates once the backend is read-only fail with an error and change nothing; the withdrawal whose header write fails reports an error and leaves the record as it was, served with its old metadata, and the next attempt withdraws it; the counter reads zero once the backend is built; each withdrawal logs one warning naming the key's hash and the headroom shortfall, and raises the counter by one, while the failed one is neither logged nor counted; in the build without the `metrics` feature the warning is still logged and the counter is not registered; and the update for the replaced object is reported not applied, and the newer object stands whole.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.REVALIDATION-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.METAABSENT-01 FDS-STORAGE.METAUPDATE-01 FDS-OBSERVABILITY.HEADROOM-01
Satisfied by	IMP-TC.COMPLIANCE.REVALIDATION-01 (durable-cache/tests/compat_revalidation.rs:22) IMP-TC.COMPLIANCE.REVALIDATION-02 (durable-cache/tests/compat_revalidation_observability.rs:17)

[TC-COMPLIANCE.VARY-01]

Vary variants stored and selected ⟨⟨*test case*⟩⟩ `draft`

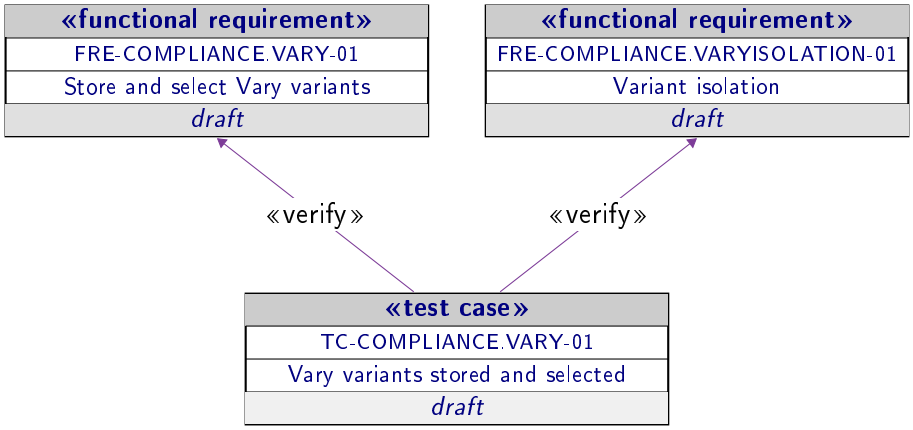


Figure 7.8: Vary variants stored and selected

Objective: Multiple variants of one primary key are stored and correctly selected.

Setup: The backend behind a `pingora-cache` `HttpCache` driven by a test harness, so that lookups, variant selection, and purges follow the framework's own call sequence; a "restart" constructs new instances over the same cache directory.

Steps:

1. Store two responses under the same primary key with different variance keys derived from `Vary`.
2. Look up with requests matching each variant, and with one matching neither.
3. Replace one variant with a newer response and apply a metadata update to it; look both variants up again. Then purge that variant's own variance-augmented key through the backend's purge operation, and look the other variant up again.
4. Restart the process and look each variant up once more.

Expected result: Each matching lookup returns its own variant; the non-matching lookup reports a miss; one variant is never served for another; mutating one variant leaves the other byte-identical; the full variant set survives the restart, each variant still retrievable under its variance-augmented key.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.VARY-01 FRE-COMPLIANCE.VARYISOLATION-01
Satisfied by	IMP-TC.COMPLIANCE.VARY-01 (durable-cache/tests/compat_vary.rs:23)

[TC-COMPLIANCE.PURGE-01]

Purge makes all variants unreachable ⟨⟨*test case*⟩⟩ `draft`

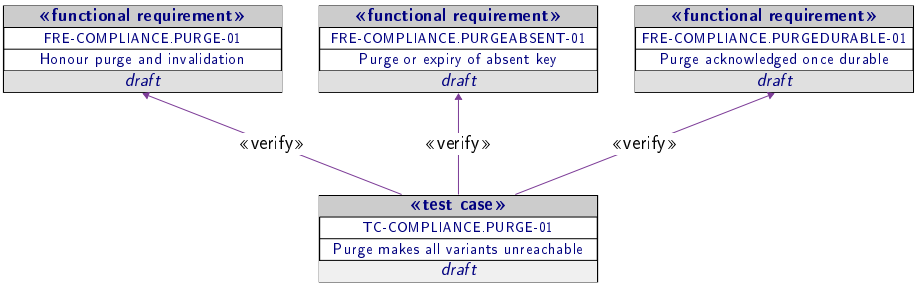


Figure 7.9: Purge makes all variants unreachable

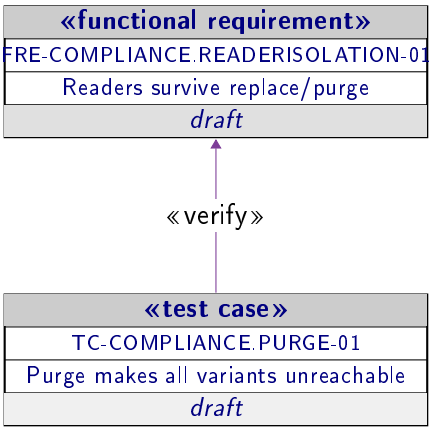


Figure 7.9: Purge makes all variants unreachable (cont.)

Objective: After a purge of a key, neither the response nor any of its variants is ever served again.

Setup: The backend behind a `pingora-cache` `HttpCache` driven by a test harness, so that lookups, variant selection, and purges follow the framework's own call sequence; a "restart" constructs new instances over the same cache directory.

Steps:

1. Store several `Vary` variants under one key.
2. Issue a purge for that key (the proxy's purge path addresses the primary slot), then look the primary and each variant up again, through the primary-slot-first lookup in `pingora-cache`.
3. Purge the same key a second time, and purge a key that was never stored.
4. Restart the process, once cleanly and once after killing it, and look the primary and each variant up again. (Purge durability across a host crash is exercised by TC-PERSISTENCE.CRASHSAFE-01.)

5. Repeat the purge while a reader is mid-stream on one variant and while a same-key store is in flight; let both run to completion.

6. Restart the process once more, letting Eviction Recovery's scan run (FDS-EVICTION.STARTUP-01), and inspect the cache directory and the eviction manager for the variant records.

Expected result: Every lookup — primary and variants — reports a miss after the purge; an invalidated response is never served; the repeated and never-stored purges are harmless no-ops that report nothing found; no purged object reappears after the restart or crash; the mid-stream reader runs to completion or fails cleanly — never another object's bytes — and its storage is not reused while referenced; the racing store publishes nothing, since the purge ended it; after the final restart, the variant records the purge orphaned are still in the cache directory, each admitted to the eviction manager with its record size, so that eviction removes them like any other object.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.READERISOLATION-01
Satisfied by	IMP-TC.COMPLIANCE.PURGE-01 (durable-cache/tests/compat_purge.rs:24)

[TC-COMPLIANCE.INCOMPLETE-01]

Incomplete responses never served ⟨⟨*test case*⟩⟩ `draft`

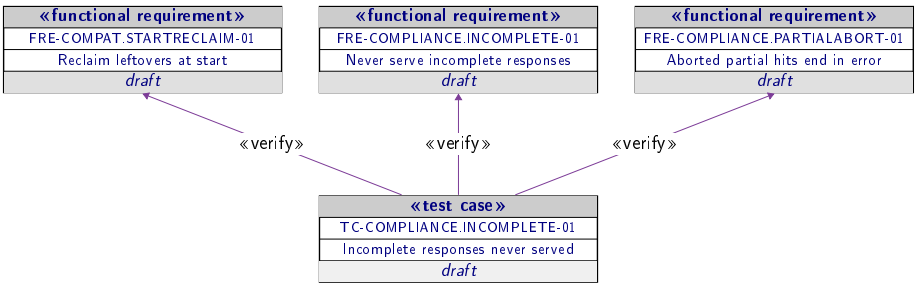


Figure 7.10: Incomplete responses never served

Objective: A store that does not run to completion is not later served as if complete.

Setup: The backend behind a `pingora-cache` `HttpCache` driven by a test harness, so that lookups, variant selection, and purges follow the framework's own call sequence; a "restart" constructs new instances over the same cache directory.

Steps:

1. Begin storing a response and, while the store is in flight, attach a follower streaming the partial hit (FRE-COMPAT.STREAMING-01, FRE-COMPLIANCE.PARTIALABORT-01).
2. Abort the store before the body is fully written (simulating an upstream disconnect), then drop its miss handler; inspect the cache directory.
3. Look the key up, including after a restart.
4. Repeat, but instead of an orderly abort kill the process mid-write (no abort path runs); restart, look the key up, and inspect the cache directory for the per-writer temporary left behind.

Expected result: Every lookup reports a miss — the only observable the `Storage` interface permits for an incomplete entry — and a partial body is never returned as a complete response; the follower's in-flight stream terminates with an error, never as a clean end of body; the orderly-aborted store's temporary is gone by the time its handler is dropped; the crash-orphaned temporary is never served or promoted, and is gone once the restarted process's restart-time scan has run. States a real host crash can leave behind are exercised by TC-PERSISTENCE.CRASHSAFE-01.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.PARTIALABORT-01 FRE-COMPAT.STARTRECLAIM-01
Satisfied by	IMP-TC.COMPLIANCE.INCOMPLETE-01 (test/compat/tests/incomplete.rs:23)

[TC-COMPLIANCE.REPLACE-01]

Stores replace atomically ⟨⟨*test case*⟩⟩ `draft`

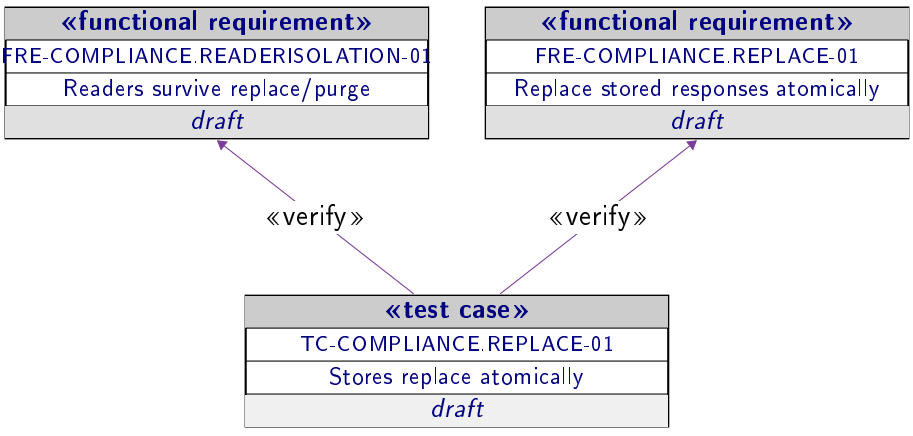


Figure 7.11: Stores replace atomically

Objective: Storing a new response under an occupied key replaces the old object atomically; readers and racing writers are never torn.

Setup: The backend driven directly through the `Storage` interface by a test harness; a "restart" constructs a new backend instance over the same cache directory.

Steps:

1. Store object A under a key and attach a reader mid-stream on A's body.
2. Store object B — different metadata and a different, larger body — under the same key; run lookups while B's store is in flight, then let it complete and look up again, including after a restart.
3. Drain the reader attached in step 1 and compare what it received.
4. Race two same-key stores (C and D) to completion and look up.

Expected result: In-process lookups during B's store return a partial hit of B (FRE-COMPAT.STREAMING-01), and a lookup that cannot see B's in-flight write returns A whole; after B's store completes, every lookup — also after the restart — returns B in its entirety, never A's metadata with B's body or any other mixture; the reader of A receives exactly A's bytes to a clean end of body (or a clean error), never bytes of B, and A's space is reclaimed only once no reader references it; after the race, lookups return exactly one winner (the last published), intact.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPLIANCE.REPLACE-01 FRE-COMPLIANCE.READERISOLATION-01
Satisfied by	IMP-TC.COMPLIANCE.REPLACE-01 (durable-cache/tests/compat_replace.rs:16)

[TC-PERSISTENCE.CRASHSAFE-01]

Host crash serves no bad content ⟨⟨*test case*⟩⟩ `draft`

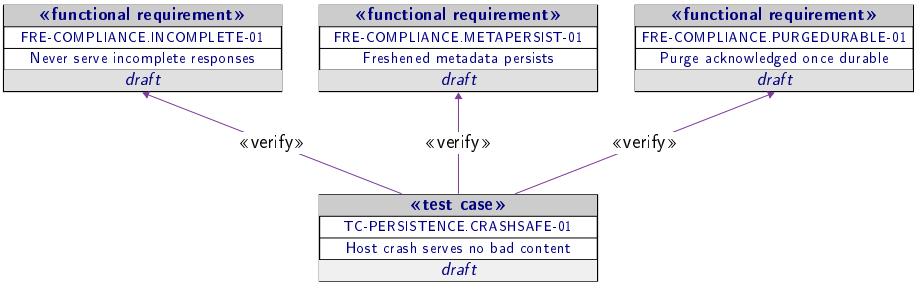


Figure 7.12: Host crash serves no bad content

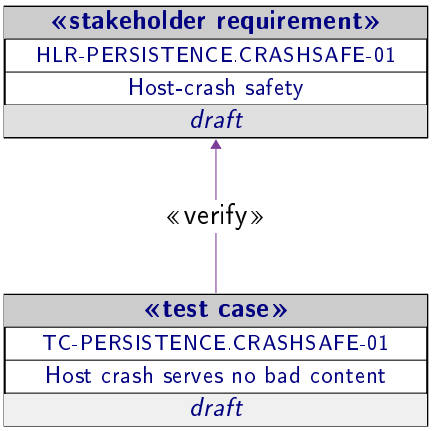


Figure 7.12: Host crash serves no bad content (cont.)

Objective: In every state a host crash or power loss could leave on disk, the restarted cache serves no object whose content differs from what was stored and no object whose purge it had acknowledged; objects that did not survive are misses.

Setup: The cache directory on an ext4 file system whose block device is a device-mapper `log-writes` target, which logs every write with its flush and FUA ordering so that the device can be replayed to any point a power failure could have left it;[2] the backend driven directly through the `Storage` interface, with the log marked after each operation the backend acknowledges.

Steps:

1. Store a set of objects of known content and varied sizes, including bodies spanning many file-system blocks; leave one further store unfinished.
2. Purge a subset of the stored objects, and apply metadata updates to another subset, both in place and withdrawing the record.
3. Replay the log to each flush and FUA point in turn. For each replayed state: mount the file system (replaying its journal), construct the backend and let its restart procedure complete, look up every key, and read every hit's body in full.

Expected result: In every replayed state: every hit's metadata and body are exactly one version the backend stored — for an updated object, the version before or after an update, never a mixture; no object whose purge was acknowledged before that state's point is served; the unfinished store and every temporary are never served; every other object is either served as stored or a miss.

View complete diagram.

[2] Linux kernel documentation: "dm-log-writes", with its replay tool at `https://github.com/josefbacik/log-writes`.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	HLR-PERSISTENCE.CRASHSAFE-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.METAPERSIST-01
Satisfied by	IMP-TC.PERSISTENCE.CRASHSAFE-01 (durable-cache/tests/persistence_crashsafe.rs:35)

[TC-PERSISTENCE.RESTART-01]

Objects survive a process exit ⟨⟨*test case*⟩⟩ `draft`

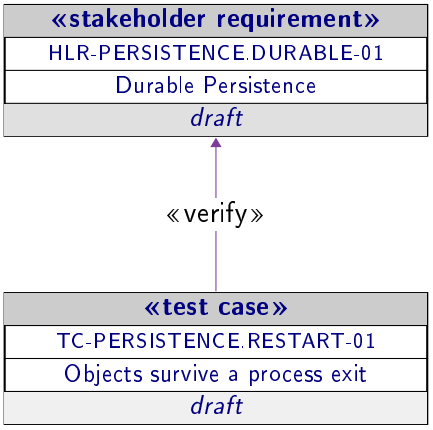


Figure 7.13: Objects survive a process exit

Objective: Every cached object whose storage completed before the cache process exited — in an orderly way or by crashing — is served by the process that starts next.

Setup: The test-subject proxy with the file-system backend, its upstream pointed at an origin that counts the requests it receives.

Steps:

1. Through the proxy, cache a set of objects: varied body sizes, `Vary` variants, and objects whose metadata was then updated by revalidation.
2. Stop the proxy gracefully; start it again on the same cache directory; request every object.
3. Cache a further set; as soon as the last store completes, kill the proxy; start it again; request every object from both sets.

Expected result: After each restart every object is served from the cache, byte for byte and with its latest metadata, and the origin receives no request for any of them.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	HLR-PERSISTENCE.DURABLE-01
Satisfied by	IMP-TC.PERSISTENCE.RESTART-01 (test/compat/tests/restart.rs:25)

[TC-PERSISTENCE.CAPACITY-01]

Cache larger than host RAM ⟨⟨*test case*⟩⟩ `draft`

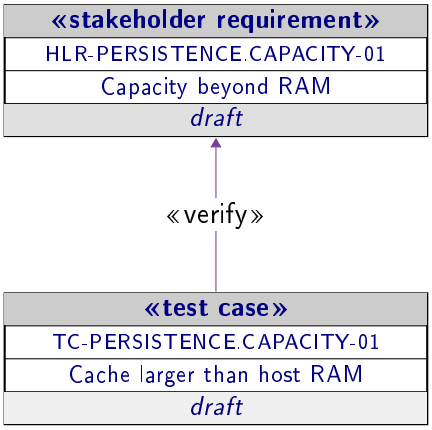


Figure 7.14: Cache larger than host RAM

Objective: The cache holds, and serves, a set of objects whose total size exceeds the RAM installed on its host.

Setup: The test-subject proxy with the file-system backend in a virtual machine with little RAM (e.g. 1 GiB) and ample disk, its eviction budget configured above the total the test stores.

Steps:

1. Cache objects through the proxy until their total size is at least four times the machine's RAM, sampling the proxy's resident memory throughout.
2. Restart the proxy and request every object.

Expected result: Every store succeeds and no object is evicted; after the restart every object is served from the cache intact; the proxy is never killed for lack of memory, and its resident memory stays well below the machine's RAM.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	HLR-PERSISTENCE.CAPACITY-01
Satisfied by	IMP-TC.PERSISTENCE.CAPACITY-01 (durable-cache/tests/measurements.rs:43)

[TC-EFFICIENCY.SPACE-01]

Space overhead measured ⟨⟨*test case*⟩⟩ `draft`

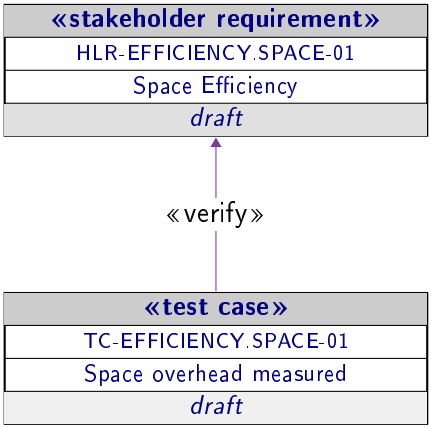


Figure 7.15: Space overhead measured

Objective: Measure the cache's on-disk space overhead as HLR-EFFICIENCY.SPACE-01 defines it and compare it with the 25 % target.

Setup: The test-subject proxy with the file-system backend on a dedicated file system, its upstream an origin serving three stated loads: (a) a mix of object sizes recorded from, or modelled on, the intended deployment; (b) only objects below 4 KiB; (c) only objects of 1 MiB and more. A tool that walks the cache directory and sums, per record, the metadata segments and body lengths the record header gives.

Steps:

1. For each load, fill the cache through the proxy until it holds at least 10^5 objects, then let it settle (no store in flight).
2. Measure the blocks the cache directory tree occupies (e.g. `du` in bytes), and the objects' own metadata and body bytes with the tool; compute the overhead as their difference over the latter.

Expected result: The overhead for each load is recorded. For the deployment load (a) it is at most 25 %. While ADR-EFFICIENCY.COMPACTION-01's conflict stands, a result above the target for loads dominated by small objects is known non-conformance, to be reported with its measured value rather than treated as a regression.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	HLR-EFFICIENCY.SPACE-01
Satisfied by	IMP-TC.EFFICIENCY.SPACE-01 (durable-cache/tests/measurements.rs:44)

[TC-PERFORMANCE.LOOKUP-01]

Lookup latency flat with size ⟨⟨*test case*⟩⟩ `draft`

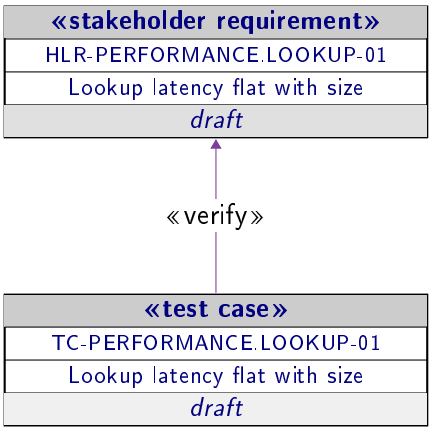


Figure 7.16: Lookup latency flat with size

Objective: The 99th-percentile lookup latency with 10^7 objects cached is no more than 25 % above that with 10^4 objects cached, as HLR-PERFORMANCE.LOOKUP-01 requires.

Setup: A dedicated test host with the cache directory on an SSD and the file system a deployment would use, idle apart from the test. The backend is driven directly through the `Storage` interface, and the time from calling `lookup` to its return — a hit with its metadata read, or a miss — is recorded per call; the rest of the lookup path in `pingora-cache` does not depend on how many objects are stored. Objects carry uniformly distributed cache-key hashes and small bodies.

Steps:

1. Store 10^4 objects.
2. *Cold measurement:* drop the operating system's page cache, then perform at least 10^5 lookups of uniformly chosen keys, half of them stored (hits) and half never stored (misses), recording each latency.
3. *Warm measurement:* repeat the same lookups immediately, so that the looked-up records and directories are in the page cache.
4. Store further objects into the same cache directory until it holds 10^7 objects, then repeat steps 2 and 3 with a fresh uniform key sample.
5. For each of the four runs, compute the 99th-percentile latency separately for hits and for misses.

Expected result: For hits and for misses, in both the cold and the warm condition, the 99th-percentile lookup latency at 10^7 objects is no more than 1.25 times the corresponding value at 10^4 objects.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	HLR-PERFORMANCE.LOOKUP-01
Satisfied by	IMP-TC.PERFORMANCE.LOOKUP-01 (durable-cache/tests/measurements.rs:45)

[TC-COMPAT.EVICTION-01]

Exact eviction state after restart ⟨⟨*test case*⟩⟩ `draft`

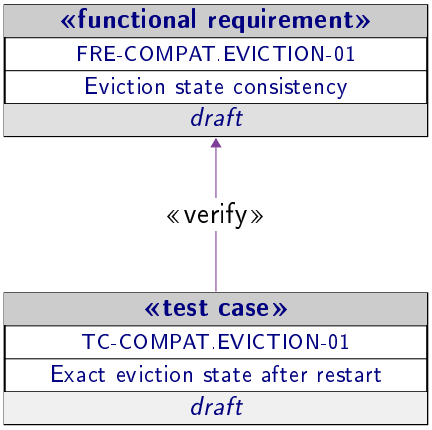


Figure 7.17: Exact eviction state after restart

Objective: Once the backend reports its restart procedure complete, the eviction manager's view of admitted objects is consistent with the durably published objects.

Setup: The backend with the stock `simple_lru` eviction manager and Eviction Recovery, driven by a test harness that can read the manager's entries and totals.

Steps:

1. Populate the cache with a known set of objects, including `Vary` variants and objects of very different sizes.
2. Before restarting, also leave on disk: an object purged during the previous run, a record damaged out of band, and a crash-orphaned per-writer temporary; and remove one variant's primary.
3. Restart the process and wait until the backend reports its restart procedure complete.
4. Inspect the manager's `total_items` and `total_size` and its per-entry view; compare each entry's size with the length of the file that stores the object.
5. Store one more object; confirm the size it is admitted with is also its record file length.

Expected result: Every durably published object, the variant whose primary was removed included, is represented exactly once with its record size and freshness

deadline; no entry exists for the purged object, the damaged record, the temporary, or the removed primary; the reported totals equal the count and aggregate record size of the published set; a store completed after the restart is admitted with the same size definition.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.EVICTION-01
Satisfied by	IMP-TC.COMPAT.EVICTION-01 (durable-cache/tests/compat_eviction.rs:26)

[TC-COMPAT.HOTUPGRADE-01]

Cache consistent across hot upgrade ⟨⟨*test case*⟩⟩ `draft`

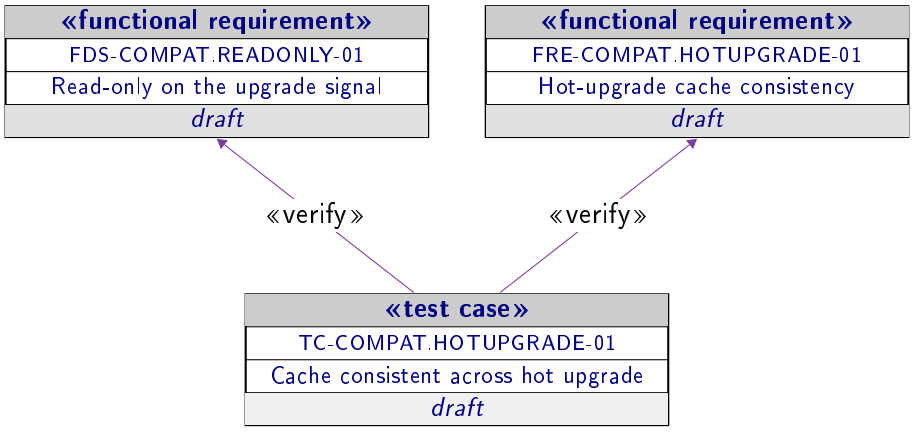


Figure 7.18: Cache consistent across hot upgrade

Objective: Once the outgoing process receives the Pingora upgrade signal, its backend makes no further durable change and refuses purges and metadata updates; the incoming process's activity never makes unreadable an object the outgoing process holds open; and the hot-upgrade helper starts the incoming process's restart scan only once the server has taken over the listening sockets and the close timeout has elapsed.

Setup: Two builds of the test-subject proxy with the file-system backend and its hot-upgrade helper installed, run as real Pingora server processes sharing one cache directory. From step 6 on, a Pingora server run inside the test harness with the helper installed, the harness standing in for the outgoing process.

Steps:

1. Run an old process serving normally, with streaming responses in progress against existing objects and a store of a slow upstream response in flight.
2. Start a new process and send the old one the Pingora upgrade signal (`SIGQUIT`), letting the listening sockets transfer.

3. Through the old process, while it still accepts or drains: let the in-flight store complete, and request a purge and trigger a revalidation metadata update for existing keys.

4. Through the new process: purge an object the old process is streaming, and drive stores, metadata updates, and eviction.

5. Let the old process finish draining and exit; restart nothing, and inspect the cache directory and the new process's lookups.

6. Start a server that takes over its listening sockets from an outgoing process, bootstrapping as a service, with the helper given that service's handle; hand a listening socket over only some time after the server has started. Then start an ordinary server, taking over no socket, with the helper installed.

7. Send one server with the helper installed the upgrade signal, though it has no socket to hand over, and another a graceful terminate.

8. Start the helper's recovery service, and shut its server down while the service waits to scan; then, over other caches, once the scan has admitted its first record, and once its walk is done, with a record removed meanwhile.

Expected result: The store the old process completed after the signal is not durably published; its purge and metadata update fail with an error and change nothing on disk; every response it was streaming completes byte for byte, including the one whose object the new process purged; the new process's lookups see only objects published before the signal or by the new process; the restart scan of step 6's first server begins no sooner than the close timeout after the hand-over, and the ordinary server's without waiting for it; in step 7, the upgrade signal makes its server's backend read-only, and the terminate leaves the other's writable; in step 8, the first service starts no scan and says so, the second scan admits no further record, and the third skips its ghost-entry checks, keeping the removed record's entry, both summary lines saying the server was shutting down.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.HOTUPGRADE-01 FDS-COMPAT.READONLY-01
Satisfied by	IMP-TC.COMPAT.HOTUPGRADE-02 (durable-cache/tests/compat_hotupgrade.rs:31) – – the IMP-TC.COMPAT.HOTUPGRADE-01 (test/compat/tests/hotupgrade.rs:37)

[TC-COMPAT.STREAMING-01]

Followers stream a partial hit ⟨⟨*test case*⟩⟩ `draft`

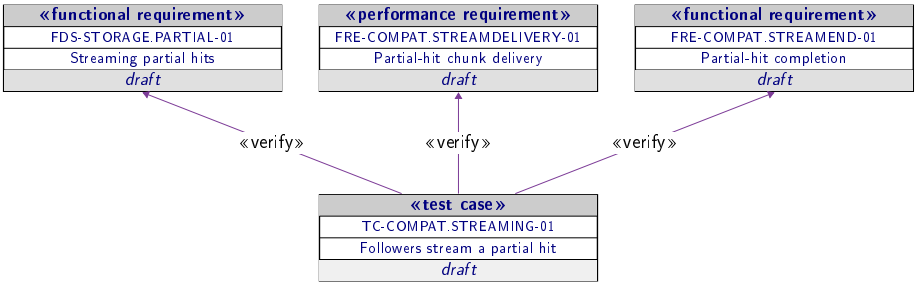


Figure 7.19: Followers stream a partial hit

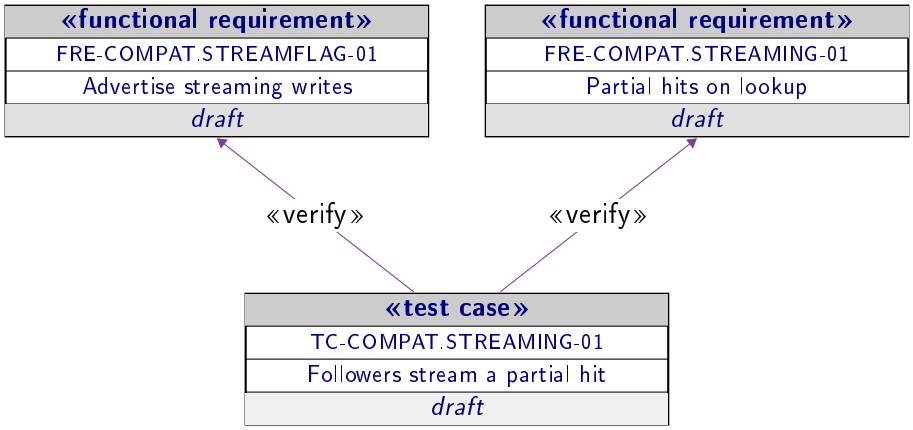


Figure 7.19: Followers stream a partial hit (cont.)

Objective: The backend advertises streaming partial writes (FRE-COMPAT.STREAMFLAG-01), and while a leader writes an object, concurrent requests are served by partial hits bound to the most recently opened writer when several are in flight (FRE-COMPAT.STREAMING-01), streamed within the chunk-delivery latency bound (FRE-COMPAT.STREAMDELIVERY-01) and ending normally when the store completes (FRE-COMPAT.STREAMEND-01), from a single upstream fetch.

Setup: A proxy with the file-system backend and no `CacheLock` configured, so that a second same-key store can open while the first is still in flight (with a lock configured, the same overlap needs the lock's age timeout to expire), on a test host that is not CPU-saturated, with at least ten followers and a body of at least a thousand chunks, so the delivery percentile rests on enough samples.

Steps:

1. Query the backend's `support_streaming_partial_write`.
2. On a second proxy instance identical but for a `CacheLock` being configured, drive a cache miss from a slow upstream; once the miss handler opens, and

before the body is written, record whether the cache lock for the key is still held.

3. Drive a cache miss from an upstream that emits a large body in throttled chunks; let the miss handler open.
4. While that store is still in flight, start a second store under the same key from a distinct upstream response, staggered so both writers' write windows overlap; let its miss handler open too.
5. Issue a lookup for the key now that both writers are open; confirm the returned partial hit is backed by the second writer's — the most recently opened — temporary object, not the first's.
6. Issue further concurrent lookups for the same key while the second store is in flight; confirm each returns a partial hit backed by that same temporary object.
7. Observe the followers' streams as the second writer appends, timestamping each chunk's `write_body` return and each follower's `read_body` return with that chunk, before that store completes.
8. Let the second store complete; drain the followers' streams (across the concluding atomic rename) and compare the received bodies to its upstream body.
9. Count upstream fetches: exactly one for each writer, including the first, whose output was superseded before ever being streamed to a follower.

Expected result: `support_streaming_partial_write` returns `true`, and the cache lock is released once the leader's miss handler opens, before its body is written. A lookup issued while several same-key writes are in flight resolves to the most recently opened one, never an earlier writer. Followers receive body chunks as that writer writes them, with at least 99 % of all chunk deliveries, across every follower, within 10 ms of the chunk's `write_body` return — no follower waits for the whole object; every follower's body is byte-identical to its upstream response and ends with a normal end of body, including for followers still reading when the object is renamed to its canonical name; each writer's upstream is fetched exactly once.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.STREAMFLAG-01 FRE-COMPAT.STREAMING-01 FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 FDS-STORAGE.PARTIAL-01
Satisfied by	IMP-TC.COMPAT.STREAMING-01 (test/compat/tests/streaming.rs:22)

[TC-COMPAT.UNSIZED-01]

Unsized body accepted and persisted ⟨⟨*test case*⟩⟩ `draft`

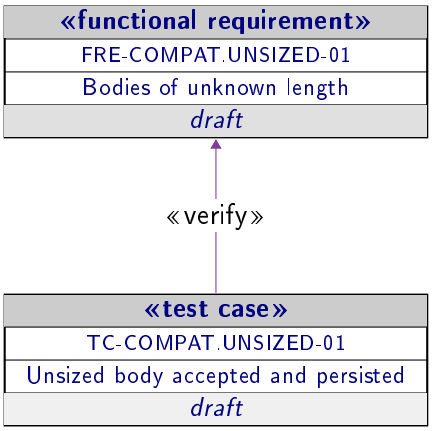


Figure 7.20: Unsized body accepted and persisted

Objective: A body whose total length is not declared when the store begins is persisted completely, and every chunk is in the store's temporary before its `write_body` call returns, so the backend holds no body bytes beyond the chunk being written, as FRE-COMPAT.UNSIZED-01 requires.

Setup: The backend driven directly through the `Storage` interface by a test harness that can read the store's temporary in the cache directory.

Steps:

1. Open a store and stream a body in irregular, unevenly-sized chunks without declaring its length, so the total length is unknowable when the store opens.
2. After each `write_body` call returns, compare the temporary's length with the length of its header, key, and metadata region plus the body bytes streamed so far.
3. Once the stream ends, retrieve the object and compare its persisted length and bytes to the total streamed.
4. Repeat with a body whose size happens to be zero (empty), and with bodies large enough to span multiple write chunks and file-system block boundaries.

Expected result: Every store succeeds; after every `write_body` call the temporary holds exactly the header, key, metadata region, and the body bytes streamed so far; the persisted object's length and bytes equal exactly what was streamed in every case, including the empty-body and multi-chunk boundary cases.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.UNSIZED-01
Satisfied by	IMP-TC.COMPAT.UNSIZED-01 (durable-cache/tests/compat_unsized.rs:23)

[TC-COMPAT.CONCURRENT-01]

Same-key writes stay isolated ⟨⟨*test case*⟩⟩ `draft`

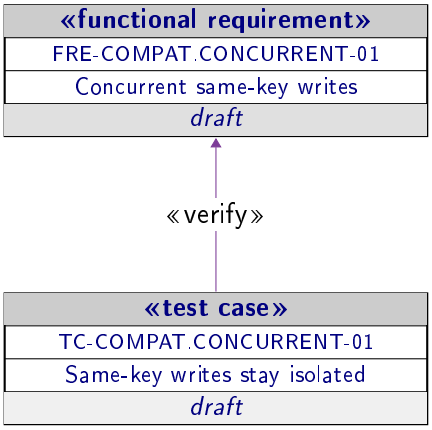


Figure 7.21: Same-key writes stay isolated

Objective: Concurrent writes of the same key never interleave their bytes, and each is durably published as a complete object or not at all, while in-process lookups may see a partial hit of one in-flight writer. (Convergence on a single winner is covered separately by TC-COMPLIANCE.REPLACE-01.)

Setup: The backend driven directly through the `Storage` interface by a test harness; a "restart" constructs a new backend instance over the same cache directory, with a second backend instance in a separate process for the cross-process case.

Steps:

1. Start two concurrent stores under the same key, each with a distinctive, easily distinguishable byte pattern and a different length, staggered so their write windows overlap.
2. While both stores are in flight, issue concurrent lookups for the key.
3. Let both stores run to completion.
4. Repeat with same-key writers started within the same process (not only across processes), exercising the same-process race the cache lock's age timeout permits.

Expected result: No lookup ever observes a body mixing bytes from more than one writer; every lookup either returns one writer's object whole (or a partial hit of one in-flight writer, per FRE-COMPAT.STREAMING-01) or reports a miss — never

a torn or interleaved result. Once both stores complete, the backend has converged on exactly one whole, uncorrupted object.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.CONCURRENT-01
Satisfied by	IMP-TC.COMPAT.CONCURRENT-01 (durable-cache/tests/compat_concurrent.rs:17)

[TC-COMPAT.FORMATUPGRADE-01]

Foreign records are misses ⟨⟨*test case*⟩⟩ `draft`

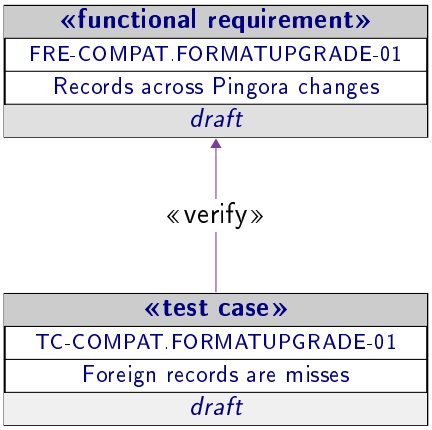


Figure 7.22: Foreign records are misses

Objective: A record written under another record format version is reported as a miss and never decoded, while a record written under the current one is returned exactly; and the `pingora-cache` release in use encodes metadata exactly as the current format version relies on.

Setup: The backend driven directly through the `Storage` interface, instrumentation recording every metadata decode, and a fixed `CacheMeta` together with the bytes `pingora-cache` 0.9.0 encodes it to.

Steps:

1. Construct the backend; store a set of objects.
2. Construct it again over the same directory; look every object up.
3. Store a fresh set; rewrite each record's `version` field to another value, re-sealing its `header_crc`; construct the backend again; look every object up; inspect the cache directory once its restart procedure completes.
4. Encode the fixed `CacheMeta` with the `pingora-cache` in use and compare the result with the fixed bytes; decode the fixed bytes and compare every field with the fixed `CacheMeta`.

Expected result: Step 2 returns every object exactly as stored. Step 3 reports every rewritten object as a miss, and the instrumentation shows no attempt to decode their metadata; the foreign records are reclaimed by the restart procedure (FDS-STORAGE.RECLAIM-01). In step 4 both comparisons are equal. A difference means the release in use changed the encoding the current format relies on, and the format version must be reconsidered before that release is adopted.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.FORMATUPGRADE-01
Satisfied by	IMP-TC.COMPAT.FORMATUPGRADE-01 (durable-cache/tests/compat_formatupgrade.rs:30)

[TC-COMPAT.WRITEFAIL-01]

Write failures fail the store ⟨⟨*test case*⟩⟩ `draft`

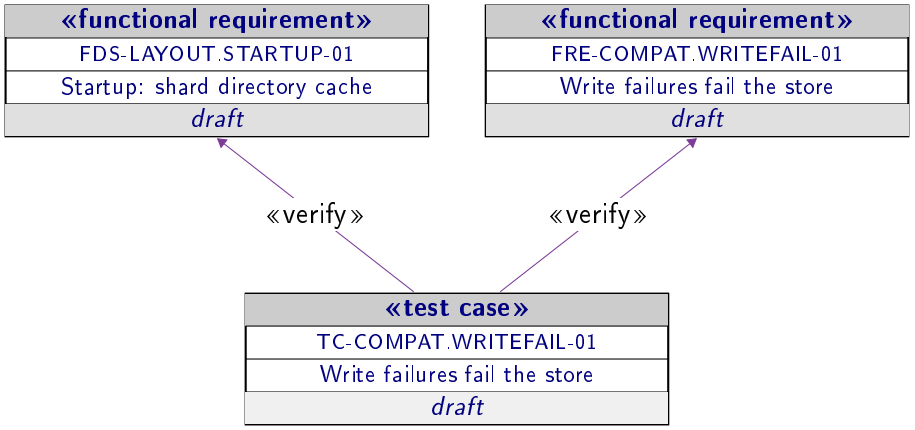


Figure 7.23: Write failures fail the store

Objective: A failure writing or synchronising a store's data is returned as an error from the call during which it occurred, and nothing is published.

Setup: The backend driven directly through the `Storage` interface, with its cache directory on a small file system (e.g. a loop-mounted image of a few megabytes) over a device-mapper `flakey` target able to fail writes on demand.

Steps:

1. Stream a body larger than the file system's free space.
2. On a file system with space, stream a complete body, switch the device to failing writes, and call `finish`.
3. Remove, from outside the backend, a shard directory the backend has already created; store an object into that shard.

4. Remove that shard directory again and make its first-level parent directory read-only, so it cannot be recreated; store another object into it.

5. After each case, look the key up.

Expected result: Step 1's `write_body` and step 2's `finish` each return an error; step 3's store succeeds after the backend recreates the directory once; step 4's store returns an error; no failed store is ever published, so each such lookup reports a miss.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.WRITEFAIL-01 FDS-LAYOUT.STARTUP-01
Satisfied by	IMP-TC.COMPAT.WRITEFAIL-01 (durable-cache/tests/compat_writefail.rs:25)

[TC-COMPAT.ABORTSPACE-01]

Aborted stores release space ⟨⟨*test case*⟩⟩ `draft`

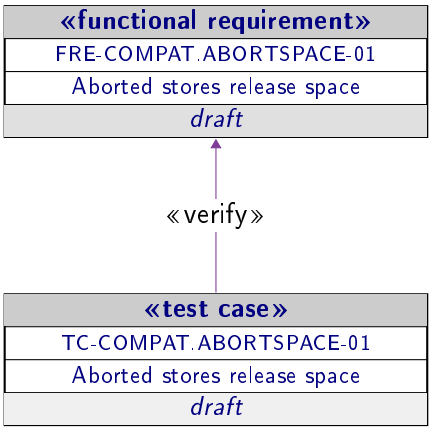


Figure 7.24: Aborted stores release space

Objective: A store that ends without being published loses its temporary no later than when its miss handler is dropped, and the storage it consumed is freed without a restart, however it ended.

Setup: The backend driven directly through the `Storage` interface, as in TC-COMPAT.WRITEFAIL-01.

Steps:

1. Stream part of a large body, then drop the miss handler without finishing, as `HttpCache` does after an upstream failure or a cancellation; inspect the cache directory and free space.

2. Repeat with a partial-hit reader attached to the store before the handler is dropped; read its stream to the end.

3. Repeat with a store ended by a write failure.

4. Repeat step 1 a thousand times in a row.

Expected result: After each dropped handler the store's temporary is gone at once, and the free space it held is available again once the backend has closed the file, without a restart; the attached reader's stream ends with an error; after the thousand aborts the cache directory's disk usage is back to what it was before them.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.ABORTSPACE-01
Satisfied by	IMP-TC.COMPAT.ABORTSPACE-01 (durable-cache/tests/compat_abortspace.rs:18)

[TC-EVICTION.RECONSTRUCT-01]

Reconstruction converges exactly ⟨⟨*test case*⟩⟩ `draft`

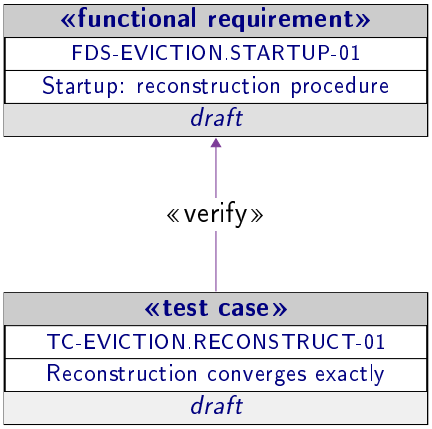


Figure 7.25: Reconstruction converges exactly

Objective: When `recover()` returns, the manager reflects the published set exactly, with record sizes — whether serving waited for it or ran alongside it — with no snapshot or delta log involved.

Setup: The backend with the stock `simple_lru` eviction manager and Eviction Recovery behind an `HttpCache`, driven by a test harness with hooks that pause `recover()` at chosen points.

Steps:

1. Populate a large known set of objects and restart the process; call `recover()` without serving, and let it return.

2. Restart again, and this time begin serving at once. With test hooks pausing `recover()` at chosen points, drive live traffic that: replaces an object after the scan has read it but before it is admitted; removes an object's record after the scan admitted it, leaving its entry in the manager, as a live eviction's purge does when it lands after the scan admitted its victim again; republishes a key after the scan admitted it and a purge removed it; and stores objects the scan never reaches.

3. Let `recover()` return; compare the manager's entries, sizes, `total_items` and `total_size` with the files on disk.

4. Trace file-system activity throughout both runs.

5. Make the backend read-only, then call `recover()`, with a test hook pausing it after it reads any record.

Expected result: After each run, every published object is represented exactly once with its current record size, no entry exists for a removed object, and the totals equal the count and aggregate record size of the published set — including after each interleaving in step 2; no snapshot file or append-only log is read or written at any point; a scan of a read-only backend returns without reading any record or admitting anything.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FDS-EVICTION.STARTUP-01
Satisfied by	IMP-TC.EVICTION.RECONSTRUCT-01 (durable-cache/tests/eviction_reconstruct.rs:20)

[TC-STORAGE.PUBLISH-01]

Atomic same-directory publication ⟨⟨*test case*⟩⟩ `draft`

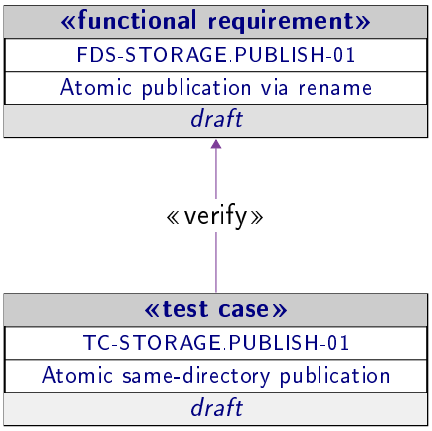


Figure 7.26: Atomic same-directory publication

Objective: The per-writer temporary and rename publish atomically, keep concurrent writers isolated, never reuse another writer's file, synchronise before publishing, report the record size, and publish nothing once the backend is read-only.

Setup: The backend driven directly through the `Storage` interface, with system calls traced (e.g. with `strace`).

Steps:

1. Start two concurrent same-key writers; confirm each writes its own temporary, named with the writer identity and counter FDS-STORAGE.PUBLISH-01 specifies, and neither's bytes appear in the other's file.
2. Before a store opens, create a file at exactly the temporary name its counter will produce next; open the store.
3. Let a store run to completion; confirm the object becomes visible under its canonical name only at the rename, and inspect the traced system-call order and the size `finish` reports.
4. Crash a writer mid-store, before rename; confirm the abandoned temporary is never served or promoted under the canonical name; restart the process and confirm the temporary is gone after its restart-time scan.
5. Trigger a replacement store under an existing key; confirm a reader already streaming the outgoing object completes undisturbed while the new object publishes, and subsequent lookups return only the new object.
6. Make the backend read-only (FDS-COMPAT.READONLY-01) while a store is in flight; let it complete.
7. Inspect the implementation and the tree layout: every temporary is created in the directory of its canonical name, and no code path publishes by copying.

Expected result: Concurrent writers' bytes are never interleaved; the pre-existing file at the temporary name is left untouched and the store uses the next counter value; the temporary's data is synchronised before the rename, and `finish` reports the published file's full length; a crash leaves no promoted incomplete object; the rename is the sole point of visibility; replacement converges to a single winner without disturbing in-flight readers of the old object; the store completing once the backend is read-only reports success, publishes nothing, and leaves no temporary; the inspection finds no cross-directory staging and no copy-based publication.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FDS-STORAGE.PUBLISH-01
Satisfied by	IMP-TC.STORAGE.PUBLISH-02 (durable-cache/tests/storage_publish.rs:24)

[TC-STORAGE.PURGE-01]

Purge unlinks canonical name only ⟨⟨*test case*⟩⟩ `draft`

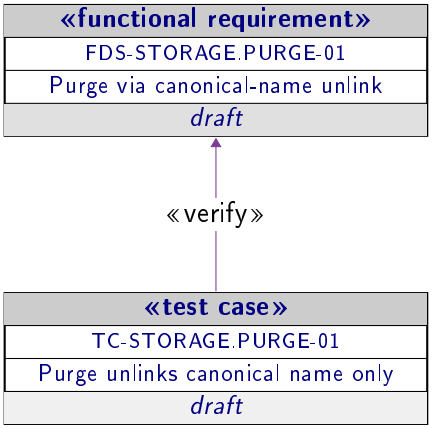


Figure 7.27: Purge unlinks canonical name only

Objective: Purge resolves and unlinks only the canonical path derived from the presented target key's combined hash, reports the outcome the framework distinguishes, is idempotent and a harmless no-op on an absent key, ends a same-key store in flight when it invalidates and leaves it alone when it evicts, and disturbs no reader already streaming the file.

Setup: The backend driven directly through the `Storage` interface by a test harness; a "restart" constructs a new backend instance over the same cache directory.

Steps:

1. Store an object together with a `Vary` variant under the same primary key; purge the primary; confirm only the primary's canonical file is removed from disk and the variant's own file is untouched.

2. Store a primary and a variant again; purge using the variant's own (varianced) `CompactCacheKey`, as `HttpCache::spawn_async_purge` can present it; confirm only the variant's canonical file is removed and the primary's is untouched.

3. Purge the same key a second time, and purge a key that was never stored; confirm both report `NotFound`, with no error and no side effect on any other object.

4. Purge a stored key through each target kind in turn — an `Active` target, and an `Exact` target holding a `KeyOnly` entry key — and confirm each removes that key's canonical file and reports `Purged(None)`, never an entry identifier (ADR-COMPAT.ENTRYID-01).

5. Purge a stored key through an `Exact` target holding an `Identified` entry key; confirm it reports `NotFound` and the file is still on disk.

6. Start a same-key store and let it reach the point of holding an open per-writer temporary, and attach a partial-hit reader to it; invalidate the key; look

the key up, with and without the store's streaming write tag; let the store complete; look the key up; then store the key afresh. Invalidate a key that holds nothing but a store in flight. Repeat the first case with a purge made to evict instead of the invalidation.

7. Attach a reader mid-stream on an already-published object; purge its key while the read is in progress; confirm the reader runs to the end of the body it already opened, never an error and never another object's bytes, and a subsequent lookup reports a miss.

8. Race an invalidation, and then a purge made to evict, against a same-key store's completion, from both orderings: the purge before the store's rename, and after it.

9. With system calls traced, purge an existing object; then, with directory synchronisation made to fail (e.g. over a device-mapper `flakey` target), purge another. With synchronisation still failing, evict an existing object, then invalidate the evicted key, and invalidate a key whose shard directory does not exist.

10. Make the backend read-only (FDS-COMPAT.READONLY-01) and purge an existing object.

11. Put a directory where a key's record would be, so that the `unlink` fails whatever the test's privileges, and purge the key.

Expected result: Only the presented target key's canonical path is ever named or opened by purge; a primary purge never touches a `Vary` variant, a variant purge never touches its primary, and a purge never touches a writer's own temporary; an `Active` and a `KeyOnly` `Exact` target resolve identically and report `Purged(None)`, while an `Identified` `Exact` target reports `NotFound` without touching the file; repeated and never-stored purges report `NotFound` harmlessly; a concurrent reader completes or fails cleanly, never crossing into another object's bytes; an invalidation ends a same-key store in flight: an untagged lookup then misses while a tagged one still finds the store, the reader attached to it gets the whole body, and the store's completion publishes nothing and reports an error, its temporary removed, while a store begun after the purge publishes as usual; the invalidation of a key holding only a store in flight reports `Purged(None)`; a purge made to evict leaves the store to publish; racing a store, an invalidation leaves the key absent in either order, and a purge made to evict leaves whichever of the two landed last; an invalidation synchronises the shard directory after the unlink, or after finding nothing, before it reports, and fails with an error when the synchronisation does, while a purge made to evict does not synchronise and so still removes its record, and an invalidation whose shard directory does not exist has nothing to synchronise and reports `NotFound`; the purge once the backend is read-only fails with an error and leaves the file in place; and the purge whose `unlink` fails reports an error and leaves the path as it was.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FDS-STORAGE.PURGE-01
Satisfied by	IMP-TC.STORAGE.PURGE-01 (durable-cache/tests/storage_purge.rs:21)

[TC-STORAGE.EXPIRE-01]

Expire keeps the body readable ⟨⟨*test case*⟩⟩ `draft`

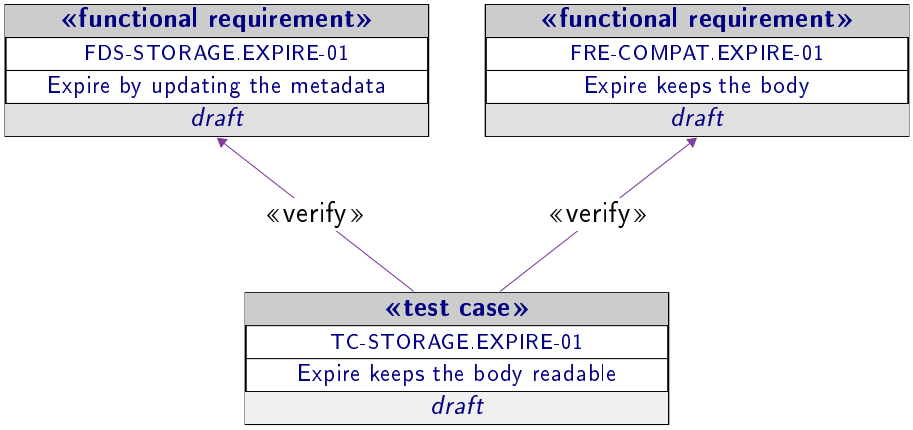


Figure 7.28: Expire keeps the body readable

Objective: An expiry stops the object being served as a fresh hit while leaving its body byte-identical and readable for the revalidation that follows, takes the in-place metadata path whatever headroom the record has left, and reports the outcome each case calls for.

Setup: The backend driven directly through the `Storage` interface by a test harness, with the record's file checked so a write in place can be told from a new file, and with the metadata headroom configurable per instance (SDS-LAYOUT.OBJECTFILE-01).

Steps:

1. Store an object with a known body and a freshness lifetime well in the future; expire its key; look the key up and read the body to the end.
2. Expire the same key a second time, and expire a key that was never stored.
3. Expire a stored key through an `Exact` target holding an `Identified` entry key.
4. With headroom configured, expire an object and compare the file's identity (device and inode) and length before and after; then repeat against an instance configured with zero headroom, for an object whose freshness deadline falls on a whole second, expired at an instant well inside a second.
5. Store an object whose response sets a `stale-while-revalidate` window; expire it; inspect the metadata the next lookup returns.

6. Start a same-key store and let it reach the point of holding an open per-writer temporary; expire the key concurrently; let the store complete; look the key up.

7. Attach a reader mid-stream on an already-published object; expire its key while the read is in progress.

8. Pause an expiry between reading the record's metadata and writing it back; from another task, run a metadata update on the same key; resume the expiry; inspect the resulting metadata.

9. Make the backend read-only (FDS-COMPAT.READONLY-01) and expire an existing object.

10. Expire a stored key whose record has been damaged, truncated, rewritten under another format version, or withdrawn (SDS-LAYOUT.OBJECTFILE-01), comparing the record's bytes before and after; look the key up.

Expected result: After an expiry the lookup still finds the record and its body reads back byte-identical, while the metadata it returns is no longer fresh at any instant after the expiry; a second expiry is idempotent and a never-stored key reports `NotFound`, neither an error nor a side effect; an `Identified Exact` target reports `NotFound` and leaves the record untouched; the expiry writes the metadata in place, leaving the file's device, inode and length unchanged, with headroom and without, the whole-second deadline included; the object's serve-stale window is still open after the expiry, so the stored body remains servable while the revalidation runs; the concurrent store's completing rename replaces the expired record with its fresh object; a reader streaming the object completes its body undisturbed; the paused expiry and the concurrent metadata update serialise on the publication lock, so the record ends carrying one of the two complete metadata sets and never a mixture; the expiry once the backend is read-only fails with an error, leaving the stored metadata unchanged; and each unreadable record is reported `NotFound` and left byte for byte as it was, and a lookup of it misses.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FRE-COMPAT.EXPIRE-01 FDS-STORAGE.EXPIRE-01
Satisfied by	IMP-TC.STORAGE.EXPIRE-01 (durable-cache/tests/storage_expire.rs:26)

[TC-STORAGE.EXECUTOR-01]

Storage never blocks the worker ⟨⟨*test case*⟩⟩ `draft`

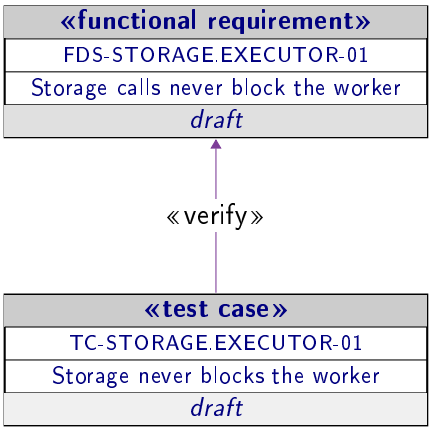


Figure 7.29: Storage never blocks the worker

Objective: No `Storage` call, hit-handler read, or miss-handler write holds the executor thread that polls it for the operation's duration, nor does the restart scan, so a request sharing that worker is not delayed by another request's file I/O.

Setup: The backend driven directly through the `Storage` interface on a *single-worker* runtime — the Pingora default for a service — with a second task on the same worker ticking on a short timer and recording the longest gap between its ticks. That gap is what a co-resident request would have waited.

Steps:

1. Start the ticking task and let it settle.
2. Drive a sequence of complete cache operations against bodies large enough that reads and writes are real work: store, look up and read the body to its end, update the metadata, expire, and purge.
3. Open a store and attach a partial-hit reader to it, then alternate `write_body` and `read_body` so the write and read paths are both active, and finish the store.
4. Read the longest inter-tick gap after each scenario.
5. Repeat both scenarios against a build whose blocking work runs inline on the worker instead of on the blocking pool.
6. Run Eviction Recovery (FDS-EVICTION.STARTUP-01) on a single worker over a damaged record, a stopped writer's temporary, and a record the eviction manager evicts as soon as it is admitted. With a test hook holding each removal where it frees its file, ask another task on that worker to answer.
7. Run Eviction Recovery on a single worker beside another task that notes each turn it gets, over records whose files are all removed between the walk and the ghost-entry check, and note each call the scan makes into the eviction manager.

Expected result: In both scenarios the longest gap stays within a bound far below what one operation's file I/O costs, showing the worker was free throughout; the inline build exceeds it in both, so the measurement is shown to detect the condition it exists to rule out. While the restart scan frees a file it removed, the other task on its worker answers, and it gets a turn between any two calls the scan makes into the eviction manager — one admission and one removal per record. The bound is a *stall* bound, not a latency figure for the operations themselves, which TC-PERFORMANCE.LOOKUP-01 measures.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FDS-STORAGE.EXECUTOR-01
Satisfied by	IMP-TC.STORAGE.EXECUTOR-01 (durable-cache/tests/executor_nonblocking.rs:27)

[TC-STORAGE.RECLAIM-01]

Orphan reclaim at restart ⟨⟨*test case*⟩⟩ `draft`

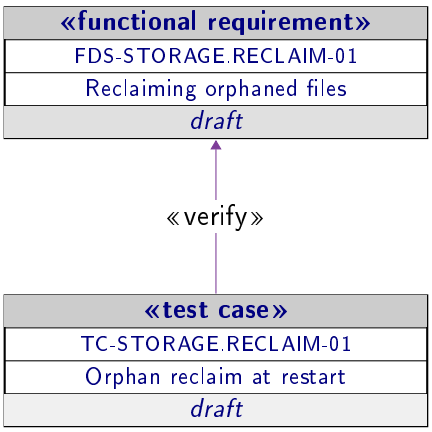


Figure 7.30: Orphan reclaim at restart

Objective: The restart-time scan reclaims a temporary exactly when its writer is not running, with that writer's lease, and a damaged, foreign or withdrawn record — never admitting what it reclaims, never reclaiming a `Vary` variant, whatever its primary holds, never unlinking a record a live store has just replaced, and reporting what it reclaimed.

Setup: The backend with the stock `simple_lru` eviction manager and Eviction Recovery, driven by a test harness with a hook that pauses the scan between classifying and unlinking an entry.

Steps:

1. Leave temporaries named after writers that are not running: one whose lease does not exist, and one whose lease exists but is not locked, as a crashed writer

leaves it. Construct the backend (running its restart procedure); confirm both temporaries are unlinked, and so is the unlocked lease.

2. Leave a temporary named after a lease the harness holds locked, as a live peer would; construct the backend; confirm the temporary and its lease are left untouched, then confirm a later scan reclaims both once the harness releases the lease.

3. Leave a record damaged out of band (as in TC-LAYOUT.OBJECTFILE-01), a record of another format version, and a withdrawn record (FDS-STORAGE.METAUPDATE-01); also copy one key's record to the path of a key never stored, and another's, under its own name, into a shard directory that name does not select; construct the backend; confirm all five are unlinked, the withdrawn one counted under its own class and the others as damaged, none is admitted, and the records at their own paths are.

4. Store four primaries, each with a `Vary` variant. Remove the first primary's file out of band (simulating a completed purge or eviction), damage the second's (as in TC-LAYOUT.OBJECTFILE-01), store the third variant with an artificially earlier `created` timestamp than its primary's own, and the fourth with one at or after it; construct the backend; confirm every variant is admitted, with the same size a directly stored object would receive, and only the damaged primary is reclaimed.

5. With a test hook pausing the scan between classifying a damaged record and unlinking it, publish a fresh record at that path through a live store; resume the scan. Then, with a metadata update paused between its two writes (FDS-STORAGE.METAUPDATE-01), run the scan over that record and let the update finish.

6. Read the reclaim counters once the backend is built; then read them and the scan's log summary after the scan.

7. Repeat steps 1 and 3 in a build without the backend's `metrics` feature; read the log and the registry.

8. Make the backend read-only, run the scan, and read its log summary.

9. Make one shard directory unlistable and one record unreadable, run the scan, and read what it returns and logs.

Expected result: A temporary is reclaimed exactly when its writer is not running, and a running writer's is never touched; a stopped writer's lease is removed and a running writer's kept; damaged, foreign, misplaced and withdrawn records are reclaimed; every variant is admitted, whatever its primary holds; no reclaimed entry, of any class, is ever admitted into the eviction manager; the fresh record published during the pause survives the resumed scan; the record read in the middle of a metadata update is admitted once the update finishes, not reclaimed; once the backend is built the counters read zero for every class, and after the scan they and the log summary report each class's reclaimed files and bytes; without the `metrics` feature the log summary is still written while no reclaim counter is registered; a scan of a read-only backend says in its summary that it stopped; the unlistable directory and the unreadable record are each skipped, named in a warning, and counted in what

the scan returns and in its summary, and every other record is admitted; the scan requires no second pass over the tree beyond TC-EVICTION.RECONSTRUCT-01's own.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FDS-STORAGE.RECLAIM-01
Satisfied by	IMP-TC.STORAGE.RECLAIM-01 (durable-cache/tests/storage_reclaim.rs:24) IMP-TC.STORAGE.RECLAIM-02 (durable-cache/tests/storage_reclaim_reporting.rs:16) – – the reporting

[TC-LAYOUT.SHARDING-01]

Shard layout and directory bound ⟨⟨*test case*⟩⟩ `draft`

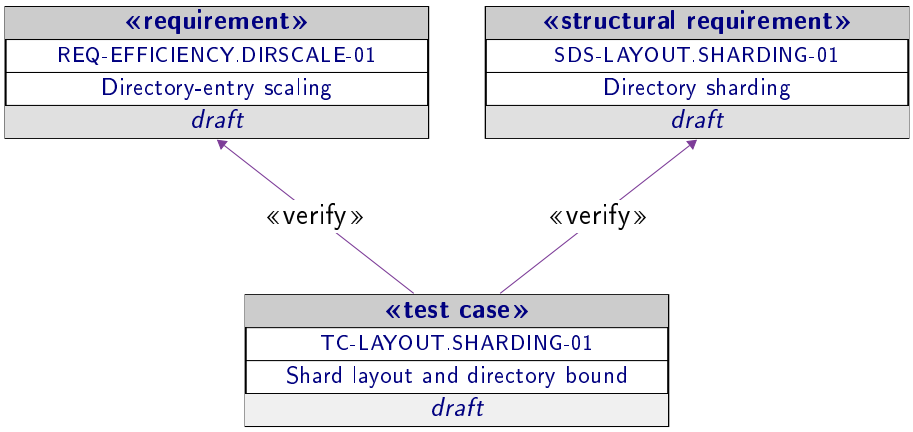


Figure 7.31: Shard layout and directory bound

Objective: The sharded path derived from an object's full `CacheKey` (used on lookup and store) is identical to the path derived from its `CompactCacheKey` (used on purge), for keys with and without a variance; the two-level split always falls on hex characters one-two and three-four of the object's name; the layout keeps every directory within the bound REQ-EFFICIENCY.DIRSCALE-01 sets; and keys that differ only in their user tag are separate objects.

Setup: The backend driven through the `Storage` interface, with a test hook that lets a test choose the name a key resolves to (a real hash cannot be steered to a chosen value).

Steps:

1. Store an object under its `CacheKey`; record the shard path the store resolves to.
2. Purge the same object using only its `CompactCacheKey`; confirm the purge resolves to the identical shard path recorded in step 1.

3. Repeat steps 1–2 for a variant: a key carrying a variance, purged through its own varianced `CompactCacheKey`.

4. Through the hash hook, repeat with combined hashes that probe the split boundary (e.g. a hash whose first four characters repeat, and one where characters two and three differ sharply); confirm the first-level directory is always exactly characters one-two and the second-level directory always exactly characters three-four, for both key forms.

5. Analysis: from the layout's fan-out, compute the expected and the high-percentile maximum entries per leaf directory for 10^8 uniformly hashed objects, adding the temporaries a leaf can hold at peak concurrency.

6. Store a large sample of objects with real hashes (e.g. 10^6); count the entries in every first- and second-level directory and compare the distribution per leaf with the analysis scaled to the sample.

7. Store two objects under keys for the same URL that differ only in their user tag; look both up; run the restart-time scan over the cache directory; then purge one and look both up again.

Expected result: The path derived from the `CacheKey` and from the `CompactCacheKey` never diverge for the same object, with or without a variance; the shard boundary is exactly hex characters one-two (first level) and three-four (second level); the analysis shows fewer than 2,000 entries in every directory at 10^8 objects; the sample shows at most 256 entries in every first-level directory and a per-leaf distribution consistent with the analysis; in step 7 each tagged key has a file of its own, each lookup returns its own body, the scan admits one eviction entry for each key, and after the purge only the purged key is a miss.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	SDS-LAYOUT.SHARDING-01 REQ-EFFICIENCY.DIRSCALE-01
Satisfied by	IMP-TC.LAYOUT.SHARDING-01 (durable-cache/tests/layout_sharding.rs:31)

[TC-LAYOUT.DIRCACHE-01]

Shard cache matches disk at start ⟨⟨*test case*⟩⟩ `draft`

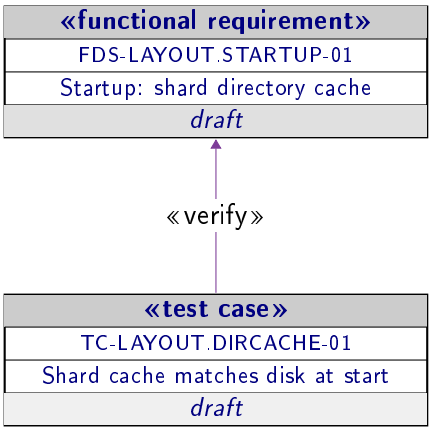


Figure 7.32: Shard cache matches disk at start

Objective: After construction, all 256 first-level directories exist and the in-memory cache exactly matches the second-level directories actually present on disk; concurrent first-time stores into the same new shard all succeed and leave it cached exactly once.

Setup: The backend driven directly through the `Storage` interface by a test harness; a "restart" constructs a new backend instance over the same cache directory.

Steps:

1. Pre-populate a subset of second-level directories directly on disk (bypassing the backend), then construct the backend; confirm all 256 first-level directories exist and the cache contains exactly the pre-populated second-level directories — no more, no fewer.

2. From multiple threads concurrently, store objects that resolve to the same, previously-unseen shard; confirm every store succeeds, the directory is created exactly once, and the cache ends up with exactly one entry for that shard.

3. Restart the backend (a fresh construction) with objects now present under previously-created shards; confirm the newly built cache reflects them and that no second-level directory creation is attempted for them (first-level directories are always ensured at construction, per FDS-LAYOUT.STARTUP-01).

Expected result: The in-memory cache never diverges from what is actually on disk; concurrent first-time stores into the same shard never fail or double-create; a restart always rebuilds an accurate cache from scratch.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	FDS-LAYOUT.STARTUP-01
Satisfied by	IMP-TC.LAYOUT.DIRCACHE-01 (durable-cache/tests/layout_dircache.rs:12)

[TC-LAYOUT.OBJECTFILE-01]

Damaged records treated as a miss ⟨⟨*test case*⟩⟩ `draft`

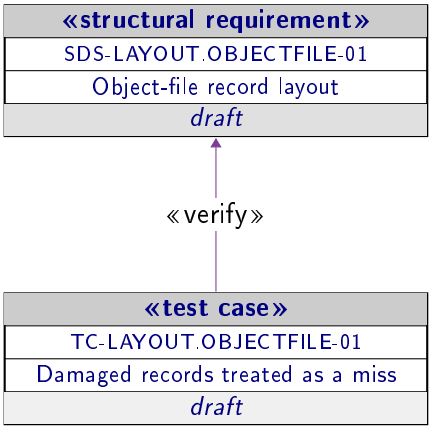


Figure 7.33: Damaged records treated as a miss

Objective: A reader accepts a published record only when its magic, version, header and metadata CRCs, and stored key all validate, its header's lengths fit the record and the file, and its length is exactly the computed body offset plus `body_len`; any single violation is a miss that leaves the file in place; a corrupted body is read to a normal end, and logged when the read-time body check is on; and a metadata read racing an in-place update sees one complete version.

Setup: The backend driven directly through the `Storage` interface by a test harness; a "restart" constructs a new backend instance over the same cache directory. The steps run in the default build and again in a build with the `body-crc-verify` feature.

Steps:

1. Store an object normally, then flip a byte in its on-disk `magic` or `version` field; look it up.
2. Store an object, then flip a byte within the header or metadata region without updating the corresponding CRC; look it up.
3. Store an object, then overwrite its `version` with another value and a matching `header_crc`, as a record written by another format revision would carry; look it up.
4. Store an object, then copy its file over another key's canonical path, so the stored key differs from the looked-up one; look up that other key.

5. Store an object, then truncate its file short of, and separately extend it past, the computed body offset plus `body_len`; look it up.

6. Store an object whose body spans several read chunks, then flip a byte within its body without changing its length; look it up and read the whole body. In the `body-crc-verify` build, repeat with the read-time body check switched on, and with it on also read an intact body whole and the corrupted body from a sought position.

7. Repeatedly apply in-place metadata updates to one object while concurrent lookups read it.

8. Store an object, then rewrite one of its header's lengths with a matching `header_crc`: `meta_capacity` so small that the metadata segments no longer fit, and, separately, `key_len` so large that the key and the body would lie past the end of the file; look it up after each.

Expected result: Each lookup in steps 1–5 and 8 reports a miss, and the damaged file is still present afterwards; every body read in step 6 ends normally, and the backend logs an error naming the file for the whole read of the corrupted body with the check on, and for no other read; every lookup in step 7 returns either the metadata before an update or after it, never a mixture and never a miss while no update is running. A store that never reaches completion is covered separately (TC-COMPLIANCE.INCOMPLETE-01): it is never visible under the canonical name at all, so this test case's scope is damage to an already-published record, not incompleteness.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	SDS-LAYOUT.OBJECTFILE-01
Satisfied by	IMP-TC.LAYOUT.OBJECTFILE-01 (durable-cache/tests/layout_objectfile.rs:24)

[TC-LAYOUT.KEYSEGMENT-01]

Key segment round trip ⟨⟨*test case*⟩⟩ `draft`

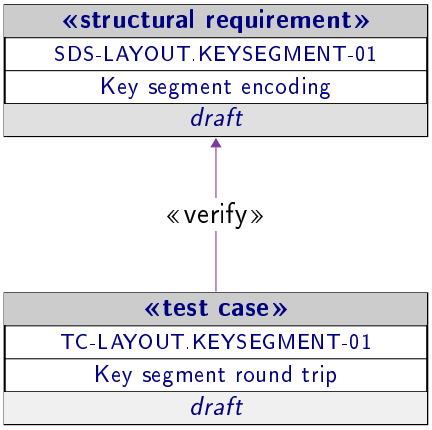


Figure 7.34: Key segment round trip

Objective: A key is stored in exactly the encoding SDS-LAYOUT.KEYSEGMENT-01 specifies and decodes back to the same key; a key segment that does not follow the encoding is rejected, and the restart-time scan reclaims its record.

Setup: For steps 1 and 2, the key segment's encoder and decoder driven directly by unit tests. For step 3, the backend driven through the `Storage` interface, and its restart-time scan run over the same cache directory.

Steps:

1. Encode keys with and without a variance hash, each with an empty and a non-empty tag; inspect each encoding, then decode it.
2. Decode a segment shorter than the primary hash and the variance flag; a segment whose flag is 1 but which ends before the variance hash does; a segment whose flag is 2; and a segment whose tag is not valid UTF-8.
3. Store an object, then set its stored key's variance flag to 2 and reseal the record's metadata and header checksums over the change; run the restart-time scan.

Expected result: In step 1 every field lies at the offset the encoding specifies, the tag runs to the end of the segment, and every key decodes to itself; every decode in step 2 fails; in step 3 the scan reclaims the record as damaged and does not admit it.

View complete diagram.

Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩
Verifies	SDS-LAYOUT.KEYSEGMENT-01
Satisfied by	IMP-TC.LAYOUT.KEYSEGMENT-01 (durable-cache/tests/layout_keysegment.rs:17)

8. Traceability Analysis

At this stage of the specification some coverage obligations are intentionally open. Each IMP obligation — on the functional designs, the structural designs, and the test cases alike — awaits the implementation itself, and closes automatically once that implementation exists: `reqman-report.py` scans the source tree for `// IMP-<SLUG>-<NN>: satisfy <TARGET-ID>` marker comments at every build and links each one back to the requirement it satisfies, alongside a pointer to the exact file and line. REQ-EFFICIENCY.FRAGMENTATION-01's satisfy obligation stays open for this version by the conflict ADR-EFFICIENCY.COMPACTION-01 declares. Forward-declared expectations on artifacts not yet written, when there are any, are tracked in **Planned Requirements** below.

The retarget of this document from Pingora 0.8.1 to 0.9.0 — the revision of HLR-COMPAT.PINGORA-01 that release calls for — landed one revision ahead of the implementation, and the implementation has since followed. The artifacts that retarget added — FRE-COMPAT.EXPIRE-01, ADR-COMPAT.ENTRYID-01, ADR-STORAGE.EXPIRE-01, FDS-STORAGE.EXPIRE-01 and TC-STORAGE.EXPIRE-01 — are closed by code built against 0.9.0, as are the ones it rewrote to match that release's interfaces, chiefly FDS-STORAGE.PURGE-01, FDS-EVICTION.STARTUP-01 and ADR-OBSERVABILITY.METRICS-01. No IMP link in the report below is stale on that account, and the one gap it reports is REQ-EFFICIENCY.FRAGMENTATION-01's, deliberately left open by ADR-EFFICIENCY.COMPACTION-01's declared conflict.

Traceability Report

Artifact Summary

Type	`draft`	Total
ADR	14	14
FDS	11	11
FRE	31	31
HLR	8	8
REQ	2	2
SDS	3	3
TC	34	34
Total	**103**	**103**

Not Yet Accepted

Artifacts whose status has not (or not yet) reached `accepted`, grouped by status:

Status: `draft` — 103 artifacts		
Type	**ID**	**Title**
ADR	ADR-COMPAT.ENTRYID-01	No storage-assigned entry IDs

Status: `draft` (continued)

Type	ID	Title
ADR	ADR-COMPAT.EVICTION-01	Eviction-state reconstruction strategy
ADR	ADR-COMPAT.READONLY-01	Read-only on the upgrade signal
ADR	ADR-EFFICIENCY.COMPACTION-01	Compaction deferred for v1
ADR	ADR-OBSERVABILITY.METRICS-01	Metrics via the prometheus crate
ADR	ADR-PERSISTENCE.DURABILITY-01	Host-crash safety via sync and CRC
ADR	ADR-PERSISTENCE.MECHANISM-01	Plain file system storage
ADR	ADR-STORAGE.ATOMICPUBLISH-01	Publish by same-directory rename
ADR	ADR-STORAGE.EXECUTOR-01	File I/O on the blocking pool
ADR	ADR-STORAGE.EXPIRE-01	Expire by rewriting the metadata
ADR	ADR-STORAGE.METAMOVE-01	Metadata moves deferred for v1
ADR	ADR-STORAGE.OBJECTFILE-01	Combined metadata and body file
ADR	ADR-STORAGE.RECLAIM-01	Reclaim via the restart-time scan
ADR	ADR-STORAGE.WRITERID-01	Writer identity in temp names
FDS	FDS-COMPAT.READONLY-01	Read-only on the upgrade signal
FDS	FDS-EVICTION.STARTUP-01	Startup: reconstruction procedure
FDS	FDS-LAYOUT.STARTUP-01	Startup: shard directory cache
FDS	FDS-OBSERVABILITY.HEADROOM-01	Headroom-exceeded observability
FDS	FDS-STORAGE.EXECUTOR-01	Storage calls never block the worker
FDS	FDS-STORAGE.EXPIRE-01	Expire by updating the metadata
FDS	FDS-STORAGE.METAUPDATE-01	Metadata update: in place or withdrawn
FDS	FDS-STORAGE.PARTIAL-01	Streaming partial hits
FDS	FDS-STORAGE.PUBLISH-01	Atomic publication via rename
FDS	FDS-STORAGE.PURGE-01	Purge via canonical-name unlink
FDS	FDS-STORAGE.RECLAIM-01	Reclaiming orphaned files

Status: `draft` (continued)		
Type	**ID**	**Title**
FRE	FRE-COMPAT.ABORTSPACE-01	Aborted stores release space
FRE	FRE-COMPAT.CONCURRENT-01	Concurrent same-key writes
FRE	FRE-COMPAT.EVICTION-01	Eviction state consistency
FRE	FRE-COMPAT.EXPIRE-01	Expire keeps the body
FRE	FRE-COMPAT.FORMATUPGRADE-01	Records across Pingora changes
FRE	FRE-COMPAT.HOTUPGRADE-01	Hot-upgrade cache consistency
FRE	FRE-COMPAT.STARTRECLAIM-01	Reclaim leftovers at start
FRE	FRE-COMPAT.STREAMDELIVERY-01	Partial-hit chunk delivery
FRE	FRE-COMPAT.STREAMEND-01	Partial-hit completion
FRE	FRE-COMPAT.STREAMFLAG-01	Advertise streaming writes
FRE	FRE-COMPAT.STREAMING-01	Partial hits on lookup
FRE	FRE-COMPAT.UNSIZED-01	Bodies of unknown length
FRE	FRE-COMPAT.WRITEFAIL-01	Write failures fail the store
FRE	FRE-COMPLIANCE.BODY-01	Faithful round-trip of the body
FRE	FRE-COMPLIANCE.FRESHNESS-01	Preserve freshness and age inputs
FRE	FRE-COMPLIANCE.HEADERS-01	Faithful header round-trip
FRE	FRE-COMPLIANCE.INCOMPLETE-01	Never serve incomplete responses
FRE	FRE-COMPLIANCE.METAABSENT-01	Update of an absent key
FRE	FRE-COMPLIANCE.METAATOMIC-01	Atomic metadata update
FRE	FRE-COMPLIANCE.METAPERSIST-01	Freshened metadata persists
FRE	FRE-COMPLIANCE.PARTIALABORT-01	Aborted partial hits end in error
FRE	FRE-COMPLIANCE.PURGE-01	Honour purge and invalidation
FRE	FRE-COMPLIANCE.PURGEABSENT-01	Purge or expiry of absent key
FRE	FRE-COMPLIANCE.PURGEDURABLE-01	Purge acknowledged once durable
FRE	FRE-COMPLIANCE.READERISOLATION-01	Readers survive replace/purge
FRE	FRE-COMPLIANCE.REPLACE-01	Replace stored responses atomically
FRE	FRE-COMPLIANCE.REVALIDATION-01	Freshen metadata
FRE	FRE-COMPLIANCE.TRAILERS-01	Trailer fields kept distinct
FRE	FRE-COMPLIANCE.VALIDATORS-01	Preserve validator fields
FRE	FRE-COMPLIANCE.VARY-01	Store and select Vary variants
FRE	FRE-COMPLIANCE.VARYISOLATION-01	Variant isolation

Status: `draft` (continued)

Type	ID	Title
HLR	HLR-COMPAT.PINGORA-01	Compatible with Pingora
HLR	HLR-COMPLIANCE.RFC-01	No added RFC 9111/9110 deviation
HLR	HLR-EFFICIENCY.SPACE-01	Space Efficiency
HLR	HLR-PERFORMANCE.LOOKUP-01	Lookup latency flat with size
HLR	HLR-PERSISTENCE.CAPACITY-01	Capacity beyond RAM
HLR	HLR-PERSISTENCE.CRASHSAFE-01	Host-crash safety
HLR	HLR-PERSISTENCE.DURABLE-01	Durable Persistence
HLR	HLR-SCOPE.V1-01	First Functional Version
REQ	REQ-EFFICIENCY.DIRSCALE-01	Directory-entry scaling
REQ	REQ-EFFICIENCY.FRAGMENTATION-01	Sub-linear small-object overhead
SDS	SDS-LAYOUT.KEYSEGMENT-01	Key segment encoding
SDS	SDS-LAYOUT.OBJECTFILE-01	Object-file record layout
SDS	SDS-LAYOUT.SHARDING-01	Directory sharding
TC	TC-COMPAT.ABORTSPACE-01	Aborted stores release space
TC	TC-COMPAT.CONCURRENT-01	Same-key writes stay isolated
TC	TC-COMPAT.EVICTION-01	Exact eviction state after restart
TC	TC-COMPAT.FORMATUPGRADE-01	Foreign records are misses
TC	TC-COMPAT.HOTUPGRADE-01	Cache consistent across hot upgrade
TC	TC-COMPAT.STREAMING-01	Followers stream a partial hit
TC	TC-COMPAT.UNSIZED-01	Unsized body accepted and persisted
TC	TC-COMPAT.WRITEFAIL-01	Write failures fail the store
TC	TC-COMPLIANCE.BODY-01	Body round-trips byte for byte
TC	TC-COMPLIANCE.FRESHNESS-01	Freshness and age inputs preserved
TC	TC-COMPLIANCE.HEADERS-01	Header fields round-trip faithfully
TC	TC-COMPLIANCE.INCOMPLETE-01	Incomplete responses never served
TC	TC-COMPLIANCE.PURGE-01	Purge makes all variants unreachable
TC	TC-COMPLIANCE.REPLACE-01	Stores replace atomically
TC	TC-COMPLIANCE.REVALIDATION-01	Metadata freshened atomically
TC	TC-COMPLIANCE.RFC-01	No regression against MemCache

Status: `draft` (continued)		
Type	**ID**	**Title**
TC	TC-COMPLIANCE.TRAILERS-01	Trailer fields kept distinct
TC	TC-COMPLIANCE.VALIDATORS-01	Validator fields preserved
TC	TC-COMPLIANCE.VARY-01	Vary variants stored and selected
TC	TC-EFFICIENCY.SPACE-01	Space overhead measured
TC	TC-EVICTION.RECONSTRUCT-01	Reconstruction converges exactly
TC	TC-LAYOUT.DIRCACHE-01	Shard cache matches disk at start
TC	TC-LAYOUT.KEYSEGMENT-01	Key segment round trip
TC	TC-LAYOUT.OBJECTFILE-01	Damaged records treated as a miss
TC	TC-LAYOUT.SHARDING-01	Shard layout and directory bound
TC	TC-PERFORMANCE.LOOKUP-01	Lookup latency flat with size
TC	TC-PERSISTENCE.CAPACITY-01	Cache larger than host RAM
TC	TC-PERSISTENCE.CRASHSAFE-01	Host crash serves no bad content
TC	TC-PERSISTENCE.RESTART-01	Objects survive a process exit
TC	TC-STORAGE.EXECUTOR-01	Storage never blocks the worker
TC	TC-STORAGE.EXPIRE-01	Expire keeps the body readable
TC	TC-STORAGE.PUBLISH-01	Atomic same-directory publication
TC	TC-STORAGE.PURGE-01	Purge unlinks canonical name only
TC	TC-STORAGE.RECLAIM-01	Orphan reclaim at restart

Traceability

[HLR] HLR-SCOPE.V1-01 First Functional Version	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Satisfied by	ADR-OBSERVABILITY.METRICS-01 ADR-PERSISTENCE.MECHANISM-01
Traced from	ADR-STORAGE.METAMOVE-01
Required coverage	✓ ADR via ⟨⟨*satisfy*⟩⟩

[HLR] HLR-PERSISTENCE.DURABLE-01 Durable Persistence	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Satisfied by	ADR-PERSISTENCE.MECHANISM-01
Derived from	FRE-COMPAT.EVICTION-01 FRE-COMPAT.FORMATUPGRADE-01 FRE-COMPAT.HOTUPGRADE-01
Verified by	TC-PERSISTENCE.RESTART-01
Required coverage	✓ ADR-PERSISTENCE.MECHANISM-01 via ⟨⟨*satisfy*⟩⟩ — *Establish the storage substrate and mechanism that persists cached objects across an exit of the cache process.* ✓ TC via ⟨⟨*verify*⟩⟩

[HLR] HLR-PERSISTENCE.CAPACITY-01 Capacity beyond RAM	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Satisfied by	ADR-PERSISTENCE.MECHANISM-01
Verified by	TC-PERSISTENCE.CAPACITY-01
Required coverage	✓ ADR-PERSISTENCE.MECHANISM-01 via ⟨⟨*satisfy*⟩⟩ — *Choose a storage substrate whose capacity is bounded by persistent storage rather than by process RAM.* ✓ TC via ⟨⟨*verify*⟩⟩

[HLR] HLR-PERSISTENCE.CRASHSAFE-01 Host-crash safety	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Satisfied by	ADR-PERSISTENCE.DURABILITY-01
Derived from	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.PURGEDURABLE-01
Verified by	TC-PERSISTENCE.CRASHSAFE-01
Required coverage	✓ ADR-PERSISTENCE.DURABILITY-01 via ⟨⟨*satisfy*⟩⟩ — *Decide which writes are synchronised to stable storage, and what integrity check detects the rest, so that a host crash can lose objects but never make the cache serve corrupted or purged content.* ✓ TC via ⟨⟨*verify*⟩⟩

[HLR] HLR-EFFICIENCY.SPACE-01 Space Efficiency	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Conflicts with	ADR-EFFICIENCY.COMPACTION-01
Derived from	FRE-COMPAT.ABORTSPACE-01 FRE-COMPAT.STARTRECLAIM-01 REQ-EFFICIENCY.FRAGMENTATION-01
Verified by	TC-EFFICIENCY.SPACE-01
Required coverage	✓ REQ via ⟨⟨*deriveReqt*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩

[HLR] HLR-PERFORMANCE.LOOKUP-01 Lookup latency flat with size	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Derived from	REQ-EFFICIENCY.DIRSCALE-01
Verified by	TC-PERFORMANCE.LOOKUP-01
Required coverage	✓ REQ via ⟨⟨*deriveReqt*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩

[HLR] HLR-COMPAT.PINGORA-01 Compatible with Pingora	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Derived from	FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.EVICTION-01 FRE-COMPAT.EXPIRE-01 FRE-COMPAT.FORMATUPGRADE-01 FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 FRE-COMPAT.STREAMFLAG-01 FRE-COMPAT.STREAMING-01 FRE-COMPAT.UNSIZED-01 FRE-COMPAT.WRITEFAIL-01
Required coverage	✓ FRE via ⟨⟨*deriveReqt*⟩⟩

[HLR] HLR-COMPLIANCE.RFC-01 No added RFC 9111/9110 deviation	
Stereotype	⟨⟨*stakeholder requirement*⟩⟩
Status	`draft`
Satisfied by	ADR-STORAGE.EXECUTOR-01 FDS-STORAGE.EXECUTOR-01
Derived from	FRE-COMPLIANCE.BODY-01 FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.METAABSENT-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.PARTIALABORT-01 FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPLIANCE.REPLACE-01 FRE-COMPLIANCE.REVALIDATION-01 FRE-COMPLIANCE.TRAILERS-01 FRE-COMPLIANCE.VALIDATORS-01 FRE-COMPLIANCE.VARY-01 FRE-COMPLIANCE.VARYISOLATION-01
Verified by	TC-COMPLIANCE.RFC-01
Required coverage	✓ FRE via ⟨⟨*deriveReqt*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩

[FRE] FRE-COMPAT.UNSIZED-01 Bodies of unknown length	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPAT.PINGORA-01
Satisfied by	ADR-PERSISTENCE.MECHANISM-01 FDS-STORAGE.PUBLISH-01
Verified by	TC-COMPAT.UNSIZED-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩

[FRE] FRE-COMPAT.CONCURRENT-01 Concurrent same-key writes	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.REPLACE-01
Derives from	HLR-COMPAT.PINGORA-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01
Traced from	ADR-STORAGE.WRITERID-01 FRE-COMPAT.HOTUPGRADE-01
Verified by	TC-COMPAT.CONCURRENT-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩

[FRE] FRE-COMPAT.EVICTION-01 Eviction state consistency	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPAT.PINGORA-01 HLR-PERSISTENCE.DURABLE-01
Satisfied by	ADR-COMPAT.ENTRYID-01 ADR-COMPAT.EVICTION-01 FDS-EVICTION.STARTUP-01
Traced from	ADR-PERSISTENCE.MECHANISM-01 ADR-STORAGE.METAMOVE-01 FRE-COMPAT.HOTUPGRADE-01 SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPAT.EVICTION-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ ADR via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPAT.HOTUPGRADE-01 Hot-upgrade cache consistency	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.EVICTION-01 FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.PURGEDURABLE-01
Derives from	HLR-COMPAT.PINGORA-01 HLR-PERSISTENCE.DURABLE-01
Satisfied by	ADR-COMPAT.READONLY-01 FDS-COMPAT.READONLY-01 FDS-EVICTION.STARTUP-01
Traced from	ADR-PERSISTENCE.MECHANISM-01 ADR-STORAGE.RECLAIM-01 ADR-STORAGE.WRITERID-01 FDS-STORAGE.RECLAIM-01 FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.FORMATUPGRADE-01
Verified by	TC-COMPAT.HOTUPGRADE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-EVICTION.STARTUP-01 via ⟨⟨*satisfy*⟩⟩ — *Realise the reconstruction procedure so the incoming process's eviction view is rebuilt from durably published objects only, keeping the shared cache consistent across the hot-upgrade window.* ✓ FDS-COMPAT.READONLY-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how the outgoing process's backend learns that the upgrade signal arrived, what it suppresses from then on (publication, purge, metadata update) and how it reports refusals, and how the incoming process's reconstruction is ordered relative to the outgoing process's last publications.*

[FRE] FRE-COMPAT.STREAMFLAG-01 Advertise streaming writes	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.STREAMING-01
Derives from	HLR-COMPAT.PINGORA-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPAT.STREAMING-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPAT.STREAMING-01 Partial hits on lookup	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPAT.PINGORA-01
Traced from	ADR-STORAGE.OBJECTFILE-01 FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.HOTUPGRADE-01 FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 FRE-COMPAT.STREAMFLAG-01 FRE-COMPLIANCE.PARTIALABORT-01 FRE-COMPLIANCE.REPLACE-01 SDS-LAYOUT.OBJECTFILE-01 SDS-LAYOUT.SHARDING-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPAT.STREAMING-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-STORAGE.PARTIAL-01 via ⟨⟨*satisfy*⟩⟩ — *Specify the in-flight write registry, the lookup order (in-flight write before the published record), and the most-recently-opened tie-break.*

[FRE] FRE-COMPAT.STREAMDELIVERY-01 Partial-hit chunk delivery	
Stereotype	⟨⟨*performance requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.STREAMING-01
Derives from	HLR-COMPAT.PINGORA-01
Traced from	ADR-STORAGE.EXECUTOR-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPAT.STREAMING-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-STORAGE.PARTIAL-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how a reader learns of each appended chunk so that it can deliver the chunk within the latency bound.*

[FRE] FRE-COMPAT.STREAMEND-01 Partial-hit completion	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.PARTIALABORT-01
Derives from	HLR-COMPAT.PINGORA-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Verified by	TC-COMPAT.STREAMING-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS-STORAGE.PARTIAL-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how each reader delivers the remainder and ends normally on completion, and why its reader stays valid across the object's durable publication.*

[FRE] FRE-COMPAT.FORMATUPGRADE-01 Records across Pingora changes	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.HOTUPGRADE-01
Derives from	HLR-COMPAT.PINGORA-01 HLR-PERSISTENCE.DURABLE-01
Traced from	SDS-LAYOUT.KEYSEGMENT-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPAT.FORMATUPGRADE-01
Required coverage	✓ SDS via ⟨⟨*satisfy*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩

[FRE] FRE-COMPAT.WRITEFAIL-01 Write failures fail the store	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.ABORTSPACE-01 FRE-COMPLIANCE.INCOMPLETE-01
Derives from	HLR-COMPAT.PINGORA-01
Satisfied by	FDS-STORAGE.PUBLISH-01
Traced from	FRE-COMPAT.ABORTSPACE-01
Verified by	TC-COMPAT.WRITEFAIL-01
Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*satisfy*⟩⟩ — *Specify how a failed write or sync during a store is detected and surfaced as an error from the call in progress, and that the store is then treated as aborted.* ✓ TC via ⟨⟨*verify*⟩⟩

[FRE] FRE-COMPAT.ABORTSPACE-01 Aborted stores release space	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.STARTRECLAIM-01 FRE-COMPAT.WRITEFAIL-01 FRE-COMPLIANCE.INCOMPLETE-01
Derives from	HLR-EFFICIENCY.SPACE-01
Satisfied by	FDS-STORAGE.PUBLISH-01
Traced from	FRE-COMPAT.STARTRECLAIM-01 FRE-COMPAT.WRITEFAIL-01
Verified by	TC-COMPAT.ABORTSPACE-01
Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*satisfy*⟩⟩ — *Specify that an unpublished store's per-writer temporary is unlinked when the store ends, including on abort and on write failure, and how attached partial-hit readers are unaffected by that unlink.* ✓ TC via ⟨⟨*verify*⟩⟩

[FRE] FRE-COMPAT.STARTRECLAIM-01 Reclaim leftovers at start	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.ABORTSPACE-01 FRE-COMPLIANCE.INCOMPLETE-01
Derives from	HLR-EFFICIENCY.SPACE-01
Satisfied by	ADR-STORAGE.RECLAIM-01 ADR-STORAGE.WRITERID-01 FDS-STORAGE.RECLAIM-01
Traced from	FRE-COMPAT.ABORTSPACE-01
Verified by	TC-COMPLIANCE.INCOMPLETE-01
Required coverage	✓ FDS via ⟨⟨*satisfy*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩

[FRE] FRE-COMPAT.EXPIRE-01 Expire keeps the body	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.REVALIDATION-01
Derives from	HLR-COMPAT.PINGORA-01
Satisfied by	ADR-STORAGE.EXPIRE-01 FDS-STORAGE.EXPIRE-01
Traced from	FRE-COMPLIANCE.PURGEABSENT-01
Verified by	TC-STORAGE.EXPIRE-01
Required coverage	✓ ADR via ⟨⟨*satisfy*⟩⟩ ✓ TC via ⟨⟨*verify*⟩⟩

[ADR] ADR-PERSISTENCE.MECHANISM-01 Plain file system storage	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.EVICTION-01 FRE-COMPAT.HOTUPGRADE-01 FRE-COMPLIANCE.INCOMPLETE-01
Satisfies	FRE-COMPAT.UNSIZED-01 HLR-PERSISTENCE.CAPACITY-01 HLR-PERSISTENCE.DURABLE-01 HLR-SCOPE.V1-01
Refined by	ADR-COMPAT.ENTRYID-01 ADR-COMPAT.EVICTION-01 ADR-COMPAT.READONLY-01 ADR-EFFICIENCY.COMPACTION-01 ADR-PERSISTENCE.DURABILITY-01 ADR-STORAGE.EXECUTOR-01 ADR-STORAGE.OBJECTFILE-01 ADR-STORAGE.RECLAIM-01 SDS-LAYOUT.SHARDING-01
Traced from	FDS-EVICTION.STARTUP-01
Derived from	REQ-EFFICIENCY.DIRSCALE-01 REQ-EFFICIENCY.FRAGMENTATION-01
Required coverage	✓ REQ-EFFICIENCY.FRAGMENTATION-01 via ⟨⟨*deriveReqt*⟩⟩ — *Bound the aggregate space overhead a plain file system imposes on objects smaller than one block — block rounding and per-file inode/directory-entry costs — so that it grows sub-linearly with the number of such objects.* ✓ REQ-EFFICIENCY.DIRSCALE-01 via ⟨⟨*deriveReqt*⟩⟩ — *Bound the number of entries that accumulate in any one directory, up to a stated cache capacity, so object lookup and maintenance stay fast as the number of cached objects grows.* ✓ ADR via ⟨⟨*refine*⟩⟩

[REQ] REQ-EFFICIENCY.FRAGMENTATION-01 Sub-linear small-object overhead	
Stereotype	⟨⟨*requirement*⟩⟩
Status	`draft`
Derives from	ADR-PERSISTENCE.MECHANISM-01 HLR-EFFICIENCY.SPACE-01
Conflicts with	ADR-EFFICIENCY.COMPACTION-01
Traced from	ADR-STORAGE.OBJECTFILE-01 FDS-STORAGE.METAUPDATE-01
Required coverage	☐ ADR via ⟨⟨*satisfy*⟩⟩

[REQ] REQ-EFFICIENCY.DIRSCALE-01 Directory-entry scaling	
Stereotype	⟨⟨*requirement*⟩⟩
Status	`draft`
Derives from	ADR-PERSISTENCE.MECHANISM-01 HLR-PERFORMANCE.LOOKUP-01
Satisfied by	SDS-LAYOUT.SHARDING-01
Verified by	TC-LAYOUT.SHARDING-01
Required coverage	✓ SDS-LAYOUT.SHARDING-01 via ⟨⟨*satisfy*⟩⟩ — *Lay out objects so that, for up to* 10^8 *uniformly hashed objects, no directory holds more than 2,000 entries.* ✓ TC via ⟨⟨*verify*⟩⟩

[ADR] ADR-STORAGE.OBJECTFILE-01 Combined metadata and body file	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Refines	ADR-PERSISTENCE.MECHANISM-01
Traces to	FRE-COMPAT.STREAMING-01 REQ-EFFICIENCY.FRAGMENTATION-01
Traced from	ADR-EFFICIENCY.COMPACTION-01
Refined by	ADR-STORAGE.ATOMICPUBLISH-01 ADR-STORAGE.EXPIRE-01 ADR-STORAGE.METAMOVE-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PARTIAL-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 SDS-LAYOUT.OBJECTFILE-01
Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify how object publication is made atomic under concurrent same-key writes and process crashes.* ✓ FDS-STORAGE.METAUPDATE-01 via ⟨⟨*refine*⟩⟩ — *Specify how a metadata update that grows past its record's reserved capacity is handled.* ✓ SDS-LAYOUT.OBJECTFILE-01 via ⟨⟨*refine*⟩⟩ — *Specify the concrete on-disk record format — header, key, metadata region, and body placement — that realises the combined metadata-and-body file.*

[ADR] ADR-PERSISTENCE.DURABILITY-01 Host-crash safety via sync and CRC	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Refines	ADR-PERSISTENCE.MECHANISM-01
Traces to	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.PURGEDURABLE-01
Satisfies	HLR-PERSISTENCE.CRASHSAFE-01
Traced from	ADR-STORAGE.EXECUTOR-01 ADR-STORAGE.EXPIRE-01
Refined by	FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 SDS-LAYOUT.OBJECTFILE-01
Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify the fdatasync of the temporary before its rename, and how a failed sync is surfaced.* ✓ FDS-STORAGE.PURGE-01 via ⟨⟨*refine*⟩⟩ — *Specify the fsync of the shard directory before an invalidating purge is reported, and that a purge made to evict is not synchronised.* ✓ FDS-STORAGE.METAUPDATE-01 via ⟨⟨*refine*⟩⟩ — *Specify that a metadata update, written in place or withdrawing the record, is not synced.* ✓ SDS-LAYOUT.OBJECTFILE-01 via ⟨⟨*refine*⟩⟩ — *Add the body CRC-32C field and how it is computed and recorded, and specify the optional read-time check that logs a mismatch at the end of a whole-body read.*

[ADR] ADR-STORAGE.ATOMICPUBLISH-01 Publish by same-directory rename	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Refines	ADR-STORAGE.OBJECTFILE-01
Traces to	ADR-STORAGE.RECLAIM-01
Satisfies	FRE-COMPAT.CONCURRENT-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPLIANCE.REPLACE-01
Refined by	ADR-STORAGE.WRITERID-01 FDS-STORAGE.PUBLISH-01
Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify the temporary's naming and placement, and the completion sequence that ends in the rename.*

[ADR] ADR-STORAGE.WRITERID-01 Writer identity in temp names	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Refines	ADR-STORAGE.ATOMICPUBLISH-01
Traces to	ADR-STORAGE.RECLAIM-01 FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.HOTUPGRADE-01
Satisfies	FRE-COMPAT.STARTRECLAIM-01
Refined by	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.RECLAIM-01
Required coverage	✓ FDS-STORAGE.PUBLISH-01 via ⟨⟨*refine*⟩⟩ — *Specify the temporary name format and exclusive creation, including what happens on a collision.* ✓ FDS-STORAGE.RECLAIM-01 via ⟨⟨*refine*⟩⟩ — *Specify the liveness test the scan applies to a temporary's writer (its lease's lock), the removal of stopped writers' leases, and the deployment constraint.*

[FDS] FDS-STORAGE.PUBLISH-01 Atomic publication via rename	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-PERSISTENCE.DURABILITY-01 ADR-STORAGE.ATOMICPUBLISH-01 ADR-STORAGE.OBJECTFILE-01 ADR-STORAGE.WRITERID-01
Traces to	FDS-STORAGE.PARTIAL-01 SDS-LAYOUT.SHARDING-01
Satisfies	FRE-COMPAT.ABORTSPACE-01 FRE-COMPAT.CONCURRENT-01 FRE-COMPAT.UNSIZED-01 FRE-COMPAT.WRITEFAIL-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPLIANCE.REPLACE-01
Traced from	FDS-STORAGE.EXECUTOR-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PARTIAL-01 FDS-STORAGE.PURGE-01 FDS-STORAGE.RECLAIM-01 FRE-COMPLIANCE.REPLACE-01 SDS-LAYOUT.OBJECTFILE-01 SDS-LAYOUT.SHARDING-01
Verified by	TC-STORAGE.PUBLISH-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩

[FDS] FDS-STORAGE.PARTIAL-01 Streaming partial hits	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-STORAGE.OBJECTFILE-01
Traces to	FDS-STORAGE.PUBLISH-01
Satisfies	FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 FRE-COMPAT.STREAMFLAG-01 FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.PARTIALABORT-01
Traced from	FDS-STORAGE.EXECUTOR-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01
Verified by	TC-COMPAT.STREAMING-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩

[FDS] FDS-STORAGE.METAUPDATE-01 Metadata update: in place or withdrawn	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-PERSISTENCE.DURABILITY-01 ADR-STORAGE.OBJECTFILE-01
Traces to	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 REQ-EFFICIENCY.FRAGMENTATION-01
Satisfies	FRE-COMPLIANCE.METAABSENT-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.REVALIDATION-01
Traced from	ADR-OBSERVABILITY.METRICS-01 ADR-STORAGE.EXPIRE-01 ADR-STORAGE.METAMOVE-01 FDS-STORAGE.EXECUTOR-01 FDS-STORAGE.EXPIRE-01 FRE-COMPLIANCE.REVALIDATION-01 SDS-LAYOUT.OBJECTFILE-01
Refined by	FDS-OBSERVABILITY.HEADROOM-01
Verified by	TC-COMPLIANCE.REVALIDATION-01
Required coverage	✓ TC-COMPLIANCE.REVALIDATION-01 via ⟨⟨*verify*⟩⟩ — *Extend this TC to also verify FDS-STORAGE.METAUPDATE-01: that an update written in place is visible to lookups when update_meta returns; that metadata outgrowing its region withdraws the record, which lookups then miss while a client already reading it gets its whole body; that the next store replaces a withdrawn record; that a withdrawal whose header write fails leaves the record as it was; that an update applies only to the response it validated; and that zero configured headroom withdraws the record on any growth.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[FDS] FDS-OBSERVABILITY.HEADROOM-01 Headroom-exceeded observability	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-OBSERVABILITY.METRICS-01 FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩

[ADR] ADR-STORAGE.METAMOVE-01 Metadata moves deferred for v1	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Refines	ADR-STORAGE.OBJECTFILE-01
Traces to	FDS-STORAGE.METAUPDATE-01 FRE-COMPAT.EVICTION-01 HLR-SCOPE.V1-01

[ADR] ADR-OBSERVABILITY.METRICS-01 Metrics via the prometheus crate	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Traces to	FDS-STORAGE.METAUPDATE-01
Satisfies	HLR-SCOPE.V1-01
Refined by	FDS-OBSERVABILITY.HEADROOM-01
Required coverage	✓ FDS-OBSERVABILITY.HEADROOM-01 via ⟨⟨*refine*⟩⟩ — *Specify each metric emitted when new metadata outgrows its record's headroom — name, type, unit, and when it changes — the accompanying warning log, and what remains when the metrics feature is disabled.*

[ADR] ADR-STORAGE.EXPIRE-01 Expire by rewriting the metadata	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Traces to	ADR-PERSISTENCE.DURABILITY-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PURGE-01
Refines	ADR-STORAGE.OBJECTFILE-01
Satisfies	FRE-COMPAT.EXPIRE-01
Refined by	FDS-STORAGE.EXPIRE-01
Required coverage	✓ FDS-STORAGE.EXPIRE-01 via ⟨⟨*refine*⟩⟩ — *Specify how a target is resolved, how the expired metadata is derived and written in place, why it always fits, which outcome each case reports, and how the operation behaves once the backend is read-only.*

[FDS] FDS-STORAGE.PURGE-01 Purge via canonical-name unlink	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-PERSISTENCE.DURABILITY-01 ADR-STORAGE.OBJECTFILE-01
Traces to	ADR-STORAGE.RECLAIM-01 FDS-STORAGE.PARTIAL-01 FDS-STORAGE.PUBLISH-01 FRE-COMPLIANCE.REPLACE-01 SDS-LAYOUT.SHARDING-01
Satisfies	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.READERISOLATION-01
Traced from	ADR-COMPAT.ENTRYID-01 ADR-STORAGE.EXPIRE-01 FDS-STORAGE.EXECUTOR-01 FDS-STORAGE.EXPIRE-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.RECLAIM-01
Verified by	TC-STORAGE.PURGE-01
Required coverage	✓ TC-STORAGE.PURGE-01 via ⟨⟨*verify*⟩⟩ — *Verify that purge unlinks only the canonical path derived from the presented target key's combined hash (a primary or a variant), for an Active target and a key-only Exact target alike; that an identified Exact target is reported not-found with nothing unlinked; that purge is idempotent and reports not-found for an absent key; that an invalidation ends a same-key store in flight, which then publishes nothing while the request writing it and a reader already streaming it still read its body, and reports a key holding only such a store purged, while a purge made to evict leaves the store to land; that it never names a writer's temporary nor disturbs a concurrent reader's already-open descriptor; that an invalidation synchronises the directory before it reports, also when it found nothing, while a purge made to evict does not; that a removal is reported carrying no entry identifier; that a purge whose unlink fails reports an error and leaves the record in place; and that it fails once the backend is read-only.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[FDS] FDS-STORAGE.EXPIRE-01 Expire by updating the metadata	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-STORAGE.EXPIRE-01
Traces to	FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PURGE-01 FRE-COMPLIANCE.READERISOLATION-01
Satisfies	FRE-COMPAT.EXPIRE-01 FRE-COMPLIANCE.PURGEABSENT-01
Verified by	TC-STORAGE.EXPIRE-01
Required coverage	✓ TC-STORAGE.EXPIRE-01 via ⟨⟨*verify*⟩⟩ — *Verify that an expiry stops the object being served as a fresh hit while leaving its body byte-identical and readable; that the stored serve-stale windows are not closed by it; that it is idempotent; that it reports not-found for an absent key, for an identified Exact target, and for a record that fails the read-time validation, a withdrawn one included, which it leaves untouched; that it writes its metadata in place, with headroom configured and without, also for a deadline on a whole second; that a concurrent store still replaces the expired record; and that it fails once the backend is read-only.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[ADR] ADR-STORAGE.EXECUTOR-01 File I/O on the blocking pool	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Traces to	ADR-PERSISTENCE.DURABILITY-01 FRE-COMPAT.STREAMDELIVERY-01
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	HLR-COMPLIANCE.RFC-01
Refined by	FDS-STORAGE.EXECUTOR-01
Required coverage	✓ FDS-STORAGE.EXECUTOR-01 via ⟨⟨*refine*⟩⟩ — *Specify which work moves to the blocking pool, what stays on the worker, and how the restart scan bounds the stall it cannot move.*

[FDS] FDS-STORAGE.EXECUTOR-01 Storage calls never block the worker	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-STORAGE.EXECUTOR-01
Traces to	FDS-EVICTION.STARTUP-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PARTIAL-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01
Satisfies	HLR-COMPLIANCE.RFC-01
Verified by	TC-STORAGE.EXECUTOR-01
Required coverage	✓ TC-STORAGE.EXECUTOR-01 via ⟨⟨*verify*⟩⟩ — *Verify that no storage operation holds the executor thread for its duration, by measuring what a task co-resident on a single-worker runtime observes while cache traffic runs.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[ADR] ADR-STORAGE.RECLAIM-01 Reclaim via the restart-time scan	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Traces to	ADR-COMPAT.EVICTION-01 FDS-EVICTION.STARTUP-01 FRE-COMPAT.HOTUPGRADE-01
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	FRE-COMPAT.STARTRECLAIM-01
Traced from	ADR-STORAGE.ATOMICPUBLISH-01 ADR-STORAGE.WRITERID-01 FDS-STORAGE.PURGE-01
Refined by	FDS-STORAGE.RECLAIM-01
Required coverage	✓ FDS-STORAGE.RECLAIM-01 via ⟨⟨*refine*⟩⟩ — *Specify the concrete detection and removal procedure for each class of file the scan reclaims, riding Eviction Recovery's existing enumeration, and the per-class count and bytes of reclaimed files each scan reports.*

[FDS] FDS-STORAGE.RECLAIM-01 Reclaiming orphaned files	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-STORAGE.RECLAIM-01 ADR-STORAGE.WRITERID-01
Traces to	FDS-EVICTION.STARTUP-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 FRE-COMPAT.HOTUPGRADE-01 SDS-LAYOUT.OBJECTFILE-01 SDS-LAYOUT.SHARDING-01
Satisfies	FRE-COMPAT.STARTRECLAIM-01
Traced from	FDS-EVICTION.STARTUP-01 SDS-LAYOUT.KEYSEGMENT-01
Verified by	TC-STORAGE.RECLAIM-01
Required coverage	✓ TC-STORAGE.RECLAIM-01 via ⟨⟨*verify*⟩⟩ — *Verify that a temporary whose writer is not running (its lease missing, or present but unlocked) is reclaimed, and one whose writer is running is left; that a stopped writer's lease is removed and a running writer's kept; that a damaged, foreign-format, misplaced, or withdrawn record is reclaimed; that a variant is admitted whatever its primary holds; that a record replaced by a live store between classification and unlink survives; that a directory or entry it cannot list or read is skipped, named in a warning, and counted; and that the log summary, and the reclaim counters when the metrics feature is enabled, report each class.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[ADR] ADR-EFFICIENCY.COMPACTION-01 Compaction deferred for v1	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Refines	ADR-PERSISTENCE.MECHANISM-01
Traces to	ADR-STORAGE.OBJECTFILE-01
Conflicts with	HLR-EFFICIENCY.SPACE-01 REQ-EFFICIENCY.FRAGMENTATION-01
Traced from	SDS-LAYOUT.OBJECTFILE-01 SDS-LAYOUT.SHARDING-01

[ADR] ADR-COMPAT.EVICTION-01 Eviction-state reconstruction strategy	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	FRE-COMPAT.EVICTION-01
Traced from	ADR-COMPAT.ENTRYID-01 ADR-COMPAT.READONLY-01 ADR-STORAGE.RECLAIM-01
Refined by	FDS-EVICTION.STARTUP-01
Required coverage	✓ FDS-EVICTION.STARTUP-01 via ⟨⟨*refine*⟩⟩ — *Specify the concrete reconstruction procedure, and the API surface that orchestrates it, realising this strategy.*

[ADR] ADR-COMPAT.ENTRYID-01 No storage-assigned entry IDs	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Traces to	ADR-COMPAT.EVICTION-01 FDS-EVICTION.STARTUP-01 FDS-STORAGE.PURGE-01
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	FRE-COMPAT.EVICTION-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[ADR] ADR-COMPAT.READONLY-01 Read-only on the upgrade signal	
Stereotype	⟨⟨*design constraint*⟩⟩
Status	`draft`
Traces to	ADR-COMPAT.EVICTION-01
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	FRE-COMPAT.HOTUPGRADE-01
Refined by	FDS-COMPAT.READONLY-01
Required coverage	✓ FDS-COMPAT.READONLY-01 via ⟨⟨*refine*⟩⟩ — *Specify the read-only operation and its integration with the execution-phase broadcast, the behaviour of each Storage call after it, and the incoming process's start condition for its restart procedure.*

[FRE] FRE-COMPLIANCE.HEADERS-01 Faithful header round-trip	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPLIANCE.RFC-01
Traced from	FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.VALIDATORS-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.HEADERS-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.TRAILERS-01 Trailer fields kept distinct	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.TRAILERS-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.BODY-01 Faithful round-trip of the body	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.BODY-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.FRESHNESS-01 Preserve freshness and age inputs	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.REVALIDATION-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.FRESHNESS-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.VALIDATORS-01 Preserve validator fields	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.REVALIDATION-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.VALIDATORS-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.REVALIDATION-01 Freshen metadata	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FDS-STORAGE.METAUPDATE-01
Derives from	HLR-COMPLIANCE.RFC-01
Traced from	FDS-COMPAT.READONLY-01 FRE-COMPAT.EXPIRE-01 FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.METAABSENT-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.VALIDATORS-01 SDS-LAYOUT.OBJECTFILE-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01 TC-COMPLIANCE.VALIDATORS-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.METAATOMIC-01 Atomic metadata update	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.REVALIDATION-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.METAPERSIST-01 Freshened metadata persists	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.REVALIDATION-01
Derives from	HLR-COMPLIANCE.RFC-01 HLR-PERSISTENCE.CRASHSAFE-01
Traced from	ADR-PERSISTENCE.DURABILITY-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01 TC-PERSISTENCE.CRASHSAFE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.METAABSENT-01 Update of an absent key	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.REVALIDATION-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	FDS-STORAGE.METAUPDATE-01
Verified by	TC-COMPLIANCE.REVALIDATION-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.REPLACE-01 Replace stored responses atomically	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FDS-STORAGE.PUBLISH-01 FRE-COMPAT.STREAMING-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01
Traced from	FDS-STORAGE.PURGE-01 FRE-COMPAT.CONCURRENT-01 FRE-COMPLIANCE.READERISOLATION-01
Verified by	TC-COMPLIANCE.REPLACE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.READERISOLATION-01 Readers survive replace/purge	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.REPLACE-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01
Traced from	FDS-STORAGE.EXPIRE-01
Verified by	TC-COMPLIANCE.PURGE-01 TC-COMPLIANCE.REPLACE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.VARY-01 Store and select Vary variants	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPLIANCE.RFC-01
Traced from	FRE-COMPLIANCE.VARYISOLATION-01
Satisfied by	SDS-LAYOUT.SHARDING-01
Verified by	TC-COMPLIANCE.VARY-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.VARYISOLATION-01 Variant isolation	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.VARY-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	SDS-LAYOUT.SHARDING-01
Verified by	TC-COMPLIANCE.VARY-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ SDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.PURGE-01 Honour purge and invalidation	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	FDS-STORAGE.PURGE-01
Traced from	FRE-COMPAT.EXPIRE-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPLIANCE.VARYISOLATION-01
Verified by	TC-COMPLIANCE.PURGE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.PURGEABSENT-01 Purge or expiry of absent key	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.EXPIRE-01 FRE-COMPLIANCE.PURGE-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	FDS-STORAGE.EXPIRE-01 FDS-STORAGE.PURGE-01
Verified by	TC-COMPLIANCE.PURGE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.PURGEDURABLE-01 Purge acknowledged once durable	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPLIANCE.PURGE-01
Derives from	HLR-COMPLIANCE.RFC-01 HLR-PERSISTENCE.CRASHSAFE-01
Traced from	ADR-PERSISTENCE.DURABILITY-01 FDS-COMPAT.READONLY-01 FRE-COMPAT.HOTUPGRADE-01
Satisfied by	FDS-STORAGE.PURGE-01
Verified by	TC-COMPLIANCE.PURGE-01 TC-PERSISTENCE.CRASHSAFE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.INCOMPLETE-01 Never serve incomplete responses	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Derives from	HLR-COMPLIANCE.RFC-01 HLR-PERSISTENCE.CRASHSAFE-01
Traced from	ADR-PERSISTENCE.DURABILITY-01 ADR-PERSISTENCE.MECHANISM-01 FRE-COMPAT.ABORTSPACE-01 FRE-COMPAT.STARTRECLAIM-01 FRE-COMPAT.WRITEFAIL-01 FRE-COMPLIANCE.PARTIALABORT-01
Satisfied by	ADR-STORAGE.ATOMICPUBLISH-01 FDS-STORAGE.PUBLISH-01 SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-COMPLIANCE.INCOMPLETE-01 TC-PERSISTENCE.CRASHSAFE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[FRE] FRE-COMPLIANCE.PARTIALABORT-01 Aborted partial hits end in error	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Traces to	FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.INCOMPLETE-01
Derives from	HLR-COMPLIANCE.RFC-01
Satisfied by	FDS-STORAGE.PARTIAL-01
Traced from	FRE-COMPAT.STREAMEND-01
Verified by	TC-COMPLIANCE.INCOMPLETE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ FDS via ⟨⟨*satisfy*⟩⟩

[SDS] SDS-LAYOUT.SHARDING-01 Directory sharding	
Stereotype	⟨⟨*structural requirement*⟩⟩
Status	`draft`
Traces to	ADR-EFFICIENCY.COMPACTION-01 FDS-STORAGE.PUBLISH-01 FRE-COMPAT.STREAMING-01
Refines	ADR-PERSISTENCE.MECHANISM-01
Satisfies	FRE-COMPLIANCE.VARY-01 FRE-COMPLIANCE.VARYISOLATION-01 REQ-EFFICIENCY.DIRSCALE-01
Refined by	FDS-LAYOUT.STARTUP-01
Traced from	FDS-STORAGE.PUBLISH-01 FDS-STORAGE.PURGE-01 FDS-STORAGE.RECLAIM-01 SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-LAYOUT.SHARDING-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩

[FDS] FDS-LAYOUT.STARTUP-01 Startup: shard directory cache	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	SDS-LAYOUT.SHARDING-01
Verified by	TC-COMPAT.WRITEFAIL-01 TC-LAYOUT.DIRCACHE-01
Required coverage	✓ TC-LAYOUT.DIRCACHE-01 via ⟨⟨*verify*⟩⟩ — *Verify the cache is rebuilt accurately from disk at every construction, and that concurrent first-time stores into the same shard never fail or double-create.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[SDS] SDS-LAYOUT.OBJECTFILE-01 Object-file record layout	
Stereotype	⟨⟨*structural requirement*⟩⟩
Status	`draft`
Traces to	ADR-EFFICIENCY.COMPACTION-01 FDS-EVICTION.STARTUP-01 FDS-STORAGE.METAUPDATE-01 FDS-STORAGE.PUBLISH-01 FRE-COMPAT.EVICTION-01 FRE-COMPAT.STREAMING-01 FRE-COMPLIANCE.REVALIDATION-01 SDS-LAYOUT.SHARDING-01
Refines	ADR-PERSISTENCE.DURABILITY-01 ADR-STORAGE.OBJECTFILE-01
Satisfies	FRE-COMPAT.FORMATUPGRADE-01 FRE-COMPLIANCE.BODY-01 FRE-COMPLIANCE.FRESHNESS-01 FRE-COMPLIANCE.HEADERS-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.TRAILERS-01 FRE-COMPLIANCE.VALIDATORS-01
Traced from	FDS-STORAGE.RECLAIM-01
Refined by	SDS-LAYOUT.KEYSEGMENT-01
Verified by	TC-LAYOUT.OBJECTFILE-01
Required coverage	✓ SDS-LAYOUT.KEYSEGMENT-01 via ⟨⟨*refine*⟩⟩ — *Specify the key segment's byte encoding — its fields' order and sizes, how the presence of a variance hash is marked, and how the tag is delimited — so that each key has exactly one encoding.* ✓ TC-LAYOUT.OBJECTFILE-01 via ⟨⟨*verify*⟩⟩ — *Verify that a reader accepts a record only when its magic, version, header and metadata CRCs, and stored key all validate, its metadata segments fit its capacity, its body offset lies within the file, and the file's length is exactly the computed body offset plus body_len; that any single violation is a miss and leaves the file in place; that a body whose checksum mismatches is read to a normal end, and is logged when the read-time check is on; and that a metadata read racing an in-place update returns one complete version.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[SDS] SDS-LAYOUT.KEYSEGMENT-01 Key segment encoding	
Stereotype	⟨⟨*structural requirement*⟩⟩
Status	`draft`
Traces to	FDS-EVICTION.STARTUP-01 FDS-STORAGE.RECLAIM-01 FRE-COMPAT.FORMATUPGRADE-01
Refines	SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-LAYOUT.KEYSEGMENT-01
Required coverage	✓ TC-LAYOUT.KEYSEGMENT-01 via ⟨⟨*verify*⟩⟩ — *Verify that keys with and without a variance hash, and with an empty and a non-empty tag, encode to the specified layout and decode back to themselves; that a segment too short for its fields, a variance flag other than 0 or 1, and a tag that is not UTF-8 are each rejected; and that the restart-time scan reclaims a record whose key does not decode.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[FDS] FDS-EVICTION.STARTUP-01 Startup: reconstruction procedure	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-COMPAT.EVICTION-01
Traces to	ADR-PERSISTENCE.MECHANISM-01 FDS-COMPAT.READONLY-01 FDS-STORAGE.RECLAIM-01
Satisfies	FRE-COMPAT.EVICTION-01 FRE-COMPAT.HOTUPGRADE-01
Traced from	ADR-COMPAT.ENTRYID-01 ADR-STORAGE.RECLAIM-01 FDS-STORAGE.EXECUTOR-01 FDS-STORAGE.RECLAIM-01 SDS-LAYOUT.KEYSEGMENT-01 SDS-LAYOUT.OBJECTFILE-01
Verified by	TC-EVICTION.RECONSTRUCT-01
Required coverage	✓ TC-EVICTION.RECONSTRUCT-01 via ⟨⟨*verify*⟩⟩ — *Verify that when recover() returns the manager holds exactly the published set with record sizes, including when stores, replacements, and purges run during the scan, and that no snapshot or delta log is read or written at any point.* ✓ IMP via ⟨⟨*satisfy*⟩⟩

[FDS] FDS-COMPAT.READONLY-01 Read-only on the upgrade signal	
Stereotype	⟨⟨*functional requirement*⟩⟩
Status	`draft`
Refines	ADR-COMPAT.READONLY-01
Satisfies	FRE-COMPAT.HOTUPGRADE-01
Traces to	FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.REVALIDATION-01
Traced from	FDS-EVICTION.STARTUP-01
Verified by	TC-COMPAT.HOTUPGRADE-01
Required coverage	✓ TC via ⟨⟨*verify*⟩⟩ ✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.RFC-01 No regression against MemCache	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	HLR-COMPLIANCE.RFC-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.HEADERS-01 Header fields round-trip faithfully	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.HEADERS-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.TRAILERS-01 Trailer fields kept distinct	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.TRAILERS-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.BODY-01 Body round-trips byte for byte	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.BODY-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.FRESHNESS-01 Freshness and age inputs preserved	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.FRESHNESS-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.VALIDATORS-01 Validator fields preserved	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.REVALIDATION-01 FRE-COMPLIANCE.VALIDATORS-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.REVALIDATION-01 Metadata freshened atomically	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-OBSERVABILITY.HEADROOM-01 FDS-STORAGE.METAUPDATE-01 FRE-COMPLIANCE.METAABSENT-01 FRE-COMPLIANCE.METAATOMIC-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.REVALIDATION-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.VARY-01 Vary variants stored and selected	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.VARY-01 FRE-COMPLIANCE.VARYISOLATION-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.PURGE-01 Purge makes all variants unreachable	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.PURGE-01 FRE-COMPLIANCE.PURGEABSENT-01 FRE-COMPLIANCE.PURGEDURABLE-01 FRE-COMPLIANCE.READERISOLATION-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.INCOMPLETE-01 Incomplete responses never served	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPAT.STARTRECLAIM-01 FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.PARTIALABORT-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPLIANCE.REPLACE-01 Stores replace atomically	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.READERISOLATION-01 FRE-COMPLIANCE.REPLACE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-PERSISTENCE.CRASHSAFE-01 Host crash serves no bad content	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPLIANCE.INCOMPLETE-01 FRE-COMPLIANCE.METAPERSIST-01 FRE-COMPLIANCE.PURGEDURABLE-01 HLR-PERSISTENCE.CRASHSAFE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-PERSISTENCE.RESTART-01 Objects survive a process exit	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	HLR-PERSISTENCE.DURABLE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-PERSISTENCE.CAPACITY-01 Cache larger than host RAM	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	HLR-PERSISTENCE.CAPACITY-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-EFFICIENCY.SPACE-01 Space overhead measured	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	HLR-EFFICIENCY.SPACE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-PERFORMANCE.LOOKUP-01 Lookup latency flat with size	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	HLR-PERFORMANCE.LOOKUP-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.EVICTION-01 Exact eviction state after restart	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPAT.EVICTION-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.HOTUPGRADE-01 Cache consistent across hot upgrade	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-COMPAT.READONLY-01 FRE-COMPAT.HOTUPGRADE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.STREAMING-01 Followers stream a partial hit	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-STORAGE.PARTIAL-01 FRE-COMPAT.STREAMDELIVERY-01 FRE-COMPAT.STREAMEND-01 FRE-COMPAT.STREAMFLAG-01 FRE-COMPAT.STREAMING-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.UNSIZED-01 Unsized body accepted and persisted	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPAT.UNSIZED-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.CONCURRENT-01 Same-key writes stay isolated	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPAT.CONCURRENT-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.FORMATUPGRADE-01 Foreign records are misses	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPAT.FORMATUPGRADE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.WRITEFAIL-01 Write failures fail the store	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-LAYOUT.STARTUP-01 FRE-COMPAT.WRITEFAIL-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-COMPAT.ABORTSPACE-01 Aborted stores release space	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FRE-COMPAT.ABORTSPACE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-EVICTION.RECONSTRUCT-01 Reconstruction converges exactly	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-EVICTION.STARTUP-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-STORAGE.PUBLISH-01 Atomic same-directory publication	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-STORAGE.PUBLISH-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-STORAGE.PURGE-01 Purge unlinks canonical name only	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-STORAGE.PURGE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-STORAGE.EXPIRE-01 Expire keeps the body readable	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-STORAGE.EXPIRE-01 FRE-COMPAT.EXPIRE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-STORAGE.EXECUTOR-01 Storage never blocks the worker	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-STORAGE.EXECUTOR-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-STORAGE.RECLAIM-01 Orphan reclaim at restart	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-STORAGE.RECLAIM-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-LAYOUT.SHARDING-01 Shard layout and directory bound	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	REQ-EFFICIENCY.DIRSCALE-01 SDS-LAYOUT.SHARDING-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-LAYOUT.DIRCACHE-01 Shard cache matches disk at start	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	FDS-LAYOUT.STARTUP-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-LAYOUT.OBJECTFILE-01 Damaged records treated as a miss	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	SDS-LAYOUT.OBJECTFILE-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

[TC] TC-LAYOUT.KEYSEGMENT-01 Key segment round trip	
Stereotype	⟨⟨*test case*⟩⟩
Status	`draft`
Verifies	SDS-LAYOUT.KEYSEGMENT-01
Required coverage	✓ IMP via ⟨⟨*satisfy*⟩⟩

Orphan Report

Artifacts with no incoming or outgoing links:

No orphan artifacts found.

Coverage Gaps

Requirements with at least one unsatisfied coverage obligation:

Type	ID	Title	Missing Coverage
REQ	REQ-EFFICIENCY.FRAGMENTATION-01	Sub-linear small-object overhead	☐ ADR via ⟨⟨*satisfy*⟩⟩

Planned Requirements

Specific requirement IDs forward-declared with `\reqexpect` but not yet linked back – either not yet written, or written without the relationship that would satisfy the obligation:

No planned requirements outstanding.

Unaccounted Artifacts

Artifacts that neither state a coverage obligation (`\reqcover`) nor are terminal (a terminal-flagged type or `\reqterminal`) nor deprecated. Each should either declare the downstream coverage it expects or be marked terminal:

No unaccounted artifacts found.

A. Storage Backend Alternatives

This appendix expands the alternatives weighed by ADR-PERSISTENCE.MECHANISM-01: how each design organises bytes on disk, what storing a streamed HTTP response looks like in it, and why it lost to the plain file system for a first version (HLR-SCOPE.V1-01).

A.1 Raw Device / Cyclone Store: Apache Traffic Server

Traffic Server's cache runs directly on raw disk spans (or span files), bypassing the file system. Each storage stripe is treated as a circular buffer: a *write cursor* advances through it, everything written lands at the cursor through a multi-megabyte *aggregation buffer* flushed sequentially, and objects larger than the target fragment size (1 MB by default) are split into *fragments* chained through directory entries — the design the documentation calls a *cyclone cache*.[1] Eviction is implicit: as the cursor wraps, it overwrites the least recently written data, and the directory entries pointing there die with it.

Example. A 2.5 MB response arriving chunk by chunk accumulates in the aggregation buffer and is cut into three fragments, each appended at the cursor; the directory chains them under the object's key. A crash between the second and third fragment leaves a truncated chain and a stale directory; on restart the directory is rebuilt by scanning the stripe, and the incomplete chain is simply never resurrected.

That machinery — aggregation buffer, fragment chaining, directory persistence, recovery scan — is what makes the model work, and none of it exists outside Traffic Server's own process: the cache is already its own internal library target,[2] but its core object is an EventSystem continuation, not a caller of one,[3] so adopting it means adopting Traffic Server's own cooperative-threading runtime alongside it, not just relinking a self-contained module. Squid's Rock store, which formally replaced its earlier COSS store in Squid 3.5,[4] solves the same duties with different trade-offs: a single database file of fixed-size slots of at most 32 KB, with a larger object spread across several slots.[5] Rebuilding machinery of either class from scratch is the largest bespoke surface among the alternatives (HLR-SCOPE.V1-01).

A.2 Embeddable Cache Engine: Meta's CacheLib

CacheLib is a C++ library combining a slab-allocated RAM cache with the *Navy* flash engine: small items go to BigHash,[6] effectively a giant fixed-bucket hash map

[1] Apache Traffic Server documentation: "Cache Architecture" (Cyclone Buffer)

[2] Apache Traffic Server source (release 10.1.4): `src/iocore/cache/CMakeLists.txt`, lines 18–39

[3] Apache Traffic Server source (release 10.1.4): `src/iocore/cache/CacheVC.h`, line 45

[4] Squid 3.5 release notes: "COSS storage type is formally replaced by Rock storage type."

[5] Squid 3.5 release notes: the rock slot size defaults to, and is at most, 32 KB, with "new support for larger than 32KB objects in both rock type cache and shared memory cache."

[6] CacheLib documentation: "Navy Overview" (Block Cache)

on the device, while larger items go to BlockCache, which divides the device into equally sized append-only regions (16 MB by default) and reclaims whole regions to make space. Allocation is size-first: `allocate` returns a writable buffer of the requested length, and data that exceeds the maximum allocation — or arrives as a stream — is stored as a parent item plus chained child items[7] added one at a time, so the total size need not be known in advance. This eviction and region-reclaim machinery is compatible with `pingora-cache`: the `EvictionManager` `pingora-cache` drives is optional,[8] so a backend can leave it unset and let CacheLib manage capacity on its own (FRE-COMPAT.EVICTION-01).

Example. Caching a streamed response means allocating a parent item for the metadata, then, per arriving chunk, allocating a chained child of that chunk's size, copying the bytes in, and linking it — workable, with every chunk boundary being the application's bookkeeping. Across a hot upgrade the model stops fitting: its persistence is a shutdown-and-reattach protocol over shared memory with one active cache instance per path,[9] so the old and new process cannot serve one store concurrently (FRE-COMPAT.HOTUPGRADE-01).

The FFI boundary between Rust and C++, together with the one-active-instance-per-path persistence model above, is what rules CacheLib out for a first version: the integration surface outweighs the engine's maturity.

A.3 Embedded Database: libmdbx/LMDB, RocksDB

These engines store *records* transactionally — copy-on-write B-trees in the LMDB family, a log-structured merge tree in RocksDB — so a cached object becomes a metadata record plus body-chunk records written as they arrive.

Example. A body that streams for minutes either holds a write transaction open for its whole duration or is committed chunk-record by chunk-record; a reader streaming a partial hit must discover freshly committed chunks and wait for the next (FRE-COMPAT.STREAMING-01) — coordination the store does not provide, so the in-memory machinery exists beside it anyway. Cross-process use during a hot upgrade is engine-specific: RocksDB admits no multi-process writers,[10] so the incoming process could not write while the outgoing one still holds the database open, while the LMDB family permits multi-process access with no more than one write transaction at a time[11] — no obstacle once the outgoing process stops writing (ADR-COMPAT.READONLY-01). Pure-Rust engines such as `redb` and `fjall` remove the FFI boundary but not the record chunking or the streaming coordination above.

What the engine genuinely buys — record packing and crash-consistent commits — is ground the file-system design cedes knowingly: packing returns as a derived efficiency requirement (REQ-EFFICIENCY.FRAGMENTATION-01), deferred for the first version (ADR-EFFICIENCY.COMPACTION-01), and object-level crash safety is recovered with file-system idioms: same-directory rename publication

[7] CacheLib documentation: "Chained items" (Chained allocations)
[8] `HttpCache::enable` takes it as an `Option`.
[9] CacheLib documentation: "Cache persistence" (Common failure scenarios)
[10] RocksDB wiki: "RocksDB FAQ" (Basic Read/Write)
[11] libmdbx README (GitHub mirror), "Gotchas"

(ADR-STORAGE.ATOMICPUBLISH-01), selective synchronisation with checksums (ADR-PERSISTENCE.DURABILITY-01), and a restart-time cleanup scan (ADR-STORAGE.RECLAIM-01).

B. The http-tests/cache-tests Test Suite

This appendix details http-tests/cache-tests, the external test suite the compliance test-case chapter's opening note relies on for TC-COMPLIANCE.RFC-01. It covers what the suite is, how it is structured, and exactly which parts of it cover which FRE-COMPLIANCE.* requirement — and which parts do not, so its adoption is not mistaken for retiring the hand-written test cases it complements. Citations below are pinned to commit b55b8bd[1] rather than main, matching this document's practice elsewhere of citing a fixed revision (e.g. \pgsrc pinning to Pingora tag 0.9.0) rather than a moving branch.

B.1 What It Is

cache-tests evaluates the behaviour of HTTP caches — browsers, proxy caches, and CDNs alike — against RFC 9111 and RFC 9110, publishing results at cache-tests.fyi interleaved with the specification text itself. It is distributed under the BSD-3-Clause licence and has seen continuous, active maintenance (over 1,100 commits as of this writing). The project is explicit that it is *not* a conformance test suite in the formal sense — it is a tool to assess how a cache behaves, with results including not just pass/fail but "suboptimal" and observation-only outcomes where the specification leaves room for implementation choice. That framing matters here: a reported failure does not, on its own, distinguish a deviation the file-system backend introduced from a pre-existing gap in the (admittedly incomplete) RFC 9111 conformance of pingora-cache itself. TC-COMPLIANCE.RFC-01 therefore never takes the suite's verdict on its own: it runs the suite against the same proxy with the stock MemCache as a reference, and counts only outcomes that are worse with the file-system backend.

B.2 Architecture

The suite has four moving parts:

- **Origin server** (test-engine/server) — a Node.js HTTP server, listening on port 8000 by default, that every test case addresses with a unique identifier and that answers with exactly the headers, timing, and body bytes that test needs. This *is* the "mocked upstream" the compliance test-case chapter's opening note calls for — it does not need to be built.
- **CLI client** (test-engine/cli.mjs) — drives requests through a cache-under-test and records, per test ID, a pass, fail, "suboptimal", or observation-only result. Invoked as npm run cli -base=<proxy-url> against a reverse proxy or CDN, or with HTTP_PROXY set against a forward proxy. Results can be exported to JSON, which is the mechanism the "run cache-tests once per backend, diff the two result sets" comparison in the compliance test-case chapter depends on.

[1] http-tests/cache-tests, commit b55b8bd (2026-07-20), the suite's main branch as of this writing

- **Browser harness** (`test-browser.html`) — runs the same tests against a browser's own cache; not applicable to this project.
- **Docker harness** (`docker/`, image `mnot/proxy-cache-tests`) — pre-wires configurations for Squid, nginx, Apache Traffic Server, Apache httpd, Varnish, Caddy, and HAProxy that each point their upstream at the origin server on the Docker host, then exposes each proxy on its own mapped port for the CLI client to target. This is a directly-imitable pattern for how the Pingora test-subject proxy should be wired: origin fixed, proxy configuration the only variable.

Each topic under `tests/*.mjs` exports a named suite of individually-identified cases (e.g. `freshness-max-age`, `vary-no-match`).

B.3 Mapping to `FRE-COMPLIANCE.*`

- FRE-COMPLIANCE.HEADERS-01 — `tests/headers.mjs`[2] (its per-header round-trip cases; its omission cases, such as dropping fields listed in `Connection`, exercise the header handling in `pingora-cache` itself, which both runs share) and `tests/cc-response.mjs`/`cc-parse.mjs`[3] for `Cache-Control` field-parsing fidelity.
- FRE-COMPLIANCE.FRESHNESS-01 — `tests/cc-freshness.mjs`[4], `tests/expires-freshness.mjs`, `tests/age-parse.mjs`[5], and `tests/heuristic-freshness.mjs`, including `freshness-max-age-max`/`-max-minus-1`/`-max-plus` (the 2147483648-second overflow cap FRE-COMPLIANCE.FRESHNESS-01 calls out by name), `age-parse-large`/`-large-minus-one`, and the `freshness-max-age-s-maxage-*` shared-vs-private precedence cases.
- FRE-COMPLIANCE.VALIDATORS-01 — `tests/conditional-etag.mjs`[6] and `tests/conditional-lm.mjs`, covering weak/strong, quoted/unquoted, and obs-text ETag variants and `If-Modified-Since` matching.
- FRE-COMPLIANCE.REVALIDATION-01 — `tests/update304.mjs`[7] (`304-lm-use-stored-*`, `304-etag-update-response-*`) and `tests/updateHead.mjs` (`head-200-update`, `head-200-freshness-update`).
- FRE-COMPLIANCE.VARY-01 — `tests/vary.mjs`[8] and `tests/vary-parse.mjs` (`vary-match`, `vary-no-match`, `vary-omit-stored`, `vary-cache-key`, and the two-way `Vary` cases).
- FRE-COMPLIANCE.BODY-01's seek fidelity — `tests/partial.mjs`[9] through its cases that serve a range from a complete stored response

[2] `tests/headers.mjs`
[3] `tests/cc-response.mjs`
[4] `tests/cc-freshness.mjs`
[5] `tests/age-parse.mjs`
[6] `tests/conditional-etag.mjs`
[7] `tests/update304.mjs`
[8] `tests/vary.mjs`
[9] `tests/partial.mjs`

(partial-store-complete-reuse-partial and its -no-last/-suffix variants). Its cases that store a 206 (Partial Content) response and reuse it (partial-store-partial-reuse-partial, partial-use-stored-headers) exercise combining partial content, which neither backend does (FRE-COMPLIANCE.INCOMPLETE-01), so both runs report them alike.

- Cacheable non-200 status codes, whose faithful round trip FRE-COMPLIANCE.HEADERS-01 requires — tests/status.mjs[10] (status-<code>-fresh/-stale, status-599-must-understand, status-200-must-understand).

B.4 Coverage Gaps

cache-tests does not cover every FRE-COMPLIANCE.* requirement; the following remain the sole responsibility of the corresponding hand-written TC-COMPLIANCE.* test case:

- FRE-COMPLIANCE.TRAILERS-01 — cache-tests drives requests over plain HTTP clients that never expose trailer fields.
- FRE-COMPLIANCE.PURGE-01, with FRE-COMPLIANCE.PURGEABSENT-01 and FRE-COMPLIANCE.PURGEDURABLE-01 — tests/invalidation.mjs[11] tests RFC 9111 §4.4 *automatic* invalidation on unsafe methods (PUT, POST, DELETE), which this specification places explicitly out of scope: pingora-cache invalidates only on an explicit purge request, never on an unsafe method, leaving that RFC 9111 duty to the proxy application. Same name, different feature — not real coverage of *explicit* purge.
- FRE-COMPLIANCE.REPLACE-01's concurrent-writer race behaviour, FRE-COMPLIANCE.READERISOLATION-01, FRE-COMPLIANCE.METAATOMIC-01, FRE-COMPLIANCE.PARTIALABORT-01, and FRE-COMPLIANCE.INCOMPLETE-01's crash/torn-write handling — cache-tests treats the cache as an HTTP black box; it cannot orchestrate concurrent same-key access, pause a store mid-body, or kill the process under test mid-store.
- FRE-COMPLIANCE.METAABSENT-01 and FRE-COMPLIANCE.VARYISOLATION-01 — reaching them needs direct Storage calls, or a per-variant mutation sequence, that an HTTP client cannot drive.
- Every restart-survival assertion, including all of FRE-COMPLIANCE.METAPERSIST-01, across the FRE-COMPLIANCE.* requirements — cache-tests cannot restart the process under test between requests.

[10] tests/status.mjs
[11] tests/invalidation.mjs